



Ultra Accelerator Link Consortium, Inc.
Specification

UALink Common Specification
Rev 2.0

Legal Notice

LEGAL NOTICE FOR THIS SPECIFICATION FROM ULTRA ACCELERATOR LINK CONSORTIUM, INC.

© 2025-2026 ULTRA ACCELERATOR LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.

This UALink Common Specification Draft Rev 2.0 (this “**UALink Specification**” or this “**document**”) is owned by and is proprietary to Ultra Accelerator Link Consortium, Inc., a Delaware nonprofit corporation (sometimes referred to as “**UALink**” or the “**UALink Consortium**” or the “**Company**”) and/or its successors and assigns.

NOTICE TO USERS WHO ARE MEMBERS OF ULTRA ACCELERATOR LINK CONSORTIUM, INC.:

Members of the UALink Consortium (sometimes referred to as a “**UALink Member**”) must be and remain in compliance with all of the following UALink Consortium documents, policies and/or procedures (collectively, the “**UALink Governing Documents**”) in order for such UALink Member’s use and/or implementation of this UALink Specification to receive and enjoy any of the rights, benefits, privileges, and/or protections of UALink Consortium membership, including without limiting those set forth in: (i) the UALink Consortium’s Intellectual Property Policy; (ii) the UALink Consortium’s Bylaws; (iii) any and all other UALink Consortium policies and procedures; and (iv) the UALink Member’s Participation Agreement.

NOTICE TO NON-MEMBERS OF ULTRA ACCELERATOR LINK CONSORTIUM, INC.:

If you are **not** a UALink Member and you have obtained a copy of this UALink Specification, you only have a right to review this document or make reference to or cite this document. Any references or citations to this document must acknowledge the Ultra Accelerator Link Consortium, Inc.’s sole and exclusive copyright ownership of this UALink Specification. The proper copyright citation or reference is as follows: “**© 2025-2026 ULTRA ACCELERATOR LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.**” When making any such citation or reference to this document you are not permitted to revise, alter, modify, make any derivatives of, or otherwise amend the referenced portion of this document in any way without the prior express written permission of the Ultra Accelerator Link Consortium, Inc.

Nothing contained in this UALink Specification shall be deemed as granting (either expressly or impliedly) to any party that is **not** a UALink Member: (i) any kind of license to implement or use this UALink Specification or any portion or content described or contained therein, or any kind of license in or to any other intellectual property owned or controlled by the UALink Consortium, including without limitation any trademarks of the UALink Consortium; or (ii) any benefits and/or rights as a UALink Member under any UALink Governing Documents. For clarity, and without limiting the foregoing notice in any way, if you are **not** a UALink Member but still elect to implement this UALink Specification or any portion described herein, you are hereby given notice that your election to do so does not give you any of the rights, benefits, and/or protections of the UALink Members, including without limitation any of the rights, benefits, privileges or protections given to a UALink Member under the UALink Consortium’s Intellectual Property Policy.

LEGAL DISCLAIMERS AND ADDITIONAL NOTICE FOR ALL PARTIES:

THIS DOCUMENT AND ALL SPECIFICATIONS AND/OR OTHER CONTENT PROVIDED HEREIN ARE PROVIDED ON AN “AS IS” BASIS. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, ULTRA ACCELERATOR LINK CONSORTIUM, INC (ALONG WITH THE CONTRIBUTORS TO THIS DOCUMENT) HEREBY DISCLAIM ALL REPRESENTATIONS, WARRANTIES AND/OR COVENANTS, EITHER EXPRESS OR IMPLIED, STATUTORY OR AT COMMON LAW, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, VALIDITY, AND/OR NON-INFRINGEMENT.

In the event this UALink Specification makes any references (including without limitation any incorporation by reference) to another standard’s setting organization’s or any other party’s (“**Third Party**”) content or work, including without limitation any specifications or standards of any such Third Party (“**Third Party Specification**”), you are hereby notified that your use or implementation of any Third Party Specification: (i) is not governed by any of the UALink Governing Documents; (ii) may require your use of a Third Party’s patents, copyrights or other intellectual property rights, which in turn may require you to independently obtain a license or other consent from that Third Party in order to have full rights to implement or use that Third Party Specification; and/or (iii) may be governed by the intellectual property policy or other policies or procedures of the Third Party which owns the Third Party Specification. Any trademarks or service marks of any Third Party which may be referenced in this UALink Specification is owned by the respective owner of such marks.

The **UALINK™** trademark, the **ULTRA ACCELERATOR LINK CONSORTIUM™** trademark, and related logos (the “**UALink Trademarks**”) are trademarks owned by Ultra Accelerator Link Consortium, Inc., a Delaware corporation, and it hereby reserves all rights, title and interest in and to all of its UALink Trademarks.

Table of Contents

Legal Notice.....	2
Table of Contents.....	4
List of Figures.....	9
List of Tables.....	12
Revision History.....	15
Preface.....	17
About This Specification.....	17
Terminology	19
1 Introduction.....	20
1.1 Multi-Node Accelerator System.....	20
1.2 Accelerator System Node	23
1.3 UALink Stack Interface Layers.....	24
1.3.1 Protocol Layer	24
1.3.2 Transaction Layer	25
1.3.3 Data Link Layer	25
1.3.4 Physical Layer.....	25
1.4 UALink Address Translation Model.....	26
1.4.1 Remote Memory Access (RMA).....	26
1.5 UALink Coherency.....	28
2 UPLI Interface Definition and Operation Rules.....	29
2.1 UPLI Interface	29
2.2 UALink Stack Component.....	33
2.3 UALink UPLI Request and Response Paths.....	35
2.4 Routing a Transaction from End-to-End.....	37
2.5 UPLI Channel Time Division Multiplexing (TDM).....	41
2.6 UALink Protocol Level Interface Flow Control and overall UALink Flow control.....	43
2.7 Interface Signals.....	46
2.7.1 Signal Groups	46
2.7.2 Common Signals	46
2.7.3 UPLI Transactions/Channel usage.....	46
2.7.4 Request Channel	49
2.7.5 Read Response/Data Channel.....	62
2.7.6 Write Response Channel	66
2.7.7 Originator Data Channel.....	69

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

2.7.8	Relationships between UPLI Channels and requirements within the Channels.....	71
2.7.9	UPLI Request, Read Response, and Write Response ordering.....	72
2.7.10	UPLI Request Single-Copy Atomicity.....	73
2.8	Data/Atomic Operands Transfer	74
3	Reliability, Availability, and Serviceability (RAS).....	84
3.1	RAS Requirements	84
3.1.1	End to End Data Protection.....	85
3.1.2	RAS Error Types	86
3.1.3	RAS Error Handling Mechanisms	87
3.1.4	UALink RAS Error Handling.....	90
4	UPLI Interface Reset, Signaling, and Connection.....	101
4.1	UPLI Interface Reset.....	101
4.2	UPLI Interface Signaling Requirements	102
4.3	UPLI Interface Control.....	103
5	Transaction Layer (TL)	105
5.1	TL Flit and Half Flit formats	105
5.1.1	TL Flit and TL Control and Data Half-Flit formats and Sequencing	105
5.1.2	TL Message Flit Format and Sequencing.....	108
5.2	TL Flit Sequencing and Packing Examples.....	109
5.3	Indicating Data Corruption in TL Data Half-Flits.	114
5.4	TL Write Flit Sequence Encoding Efficiency Examples.....	115
5.4.1	Single WriteFull Request and Single WriteFull Response.....	116
5.4.2	WriteFull Requests and Compressed WriteFull Responses	116
5.4.3	Compressed WriteFull Requests and Compressed WriteFull Responses.....	117
5.4.4	Maximum Efficiency WriteFulls.	118
5.4.5	Maximum Efficiency WriteFulls with Authentication.....	119
5.4.6	Maximum Efficiency Reads.	120
5.4.7	Maximum Efficiency Reads with Authentication.....	121
5.4.8	Maximum Efficiency With Mixed Reads and Writes.....	122
5.5	TL Tx and Rx Dataflow and Tx and Rx Compression Caches.....	124
5.6	TL Tx and Rx Address Cache Synchronization.....	127
5.6.1	CLOAD and CWAY control signals.....	128
5.6.2	Address Cache sequencing at the Tx Address Cache and Rx Address Cache.....	128
5.7	TL Control Half-Flit Request/Response Field packing limits.....	129
5.8	TL Flow Control.....	133

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

5.9	TL Control Field Bit Assignments and Legal TL Message Flit Types.....	136
5.9.1	Uncompressed Request Field.....	137
5.9.2	Uncompressed Response Field.....	138
5.9.3	Compressed Request Field.....	140
5.9.4	Compressed Response Field for Single Beat Read Response.....	142
5.9.5	Compressed Response Field for a Write or Multi-Beat Read Response.....	144
5.9.6	Flow Control/NOP Field.....	146
5.10	Recommended TL backoff modes.....	147
6	UALink In-Network Collectives.....	148
6.1	Introduction.....	148
6.1.1	BROADCAST Collective	149
6.1.2	REDUCE Collective.....	150
6.1.3	ALL-REDUCE Collective	151
6.1.4	REDUCE-SCATTER Collective.....	152
6.2	Collective Primitives	154
6.2.1	Switch Structures to support Collective Primitives	155
6.2.2	Collective Primitive Request/Response Flows	157
6.3	Block Collectives	161
6.3.1	Switch Block Collective Queues.....	162
6.3.2	BlockCollectiveAllocate Request.....	163
6.3.3	BlockCollectiveDeallocate Request.....	164
6.3.4	BlockCollectiveInvoke Request.....	165
6.3.5	Block Collective Normal and Strided Accesses/Address Masking.....	168
6.3.6	Address Masking	168
6.3.7	Block Collective Control Block Parameters and Format.....	169
6.3.8	Switch Structures to support Block Collectives.....	172
6.3.9	Block Collective Request/Response Flows.....	173
6.4	Relationships between the Collectives, Collective Primitives, and Block Collectives	179
6.5	Reproducibility.....	180
6.6	Rounding Modes and Stochastic Rounding	181
6.7	In Network Collective Optionality	182
7	Manageability Requirements.....	183
7.1	UALink Accelerators and System Nodes.....	183
7.2	UALink Switches and Switch Platforms	184
7.3	UALink Pod Controller	184

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

7.4	UALink Virtual Pods	187
7.5	Manageability Workflows	188
8	Security	189
8.1	References	189
8.2	System Overview	189
8.3	Security model.....	190
8.3.1	Security objectives:.....	190
8.3.2	Trusted Computing Base (TCB).....	191
8.3.3	Adversary profile and capabilities	191
8.3.4	Security Assumptions.....	192
8.3.5	Threat model.....	192
8.4	UALink System Security.....	193
8.5	Encryption and authentication scheme for UALink.....	197
8.5.1	Control Half-Flit field encryption and authentication.....	198
8.5.2	Data authentication and encryption.....	206
8.5.3	ISOLATE response handling	206
8.5.4	Modes of operation.....	206
8.5.5	Poisoned data handling with security enabled.....	206
8.5.6	Ordering requirements imposed due to AES-GCM.....	207
8.5.7	Authentication and Encryption/Decryption Implementation in an UALink port	207
8.5.8	Initializing encrypted and authenticated transmission and reception.....	230
8.5.9	Refreshing an expired key	232
8.5.10	Safeguarding UALink configuration to ensure confidentiality and integrity	232
8.5.11	Integrity failure handling.....	233
8.5.12	Switch requirements	233
8.5.13	Datapath protection.....	234
8.6	Security during collective operations	236
8.6.1	Switch Port Identifier	238
8.6.2	Encryption and authentication scheme for Accelerator-Switch link.....	239
8.6.3	Ordering Requirements.....	240
8.6.4	Encryption Keys and Security State Management.....	240
8.6.5	Detecting collective traffic.....	242
8.6.6	Configuring and starting Accelerator-Switch link protection	243
8.6.7	Handling integrity failure in Accelerator-Switch link.....	244
8.7	Keyrollmsg formats	244

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

8.8	Specification version compatibility.....	245
9	UALink Switch Requirements.....	246
9.1	Overview.....	246
9.2	Bifurcation support	246
9.3	Lossless Request and Response delivery	246
9.4	Non-blocking architecture.....	247
9.5	Forward progress guarantee	247
9.6	Ordering and Virtual Channels	247
9.7	Routing Table Structure.....	247
9.7.1	Routing Table Instances	248
9.7.2	Egress port reachability	248
9.8	Configuration	248
9.9	UALink Switch Recommendations and Goals.....	248
9.9.1	Debug	248
9.9.2	Latency goals.....	249
9.9.3	Performance goals	249

List of Figures

Figure 0-1 UALink Connectivity Overview	17
Figure 1-1 UALink Based Multi-Accelerator System	21
Figure 1-2 Accelerator communication over a direct link and over a Switch	23
Figure 1-3 UALink Stack	24
Figure 1-4 UALink cross-domain address translation model	26
Figure 1-5 Translation Process	27
Figure 2-1 UALink Protocol Level Interface	29
Figure 2-2 Extending a UPLI interface through Intermediate UPLI Interfaces	30
Figure 2-3 UALink Protocol Level Interface Channels and control signals	31
Figure 2-4 UALink Stack Component	33
Figure 2-5 Connected UALink Stack Components	34
Figure 2-6 End-to End UALink Connection Between two Accelerators	35
Figure 2-7 UPLI Request and Response Flows	36
Figure 2-8 Example system with 32 Accelerators with 32 x1 UALink Links	37
Figure 2-9 Read Request end-to-end flow with Response	38
Figure 2-10 UALink Station with x1 bifurcation	41
Figure 2-11 Flow control loops	45
Figure 2-12 Four DoubleWorld Read Request Not Straddling a 64-Byte Boundary	75
Figure 2-13 Four DoubleWord Read Request Straddling a 64-byte boundary	75
Figure 2-14 Single Byte Read	76
Figure 2-15 Six Byte Read Access Not Straddling a 64-Byte Boundary	76
Figure 2-16 Four Byte Read Access Straddling a 64-Byte Boundary	77
Figure 2-17 Four Doubleword Write Request Not Straddling a 64 Byte Boundary	78
Figure 2-18 Sixteen Doubleword Write Request Not Straddling a 64-Byte Boundary	78
Figure 2-19 Three Doubleword Write Request Straddling a 64 Byte Boundary	79
Figure 2-20 A 128 Byte Write Full Request (Requires Two Beats)	79
Figure 2-21 Single Operand (Eight-Byte Operands) Atomic	81
Figure 2-22 Single Operand (Four-Byte Operands) Atomic	81
Figure 2-23 Double Operand (Eight-Byte Operands) Atomic, Data Returned in High 32 Bytes.	82
Figure 2-24 Double Operand (Four-Byte Operands) Atomic, Data Returned in Low 32 Bytes.	83
Figure 3-1 UALink End to End Data Protection	85
Figure 3-2 UPLI Control Error detected at an Originator or Completer not on a UALink TL on an Accelerators	91
Figure 3-3 UPLI Control Error detected at a UALink TL on an Accelerator	92
Figure 3-4 UPLI Interface Control Error detected at the UALink TL on the Switch	93
Figure 3-5 UPLI Control Error detected at the UPLI Completer on a Switch	93
Figure 3-6 UPLI Control Error detected within the Switch Core at the UPLI Originator	94
Figure 3-7 UALink Link goes down	95
Figure 3-8 PHYs Inform UALink TLs in Accelerator and Switch Which Enter Drop Mode	95
Figure 3-9 Placing the Accelerator UPLI Originator Into Isolation Mode	96
Figure 3-10 Other Accelerators Time Out	96
Figure 3-11 Link Down Error Processing Complete	97
Figure 3-12 Inc Engines issuing INC Requests	98
Figure 3-13 Accelerators Responding to INC Request With An Error on One Accelerator	99
Figure 3-14 Switch Returning Status for INC Operation	100
Figure 4-1 UPLI Interface Reset Requirements	101
Figure 4-2 UPLI Connection Handshake Protocol – Originator connects first	104

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Figure 4-3 UPLI Connection Handshake Protocol – Completer connects first	104
Figure 4-4 UPLI Connection Handshake Protocol – Originator and Completer connecting concurrently	104
Figure 5-1: TL Flit connections to UPLI interfaces	105
Figure 5-2: UALink TL Tx Dataflow	124
Figure 5-3: UALink TL Rx Dataflow	125
Figure 5-4: Indexing of Accelerator Tx Address Caches/Switch Rx Address Caches	127
Figure 5-5: Indexing of Switch Tx Address Caches/Accelerator Rx Address Caches	127
Figure 5-6 TL Receive Catch Buffer Dataflow	131
Figure 5-7: Flow Control Field Relation to Credit Channels	133
Figure 6-1 BROADCAST Collective	149
Figure 6-2 REDUCE Collective	150
Figure 6-3 ALL-REDUCE Collective	151
Figure 6-4 REDUCE-SCATTER Collective	152
Figure 6-5 REDUCE-SCATTER implemented with repeated REDUCE Collective	153
Figure 6-6 Collective Primitives Support Structures	156
Figure 6-7 Collective Primitive Request Flow	157
Figure 6-8 Collective Primitive Response Flow	159
Figure 6-9 Control Block Queue and Control Logic	162
Figure 6-10 ReqAddr field for BlockCollectiveAllocate Request	163
Figure 6-11 Low Order 8 bytes of Data Conveyed with BlockCollectiveAllocate	163
Figure 6-12 ReqAddr field for BlockCollectiveDeallocate Request	164
Figure 6-13 Low Order 8 bytes of Data Conveyed With CollectiveBlockDeallocate	164
Figure 6-14 ReqAddr field for BlockCollectiveInvoke Request	165
Figure 6-15 Base Address of the Block Collective	166
Figure 6-16 Low Order 16 bytes of BlockCollectiveInvoke Status Buffer	166
Figure 6-17 Non-Strided Block Collective Memory Access Pattern	168
Figure 6-18 Strided Block Collective Memory Access Pattern	168
Figure 6-19 Block Collective Address Mask Register	169
Figure 6-20 Block Collectives Support Structures	172
Figure 6-21 Block Collective Invocation.	173
Figure 6-22 Block Collective Input Buffer Read(s)	174
Figure 6-23 Block Collective Response Flow	175
Figure 6-24 Block Collective Output Buffer Write(s)	176
Figure 6-25 Block Collective Output Buffer Write Response(s)	177
Figure 6-26 Block Collective Status Buffer Update	178
Figure 6-27 Collective Primitives and Block Collectives Relationships	180
Figure 7-1 UALink System Node	184
Figure 7-2. A UALink Switch Platform	184
Figure 7-3. A UALink Pod managed by a Pod Controller	187
Figure 7-4. A UALink Pod partitioned into three Virtual Pods	188
Figure 8-1 System level view of confidential computing in a pod	195
Figure 8-2 Encryption and Authentication touch points in UALink stack	197
Figure 8-3 Example of UALink TL flit with the "Tag" half flit	198
Figure 8-4 Security related state elements in UALink port TX and RX for a 1024 accelerator system	208
Figure 8-5 A 4 accelerator UALink system example	210
Figure 8-6 KDF-KMAC256 (K, X) function	211
Figure 8-7 Stream Key derivation flow in UALink port TX	216

List of Figures

10

Figure 8-8 Master key swap flow in TX.....	218
Figure 8-9 Master key swap flow in RX.....	220
Figure 8-10 Source accelerator - Destination accelerator interactions for key derivation & swap flow.....	221
Figure 8-11 PCRC generation and integration during encryption flow on the transmitter side.....	235
Figure 8-12 PCRC Calculation during the decryption flow on the receiver side.....	236
Figure 8-13 Security architecture of a Virtual Pod that uses collectives.....	238
Figure 8-14 Accl-Switch link protection on a system with two Virtual Pods.....	240

List of Tables

Table 2-1 Common Signals	46
Table 2-2 Request Channel Signals	49
Table 2-3 Min and Max function definitions for IEEE DataTypes	52
Table 2-4 Min and Max Function definitions for FP8 Datatypes (E5M2, E4M3)	53
Table 2-5 Reduction Operation/Datatypes	53
Table 2-6 UALink Rounding Modes	55
Table 2-7 Read, ReqAttr Usage	55
Table 2-8 BlockRead, ReqAttr Usage.....	55
Table 2-9 ReadReduce, ReqAttr Usage	56
Table 2-10 Atomic Request ReqAttr Usage.....	56
Table 2-11 Atomic OpSize Encoding.....	56
Table 2-12 Commands.....	57
Table 2-13 UPLI Write Message Request Types.....	61
Table 2-14 KeyInfo Definition	61
Table 2-15 Read Response/Data Channel Signals	62
Table 2-16 RdRspStatus [3:0] Encoding for Predefined Commands	64
Table 2-17 RdRspStatus[3:0] Encoding for Vendor Defined Command.....	65
Table 2-18 Write Response Channel Signals.....	66
Table 2-19 WrRspStatus[3:0] Encoding for Predefined Commands	68
Table 2-20 WrRspStatus[3:0] Encoding for Vendor Defined Commands.....	68
Table 2-21 Originator Data Channel Signals	69
Table 5-1: TL Flit organization.....	106
Table 5-2: Control Half Flit Field Footprints and Sizes.....	106
Table 5-3: TL Flit with Message Indicator Bits.....	109
Table 5-4: Message TL Half-Flit.....	109
Table 5-5: An 8-sector Request (Read) followed by a Mandatory NOP Half Flit	110
Table 5-6: A 4 sector Request (WriteFull 256 bytes) followed by a Control Half Flit with Flow Control.	110
Table 5-7: A 4 sector Request (Write 192 bytes) followed by a Control Half Flit with Flow Control	111
Table 5-8: A 2-sector Request (64 byte write), NOP, FC, and 4-sector AtomicR request.....	111
Table 5-9: A 2-sector Request (64 byte write), NOP, FC, and 4-sector AtomicR request.....	112
Table 5-10: A 2-sector Request (256 byte write), 2-sector Request (256-byte WriteFull), and a 4-sector AtomicNR	112
Table 5-11: Authentication mode example matching the prior example.	113
Table 5-12: Read Requests and Associated Read Responses.....	114
Table 5-13: 256-byte WriteFull with corrupt second 64-byte beat.	115
Table 5-14: An example illustrating corrupted Atomic Operand data.....	115
Table 5-15 A single 4-sector Request (WriteFull 256 bytes) and a 2-sector Write Response.....	116
Table 5-16 Three 4 sector WriteFull 256 byte Requests and three Compressed Write Responses.....	117
Table 5-17 Four 2 sector WriteFull 256-byte Requests and four Compressed Write Responses. ...	118
Table 5-18: Five 2 sector WriteFull 256-byte Requests and Five Compressed Write Responses....	119
Table 5-19: Two 2 sector WriteFull 256-byte Requests and Two Compressed Write Responses... ..	120
Table 5-20: Five 2 sector Read 256-byte Requests and Five Compressed Read Responses.....	121
Table 5-21: Two 2-sector Read 256-byte Requests and Two Compressed Read Responses.	122
Table 5-22 Three Write, Two Read Maximum Efficiency Example	123
Table 5-23 Address Cache Loading Request Availability.....	128

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Table 5-24 Request Packing without Data Half-Flits	130
Table 5-25 Request Pacing with Data Half-Flits	130
Table 5-26: Flow Control Field Signals.....	134
Table 5-27: TL Control Half-Flit Message Type Values.....	136
Table 5-28: Legal TL Message Half-Flits	136
Table 5-29 Uncompressed Request Field Signals	137
Table 5-30 Uncompressed Response Field Signals	138
Table 5-31 Compressed Request Field Signals	140
Table 5-32 Compressed Request Usage Restrictions.	141
Table 5-33 Compressed Request Command Encoding.....	141
Table 5-34 Compressed Response for Single Beat Read, AtomicR, and ReadReduce Field Signals.	142
Table 5-35 Compressed Response for Single Beat BlockRead Field Signals	142
Table 5-36 Compressed Response for Single-Beat Read Response Usage Restrictions.	143
Table 5-37 Compressed Response for a Write, WriteFull, AtomicNR, WriteMulticast, WriteFullMulticast, AtomicNRMulticast, or for a Multi-Beat Read, AtomicR, or ReadReduce Request Field Signals	144
Table 5-38 Compressed Response for a BlockWriteFull or Multi-Beat BlockRead Field Signals	145
Table 5-39 Compressed Response for Write or Multi-Beat Read Response Usage Restrictions.	145
Table 5-40 Flow Control/NOP Field	146
Table 6-1 Read/Write Response Reduction.....	160
Table 6-2 BlockCollectiveAllocate Status Field Indications.....	164
Table 6-3 BlockCollectiveDeallocate Status Field Indications.....	165
Table 6-4 Block Collective Invoke Status Buffer Field Definitions.....	166
Table 6-5 BlockCollectiveAllocate IStatus Field Indications	167
Table 6-6 Block Collective Control Block Fields	170
Table 6-7 Collective Type Parameter.....	171
Table 6-8 Block Collective Parameter Arrangement	171
Table 8-1 Possible attack types	191
Table 8-2 Threat model.....	193
Table 8-3 Request channel signals that are encrypted and authenticated.....	198
Table 8-4 Read response channel signals that are authenticated and encrypted	201
Table 8-5 Write response channel signals that are authenticated and encrypted.....	203
Table 8-6 Originator data channel signals that are authenticated and encrypted.....	205
Table 8-7 88-bit Context value in KMAC256.....	212
Table 8-8 96-bit IV construction for Accl-Accl link protection with multi-path disabled.....	212
Table 8-9 96-bit IV Construction when Accelerator is TX and Switch is RX.....	213
Table 8-10 96-bit IV Construction when Switch is TX and Accelerator is RX	213
Table 8-11 Request Channel Fields for AAD and MSG - Non Write or UPLI Write Message Request	223
Table 8-12 Originator Data Channel Fields for AAD and MSG (partial-word write or UPLI Write Message Request).....	223
Table 8-13 Originator Data Channel Fields for AAD and MSG (Full Write)	223
Table 8-14 Read Response channel fields for AAD and MSG	223
Table 8-15 Write Response channel fields for AAD and MSG	223
Table 8-16 Mapping Request Channel AAD bits.....	224
Table 8-17 Mapping Request Channel MSG bits for non write and non UPLI Write Message	225
Table 8-18 Mapping Request and Originator Data channels AAD bits for Wr Req or UPLI Write Message	225

Table 8-19 Mapping MSG bits for Write Request with Byte Enable or UPLI Write Message of the 1st beat.....	226
Table 8-20 Mapping MSG bits for Full-word Wr Req (w/o Byte Enable) of the 1st beat.....	227
Table 8-21 Mapping AAD bits for Originator data non 1st beat	227
Table 8-22 Mapping MSG bits for Full-word Wr Req (w. Byte Enable) non 1st beat.....	228
Table 8-23 Mapping MSG bits for Full-word Wr Req (w/o Byte Enable) non 1st beat.....	229
Table 8-24 Mapping Read Response Channel AAD bits	229
Table 8-25 Mapping MSG bits for Read Response	230
Table 8-26 Mapping Read Response Channel AAD bits	230
Table 8-27 Context for Deriving Accelerator TX and Switch RX keys	242
Table 8-28 Context for Deriving Switch TX and Accelerator RX keys	242
Table 29 Possible operational modes for various specification versions.....	245

Revision History

Rev	Date	Changes
UALink 200		
0.49	10/23/2024	Initial pre-release of UALink200 specification to test new Word template.
0.50	10/28/2024	Added initial Fault Management and Telemetry section.
0.70	12/01/2024	Added TL Chapter details. Various Changes in other sections.
0.71	12/9/2024	Reworked Credit Return interfaces for UPLI and clarified Credit Return interface for TL. Other small changes in TL section.
0.90	1/5/2025	Added new Security and Switch Requirements Sections. Added Authentication tag support in UPLI/TL. UALink operation Single-Copy-Atomicity defined. DL Link Level Replay support and other DL changes. Restructured the Manageability Requirements section.
0.91	1/8/2025	Rebuild of 0.90 to include the TL section after the mis-build in 0.90
0.92	2/4/2025	Added support in UPLI section for Authorization Tags and UPLI Write Message Requests. Additional detail on Vendor Defined Commands (to be completed in next release). Clarifications on Ordering and Single-Copy-Atomicity of UPLI Requests. Corrections to TL Flit packing and additional examples in TL section. Manageability/Security chapters content reorganized to remove redundancy and terminology harmonized. Details for handling poisoning of data in security modes. Clarifications in security chapter on various implementation flows and implementation details.
0.93	2/16/2025	Clarifications on Vendor Defined Command Semantics. Clarification on Ordering Rules (particularly with regard to Vendor Defined Commands). Additional TL Flit packing examples (including Reads and mixtures of Reads and Writes). UPLI and TL Credit Return Interfaces restructured and significantly simplified. ISOLATE Write Response further clarified. Definitions for Pod/Virtual Pod/Switch/Physical Switch. Added Vendor Defined UPLI Write Messages. Suggested backoff modes for TL Control Half-Flit packing.
1.0RC	2/19/2025	Various corrections to terminology. Corrections to Terminology Table. Clarifications on initial credit release done signals on UPLI interface. Bit lanes and FTYPEs assigned for TL Flit packets. Minor corrections in security an PHY sections.
1.0	4/1/2025	Minor bug fixes and modifications for normative language.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Draft Rev1.5 v0.9	1/13/2026	First draft review copy of 1.5 Common specification. Moves DL/PL content out to separate specifications. Add support for In-Network-Collectives (INC).
Draft Rev1.5 v0.91	2/13/2026	Additional clean up throughout including significant additions/clarifications to INC section and inclusion of existing approved 1.0 errata.
Draft Rev1.5 v0.92	2/15/2026	Additional 1.0 errata and other minor clean up.
Draft Rev1.5 Release Candidate 1.0	2/26/2026	Additional errata and INC clean up.
Draft Rev2.0 Release Candidate 1.0.1	3/31/2026	Corrections to BLockCollectiveAllocate/ BlockCollectiveDeallocate commands in ReqCmd table. Added SSize2 parameter to Block Collectives along with corrections to X_SIZE, Length parameters for tiled accesses and accesses with differing input and output datatypes. Corrected INC reduction operators table. Correction to FTYPE=0x7 table in Transaction Layer section. Numerous small typo corrections. Added KeyInfo field to the UPLI KeyRoll UPLI Messages. Added Optionality rules for In Network Collectives. Added Checking rules for In Network Collectives. Added rule for setting CLOAD.
Rev2.0	4/6/2026	Created final release version.

Preface

About This Specification

This specification is intended to define a set of protocols and interfaces that enable the creation of systems comprised of multiple System Nodes targeting AI applications. A System Node typically contains one or more Host CPUs and one or more Accelerators connected within the System Node utilizing an implementation specific set of interconnects such as CXL®, PCIe®, XGMI, CHI c2c, AMD Infinity Fabric®, etc. These System Nodes can be and often are coherent within the System Node, meaning that each Accelerator and each Host CPU can directly and coherently access all Host and Accelerator memory within the given System Node though this is not required. The exact configuration and number of Accelerators and Hosts within a System Node and the nature of the coherence and accessibility to memory within the node is implementation specific and is not mandated by this specification. Each System Node is typically managed under the control of a single OS image (System Nodes are also referred to as “OS Domains”).

The protocols and interfaces defined in this specification are intended to support low latency Accelerator-to-Accelerator communication across System Nodes using direct read, write, and atomics transactions. These protocols and interfaces do not, however, allow for Host CPU accesses to memory attached to device or host in another remote System Node.

The interfaces described in this specification are the UALink Protocol Level Interface (UPLI) and the Ultra Accelerator Link (UALink) interface. The UALink Protocol Level Interface is a point-to-point, on-chip interface comprised of various channels that transfer UPLI transactions consisting of Requests, Read data, Write data, and Request Responses between an Originator and a Completer. The Ultra Accelerator Link is a high-bandwidth point-to-point serial interface providing a connection between Accelerators and Switches that allows UPLI transactions to be transferred between Originators and Completers in Accelerators within and across System Nodes. This specification is primarily intended to create a switching ecosystem for Accelerators.

The figure below Figure 0-1 UALink Connectivity Overview, illustrates a portion of a simple example system illustrating two (of possibly many) System Nodes (SN0 and SN1) each illustrating one (of possibly many) Host/Acc pairs in each of the System Nodes.

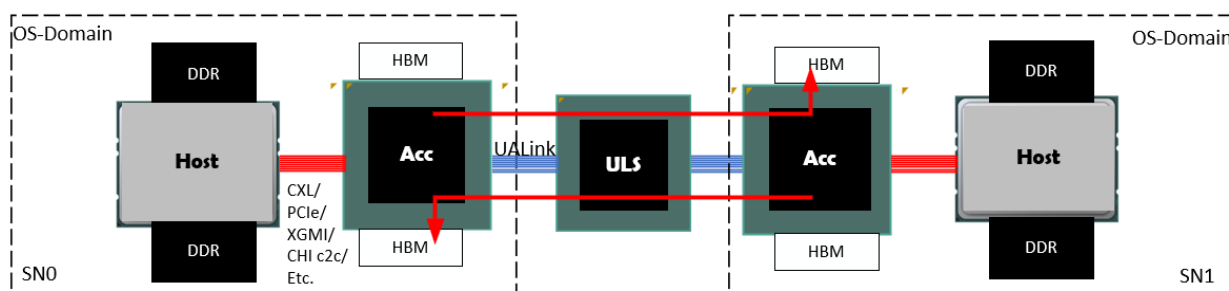


Figure 0-1 UALink Connectivity Overview

The illustrated Host and Accelerator (Acc) in each of the system nodes are connected using an implementation chosen interconnect such as CXL, PCIe, AMD Infinity Fabric, XGMI, CHI c2c, etc. (red) that can and often allows for Host and Acc access to all memory within the node or at least on the connected Host and Accelerator. The Accelerator is further attached to a set of UALink Links (blue) that connect to a Switch and then on to another set of UALink links to the remote Accelerator

allowing UPLI transactions to be routed between Accelerators in different System Nodes (accesses can also be routed between Accelerators in the same System Node).

In a typical system, Requests from a given Accelerator in one System Node to a remote Accelerator in a different System Node may access any Accelerator memory or Host Memory in the Remote System node. This is illustrated above by red arrows above showing each Accelerator accessing the remote Accelerator's memory (Host and Accelerator accesses to local Host and Accelerator memory are not shown). Hosts accessing any memory in any remote System Nodes shall not be supported.

This version of the specification does not define or enable attaching devices to the Switches. It does not define or enable how to perform in-memory, or near-memory compute.

Terminology

The following terms are used in this specification:

Term	Definition
UALink	Ultra Accelerator Link
UPLI	UALink Protocol Level Interface
DL	Data Link Layer
DL Channel	Logical channels with in DL, one for TL Flits, one for DL UART messages
SH	Segment header, used with in a DL Flit
FH	Flit Header, for DL Flit
DL alternative sector	Sectors in a DL Flit that are sued for non TL Flits
DL message	Message that starts and terminates at the DL
CRC	Cyclic redundancy check
RS	Reconciliation Layer, interface between PCA and DL
AM	Alignment marker, used for alignment of PCS lanes
PCS	Physical Coding Sublayer
FEC	Forward error correction
PMA	Physical Medium Attachment Interface
GAUI	Gigabit unit attachment
VDCI	Voltage Domain Crossing Interface
SPA	System Physical Address
Field	A group of one or more signals that share a name and encode a specific piece of information. Signals within a field are numbered according to binary significance.
SOC	System on Chip.
SPC	Symbols Per Clock
Word	Two Bytes
Doubleword	Four Bytes
UART	Universal Anonymous Receiver Transmitter. A DL mechanism for F/W on either end of the link to exchange information.
Pod	The collection of all the Accelerators and Switches physically connected though UALink via Switches.
Virtual Pod	Virtual Pods are a non-overlapping partition of a Pod where the Accelerators within a Virtual Pod may communicate with other Accelerators in the Virtual Pod, but with no other Accelerators in the Pod.
Availability	Security objective ensuring a resource (e.g., network) is functioning and data is accessible when needed
UALink Network	The physical network of UALink Links and Switches connecting the Accelerators in a Pod.
CC	Confidential Computing
Confidentiality	Security objective ensuring data are only readable by an authorized party
Front end network	Network used by OS domains to communicate and establish a Tenant TCB.
Infrastructure provider	An organization that maintains computing resources such as servers, storage, networking and virtualization and provides them to the users on demand.
Integrity	Security objective ensuring data are only writeable and modifiable by an authorized party
Pod Controller	Central controller software responsible for managing the lifecycle of the Pod including topology discovery, configuration, resource management, virtual pod creation and management and Pod health monitoring. The Pod Controller is typically owned by the Infrastructure provider.
Port encryption engine	A port encryption engine has at least one association (key, IV/count/sequence #, etc.) and enough encryption/decryption capability to keep up with line rates. Additionally, based on implementation, the port encryption engine may have buffering associated with each association such that it can pre-compute counter encryption values. An accelerator requires a port encryption engine per port.
Switch	An entity that can switch UALink traffic between a set of Ports equal in number to the number of Accelerators in the Pod.
Physical Switch	A physical hardware entity that can be used to implement a Switch and which can have Ports equal to he number of Accelerators in the Pod or Ports equal to an integer multiple greater than one of the number of Accelerators in the Pod.
TCB	Trusted Computing Base – The set of hardware and software components that are trusted to meet the security objectives of a feature.
TEE	Trusted Execution Environment. It is responsible for bringing an accelerator into the TCB and for UALsec configuration (e.g., key establishment). In the context of CC, TEE examples include Intel TDX, AMD SEV and ARM CCA).
Tenant	User of the infrastructure computing resources such as AI Cluster. In an AI cluster, the Tenant is typically assigned a set of accelerators (i.e., a Virtual Pod) to run its workload.
TVM	Trusted Execution Environment Virtual Machine. This is a confidential computing VM running in a TEE. One or more accelerators are assigned to a TVM which is responsible for secure configuration and management of those accelerators.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification Rev 2.0

TSM	TEE Security Manager. This is SW/FW component on the host which establishes secure interface with the device and is responsible for configuring and helping enforce TEE IO security policies on the host side. It is in the TCB of all TVMs
DSM	Device Security Manager. This is a FW component on the device that enforces device security policies. It communicates with the TSM over a secure channel and receives commands from TSM for configuring and enforcing device security policies.
System Node	Hardware platform that hosts Accelerators, alongside one or more Central Processing Unit(s) (CPU(s)) and one or more network interface(s). The System Node is the boundary of an OS Domain, and UALink System Nodes host a Node Management Agent.
Switch Platform	Hardware platform that hosts Switches, a Switch Management Agent, and a network interface. When present, Switch Platforms are distinct from System Nodes.
Node Management Agent	Firmware/Software component that manages Accelerators in a System Node under the direction of the Pod Controller.
Switch Management Agent	Firmware/Software component that manages Switches under the direction of the Pod Controller.

In addition to the hardware requirements laid out by this specification, the complementary *Ultra Accelerator Link Manageability Specification* documents the requirements for firmware and software to manage and operate an Ultra Accelerator Link Pod.

1 Introduction

1.1 Multi-Node Accelerator System

The main purpose of this specification is to enable low latency and efficient communication between Accelerators. The Accelerators and the bandwidth allocated to each Accelerator can scale to meet the requirements of AI applications. illustrates an example system with multiple nodes, where each node has a Host processor and four Accelerators. The system has 'M' Accelerators in total and each Accelerator has 'N' Ports. The 'N' Ports are assumed to be symmetric, and traffic is spread across all the ports. Usually, a single OS image controls and manages each System Node (System Nodes are also called "OS Domains"). A set of UALink Switches connects the Accelerators together.

The UPLI allows up to 1024 Accelerators or endpoints in a system to communicate using a 10-bit Identifier. The 10-bit Source and Destination Accelerator Identifiers are used by the Switch to route Requests and Responses between a sender and a receiver. All Requests shall carry Source and Destination Accelerator Identifiers, Responses also carry Source and Destination Accelerator Identifiers, but only need the Destination Identifier for routing, the Source Identifier is retained to aid in debugging.

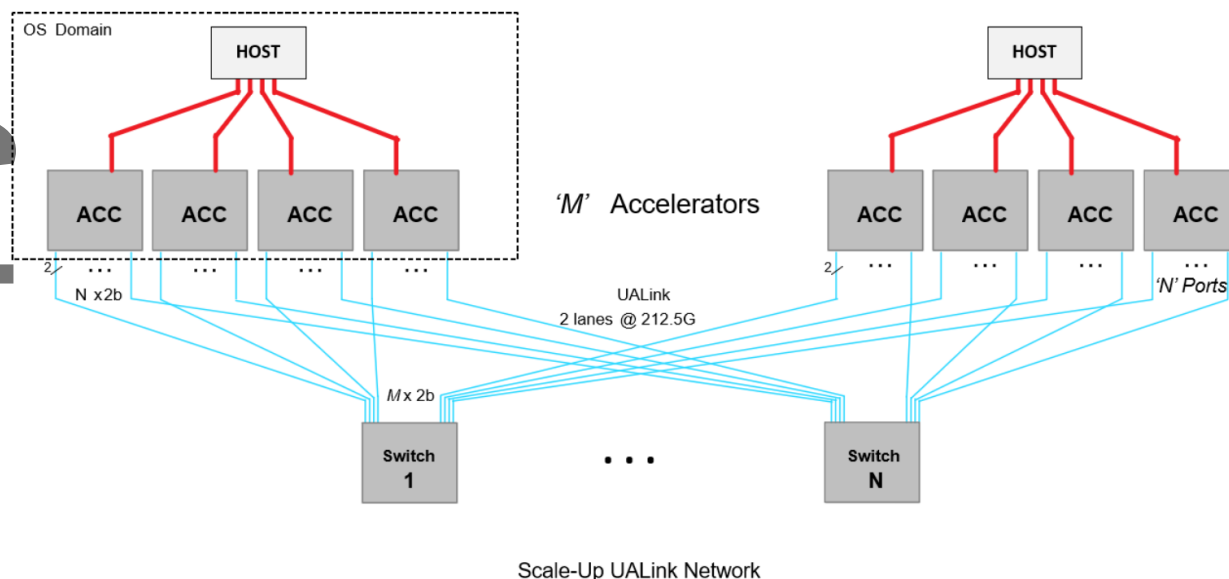


Figure 1-1 UALink Based Multi-Accelerator System

As shown in Figure 1-1, the Accelerator Fabric Switch connects 'M' Accelerators through UALink Links that consist of UALink Lanes. A Lane is a pair of signals, one for transmit and one for receive, and UALink Lanes can be grouped into a one Lane Link (a x1 Link), two Lane Link (a x2 Link), and a four Lane Link (a x4 Link). The number of Lanes per Switch and the bifurcation capability of the Switches and Accelerators shall determine how many Accelerators can be connected per Switch.

A Pod consists of the largest number of Accelerators that are to be connected via UALink Switches. A Switch is defined as a logical entity having a number of ports (radix) equal to the number of Accelerators in the Pod. Each Port on the Switch shall connect to a distinct Accelerator. Unless partitioned to create Virtual Pods, a Switch can route traffic on any Port on the Switch to any other Port on the Switch (if the Switch is partitioned to create Virtual Pods, traffic is restricted to remain within the respective Virtual Pods). The number of Switches shall equal the number of Ports on the Accelerators (all Accelerators in the Pod should have the same number of Ports).

With these constraints, the UALink Switches connect the Accelerators in a Pod in a way that each Port on an Accelerator may intercommunicate with only a single port on each other Accelerator.

A Virtual Pod is a group of one or more Accelerators in the Pod that may communicate amongst themselves but not with any other Accelerator in the Pod. The Pod may be divided into Virtual Pods by partitioning the Switches into non-overlapping subsets of Ports on each Switch. The Ports within a subset can communicate with one another but not with any Port outside the subset. Switches shall provide a mechanism to configure partitions.

The Switches in a Pod may be realized in hardware utilizing Physical Switches that have a Radix equal to the number of Accelerators in the Pod in which case the partitioning of the Physical Switch directly creates the Virtual Pods. If, however, the Physical Switch has a radix equal to an integer multiple greater than one of the number of Accelerators in the Pod, the Physical Switch shall first be partitioned into a number of Switches. These Switches may then be further partitioned to create the Virtual Pods.

All Accelerators in a Pod have a unique Accelerator ID, regardless of Physical Switch partitioning or Virtual Pod partitioning. All Accelerator Ports, and thus also all Switch Ports, in a Virtual Pod, share identical security (encryption/authentication) settings.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

This Specification shall supports a max data rate of 200 GT/s per Lane and a max link width of 4 Lanes. A UALink Station (or simply Station) is defined as a group of 4 UALink Lanes. A UALink Station may be bifurcated to connect to one x4-UALink Links (or simply Link), two x2 Links, or four x1 Links. The UALink Links shall attach between UALink ports on two different Devices (in this figure, a port at the ACC and a port at the UALink Switch). The maximum bandwidth for each UALink Station shall be 800 Gigabits /s (Gbps).

The signaling rate is usually higher (212.5 GT/s) to accommodate the bandwidth consumed by Ethernet Layer1 for Forward Error Correction Code (FEC) and additional Layer1 encoding.

1.2 Accelerator System Node

An Accelerator System Node may be comprised of one or more host processors, one or more Accelerators, and devices under a single OS domain. An Accelerator can communicate to another Accelerator either through a direct UALink link or through a UALink Switch. Communication between Accelerators inside a system node is called in-domain communication, i.e. within an OS-domain. Communication between Accelerators in differing system nodes is referred to as cross-domain communication.

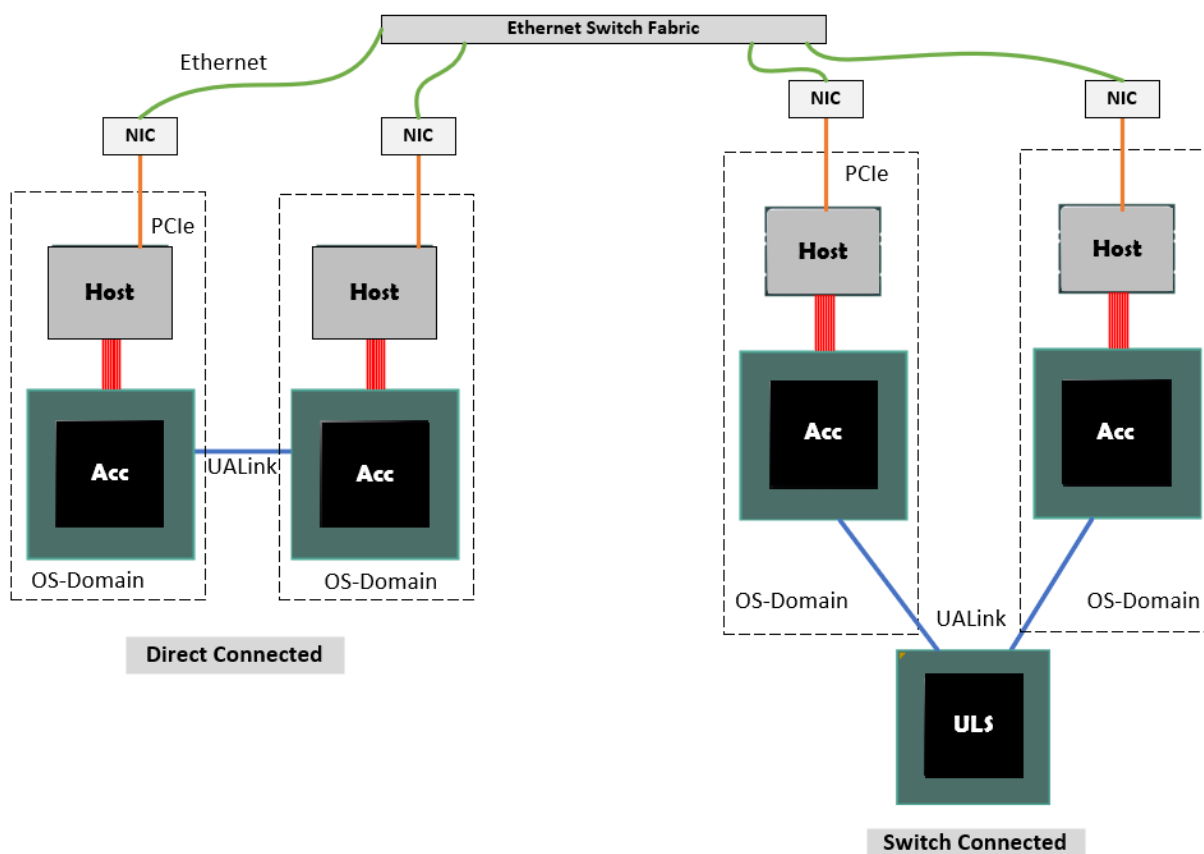


Figure 1-2 Accelerator communication over a direct link and over a Switch

UALink Switches shall enable a direct load/store access model for a scale-up Accelerator Pod with up to 1024 Accelerators. An Ethernet switched network shall enable the data center scale-out cluster of many thousands of Accelerators through Ethernet switches. This may be enabled through a front-side NIC attached to the host.

1.3 UALink Stack Interface Layers

The UALink Link carries messages between a sender and receiver. UALink is a symmetrical protocol with the same set of messages and channels supported in both transmit and receive paths. These messages traverse through multiple functional layers of the UALink stack.

A UALink stack shall be comprised of a

- Protocol Layer
- Transaction Layer
- Data Link Layer and
- Physical Layer

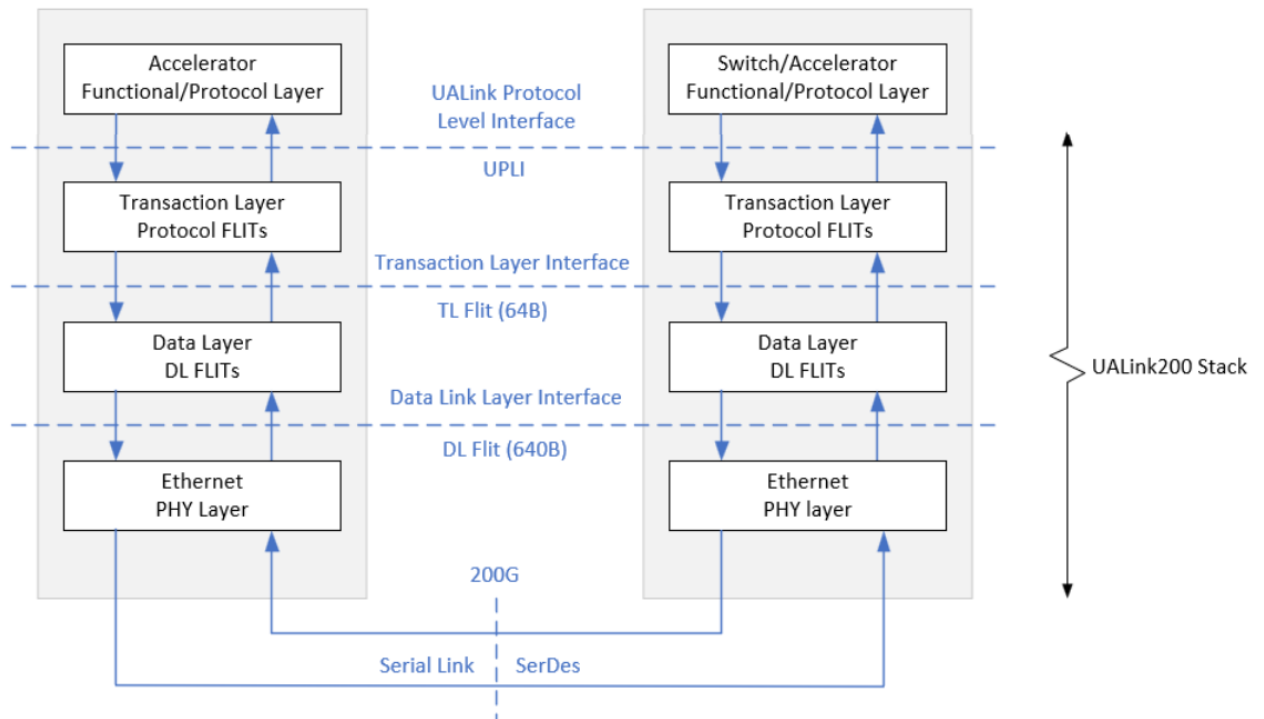


Figure 1-3 UALink Stack

1.3.1 Protocol Layer

The protocol layer for UALink is called UALink Protocol Level Interface (UPLI). UPLI defines a logical signaling interface and a protocol by which devices can exchange data and control information through a set of Request and Response messages. The UALink Specification fully defines the UPLI Protocol and expects that implementations that follow this protocol will be compatible with UALink Switches. The UPLI Protocol has built-in flexibility to allow vendors to create custom protocol messages for communication between Accelerators that are the same kind without any modification to the UALink Switches. The UALink Protocol Level Interface is the primary interface which implementations may develop to while typically using third party vendor supplied IP for the rest of the stack.

1.3.2 Transaction Layer

The Transaction Layer (TL) shall connect to two UPLI Interfaces, one sourced from a UPLI Originator and one sourced from a UPLI Completer. The TL shall drive a 64-byte Outbound Transmit (Tx) Flit to the UALink DL and shall receive from the UALink DL a 64-byte Inbound Receive (Rx) Flit from the UALink DL. The UPLI channels driven into the TL from both UPLI interfaces shall be packaged into 64-byte Outbound Transmit (Tx) Flit which shall be transmitted to the UALink DL. Similarly, the Receive (Rx) 64-byte TL Flit from the UALink DL shall be unpacked into Request, Read Response/Data, Orig Data, and Write Response channels for the two attached UPLI Interfaces.

1.3.3 Data Link Layer

The data link layer receives 64-Byte Flits from the Transaction Layer (TL) and shall package these Flits into 640-Byte Flits in the egress direction and shall send them to the Physical layer (PL). Similarly, in the ingress direction the Data Link Layer (DL) shall receive 640-Byte Flits from the PL and shall unpack them into 64-Byte Flits and then shall send them to the transaction layer (TL). The DL shall provide a control message service used for coordinating changes to the link, i.e., online/offline, and other features. The DL shall provide a UART mechanism for firmware-controlled sequences to be passed across the link.

1.3.4 Physical Layer

The Physical Layer (PL) is based on IEEE 802.3dj (D1.4 at the time of writing). The PL shall support the following rates based on 200G serial: 200GBASE-KR1/CR1, 400GBASE-KR2/CR2, and 800GBASE-KR4/CR4. The PL shall also support the following rates based on 100G serial, 100GBASE-KR1/CR1, 200GBASE-KR2/CR2, 400GBASE-KR4/CR4. To reduce latency at the 200G serial rates, 1-way and 2-way code word interleave modes are optionally supported, in addition to the standard 4-way interleave. To improve latency each 640-Byte DL Flit shall be packed uniquely into a single 680-Byte code word. The additional 40-bytes shall be for FEC overhead and 256B/257B line coding. Achieving DL Flit to code word alignment does require changes to a standard Ethernet PCS, regarding alignment marker insertion and removal. The alignment markers on the wire are unchanged from IEEE 802.3 definition, only the mechanism for how the alignment markers are inserted and removed changes. Ethernet Retimers shall be compatible with UALink provided they use the recovered clock for forwarding the data. This is the most common mechanism. Adding or removing Idle codes would require FEC decode and encode and a large latency penalty. In addition, this would break the DL Flit to code word association required for UALink. Auto negotiation and link training is unchanged from 802.3.

1.4 UALink Address Translation Model

Figure 1-4 shows the UALink network, which allows data to move between devices. It supports data transfers within and across system nodes. Accelerators may use a System Physical Address (SPA) to access memory within a System domain and may use a Network Physical Address (NPA) to access memory in a different System domain. An implementation can also opt for a global addressing model that is flat to simplify the translation process. This section provides a brief overview of a cross-domain address translation model. It is only for illustration. This specification leaves the address translation as an implementation choice as Switches use identifier-based routing. In this example, the source Accelerator uses the Memory Management Unit (MMU) to translate a Guest Virtual Address (GVA) to a Network Physical Address (NPA). At the destination node, a link MMU is used to translate NPA to a local SPA.

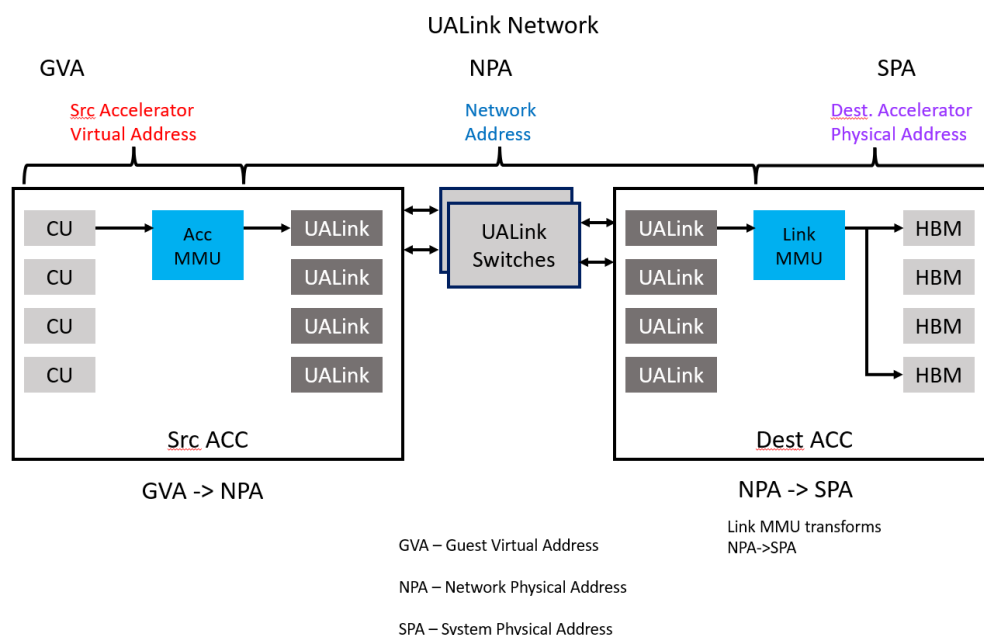


Figure 1-4 UALink cross-domain address translation model

1.4.1 Remote Memory Access (RMA)

Distributed applications which span many Accelerators need the ability to securely access memory on remote system nodes. The first step in this process is the ability to import memory from a target node. This usually happens through an OpenSHMEM or a custom shared memory library that can exchange pointers between an importer and an exporter. The library handles a partitioned global address space (PGAS) that covers memory across multiple system nodes. The exchanged pointer between a receiver and sender consists of an address handle and physical Accelerator identifier within a Pod. The use of an address handle instead of an actual address provides more security. The pointer exchange process is expected to take place through the front side Ethernet network connected to the host.

In Figure 1-5, the source Accelerator, which imports memory, creates a Page Table Entry (PTE) in the Accelerator's memory management unit (MMU) which includes the address handle and the

Accelerator identifier. The exporting or destination Accelerator creates a new page table entry in its link MMU. This includes the address handle and the source Accelerator identifier.

Figure 1-5 below illustrates the translation process at the source and the destination Accelerators. Applications running on the compute elements use Guest Virtual Address. These accesses from the Compute Unit (CU) with many compute elements go through the MMU to translate virtual address to a physical address.

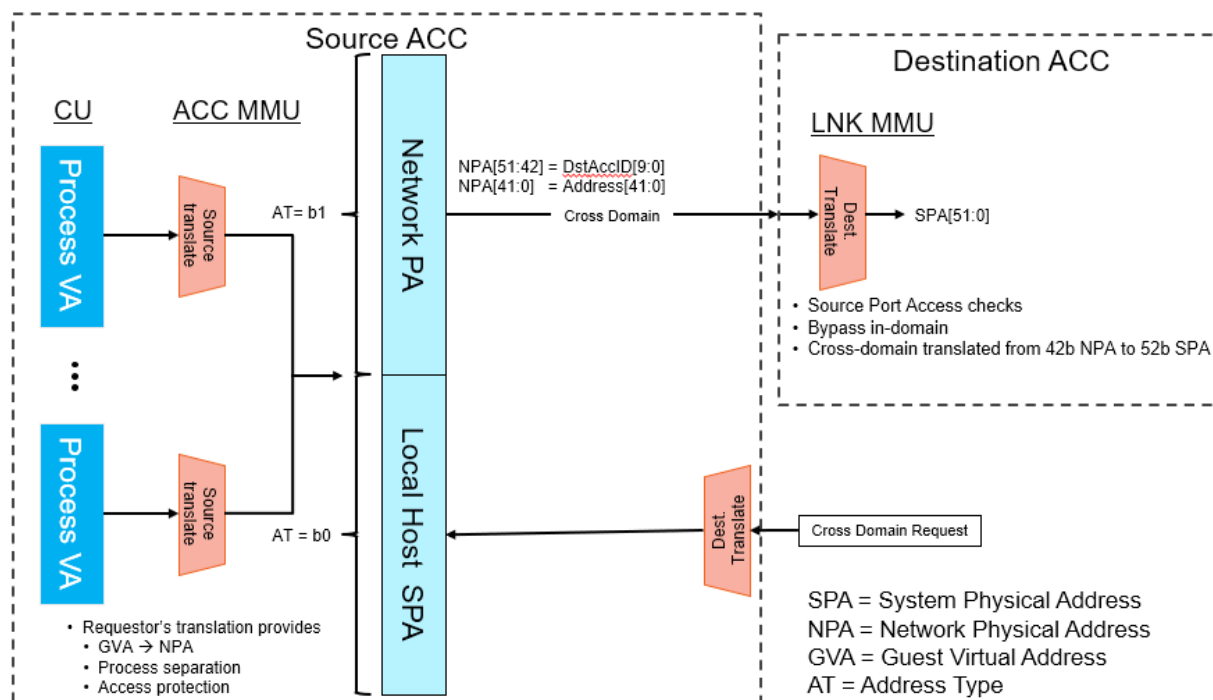


Figure 1-5 Translation Process

In addition to the address, the PTE also adds a bit to identify the type of physical address. The two types of physical address supported are System Physical Address (SPA) which is the local address within a domain to access system memory and the other is Network Physical Address (NPA) which contains the address handle and target identifier. The UALink network routes Requests and Responses using the source and destination identifiers. Accelerators must drive the identifiers for both in-domain and cross-domain accesses. At the destination Accelerator, NPA is translated through an UALink link MMU to the local SPA of the target system node.

1.5 UALink Coherency

UALink does not support snoop transactions for keeping hardware coherence among Accelerators. Hardware coherence between host processors and Accelerators within a system node shall be handled through host side connections. Since AI/ML workloads typically involve many Accelerators, software coherence enables applications to scale efficiently across scale-up Pods and scale-out clusters. There is no significant benefit in adding complexity to carry snoop messages on UALink to only enable hardware coherence amongst Accelerators within a system node. Hence Accelerators that cache data from a peer memory within or across system nodes shall be expected to keep coherence through software by clearing caches at the right kernel boundaries.

UALink shall support an I/O coherency model with the following semantics:

- Read from a peer memory shall get the most recent coherent copy of data from memory or a cache within its system node.
- Writes to a peer memory shall invalidate all cache copies within its system node. Partial writes shall fetch any cached data in the system and merge with the data from write. The most recent copy of the data shall be written back to memory.

Hardware coherency within a system node (OS-domain) amongst the host processors and Accelerators is not specified by UALink. Implementations are expected to handle coherency through implementation-specific hardware or software methods.

2 UPLI Interface Definition and Operation Rules

2.1 UPLI Interface

The UPLI defines a logical signaling interface and a protocol by which devices can exchange data and control information through a variety of Request and Response messages. The UPLI Protocol has the following features and benefits:

- Split Requests / Responses improve utilization of interface bandwidth.
- Per UPLI interface port transaction identification tags to support multiple outstanding Requests on the interface port.
- Reads, Writes, and Atomic memory operations

Data transfer is effected through Request / Response message pairs. A Request and its subsequent Response form a Transaction. The following figure illustrates a UPLI Interface:

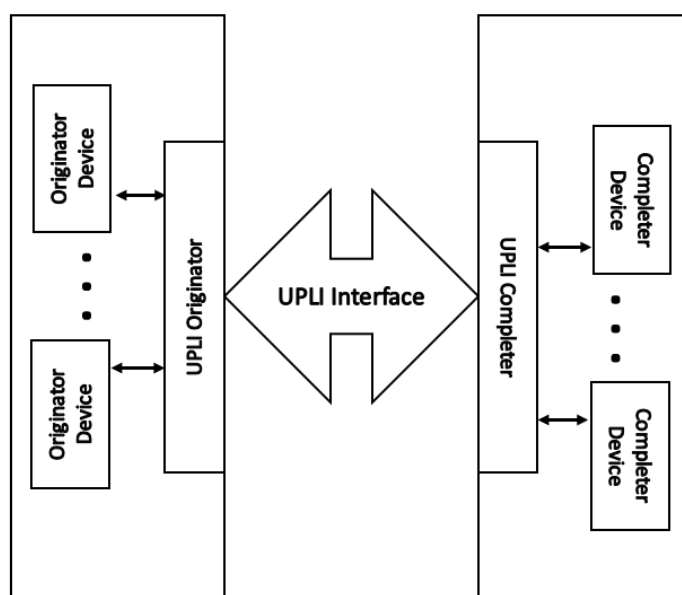


Figure 2-1 UALink Protocol Level Interface

In the UALink Protocol Level Interface, a device that initiates a Transaction by making a Request is called an Originator Device. A device that can respond to a Request is called a Completer Device. Requests are also called Commands. The terms Command and Request are used interchangeably in this specification.

The UALink Protocol Level Interface defines the Requests that can be issued by an Originator and the Responses that shall be returned to the Originator for each Request type.

An Originator Device attaches to the UALink Protocol Level Interface via a UPLI Originator. Similarly, a Completer Device attaches to the UALink Protocol Level Interface via a UPLI Completer.

The Originator Device shall initiate a transaction by issuing a Read, Write, Atomic, UPLI Write Message, or Vendor Defined Command Request. When the Request is completed, the Completer Device shall issue a Response to the Originator Device. Transaction Tags allow the Originator Device to pair Requests with subsequent Responses. All Requests shall require a Response. In most

of the figures in this specification, the Originator Devices and Completer Devices will not be shown for clarity.

A UPLI interface can be (and is typically) extend through a series of intermediate UPLI interfaces with Intermediate Logic Blocks as show in Figure 2-2 Extending a UPLI interface through Intermediate UPLI Interfaces below:

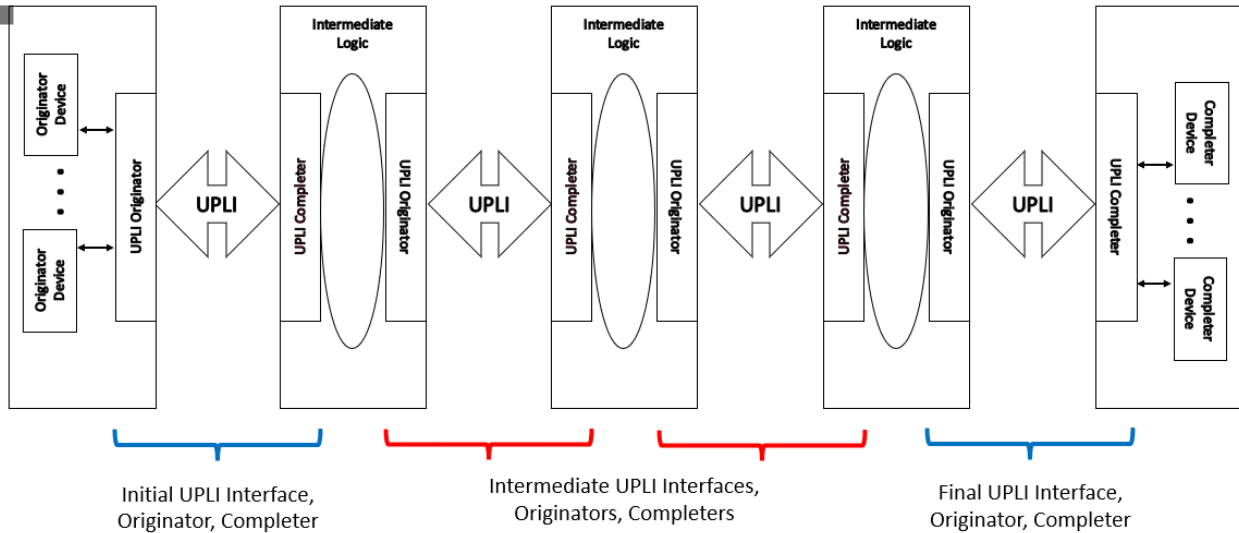


Figure 2-2 Extending a UPLI interface through Intermediate UPLI Interfaces

The Initial UPLI Interface connected to the Originator Devices is connected to a set of Intermediate Logic Blocks and Intermediate UPLI Interfaces that then ultimately connect to the Final UPLI Interface which is connected to the Completer Devices.

As a typical use case, Figure 2-2 Extending a UPLI interface through Intermediate UPLI Interfaces corresponds to the single path from a specific Source Accelerator to a specific Destination Accelerator in a UALink use case utilizing a Switch. The first Intermediate Logic Block above represents the logic between a Source Accelerator and a Switch to implement a Transaction Layer Interface, a Data Link Layer Interface, a Physical Interface, and then another Data Link Layer and Transaction Layer Interface between the Source Accelerator and the Switch. The second Intermediate Logic Block represents the Switch core that routes UPLI Requests and Responses. While this figure represents only one path through the Switch from a specific Source Accelerator to a specific Destination Accelerator, the Switch core generally connects to multiple UPLI interfaces, not shown here, from multiple Source Accelerators. The third Intermediate Logic Block represents the Transaction Layer, Data Link Layer, and Physical Interfaces connecting the Switch to the Destination Accelerator.

Generally speaking, but subject to rules explained later in 2.7.9 , an Intermediate Logic Block may reorder Requests and Responses received from one attached UPLI Interface before forwarding them on to the other attached UPLI Interface.

The UALink Protocol Level Interface contains four channels: The Request (Req) Channel, the Read Response/Data (RdRsp) Channel, the Originator Data (OrigData) Channel, and the Write Response (WrRsp) Channel.

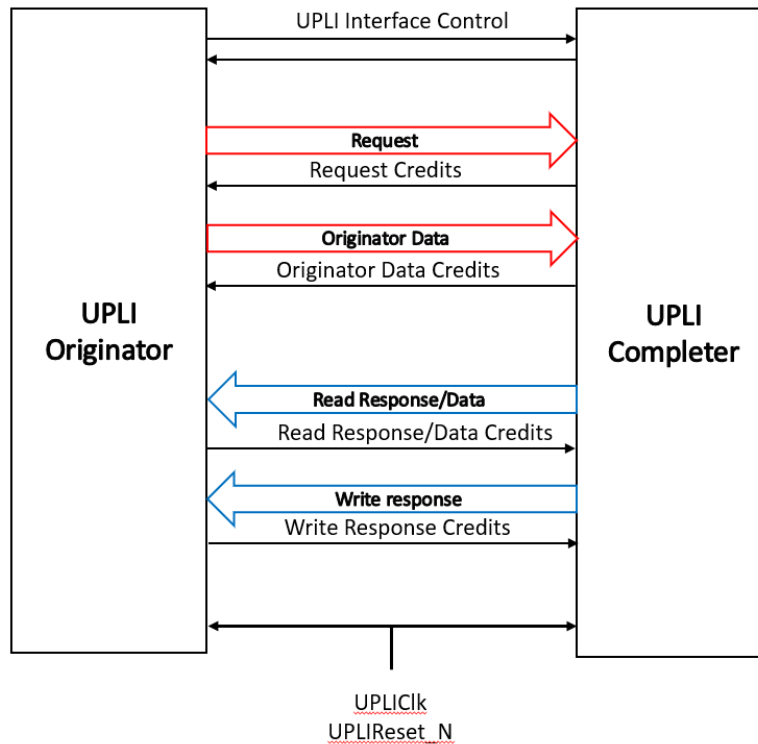


Figure 2-3 UALink Protocol Level Interface Channels and control signals

The Request Channel shall contain a Request Address, the Request Type (Read, Write, Atomic, UPLI Write Message, Vendor Defined Command), fields controlling the size of the Request, a Source Accelerator Identifier, a Destination Accelerator Identifier, a Transaction Tag to uniquely identify the Request among outstanding Requests on the Request Channel, and various additional control and parity signals.

The Originator Data Channel shall contain 64 bytes of data for a Beat of a Write Request, Vendor Defined Write class Request, or UPLI Write Message Request (a Write Request, Vendor Defined Write class Request or UPLI Write Message Request may require up to four Beats) or Operand data for an Atomic Request or Vendor Defined Atomic class Request, Byte Enables for data or Operand Data, an indication of which Beat is present, and various additional control and parity signals. The Source Accelerator and Destination Accelerator for the Request are specified by the Request in the Request Channel associated with the Data Beats.

The Read Response/Data Channel shall contain 64 bytes of data for a Beat of the Read Response (a Read Response may require up to four Beats), a Source Accelerator Identifier, a Destination Accelerator Identifier (a swap of the original Request Source and Destination values), a Transaction Tag to identify the Request among outstanding Read Requests from the Request Channel, indicators of the Read Response status, an indication of which Beat is present, and various additional control and parity signals.

The Write Response Channel shall contain a Source Accelerator Identifier, a Destination Accelerator Identifier (a swap of the original Request Source and Destination values), a Transaction Tag to identify the Request among outstanding write Requests from the Request Channel, indicators of the Write Response status, and various additional control and parity signals.

For each of these channels, there is a subset of signals for Credit Management. These signals flow from the Receiver of the Channel to the Source of the Channel. These Credit Management Signals return Credits from the Receiver of the Channel to the Source of the Channel to allow the source to emit new valid Beats on the Channel. The source of the Channel shall not emit a new Beat without possessing the appropriate Credit(s).

The UPLIClk signal is the clock delineating the cycle boundaries on the clock synchronous UALink Protocol Level Interface. The UPLIReset_N signal is a low-active signal to reset the UALink Protocol Level Interface.

In this specification, similar signals that occur in different channels may be referred to using a shorthand notation. For example, The ReqVld signal in the Request Channel, the RdRspVld signal in the Read Response/Data Channel, the WrRspVld signal in the Write Response Channel, and the OrigDataVld signal in the Originator Data Channel are collectively referred to by the following notation: *Vld. These signals indicate, in each channel, whether a valid “Beat” of information is present on the channel.

Other examples include:

*Data: (RdRspData signal in the Read Response/Data Channel; OrigData signal in the Originator Data Channel)

*CreditVld: (ReqCreditVld signal in the Request Channel; RdRspCreditVld signal in the Read Response/Data Channel; WrRspCreditVld signal in the Write Response Channel; OrigDataCreditVld signal in the Originator Data Channel).

2.2 UALink Stack Component

Figure 2-4, below illustrates the UALink Stack Component which consists of an independent UPLI Originator and an independent UPLI Completer and logic units to implement the UALink Protocol Layers. The UPLI Completer and UPLI Originator within a given UALink Stack shall not communicate directly with one another. Rather, the UPLI Completer and UPLI Originator are connected to an UALink TL (Transport Layer).

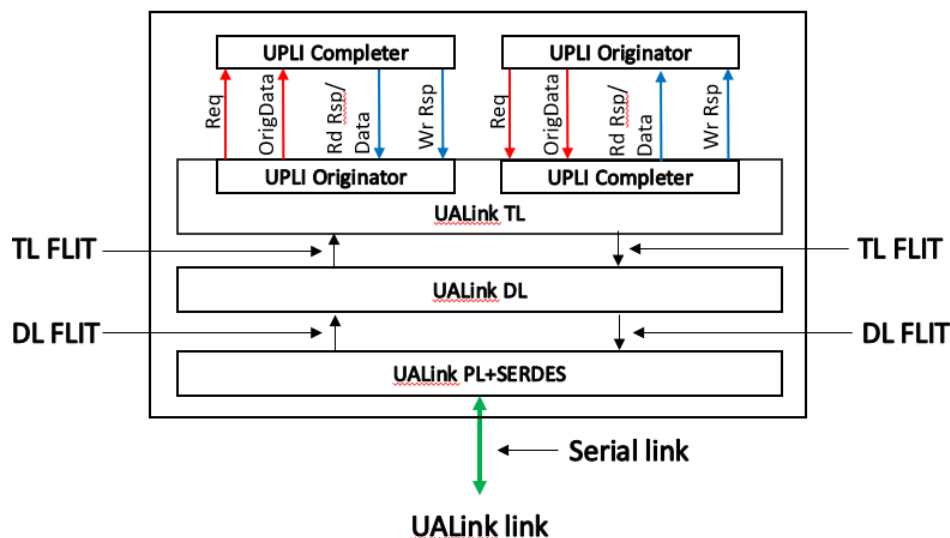


Figure 2-4 UALink Stack Component

The UALink TL converts the UPLI Protocol Channels driven by the UPLI Originator and UPLI Completer (Req and OrigData for the UPLI Originator and Rd Rsp/Data and Wr Rsp for the UPLI Completer) into a TL Flit that is passed to the UALink DL Layer. Similarly, the UALink TL receives a TL Flit from the UALink DL that is unpacked into the UPLI Protocol Channels received by the UPLI Completer and the UPLI Originator (Req and OrigData for the Completer and Rd Rsp/Data and Wr Rsp for the Originator).

The UALink DL receives several TL Flits and adds CRC protection and a header to the Flit to form a DL Flit and passes this DL Flit on to the UALink PL. Similarly, the UALink DL receives DL Flits from the UALink PL, strips the CRC and header from that Flit, and forms TL Flits that are passed on to the UALink TL.

The UALink PL receives DL Flits and produces a code word with FEC encoding that is serialized and transmitted to a UALink PL in a connected UALink PL Stack Component. Similarly, the UALink PL receives a serialized code word with FEC from the UALink PL in the connected UALink Stack Component performs FEC decode and converts that into DL Flit.

To create an interface between an Accelerator and the Switch, two UALink Stack Components are connected as shown below in Figure 2-5, this overall interface shall be bi-directional and symmetric with the UPLI Originator in the Accelerator communicating with the UPLI Completer in the Switch and the UPLI Originator in the Switch communicating with the UPLI Completer in the Accelerator.

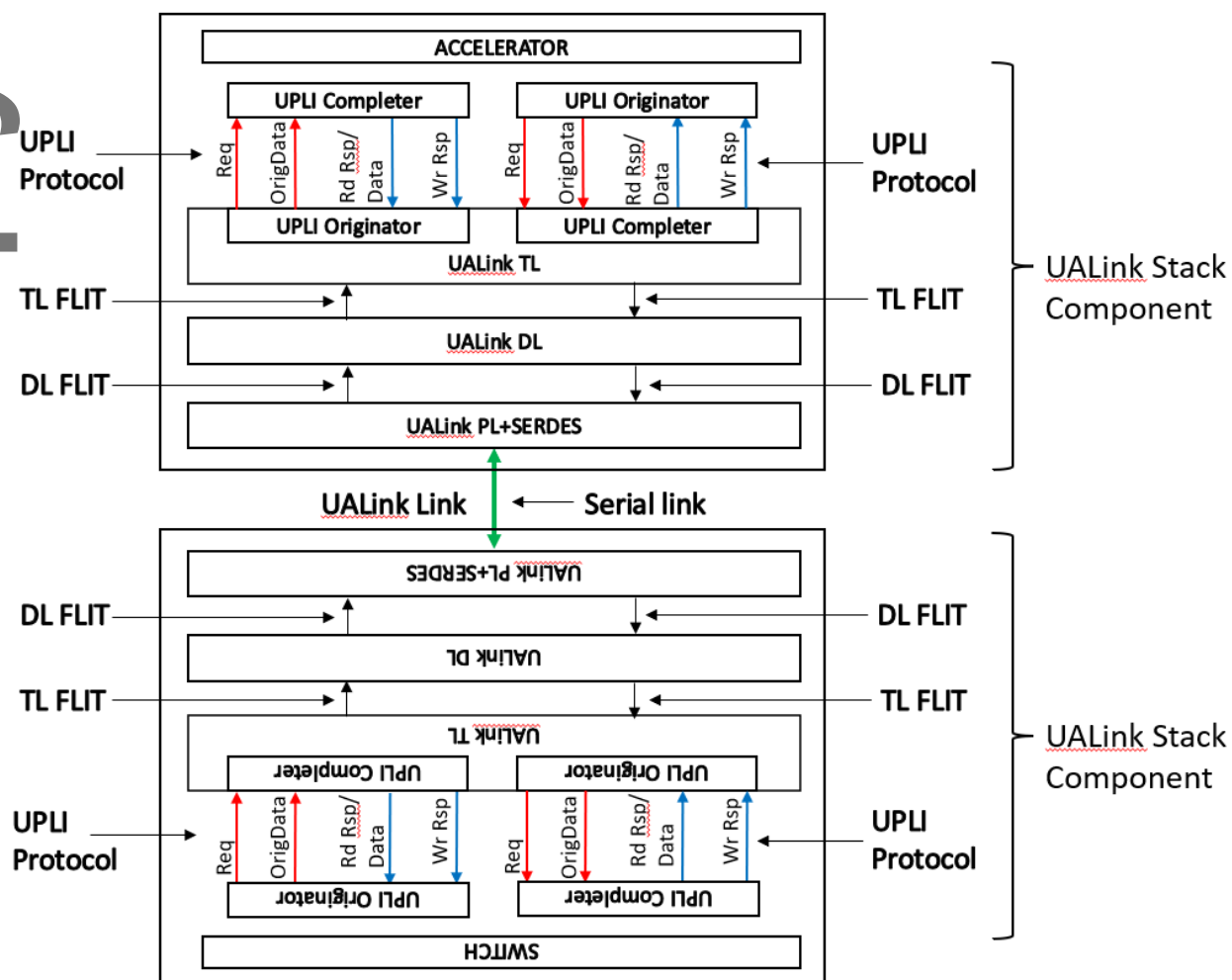


Figure 2-5 Connected UALink Stack Components

2.3 UALink UPLI Request and Response Paths

Figure 2-6, shows a logical representation of two different Accelerators (ACC "A" and ACC "B") connected through different pairs of UALink Stack Components through a Switch:

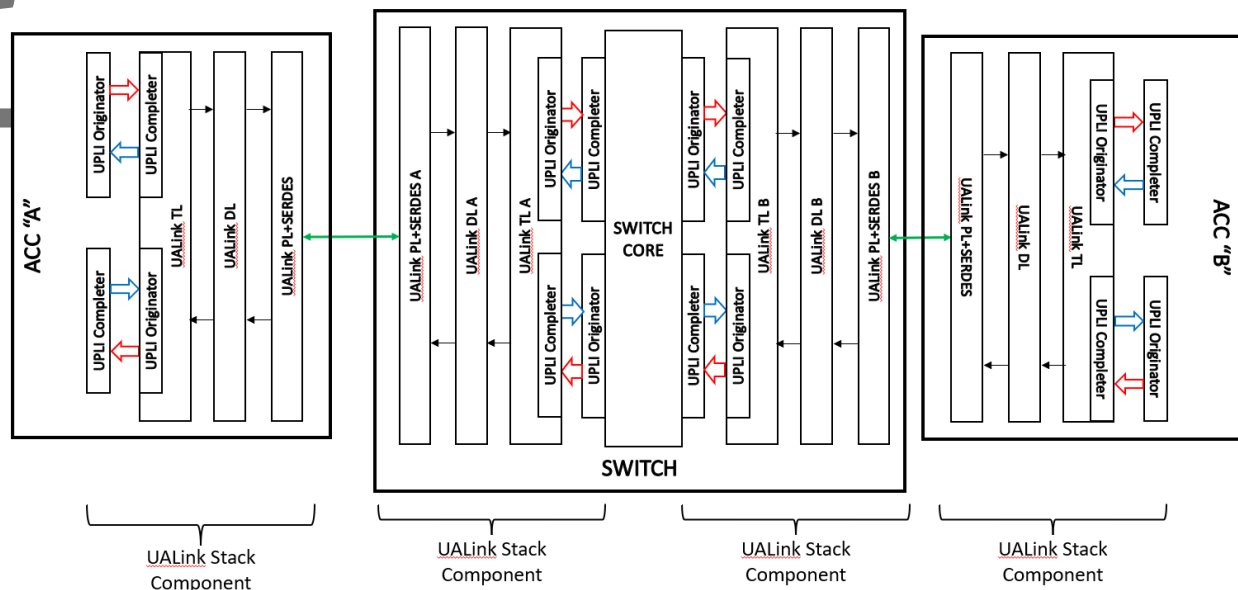


Figure 2-6 End-to-End UALink Connection Between two Accelerators

To issue a Request from Accelerator A to Accelerator B, the UPLI Originator in Accelerator A shall issue a Request on the Request Channel (and if the Request is a Write, UPLI Write Message, Vendor Defined write Class Command also issues the first Beat of data on the Originator Data Channel. Atomic or Vendor Defined Atomic write Class Commands issue Operand Data on the Originator Data Channel on the first Beat of data on the Originator Data Channel). The UALink TL receives the Request and shall package it into a TL Control Half-Flit (Data Beats shall be packaged as subsequent Data Half-Flits). TL Half-Flits are combined to create a TL Flit and a TL Flit shall consist of two TL Half-Flits. The TL Flit is then modified by the UALink DL to add FEC and CRC and becomes a DL Flit which is then passed through the PHY on Accelerator A as a bitstream to PHY A on the Switch. The PHY A on the Switch is one of several possible PHYs on the Switch where traffic from Accelerator A can be received. The bitstream is reconstituted into the DL Flit by PHY A and passed through UALink DL A and converted to a TL Flit. Finally, the TL Flit passes through UALink TL A and is presented to the UPLI Completer in the Switch attached to UALink TL A.

The UPLI Completer at the Switch attached to UALink TL A shall package the UPLI Protocol elements and the Switch Core shall route these elements to the UPLI Originator attached to UALink TL B that can deliver the Request to Accelerator B. The process of transiting the two UALink Stack Components to reach Accelerator B from the Switch shall be the same process as transiting from Accelerator A to the Switch and terminates at the UPLI Completer in Accelerator B.

When the Request has been processed in Accelerator B, Responses (and data for Read, Vendor Defined Read class Commands, Atomic Requests, or Vendor Defined Atomic class Commands) shall be issued by the UPLI Completer on Accelerator B and pass through the two UALink Stack Components to reach the UPLI Originator connected to UALink TL B. These Responses are routed through the Switch Core and then pass through the two UALink Stack Components starting at the

Switch UPLI Completer attached to UALink TL A and terminate at the UPLI Originator in Accelerator A.

The following diagram, Figure 2-7, illustrates the flow of Responses and Requests for a pair of Accelerators. For clarity, this figure omits the UALink DL and UALink PHY blocks. Requests initiated by the Accelerator A UPLI Originator are ultimately sent to the UPLI Completer in Accelerator B. However, the Accelerator A Request is first received by the UPLI Completer in the Switch and is then routed to the UPLI Originator in the Switch ultimately connected to Accelerator B which passes the Request on to Accelerator B's UPLI Completer.

In a similar fashion, the Response for Accelerator A's Request is ultimately sent from the UPLI Completer in Accelerator B to the UPLI Originator in Accelerator A via first the UPLI Originator at the Switch connected to Accelerator B and then the UPLI Completer in the Switch ultimately connected to Accelerator A.

The requests and Responses for Accelerator B's Requests and Responses follow paths that are symmetrical to Accelerator A's requests and Responses, but in the opposite direction.

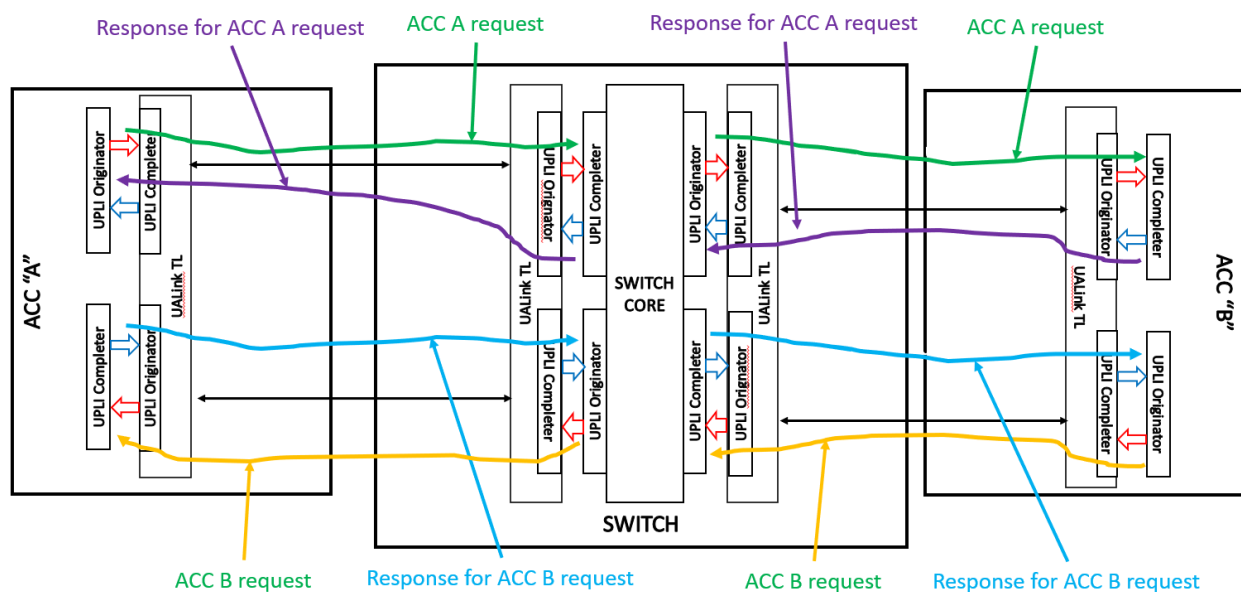


Figure 2-7 UPLI Request and Response Flows

2.4 Routing a Transaction from End-to-End

This Section illustrates the flow of a Request from an Originator Device to a Completer Device and the corresponding Response from the Completer Device to the Originator Device.

A UALink station has four UALink lanes which can be bifurcated into one x4, two x2, or four x1 UALink Ports. All UALink stations in a Pod (both on the Switches and at the Accelerators) must be bifurcated in the same manner.

The number of UALink Ports on each Accelerator shall be equal. The Port identifier within a station shall start at 0 and shall be numbered consecutively. UALink Stations shall be numbered contiguously, and a Request or Response is routed from a Source Port of a Switch to a Destination Port of a Switch by routing from the Source Port on the Source Station to the Target Station and then to the Target Port within the station. How a Request or Response is routed from within a Switch is ultimately up to the implementation.

In a Pod, a Physical Switch shall have at least as many Ports as the number of Accelerators in the Pod and the number of Ports on a Physical Switch should equal the Number of Accelerators in the Pod (when the number of Ports on a Physical Switch matches the Number of Accelerators in the system – as shown below in Figure 2-8 Example system with 32 Accelerators with 32 x1 UALink Links -- the concept of Switch and Physical Switch are equivalent) . This lets each Switch connect to every Accelerator in the Pod via a specific Port. The example system shown below in Figure 2-8 illustrates a Pod with 32 Accelerators numbered Accelerators 0, 1, to 31 and 32 Switches numbered 0, 1, to 31. For clarity, certain of the UALink Links have been shown while the other links are omitted.

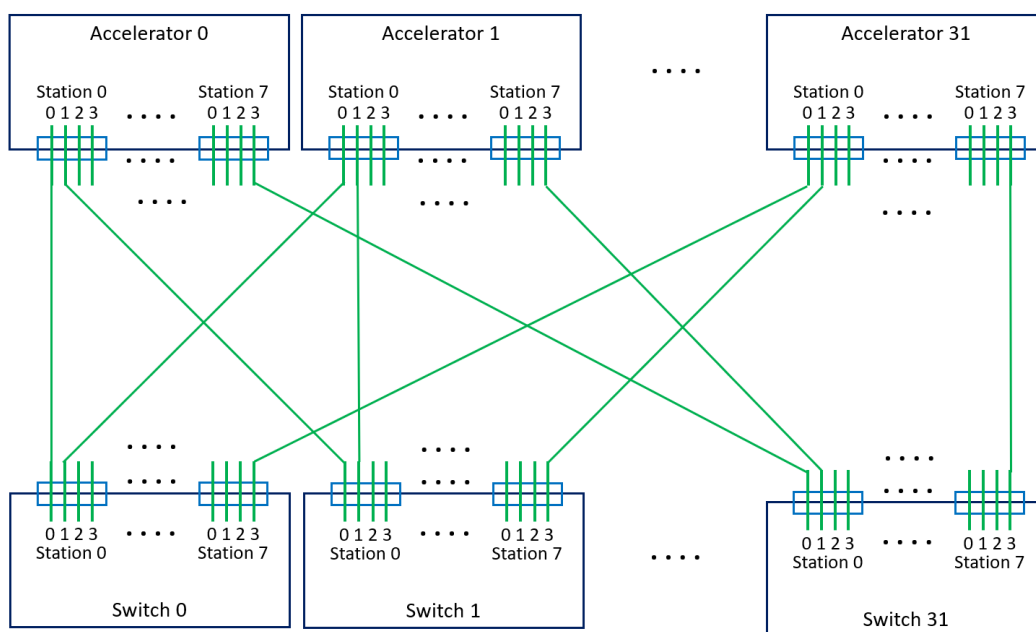


Figure 2-8 Example system with 32 Accelerators with 32 x1 UALink Links

The Accelerators and Switches shall use a specific subset of signals in the various UPLI Channels to route and process Requests and data through the system. The rest of the signals within a Channel are conveyed, usually unmodified, with the Transaction. However, specific signals may be modified at various points as the Transaction progresses through the Pod. In addition to the routing signals

and the other signals, a set of Credit Management signals shall provide flow control within a each UALink Channel within any given UALink Protocol Level Interface.

Specifically, the Request Channel (Req) shall contain two 10-bit signals, “ReqSrcPhysAccID” and “ReqDstPhysAccID” that specify the Source Accelerator and Destination Accelerator of the Request. In addition, the Request Channel shall contain a 2-bit signal “ReqPortID” that shall control, at certain UALink Protocol Level Interfaces, the routing of the Request onto the bifurcated UALink Links at the associated UALink Stack Component and shall control the Time Division Multiplexing (TDM) of the UALink Protocol Level Interfaces, as described in a later section, at all UALink Protocol Level Interfaces.

The Read Response/Data (RdRsp) Channel shall contain signals “RdRspSrcPhysAccID”, “RdRspDstPhysAccID”, and “RdRspPortID” and the Write Response (WrRsp) Channel shall contain signals “WrRspSrcPhysAccID”, “WrRspDstPhysAccID”, and “WrRspPortID”. These signals shall function in those UPLI Channels in an analogous way to the respective signals in the Request Channel.

A sample Read Request to address “X” from an Accelerator A (numbered “0”) to an Accelerator B (numbered “31”) in a Pod connected as shown above (Figure 2-8 Example system with 32 Accelerators with 32 x1 UALink Links) will be used to illustrate the processing of a transaction using the following figure (Figure 2-9 Read Request end-to-end flow with Response):

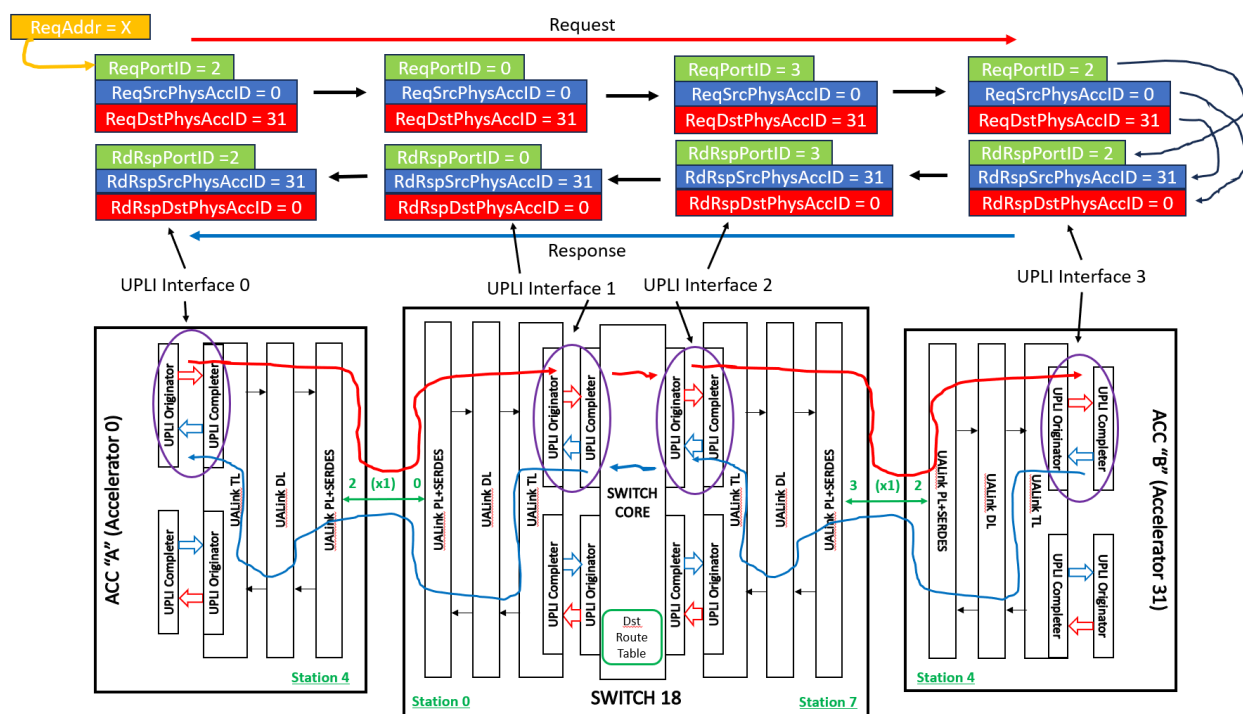


Figure 2-9 Read Request end-to-end flow with Response

The Read Request begins with an Originator Device (not shown) forming a Read Request to address “X”. Conceptually, Requests can go out any UALink Link on the Accelerator (each UALink Link is attached to a Switch which is capable of routing any Request to any Destination Accelerator). However, generally speaking, to enforce ordering for Requests all accesses within any given 256-byte region of memory must be routed out the same UALink Port (in this example, arbitrarily chosen to be Station 4, Port 2 for the address “X” – see section 2.7.9 for complete details).

The Read Request is formed at the Originator Device and is issued to the UPLI Originator at Station 4 with an indication to use UALink Port 2, an indication of the Source Accelerator (0 in this example), an indication of the Destination Accelerator (31 in this example), and any other information associated with the Read Request. The UPLI Originator issues the Request on UALink Protocol Level Interface 0 with ReqPortID set to '2', ReqSrcPhysAccID set to '0' and ReqDstPhysAccID set to '31'. The ReqSrcPhysAccID and ReqDstPhysAccID fields shall remain unchanged from the initial Originator in the Source Accelerator A to the final UPLI Completer in the Destination Accelerator B.

The Read Request is then conveyed across the UALink Link attached to Station 4, Port 2 (as controlled by the ReqPortID value) on Accelerator 0. The ReqPortID value shall not be propagated over the UALink Link but rather instead be locally generated at the Switch based on the Port the Read Request entered the Switch. The Read Request enters Switch 18 (the UALink Link at Station 4, Port 2 on all Accelerators are attached to Switch 18 in this example topology) at Station 0, Port 0 (The UALink Links at Station 0, Port 0 on all Switches are attached to Accelerator 0 in this example topology). Based on the entry port at the Switch, a new value for ReqPortID of 0 is assigned and used at UALink Protocol Level Interface 1 in the Switch. The ReqPortID value at Ultra Link Protocol Interface 1 only controls the TDM for the Read Request while transiting UALink Protocol Level Interface 1 into the Switch core.

Once in the Switch core, a look up, based on the ReqDstPhysAccID signal, is performed in the Destination Route Table (Dst Route Table) to determine the proper Station and Port the Read Request should be routed through to get to the destination Accelerator (in this case, Station 7, Port 3 – Accelerator 31 is connected to Station 7, Port 3 on all Switches in this example topology) and routes the Read Request to that station (Station 7) and presents the Read Request on UALink Protocol Level Interface 2 with the newly assigned ReqPortID value of 3.

The UPLI Originator at UALink Protocol Level Interface 2 issues the Read Request which exits the Switch on Port 3 and drops the ReqPortID value. The Read Request then enters Accelerator 31 at Station 4, Port 2 and the value of ReqPortID is regenerated to a value of '2' indicating the Port on which this Request entered Accelerator 31. This regenerated ReqPortID value is used when the Request is issued on UALink Protocol Level Interface 3 and controls the TDM for that interface. The UPLI Completer at UALink Protocol Level Interface 3 retains the value of ReqPortID, ReqSrcPhysAccID, and ReqDstPhysAccID while the Request is being processed by a Completer Device in the Destination Accelerator.

When the information for a Read Response is ready (Read Responses may contain more than one Response Beat), the UPLI Completer at UALink Protocol Level Interface 3 forms a Response Beat with RdRspPortID set to the retained ReqPortID value of '2' and uses the retained Station value to deliver the Response Beat to Station 4. This allows the Completer at UALink Protocol Level Interface 3 to route the Response Beat to the correct output Port and Station without referring to the address of the original Read Request. At UALink Protocol Level Interface 2, the RdRspPortID signal has the regenerated value of '3' reflecting the port the Read Response entered the Switch on.

At UALink Protocol Level Interface 1, the RdRspPortID signal value '0' indicates the Port on Station 0 where the Read Response should exit. This value is obtained from a lookup of the Destination Route Table indexed by RdRspDstAccPhysID at UALink Protocol Level Interface 2 which indicates the destination Accelerator is connected to Port 0, Station 0 on this Switch. Finally, the value of RdRspPortID at UALink Protocol Level Interface 0 is '2', matching the Port the Read Response entered the Source Accelerator. For Read Responses containing more than one Beat of Data, the same process is followed for each Beat.

Writes Requests shall act similarly to Read Requests with the exception that data is issued on the Originator Data Channel to the Destination Accelerator with a single Beat Write Response containing no data being returned.

Evaluation Copy

2.5 UPLI Channel Time Division Multiplexing (TDM)

Each UALink Station driven by an UALink Stack Component can interface to 8 UALink Lanes that can be bifurcated into four x1 UALink Links, two x2 UALink Links, or one x4 UALink Link. The UALink Ports attached to the UALink Links shall be numbered “0” for a x4 bifurcation, “0, 1” for a x2 bifurcation, and “0, 1, 2, 3” for a x1 bifurcation.

Each Channel in an UALink Protocol Level Interface shall be Time Division Multiplexed (TDM), meaning that for a given cycle, the signals within the UALink Protocol Level Interface channel (aside from the Credit Management signals) are associated with a specific bifurcated Port on the UALink Stack Component attached to a UALink Link. Figure 2-10 illustrates an example UALink Stack Component with two associated UALink Protocol Level Interfaces with the UALink station bifurcated into four x1 UALink Links:

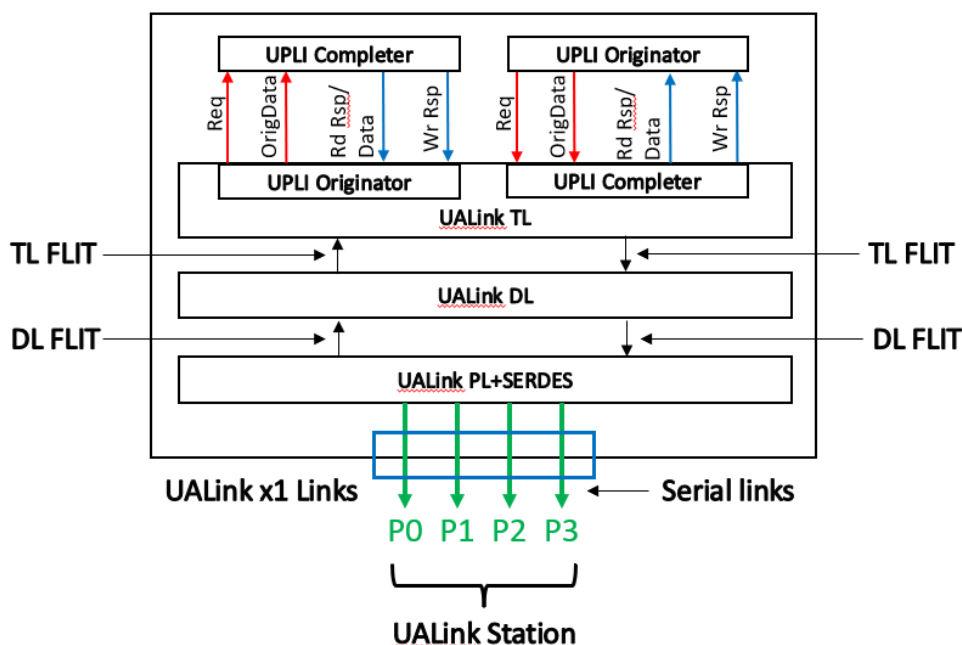


Figure 2-10 UALink Station with x1 bifurcation

Each Channel in the UALink Protocol Level Interface in the UALink Stack Component shall have a unique “Valid” signal (“ReqVld” for the Request Channels, “OrigDataVld” for the Originator Data Channels, “RdRspVld” for the Rd Response/Data Channels, and “WrRspVld” for the Write Response Channels) that indicates when valid data is being driven onto the Channel. The “PortID” signal (“ReqPortID” for the Request Channels, “OrigDataPortID” for the Originator Data Channels, “RdRspPortID” for the Rd Response/Data Channels, and “WrRspPortID” for the Write Response Channels) for a given UALink Protocol Level Interface channel shall signify the TDM value for the current cycle when the “Valid” signal for that Channel is asserted and shall be ignored otherwise.

The TDM value shall cycle through a sequence of all the possible Port values (based on the bifurcation) in ascending order, wrapping back to 0 at the maximum value, every ‘N’ cycles, where N is the number of ports in the station. Valid cycles for a given PortID may only be presented on any UALink Protocol Level Interface Channel every N cycles.

The TDM cycle for the Request Channel and the Originator Data Channel shall be set by the “PortID” signal value on the first ReqVld Beat for that Request Channel (whether the Request is a Read or

Vendor Defined Read class Command where the OrigData field is unused or is a Write, Atomic, UPLI Write Message, Vendor Defined Write class Command, Vendor Defined Atomic class Command, or Atomic Request where the OrigData field is used) and shall remain fixed thereafter for those channels (the Request and Originator Data Channel must be on the same TDM phase to allow the Req and OrigData Channel to both be valid for the first beat of any Request that issues data on the Originator Data Channel). For example, if after reset, if the first valid packet on a Request Channel has a ReqPortID value of '2', the TDM for that Request Channel and Originator Data starts on that cycle with a value of 2 and cycles through the values in the following fashion: 2, 3, 0, 1, 2, 3, 0, 1, . . . thereafter (for a x1 bifurcation. For a x2 bifurcation, the sequence is 0, 1, 0, 1, and for a x4 bifurcation the sequence is 0, 0, 0.....). The TDM cycle for the Read Response/Data and Write Response Channels shall be set independently based on the PortID value for the initial Read and Write Response Beats respectively.

The PortID signal for any subsequent "Valid" cycle on the Channel shall match the established TDM value for the current cycle. The explicit PortID signal removes the need for an explicit synchronization sequence to establish the TDM value and allows a receiver to simply use PortID to identify the TDM cycle without having to explicitly track the current TDM cycle (though the receiver implementation can choose to track the TDM cycle to check for the correct TDM cycle value on received Beats). Finally, an explicit PortID signal is useful for debug.

The UALink TL shall be responsible for assembling the various valid packets for differing Ports on the differing UPLI Channels into outbound TL Flits and placing the various received TL Flits onto the correct UPLI TDM cycles on each UPLI Interface.

2.6 UALink Protocol Level Interface Flow Control and overall UALink Flow control

Flow control in the UALink Protocol Level Interface shall utilize a Credit-based control mechanism with Credits being per Port, per UPLI Channel. At initialization or after reset of the UPLI Interface, the Sender side of a UPLI Channel shall have no Credits and the Receiver side of a UPLI channel shall issue an initial set of Credits to the Sender side.

The Receiver shall indicate a Credit or Credits is being initially issued (or, after initialization, being returned) using the following signals:

- A *CreditVld[3:0] signal (ReqCreditVld[3:0], OrigDataCreditVld[3:0], RdRspCreditVld[3:0], WrRspCreditVld[3:0]) shall indicate a Credit or Credits is being returned to the Sender and for what port (*CreditVal[0]=1 indicates Port 0 and so on). Credits may be returned on any Port of set of Ports simultaneously by asserting the appropriate signals within *CreditVld[3:0]. Credit returns shall be independent of the Time Division Multiplexing of the UPLI interface (i.e. Credits for any Port may be returned in any cycle).
- A *CreditPool[3:0] signal (ReqCreditPool[3:0], OrigDataCreditPool[3:0], RdRspCreditPool[3:0], WrRspCreditPool[3:0]) shall indicate the “type” of the Credit being returned (‘0’ indicates a VC Credit, ‘1’ indicates a Pool Credit) and for what Port (*CreditPool[0] indicates the Credit Type for Port0 and so on). This signal shall only be considered valid when the corresponding *CreditVld[3:0] signal is asserted.
- A set of *CreditPort[0,1,2,3]VC[1:0] signals (ReqCreditPort0VC[1:0], ReqCreditPort1VC[1:0], ReqCreditPort2VC[1:0], ReqCreditPort3VC[1:0], OrigDataPort0VC[1:0], OrigDataPort1VC[1:0], OrigDataPort2VC[1:0], OrigDataPort3VC[1:0], RdRspCreditPort0VC[1:0], RdRspCreditPort1VC[1:0], RdRspCreditPort2VC[1:0], RdRspCreditPort3VC[1:0], WrRspCreditPort0VC[1:0], WrRspCreditPort1VC[1:0], WrRspCreditPort2VC[1:0], WrRspCreditPort3VC[1:0]) shall indicate the Virtual Channel of the credit being returned for the Channel and Port associated with the signal (for example, ReqCreditPort0VC[1:0] is the Virtual Channel of the credit being returned for the Request Channel on Port0 and similarly for the other signals). This signal shall only be considered valid when the corresponding *CreditVld[3:0] signal is asserted.
- A set of *CreditPort[0,1,2,3]Num[1:0] signals (ReqCreditPort0Num[1:0], ReqCreditPort1Num[1:0], ReqCreditPort2Num[1:0], ReqCreditPort3Num[1:0], OrigDataPort0Num[1:0], OrigDataPort1Num[1:0], OrigDataPort2Num[1:0], OrigDataPort3Num[1:0], RdRspCreditPort0Num[1:0], RdRspCreditPort1Num[1:0], RdRspCreditPort2Num[1:0], RdRspCreditPort3Num[1:0], WrRspCreditPort0Num[1:0], WrRspCreditPort1Num[1:0], WrRspCreditPort2Num[1:0], WrRspCreditPort3Num[1:0]) that shall indicate the number of Credits being returned for the channel and port associated with the signal (for example ReqCreditPort0Num[1:0]+1 is the number of Credits being returned for the Request Channel on Port0 and similarly for the other signals).. This signal shall only be considered valid when the corresponding *CreditVld[3:0] signal is asserted.

At initialization or after Reset, the Receive Side of the UPLI Channel will release a set of Credits corresponding to the available buffering at the Receive side of the UPLI Channel.

The Receive side may issue a set of Pool credits equal to the number of Pool buffers in the Receive side. The Receive side Pool buffers indicated by these Credits shall be able to process UPLI beats for

any Virtual Channel. The Receive side may issue a set of VC credits associated with specific Virtual Channels equal to the number of VC buffers in the Receive side for each of the Virtual Channels supported by the Receive side VC buffers. A Receive Side VC buffer need only process UPLI beats for a specific Virtual Channel.

The Receiver Side shall release one of only Pool Credits, only Virtual Channel Credits, or some combination of Pool Credits and Virtual Channel Credits.

When the Receiver side is done releasing the initial Credits for the given Channel, the Receiver side asserts the *CreditInitDone[3:0] signals (ReqCreditInitDone[3:0], OrigDataCreditInitDone[3:0], RdRspCreditInitDone[3:0], WrRspCreditInitDone[3:0]) as each port finishes. The Receiver side shall not assert *CreditInitDone until at least one cycle after the last Credit is released for that Channel. Each signal in *CreditInitDone[3:0] is associated with a given Port (*CreditInitDone[2] corresponds to Port 2 on the associated Channel). The *CreditInitDone[3:0] signals may be asserted independently and once asserted shall remain asserted until the UPLI Interface is reset or powered off.

The Sender side shall monitor the *CreditInitDone signal until that signal has been asserted an implementation specific number of cycles greater than 1. Once that has occurred, the Sender side shall consider the initial issuance of the Credits to be complete and shall no longer monitor the signal.

The monitoring of the signal for an implementation specific sized burst of asserted values ensures that an error on this signal that is shorter in duration than the burst length will not errantly terminate the initial credit release early. Further, ignoring the signal after this burst will ensure any subsequent errors on the signal do not impact the Channel.

Once the initial credit release is complete for a given port and Channel, the Send side may start issuing UPLI Beats for that Port and Channel if all other constraints are met. For example, when the Sender wishes to issue a write, it shall have the necessary credits in both the Request Channel (to issue the Request) and the Originator Data Channel (to issue the Beats contiguously and starting with the Request) before the Request can be issued. When a Beat is issued, the Send side shall set the *VC[1:0] signal (ReqVC[1:0], OrigDataVC[1:0], RdRspVC[1:0], WrRspVC[1:0]) to indicate the Virtual Channel of the Beat and shall set the *Pool signal (ReqPool, OrigDataPool, RdRspPool, WrRspPool) to indicate if a Virtual Channel Credit or a Pool Credit was used to issue the beat.

If any Pool Credits were initially released, the sender side shall issue UPLI beats for differing Virtual Channels using those Pool Credits (indicated by setting *Pool = b'1') according to an allocation of those Pool Credits to the various Virtual Channels that the Sender selects (and which the Sender may vary over time). If any Virtual Channel Credits were initially released, the Sender side shall issue UPLI beats using Virtual Channel Credits for the Virtual Channels (indicated by setting *Pool = b'0') in accordance with the number of Virtual Channel Credits initially released for each Virtual Channel.

The Sender Side may independently choose the type of Credit used for each beat of a Request. For example, for a Write Request, the Credit used to issue the Request could be a Pool Credit and the Credits used to issue the data beats on the OrigData Channel could be a mixture of Pool and VC Credits.

When a UPLI Beat is received at the Receive side, the Virtual Channel of the Beat (*VC[1:0]) and the Pool indication (*Pool) shall be recorded by the Receive side and these values are played back to the Sender when returning the Credit for that beat. The Sender side shall return the Credit to the appropriate Pool or Virtual Credit count.

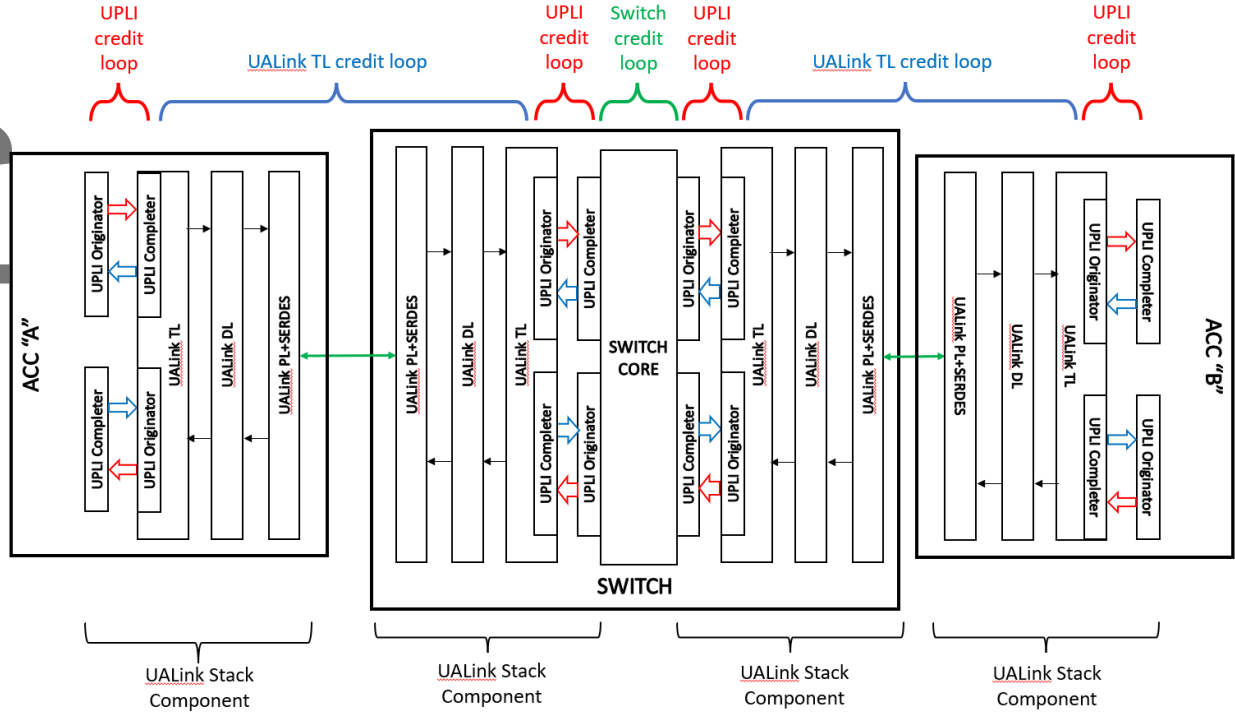


Figure 2-11 Flow control loops

The figure above (Figure 2-11) illustrates the various flow control Credit loops in the full path between two Accelerators. UALink Protocol Level Interface Credit controls are used at each instance of the UALink Protocol Level Interface along the path. A distinct independent Credit loop mechanism from one UALink TL to the next UALink TL is used to pace Requests and Responses between the UALink TL's.

2.7 Interface Signals

This section describes the signals that make up the UALink Protocol Level Interface and how transactions are conveyed on the various channels of the UALink Protocol Level Interface.

2.7.1 Signal Groups

The UALink Protocol Level Interface shall be organized in the following groups of signals:

- Common signals
- UALink Protocol Level Interface Control signals
- Request Channel
- Read Response/Data Channel
- Write Response Channel
- Originator Data Channel

In this specification, similar signals that occur in different channels may be referred to using a shorthand notation. For example, The ReqVld signal in the Request Channel, the RdRspVld signal in the Read Response/Data Channel, the WrRspVld signal in the Write Response Channel, and the OrigDataVld signal in the Originator Data Channel are collectively referred to by the following notation: *Vld. These signals shall indicate, in each Channel, whether a valid Beat of information is present on the Channel.

Other examples include:

*Data: (RdRspData signal in the Read Response/Data Channel; OrigData signal in the Originator Data Channel)

*CreditVld: (ReqCreditVld signal in the Request Channel; RdRspCreditVld signal in the Read Response/Data Channel; WrRspCreditVld signal in the Write Response Channel; OrigDataCreditVld signal in the Originator Data Channel).

2.7.2 Common Signals

Table 2-1 lists the signals that make up the Common Signal group. SOC-pervasive logic shall supply these to each UALink Protocol Level Interface.

Table 2-1 Common Signals

Name	Size	Description
UPLIClk	1	Interface clock, sourced from the SOC pervasive logic. UALink Protocol Level Interface signals are synchronous to this clock as well as the UPLIReset_N, *ClkReq, and *ClkAck signals.
UPLIReset_N	1	Interface reset signal, active LOW and is asserted and de-asserted and asserted synchronously to UPLIClk.

The rising edge of UPLIClk shall determine the timing of information transfer across the interface. In general, all signals shall be stable prior to and immediately following the rising edge of UPLIClk.

UPLIReset_N shall be a negative-active signal (that is, the signal is asserted when the voltage level of the signal is low.) and may be asserted at any time.

2.7.3 UPLI Transactions/Channel usage

A Transaction on the UALink Protocol Level Interface can contain a Read, Write, a Vendor Defined Command, a UPLI Write Message Request, a Vendor Defined UPLI Write Message Request, or an

Atomic. These Transactions read from memory, write to memory, atomically modify memory while optionally returning the original value in memory, or deliver a UPLI message, or performs a Vendor defined operation.

A UPLI transaction consists of multiple cycles (also called “Beats”), which shall occur in a specific order on a specific subset of the UPLI channels (Request, Read Response/Data, Write Response, Originator Data) in the UALink Protocol Level Interface depending on the type of the UPLI Transaction. Transactions shall be issued at the UPLI Originator and shall be processed at the UPLI Completer. The UPLI Completer shall return a Response to the Request on either the Read Response/Data Channel or the Write Response Channel, but not both. Which Channel is used to return a Response shall depends on the transaction type.

Each UPLI transaction shall be initiated by a Request in the Request Channel indicated by the assertion of the Valid Signal (ReqVld) in the Request Channel (a Beat). A Request Beat shall contain other information about the Request involved with routing and identifying the Request such as the Port ID (ReqPortID), the Source Accelerator and Destination Accelerator physical IDs (ReqSrcPhysAccID and ReqDstPhysAccID), and a Transaction Tag to identify the Request from among other Requests (ReqTag). In addition, the Request Beat shall also contains information about the Request such as the Address (ReqAddr), the Command Type (ReqCmd), an indication of the number of doublewords to transfer (ReqLen), and attributes about the Request (ReqAttr) among other fields.

The data returned for a Read Transaction shall be conveyed from the Completer to the Originator as data Beats on the Read Response/Data Channel along with fields providing additional information about the Read Response. This information shall include the status of the Read Transaction (RdRspStatus), which Beat of data is being returned in this beat (RdRspOffset), the total number of Data Beats in the Response (RdRspNumBeats), whether the Beat has been corrupted (RdRspDataError), and if the current Beat is the last Beat to be transmitted (RdRspLast), A Read Response may contain one to four Data Beats.

Read Response Beats shall occur on the Read Response/Data Channel after the Read Request has been received and processed by the Final Completer.

For Write Transactions, the Data shall be conveyed from the Initial Originator to the Final Completer as one to four Data Beat(s) on the Originator Data Channel, each Beat shall Contain the Port ID (OrigPortID) field to provide partial routing information. The first Originator Data Beat for a given transaction shall occur in the same cycle as the Request for the Write. This shall allow the Originator Data beats to inherit the remaining routing and identifying information (ReqSrcPhysAccID, ReqDstPhysAccID, ReqTag) from the Write Request in the Request Channel as well as the number of Data Beats in the Transaction (ReqNumBeats).

A Write Request Beat shall also include information such as a set of Byte Enables to indicate the bytes to be updated in memory (OrigDataByteEn), which Beat of Data is being sent in this Beat (OrigDataOffset), whether this Beat has been corrupted (OrigDataError), and if the current Beat is the last Beat to be transmitted (OrigDataLast).

In the UALink Protocol Level Interface, an Atomic Request can either return the initial value of memory before the modification and atomically modify memory (referred to as an AtomicR command) or only perform the modification of memory (referred to in UPLI as an AtomicNR command).

In addition, all atomic commands (AtomicR and AtomicNR) shall require one or two operands to control the atomic update of memory. For example, an “Atomic add” would require one parameter

to specify the value to Atomically add to memory location(s) and a “Compare-and-Swap” would require two parameters: one for the value to initially compare the memory location to, and a value to swap into the memory location if the initial comparison matched. The exact semantics of the various AtomicR/AtomicNR Commands and the number of parameters they need is not specified by this Specification; however all Atomic Commands shall be limited to no more than two parameters.

In the case of an Atomic Request for either an AtomicR Command or an AtomicNR Command, the Transaction proceeds as described above for a Write Request with the exception that the OrigData Channel shall be used to transfer operand data for the Atomic instead of transferring write data. For an AtomicNR Command, the Completer shall provide a Response on the Write Response Channel. For an AtomicR command, the Completer shall return the values initially read from memory as data on the Read Response/Data Channel along with status information about the transaction.

2.7.4 Request Channel

An Originator shall use the Request Channel to request that data be written to or read from the system memory space. Table 2-2, Request Channel Signals, lists the signals that shall make up a Request Channel.

The column labeled Driver indicates whether the Originator or the Completer drives the signal.

Table 2-2 Request Channel Signals

Name	Size (bits)	Driver	Description
Request Channel Information and Control Signals These signals are Time Division Multiplexed (TDMed) according to ReqPortID value.			
ReqVld	1	Originator	Request valid. This signal indicates that the Originator is presenting valid information (a Beat) on this channel.
ReqPortID	2	Originator	Request Port ID. Where the Originator drives the Request Channel to a TL, indicates the Port associated with that TL that the Request Channel Beat is to be presented on. For all Originators, ReqPortID indicates the TDM cycle for the Request Channel.
ReqASI	2	Originator	Request Address Space Identifier. For any Request that is not a UPLI Write Message Request or Vendor Defined Command, identifies the address space. For a UPLI Write Message Request, this field is either Reserved or specifies a specific function for the type of the UPLI Write Message Request (the type of UPLI Write Message is specified in the ReqMetaData field) and for a Vendor Defined Command this signal is vendor defined.
ReqAuthTag	64	Originator	Request Authorization Tag. Authorization Tag for the Request. See the Chapter on Security for more details about this signal and Authorization. When Authorization is not active for the Request or ReqVld is de-asserted, this field shall be driven to zero.
ReqSrcPhysAccID	10	Originator	<u>For a ReadReduce Request issued from an Accelerator:</u> Bits [9:7] indicate the rounding mode for the Reduction, and bits [6:0] indicate the Reduction Operation/Data Type/Intermediate Precision (if applicable)/Result Data Type (if applicable) for the ReadReduce. <u>For all other Requests issued by an Accelerator or any Request issued from a Switch:</u> Request Source Physical Accelerator ID. Physical Accelerator ID for the Source Accelerator of the Request (i.e. the Accelerator with the Initial UPLI Originator that initiated the Request).
ReqDstPhysAccID	10	Originator	<u>For ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMulticast, BlockRead, BlockWriteFull, and Invoke Block Collective Invoke Requests:</u> This signal shall contain the 10-bit GroupID for the In-Network-Collective (INC) the Request is targeting. Conveying the GroupID on this wire to the destination Accelerator allows implementations to utilize this information in implementation specific memory protection schemes for In-Network-Collectives (INC). Information from the Group Table in the Switch will be used to provide routing of the various INC Requests to the Destination Accelerators. <u>For all other Requests:</u> Request Destination Physical Accelerator ID. Physical Accelerator ID for the Destination Accelerator of the Request (i.e. the Accelerator with the Final UPLI Completer that will satisfy the Request)
ReqTag	11	Originator	Request Tag. This field is the Transaction Tag used to uniquely identify each outstanding Request (tag shared across all command types) from the Source Accelerator UALink Port within the Station that issued the Request for non Block* Requests. For Block* Requests, the ReqTag field contains the Tag value assigned by the Switch when issuing the Request.
ReqNumBeats	2	Originator	Request Number of Data Beats. Indicates the number of Data Beats transferred on the OrigData Channel for this Request for a Write, Vendor Defined Write class Command, Atomic, Vendor Defined Atomic class Command or UPLI Write Message. The number of Data Beats transferred is (ReqNumBeats+1). This field is intended to relieve Switches of the need to compute the number of Beats transferred on the OrigData Channel for the Request from other fields in the interface. Switches shall rely on this field to

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Name	Size (bits)	Driver	Description
			determine the number of Beats for Requests transferred on the OrigData Channel. This field shall only valid for Requests with ReqCmd[5] = 1 (Atomics, Writes, Vendor Defined Write class Commands, UPLI Write Message Requests, and Vendor Defined UPLI Write Message Requests). For Requests with ReqCmd[5] = 0 that are not Read class Vendor Defined Commands (Reads), this field shall be driven to 0. For Requests that are Read class Vendor Defined Commands, this signal shall be valid and may be, but is not required to be, used to compute the number of Data Beats returned on the Read Response.
ReqAddr	57	Originator	Request Address. For any Request that is not a UPLI Write Message Request, Vendor Defined Command, or Vendor Defined UPLI Write Message, this field shall specify the Request address. For a UPLI Write Message Request, this field is either Reserved or specifies control information for the UPLI Write Message (the type of the UPLI Write Message shall be specified in the ReqMetadata field). Unless the type of UPLI Write Message defines this field as carrying an address, this field does not specify an address for the UPLI Write Message. UPLI Write Message data is issued as one, two, three, or four 64-byte beats of data without regard to the value in the ReqAddress field. For a Vendor Defined Command, this signal is vendor defined. For Vendor Defined Commands, Vendor Defined UPLI Write Messages, this field is Vendor Defined. For all Request types, the low-order two bits of this field are set to b'00'.
ReqCmd	6	Originator	Request command. This field indicates the Command for this Request and is detailed below.
ReqLen	6	Originator	Request Length. For any Request that is not a UPLI Write Message, Vendor Defined UPLI Write Message or Vendor Defined Command, this field indicates number of doublewords of data to be transferred by the transaction Request. The requested transfer size is (ReqLen + 1) doublewords. For a WriteFull Request, this field shall indicate a transfer of either 64, 128, 192, or 256 bytes corresponding to one, two, three or four 64-byte data beats. For a UPLI Write Message Request, this field is either Reserved or specifies a specific function for the type of the UPLI Write Message Request (the type of UPLI Write Message is specified in the ReadMetaData field). For a Vendor Defined Command or Vendor Defined UPLI Write Message Request, this signal is vendor defined.
ReqAttr	8	Originator	Request Attributes. For any Request that is not a UPLI Write Message, a Vendor Defined UPLI Write Message, a Vendor Defined Command, a BlockRead, or a ReadReduce this field shall specify extended transaction attributes to be used at the Destination Accelerator. The encoding of this field depends on the Request and is detailed below. For a UPLI Write Message Request, this field is either Reserved or specifies a specific function for the type of the UPLI Write Request (the type of the UPLI Write Message is defined by the ReqMetadata field). For a Vendor Defined Command or Vendor Defined UPLI Write Message, this field is vendor defined. For a BlockRead Request, this field is interpreted at byte enables for the read and must be set to x'FF' as all BlockRead Requests transfer 256 bytes with a fully populated first and last doubleword. For a ReadReduce Request from the Source Accelerator to the Switch, the ReqAttr field indicates a Stochastic Seed value for ReadReduce that is used for Add reductions when the rounding mode is Stochastic. For a ReadRequest from the Switch to the destination Accelerator, this field is set to x'FF' to indicate to the Completer Device in the Destination Accelerator to read all the bytes in the first and last doubleword (or the single doubleword if the first and last doubleword are the same).
ReqMetaData	8	Originator	For any Request that is not a UPLI Write Message Request, Vendor Defined UPLI Write Message Request or a Vendor Defined Command, this field shall be an implementation defined control information field conveyed to the Final Completer with the Request. For a UPLI Write Message or Vendor Defined UPLI Write Message, this field specifies the type of the UPLI Write Message as described below. For a Vendor Defined Command, this field is Vendor Defined.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Size (bits)	Driver	Description
ReqVC	2	Originator	Request Virtual Channel. Specifies the Virtual Channel of the Request. In UALink 1.0, this field is informational only to the Switch with the exception that this value is returned to the Originator in the Credit Return signals. The Switch has one virtual channel and passes this field through to the Destination Accelerator unchanged.
ReqPool	1	Originator	Request Pool. Specifies whether the Request was issued using a Pool Credit (ReqPool = 1) or a Virtual Channel Credit (ReqPool = 0).
Request Channel Credit Return signals These signals are not Time Division Multiplexed (TDMed). A Credit or Credits for any port may be returned on any cycle.			
ReqCreditInitDone	4	Completer	Request Credit Initialization Done. When Asserted, no earlier than the cycle after the last Credit is released, indicates to Originator that the initial Credit release for the Port associated with the individual signal is complete (ReqCreditInitDone[2] indicates Port 2 is complete). The individual "done" signals can be asserted independently and once asserted remain asserted until the interface is reset or powered off. The Originator shall monitor the ReqCreditInitDone signal until that signal has been asserted an implementation specific number of cycles greater than 1. Once that has occurred, the Originator shall consider the initial issuance of the Credits to be complete and shall no longer monitor the signal.
ReqCreditVld	4	Completer	Request Credit Valid. Indicates a Credit or Credits is being released to the Originator. Credits are per port and a Station may have 1, 2 or 4 ports. ReqCreditVld[n] corresponds to Port[n]. Credits can be released on more than one port simultaneously.
ReqCreditPool	4	Completer	Request Credit Pool. Indicates the type of Credit indicated on ReqPool for the Request that consumed the Pool or Virtual Channel Credit or Credits being released. If '0', the Credit or Credits being released is a Virtual Channel Credit and if '1', the Credit or Credits released is a Pool Credit. The ReqVC value for the Request corresponding to the returned Credit or Credits is placed on the appropriate ReqCreditPort[0,1,2,3]VC signal.
ReqCreditPort0VC	2	Completer	Request Credit Port 0 Virtual Channel. Indicates the Virtual Channel indicated on ReqVC for the Request that consumed the Pool or Virtual Channel Credit or Credits being released for Port0. Valid when ReqCreditVld[0] is asserted.
ReqCreditPort1VC	2	Completer	Request Credit Port 1 Virtual Channel. Indicates the Virtual Channel indicated on ReqVC for the Request that consumed the Pool or Virtual Channel Credit or Credits being released for Port1. Valid when ReqCreditVld[1] is asserted.
ReqCreditPort2VC	2	Completer	Request Credit Port 2 Virtual Channel. Indicates the Virtual Channel indicated on ReqVC for the Request that consumed the Pool or Virtual Channel Credit or Credits being released for Port2. Valid when ReqCreditVld[2] is asserted.
ReqCreditPort3VC	2	Completer	Request Credit Port 3 Virtual Channel. Indicates the Virtual Channel indicated on ReqVC for the Request that consumed the Pool or Virtual Channel Credit or Credits being released for Port3. Valid when ReqCreditVld[3] is asserted.
ReqCreditPort0Num	2	Completer	Request Credit Port 0 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, ReqCreditPort0Num+1). Only valid when ReqCreditVld[0] is asserted.
ReqCreditPort1Num	2	Completer	Request Credit Port 1 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, ReqCreditPort1Num+1). Only valid when ReqCreditVld[1] is asserted.
ReqCreditPort2Num	2	Completer	Request Credit Port 2 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, ReqCreditPort2Num+1). Only valid when ReqCreditVld[2] is asserted.
ReqCreditPort3Num	2	Completer	Request Credit Port 3 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, ReqCreditPort3Num+1). Only valid when ReqCreditVld[3] is asserted.
Request Channel Parity Signals			
ReqVldParity	1	Originator	Request Valid Parity. Even parity across ReqVld. Checked every cycle.
ReqAuthTagParity	1	Originator	Request Authorization Tag Parity. Even Parity across ReqAuthTag. Checked only when ReqVld is asserted.
ReqAddrParity	1	Originator	Request Address Parity. Even Parity across ReqAddr. Checked only when ReqVld is asserted.
ReqParity	1	Originator	Request Parity. ReqParity provides even parity over ReqTag, ReqLen, ReqAttr, ReqCmd, ReqMetaData, ReqVC, ReqASI, ReqSrcPhysAccID, ReqDstPhysAccID, ReqPortID, ReqNumBeats, ReqPool. Checked only when ReqVld is asserted.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Name	Size (bits)	Driver	Description
ReqCreditVldParity	1	Completer	Request Credit Valid Parity. Even parity across ReqCreditVld. Checked every cycle.
ReqCreditParity	1	Completer	Request Credit Parity. Even parity across ReqCreditPort0VC, ReqCreditPort1VC, ReqCreditPort2VC, ReqCreditPort3VC, ReqCreditPort0Num, ReqCreditPort1Num, ReqCreditPort2Num, ReqCreditPort3Num, and ReqCreditPool. Checked in beats where ReqCreditVld is non-zero.

An Originator Device shall use the Request Channel to request the transfer of data to or from the system memory space. The amount of data requested shall be no more than 256 bytes and shall not cross a 256-byte boundary. Up to 64 bytes of data can be transferred in one clock cycle, referred to as a Beat. Transfers larger than 64 bytes are transferred in a series of Beats (also referred to as a Burst).

2.7.4.1 ReqASI

The use of the ReqASI (Request Address Space Identifier) field shall be defined by the Accelerators and is implementation dependent. A possible use of the ReqASI field is to identify Requests that are in-node communication vs cross-node communication. This allows the Accelerators, for example, to identify the different sets of Requests and process translation, protection restrictions, maximum address sizes, etc. differently for the different sets of Requests.

2.7.4.2 ReqTag

Each Originating Device may not reuse a ReqTag (Request Tag) until the last Beat of the Response has been received. Request Tags shall be unique per port.

2.7.4.3 ReqSrcPhysAccID

For any request other than a ReadReduce Request issued by an Accelerator (including ReadReduce Requests subsequently issued by a Switch), the ReqSrcPhysAccID field shall contain the Physical Accelerator ID of the Accelerator that originally issued the Request.

For a ReadReduce Request issued by an Accelerator, bits [6:0] (the ReduceOP/DataType field) of this signal shall indicate the Reduction Operator/DataType/Intermediate DataType (if applicable)/Final DataType (if applicable). Incoming data to the reduction shall be in the format specified by DataType. The incoming data shall be processed in the precision specified by DataType if no Intermediate DataType is specified and shall be converted to the Intermediate DataType before reduction if an Intermediate DataType is specified. If an additional Final DataType is specified, the output data shall be written in this format. If no Final DataType is specified, the data shall be down-converted to the original DataType before being returned.

The Reduction operations shall support three different “MIN” and “MAX” functions for the IEEE floating point datatypes FP64, FP32, FP16 and for the BF16 floating point datatype:

- IEEE 754-2008 maxNum(), minNum()
- IEEE 754-2019 maximum(), minimum()
- IEEE 754-2019 maximumNumber(), minimumNumber()

as shown in the following table (Table 2-4) with A < B:

Table 2-3 Min and Max function definitions for IEEE DataTypes

src0	src1	IEEE754-2008 maxNum() IEEE754-2008 minNum()	IEEE754-2019 maximum() IEEE754-2019 minimum()	IEEE754-2019 maximumNumber() IEEE754-2019 minmumNumber()
------	------	--	--	---

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

qNaN-0	qNaN-1	qNaN-0 or qNaN-1	qNaN-0 or qNaN-1	qNaN-0 or qNaN-1
qNaN-0	sNaN-1	qNaN-0 or quieted NaN-1	qNaN-0 or quieted NaN-1	qNaN-0 or quieted NaN-1
qNaN-0	B	B	qNaN-0	B
sNaN-0	qNaN-1	Quieted NaN-0 or qNaN-1	Quieted NaN-0 or qNaN-1	Quieted NaN-0 or qNaN-1
sNaN-0	sNaN-1	Quieted qNaN-0 or quieted qNaN-1	Quieted qNaN-0 or quieted qNaN-1	Quieted qNaN-0 or quieted qNaN-1
sNaN-0	B	Quieted qNaN-0	Quieted qNaN-0	B
A	qNaN-1	A	qNaN-1	A
A	sNaN-1	Quieted qNaN-0	Quieted NaN-1	A
A	B	MAX: B MIN: A	MAX: B MIN: A	MAX: B MIN: A

The “MIN” and “MAX” functions for FP8 Datatypes (E5M2, E4M3) are show in the following table (Table 2-6) with A < B:

Table 2-4 Min and Max Function definitions for FP8 Datatypes (E5M2, E4M3)

src0	src1	FP8maximum() FP8minimum()	FP8maximumNumber() FP8minimumNumber()
NAN-0	NAN-1	NAN-0 or NAN-1	NAN-0 or NAN-1
NAN-0	B	NAN-0	B
A	NAN-1	NAN-1	A
A	B	MAX: B MIN: A	MAX: B MIN: A

The following table, Table 2-5, lists the supported Reduction Operations and DataTypes:

Table 2-5 Reduction Operation/Datatypes

ReduceOP/DataType[6:0]	Reduction/Datatypes
x'00'	Min/UnsignedInt8
x'01'	Min/SignedInt8
x'02'	Min/UnsignedInt16
x'03'	Min/SignedInt16
x'04'	Min/UnsignedInt32
x'05'	Min/SignedInt32
x'06'	Min/UnsignedInt64
x'07'	Min/SignedInt64
x'08'	Max/UnsignedInt8
x'09'	Max/SignedInt8
x'0A'	Max/UnsignedInt16
x'0B'	Max/SignedInt16
x'0C'	Max/UnsignedInt32
x'0D'	Max/SignedInt32
x'0E'	Max/UnsignedInt64
x'0F'	Max/SignedInt64
x'10'	Add/UnsignedInt8, SignedInt8
x'11'	Add/UnsignedInt16, SignedInt16
x'12'	Add/UnsignedInt32, SignedInt32
x'13'	Add/UnsignedInt164, SignedInt64
x'14'	BitwiseAND (All Datatypes)
x'15'	BitwiseOR (All Datatypes)

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

ReduceOP/DataType[6:0]	Reduction/Datatypes
x'16'	BitwiseXOR (All Datatypes)
x'17'-x'2F'	<i>Reserved</i>
x'30'-x'3F'	Reserved for Vendor Defined Commands
x'40'	Add/FP64
x'41'	Add/FP32
x'42'	Add/FP16
x'43'	Add/BF16
x'44'	Add/FP16/FP32/FP16
x'45'	Add/BF16/FP32/BF16
x'46'	Add/E5M2/FP16/E5M2
x'47'	Add/E5M2/BF16/E5M2
x'48'	Add/E5M2/FP32/E5M2
x'49'	Add/E4M3/FP16/E4M3
x'4A'	Add/E4M3/BF16/E4M3
x'4B'	Add/E4M3/FP32/E4M3
x'4C'	Add/E5M2/FP16/FP16
x'4D'	Add/E5M2/BF16/BF16
x'4E'	Add/E5M2/FP32/FP32
x'4F'	Add/E4M3/FP16/FP16
x'50'	Add/E4M3/BF16/BF16
x'51'	Add/E4M3/FP32/FP32
x'52'	Add/FP16/FP32/FP32
x'53'	Add/BF16/FP32/FP32
x'54'	MaxNum/FP64
x'55'	MaxNum/FP32
x'56'	MaxNum/FP16
x'57'	MaxNum/BF16
x'58'	MinNum/FP64
x'59'	MinNum/FP32
x'5A'	MinNum/FP16
x'5B'	MinNum/BF16
x'5C'	maximum/FP64
x'5D'	maximum/FP32
x'5E'	maximum/FP16
x'5F'	maximum/BF16
x'60'	minimum/FP64
x'61'	minimum/FP32
x'62'	minimum/FP16
x'63'	minimum/BF16
x'64'	maximumNumber/FP64
x'65'	maximumNumber/FP32
x'66'	maximumNumber/FP16
x'67'	maximumNumber/BF16
x'68'	minimumNumber/FP64
x'69'	minimumNumber/FP32
x'6A'	minimumNumber/FP16
x'6B'	minimumNumber/BF16
x'6C'	FP8maximum/E5M2
x'6D'	FP8minimum/E5M2
x'6E'	FP8maximum/E4M3
x'6F'	FP8minimum/E4M3
x'70'	FP8maximumNumber/E5M2
x'71'	FP8minimumNumber/E4M3
x'72'	FP8minimumNumber/E5M2
x'73'	FP8minimumNumber/E4M3
x'74'-x'7F'	Reserved for Vendor Defined Commands

For a ReadReduce Request issued by an Accelerator, bits [9:7] of the ReqSrcPhysAccID signal shall indicate the rounding mode for an Add reduction and shall be reserved (and take on a b'000' value)

for any reduction other than an Add. The rounding modes shall be those indicated in the following table :

Table 2-6 UALink Rounding Modes

RoundMode[2:0]	Rounding Mode
x'0'	(RNE) Round to nearest, ties to even.
x'1'	(RNA) Round to nearest, times away from zero
x'2'	(RZ) Round towards zero (truncation)
x'3'	(RU) Round towards +infinity (rounding up or ceiling)
x'4'	(RD) Round towards -infinity (round down or floor)
x'5'	Reserved
x'6'	Reserved
x'7'	(SR) Stochastic Rounding

2.7.4.4 ReqAddr

The ReqAddr (Request Address) field shall provide the byte address of the first doubleword being transferred. The low-order two bits of this field (ReqAddr[1:0]) shall be set to b'00' for all Requests.

2.7.4.5 ReqLen

The ReqLen (Request Length) field shall specify the number of doublewords to be transferred. The transfer size indicated shall be (ReqLen + 1).

For RedReduce Collective Primitive Requests with an Add reduction function that returns data elements of a different size than the input data elements read from the Accelerators in the Group, the ReqLen field shall indicate the number of doublewords to be returned to the Requesting Accelerator. The replicated ReadReduce Requests issued to the destination Accelerators shall read a smaller amount of doublewords consistent with the size of the input datatype's size relative to the output datatype's size.

2.7.4.6 ReqAttr

ReqAttr (Request Attributes) shall provide specific attribute information about the Transaction Request. Encoding shall depend on the type of the Request as shown in the following tables.

Table 2-7 Read, ReqAttr Usage

Bit Range	Sub-field
ReqAttr[7:4]	Byte enables, last DW. Ignored if the last DW is also the first DW.
ReqAttr[3:0]	Byte enables, first DW

Table 2-8 BlockRead, ReqAttr Usage

Bit Range	Sub-field
ReqAttr[7:4]	Byte enables, last DW. Ignored if the last DW is also the first DW. The field shall be set to b'1111'.
ReqAttr[3:0]	Byte enables, first DW. The field shall be set to b'1111'.

BlockRead Requests shall transfer 256 bytes of data and therefore the byte enables for the first and last 4-byte doubleword must all be set.

The ReadReduce Collective Primitive ReqAddr, ReqLen, ReqAttr shall indicate a region of memory whose starting address is aligned to the size of the data elements the ReadReduce returns and whose size is a non-zero integer multiple of the size of the data elements returned (Table 2-9).

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Table 2-9 ReadReduce, ReqAttr Usage

Bit Range	Sub-field
ReqAttr[7:4]	From Source Accelerator to Switch: this field contains the high-order four bits of the stochastic seed value. From the Switch to the Destination Accelerator: this field contains a b'1111' value that the Completer Device in the Destination Accelerator shall interpret as reading all 4-bytes unless the last DW is also the first DW, in which case this field is Ignored.
ReqAttr[3:0]	From Source Accelerator to Switch: this field contains the low-order four bits of the stochastic seed value. Byte enables, first DW. The field shall be set to b'1111'

For a ReadReduce Request, the ReqAttr field from the Source Accelerator to the Switch transfers an 8-bit Stochastic Seed value to the Switch that is used when the rounding mode is Stochastic Rounding. Between the Switch and the destination Accelerator, the ReqAttr field is set to an all 1's value to indicate that the byte enables are active for all the bytes. The Completer Device in the Destination Accelerator shall treat the ReadReduce ReqAttr values in the same manner as the ReqAttr field is treated for a Read Request (the memory accessed by a ReadReduce shall be well-behaved – i.e. all bytes are present – and idempotent – i.e. reads do not have any side-effects and therefore bytes outside those directly being requested can safely be read by the ReadReduce and are returned in an aligned fashion in the data beats in the Read Response).

Table 2-10 Atomic Request ReqAttr Usage

Bit Range	Sub-field
ReqAttr[7:4]	Operation Type (OpType[3:0]).
ReqAttr[3:2]	Operand Size (OpSize).
ReqAttr[1]	<i>Reserved</i>
ReqAttr[0]	Operation Type (OpType[4]).

The Operation Type for the AtomicNR, AtomicR, AtomicNRMulticast Requests shall be encoded in OpType[4:0] (ReqAttr[0] concatenated with ReqAttr[7:4]). The precise semantics for the read-modify-write and whether the read-modify-write uses one or two operations is implementation defined.

The operand size OpSize is encoded in ReqAttr[3:2] field as shown below.

Table 2-11 Atomic OpSize Encoding

OpSize	Operand Size
00b	32 bits
01b	64 bits
10b	16-bits
11b	8-bits

For Writes, the ReqAttr field may be used in an implementation specific manner to convey additional metadata for writes.

For Vendor Defined Commands, the ReqAttr field shall be Vendor Defined.

For UPLI Write Message Requests and Vendor Defined UPLI Write Message Requests, the ReqAttr field shall be defined as shown below in Table 2-13.

2.7.4.7 ReqCmd

The ReqCmd (Request Command) field shall indicate the Command for this Request. When ReqCmd[5] is 1b, the Originator shall transfer data on an Originator Data Channel. For such Commands, in addition to obtaining a Credit from the Request Channel flow control to issue the Request, one or more Credits shall be obtained for the Originator Data Channel.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification Rev 2.0

The legal Command types and legal encoding for the ReqCmd field of the Request Channel is shown in Table 2-12, below.

The column labeled “Data Channels” lists the Data Channel or Channels utilized by the command and for what purpose. The Resp Chan column indicates whether the Response to the command is returned to the Originator on a Read Response/Data Channel or a Write Response Channel.

ReqCmd [5] = 1 indicates that the command will transfer data on an Originator Data Channel.

- ReqCmd [5:4] = 00 indicates Read type Requests (i.e. Requests that do not have data or byte masks with the Request).
- ReqCmd [5:4] = 10 indicates Write type Requests (i.e. Requests that provide data and possibly byte masks with the Request).
- ReqCmd [5:4] = 11 indicates Atomic type Requests (i.e. Requests that provide Operand Data and byte masks with the Request).

Table 2-12 Commands

UPLI Command	ReqCmd	Data Channels	Resp Chan	Description
<i>Reserved</i>	00 0000b/00h - 00-0010b/02h			
Read	00 0011b/03h	Rd Rsp/Data (read data)	Read	Read. I/O Coherent read, up to 256 bytes (4 beats).
ReadReduce	00_0100b/04h	Read Rsp/Data (read data)	Read	Collective Primitive Read, up to 256 bytes (4 beats). Issued by both the Source Accelerator and the Switch.
BlockRead	00 0101b/05h	Read Rsp/Data (read data)	Read	BlockRead. Issued by Switch to perform a memory read on behalf of a Block Collective.
<i>Reserved</i>	00 0110b/06h - 00 0111b/07h			
Reserved for Vendor Defined Commands	00 1000b/08h - 00 1111b/0Fh			Command encodings reserved for Read class Vendor Defined Commands.
<i>Reserved</i>	01 0000b/10h - 10-0010b/22h			
BlockCollectiveInvoke	10-0000b/20h	Orig Data (write data)	Write	Block Collective Invoke request issued by an Accelerator to initiate a Block Collective.
BlockCollectiveAllocate	10-0001b/21h	Orig Data (write data)	Write	Block Collective Allocation request issued by an Accelerator to allocate entries in the Switch Block Collective Control Block Queue.
BlockCollectiveDeallocate	10-0010b/22h	Orig Data (write data)	Write	Block Collective Deallocation request issued by an Accelerator to deallocate entries in the Switch Block Collective Control Block Queue.
BlockWriteFull	10 0011b/23h	Orig Data (write data)	Write	BlockWriteFull issued by Switch to perform writes to memory on behalf of a Block Collective.
<i>Reserved</i>	10 0100b/24h - 10 0101b/25h			
WriteMulticast	10 0110b/26h	Orig Data (write data)	Write	Collective Primitive Write, I/O Coherent write, up to 256 byte (4 beats). Issued both by Source Accelerator and the Switch.
WriteFullMulticast	10 0111b/27h	Orig Data (write data)	Write	Collective Primitive Write, I/O Coherent store of one or more full (i.e. all Byte Enables for all Beats are asserted) Data Beats. Issued both by Source Accelerator and the Switch
Write	10 1000b/28h	Orig Data (write data)	Write	Write. I/O Coherent write, up to 256 bytes (4 beats).

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

WriteFull	10 1001b/29h	Orig Data (write data)	Write	WriteFull. I/O Coherent store of one or more full (i.e. all Byte Enables for all Beats are asserted) Data Beats.
UPLI Write Message	10 1010b/2Ah	Orig Data (write data)	Read or Write	UPLI Write Message, issues a UPLI message from the Source Accelerator Originator to the Destination Accelerator Completer. The type of the UPLI message is encoded in the ReqMetaData field. Depending on the type of UPLI message, the Response is returned on the Read or Write Response Channel.
<i>Reserved</i>	10 1011b/2Bh - 10-1011b/2Bh			
Reserved for Vendor Defined Commands	10-1100b/2Ch - 10-1111b/2Fh			Command encodings reserved for Write class Vendor Defined Commands.
AtomicR	11 0000b/30h	Rd Rsp/Data (returned read data) OrigData (atomic operands)	Read	Atomic Return Data. I/O Coherent atomic read-modify-write with data returned.
<i>Reserved</i>	11-0001b/31h			
AtomicNR	11 0010b/32h	OrigData (atomic operands)	Write	Atomic No-Return Data. I/O Coherent atomic read-modify-write with no data returned.
AtomicNRMulticast	11 0011b/33h	OrigData (atomic operands)	Write	Collective Primitive Atomic No-Return Data. I/O Coherent atomic read-modify-write with no data returned. Issued both by Source Accelerator and the Switch.
<i>Reserved</i>	11-0100b/34h - 11-0110b/3Bh			
Reserved for Vendor Defined Commands.	11-1100b/3Ch - 11-1111b/3Fh			Command encodings reserved for Atomic classVendor Defined Commands.

Reads

The Read Commands shall be the following:

- Read
- ReadReduce
- BlockRead

Originators shall perform Read Requests to read data. Read Requests shall be up to 256 bytes in size and shall not cross a 256-byte boundary. Read Requests shall be considered complete on UPLI when the read Response is returned.

Originators shall perform a ReadReduce Request to read data in response to a Collective Primitive. ReadReduce Requests shall be up to 256 bytes in size and shall not cross a 256-byte boundary. The Switch shall replicate the read to each Rank in the Collective Primitive and shall reduce the data from the Ranks according to a reduction function to produce a single Read Reponse. ReadReduce Requests shall be considered complete on UPLI when the Read Response for the ReadReduce is returned.

Originators shall perform a BlockRead Request to read data in response to a Block Collective. Block Read Requests shall be 256 bytes in size and shall be aligned on 256-byte boundares (so as not to cross a 256-byte boundary). BlockRead Requests shall be considered complete on UPLI when the Read Response for the Block Read is returned.

A Read Request, ReadReduce Request, and a Block Read Request shall participate in the local coherence protocol in the Destination Accelerator and shall return a coherent value for the read from that Accelerator.

Writes

The Write Commands shall be the following:

- Write
- WriteFull
- BlockCollectiveAllocate (issued as a write class command)
- BlockCollectiveDeallocate (issued as a write class command)
- BlockCollectiveInvoke (issued as a write class command)
- WriteMulticast
- WriteFullMulticast
- BlockWriteFull

Originators shall perform Writes to write data to system memory on the destination Accelerator. Data shall be sent on the Originator Data Channel with valid bytes indicated by the OrigDataByteEn field. Write Requests shall be up to 256 bytes (4 OrigData beats) and shall not cross a 256-byte boundary.

WriteFull Requests shall be Writes that are restricted to being 64, 128, 192, or 256 bytes in length and shall have all Byte Enables active for all the bytes referenced by the WriteFull Request (this allows the Transaction Layer to not transmit the Byte Enables for the write and have the TL at the Destination Accelerator reconstruct the Byte Enables saving link bandwidth).

Write Requests and WriteFull Requests shall participate in the local coherence protocol in the Destination Accelerator and shall update memory and those caches, if any, necessary to perform a coherent update at the Destination Accelerator.

The BlockCollective Allocate, Deallocate, Invoke Requests are used to manage and invoke Block Collectives.

The WriteMulticast and WriteFullMulticast shall be processed in the same way as a Write and a WriteFull, respectively. The BlockWriteFull shall be processed in the same manner as a WriteFull Request. All BlockWriteFull Requests shall be 256 bytes in length with all Byte Enables set. The ReqDstPhysAccID field for a BlockWriteFull either from the Accelerator to the Switch or from the Switch to the Accelerator shall contain the GroupID of the Block Collective.

Atomics

The Atomics Commands are the following:

- AtomicR
- AtomicNR
- AtomicNRMulticast

Originators shall issue Atomic Commands to perform atomic (at the Completer) read-modify-write operations. The AtomicR command shall performs an atomic read-modify-write operation and shall return the initial value read from memory. The AtomicNR Command performs an atomic read-modify-write operation in memory and shall not return the initial value read from memory. The exact semantics of how memory is modified and whether the Atomic command requires one or two operands is implementation specific and not defined in this specification.

An Atomic Request shall participate in the local coherence protocol in the Destination Accelerator and shall update memory and whatever caches necessary to perform a coherent update at the Destination Accelerator. If the Atomic is an AtomicR, a coherent value for the location(s) before the update shall be returned.

AtomicNRMulticast requests shall be processed in the same manner as an AtomicNR.

UPLI Write Messages

The UPLI Write Messages shall consist of the following command:

- UPLI Write Message

The UPLI Write Message shall be a message from the UPLI Originator in the Source Accelerator to the UPLI Completer in the Destination Accelerator. This message shall be conveyed as a Request with semantics similar to an Atomic. The UPLI Write Message shall convey a one to four 64-byte Data Beat payload on the OrigData Channel that may contain no content, the ReqMetaData field shall be used to signify which specific UPLI Write Message type the message is (up to 256 choices). For each UPLI Write Message the required Response for the UPLI Write Message shall be conveyed on either the Read Response/Data Channel or the Write Response Channel based on the UPLI Write Message type (similar to how AtomicR and AtomicNR requests are handled). If the Response to the Write UPLI Message is conveyed on the Read Response/Data Channel, the number of 64-byte Data Beats in the Response is specific to the type of the UPLI Write Message, may vary from a minimum of one Beat to four Beats, and does not have to match the number of 64-byte Data Beats transferred with the UPLI Write Message Request. A set of the values of the ReqMetaData field (0xF0-0xFF) for a UPLI Write Message Request specify Vendor Defined UPLI Write Message types.

The semantic effect(s), if any, in the Source Accelerator or Destination Accelerator due the UPLI Write Message or Vendor Defined UPLI Write Message and/or the associated Response are specific to the particular type of the UPLI Write Message or Vendor Defined UPLI Write Message.

A UPLI Write Message Request shall not cross a 256-byte boundary.

Vendor Defined Commands

The UPLI Interface defines three different classes of Vendor Defined Commands (VDCs): Read class VDCs (ReqCMD = 0x08h-0x0Fh), Write class VDCs (ReqCMD = 0x2Ch-0x2Fh), and Atomic class VDCs (ReqCMD = 0x3Ch-0x3Fh). For each Vendor Defined Command, the following UPLI Request Interface signals shall be Vendor defined: ReqASI, ReqAddr, ReqLen, ReqAttr, and ReqMetaData. The remaining signals in the UPLI Request Interface shall retain the same functionality as for a non-VDC.

The number of UPLI Data Beats transferred on the OrigData channel for a Write class or for an Atomic Class Vendor Defined Request shall be controlled solely by the value of the ReqNumBeats signal in the Vendor Defined Request.

For a Read class Vendor Defined Command or for an Atomic class Vendor Defined Command that returns data, the number of Beats of data returned in the associated Read Response shall be an implementation specific function of signals (possibly including ReqCMD and ReqNumBeats) in the Vendor Defined Command. The number of Data Beats returned in the associated Read Response shall be controlled solely by the RdRspNumBeats signal.

The layout and contents of the Data Beats for Vendor Defined Commands are vendor defined.

Any VDC that accesses memory shall be issued on the same port on the Accelerator that the non-VDCs that have an address within the same 256-byte memory region accessed by the VDC.

No VDC shall access and/or alter more than one 256-byte region of memory.

Read class VDCs shall not issue Data Beat(s) on the OrigData channel and the Response for a Read class VDR shall be returned on the Read Response channel.

Write class VDCs shall issue Data Beat(s) (with associated Byte Enables) on the OrigData Channel and the Response for a Write class VDC shall be returned on the Write Response channel.

Atomic class VDCs shall issue Data Beat(s) on the OrigData Channel and the Response for an Atomic VDC shall be returned on either the Read Response Channel or the Write Response Channel.

2.7.4.8 ReqMetaData

For Requests that are not UPLI Write Message Requests, the ReqMetaData field shall convey implementation defined control information.

For a UPLI Write Message Request, the ReqMetadata field shall indicate the particular type (up to 256) of the UPLI Write Message. The following table indicates the legal UPLI Write Message Request types, how the various fields that may be Reserved or use for the UPLI Write Message and defined for each type, and the number of 64-byte OrigData Beats are sent with the Request:

Table 2-13 UPLI Write Message Request Types

ReqMetaData value	Type	ReqAddr	ReqASI	ReqAttr	# Write Beats or # Read Beats/ Content	Response Channel/ # Beats if Read
0x00	NOP	Reserved	Reserved	Reserved	1 / Null (all zeros)	Write
0x01	KeyRollMsg.ReqChannel	Reserved	Reserved	KeyInfo	1 / Null (all zeros)	Write
0x02	KeyRollMsg.RdRspChannel	Reserved	Reserved	KeyInfo	1 / Null (all zeros)	Read
0x03	KeyRollMsg.WrRspChannel	Reserved	Reserved	KeyInfo	1 / Null (all zeros)	Write
0x04-0xEF	Reserved					
0xF0-0xFF	Vendor Defined UPLI Write Messages	Vendor Defined	Vendor Defined	Vendor Defined	Vendor Defined	Vendor Defined

To ease compatibility issues with future version of the UALink specification, implementations should consume bits of ReqMetaData from the low order bit and leave as many high-order bits of ReqMetaData unused as possible (this will allow future version of the UALink specification to more easily reclaim bits of ReqMetaData for new predefined uses if necessary).

The key rolling message shall contain a KeyInfo field in ReqAttr as defined below:

Table 2-14 KeyInfo Definition

Bit Range	Sub-Field
ReqAttr[0]	Swap Hint
ReqAttr[1]	INC KeyRoll
ReqAttr[7:2]	Reserved

2.7.5 Read Response/Data Channel

The Read Response/Data Channel shall provide the Read Response and Read Response Data for a specific Read Request.

Data shall be returned in the RdRspData field in one or more Beats. A Response shall be matched to the Request that evoked it based on the information in the RdRspTag fields. For a Read Response to correspond to a Read Request the RdRspTag shall match the ReqTag of the Request that initiated the Data Transfer.

The column labeled Driver indicates whether the Originator or the Completer drives the signal.

Table 2-15 Read Response/Data Channel Signals

Name	Size	Driver	Description
Read Response and Data Channel Information and Control signals These signals are Time Division Multiplexed (TDMed) according to the RdRspPortID value.			
RdRspVld	1	Completer	Read Response Valid. This signal indicates that the Completer is presenting valid information (a Data Beat) on this Channel.
RdRspPortID	2	Completer	Read Response/Data Port ID. Where the Completer drives the Read Response/Data Channel to a TL, indicates the Port associated with that TL that the Read Response/Data Channel Beat is to be presented on. For all Completers, RdRspPortID indicates the TDM cycle for the Read Response/Data Channel.
RdRspAuthTag	64	Completer	Read Response Authorization Tag. Authorization Tag for the Read Response. See the Chapter on Security for more details about this signal and Authorization. When Authorization is not active for the Response or RdRspVld is de-asserted, this field shall be driven to zero.
RdRspSrcPhysAccID	10	Completer	Read Response Source Physical Accelerator ID. The source Accelerator ID of the Read Response. This field should contain the value of ReqDstPhysAccID for the Request that caused this Read Response (the Destination Accelerator of the original Request is the source Accelerator of the Read Response). This information is not required for functionality but is useful for debug (the Read Response is routed using the RdRspDstPhysAccID field). Carrying a correct value in this field is optional (i.e. UALink TL or UALink DL may compress this field out). When the field does not contain an accurate value, the field should be driven to zero. Because this field may be inaccurate, it may not be used for a functional purpose.
RdRspDstPhysAccID	10	Completer	Read Response Destination Physical Accelerator ID. The Destination Accelerator ID of the Read Response. This field shall contain the value of ReqSrcPhysAccID for the Request that caused this Read Response (the Source Accelerator of the original Request is the Destination Accelerator of the Read Response). This field is used to route the Read Response back to the Accelerator that originally sourced the Request.
RdRspTag	11	Completer	Read Response Transaction Tag. This field contains the value of ReqTag field from the initial Request.
RdRspNumBeats	2	Completer	Read Response Number of Data Beats. Indicates the number of Beats for this Read Response. The number of beats transferred is (RdRspNumBeats+1). This signal is intended to allow Switches to gather multiple Beats together more easily into a larger entity for routing and relieve Switches of the need to compute the number of Beats being transferred. Switches shall rely on this field to determine the number of beats being transferred for Read Responses.
RdRspData	512	Completer	Read Response Data. If the Read was successful, this field carries the requested data. The transfer of the requested data may require more than one Data Beat.
RdRspStatus	4	Completer	Read Response status. This field indicates the status of the Read Request. RdRspStatus is required to be the same across all beats of a Response.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Size	Driver	Description
RdRspOffset	2	Completer	Read Response Offset. Indicates the index of the current Data Beat. Data to be returned to the Originator is transferred from the Completer in the RdRspData field in one or more Data Beats. Each Beat is sequentially assigned a number starting at 0 and ending with n-1, based on the order of the data in memory; n is the total number of Beats required to transfer the data to be returned. When the transfer requires more than one Beat and the transfer is in Multi-Beat Mode, the Completer must begin with RdRspOffset 0 and continue in Beat order. In single-beat mode, the Beats may come in any order. This field provides the number of the Beat being presented on this transfer cycle.
RdRspLast	1	Completer	Read Response Last. This signal is asserted with the last Response Data Beat of a data transfer.
RdRspDataError	1	Completer	Read Response Data Error. This field is sent with the same timing as RdRspData and indicates the status (error – or “poisoned” – or not) of the Beat. This field shall be ignored when RdRspVld is not asserted. This field may have differing values in the different beats unlike RdRspStatus and this field shall not be used to indicate errors other than parity issues with Data or Byte Enable fields. As one example, Data Response Beats with a RdRspStatus of DECODE_ERROR should provide beats of a fixed data pattern – say all 0xF’s – and have RdRspDataError = 0 for these Beats unless a parity error occurred during transmission. The DECODE_ERROR is indicated solely by the RdRspStatus field.
RdRspTypeInfo	2	Completer	Read Response Type Info. This signal shall indicate the type of the Request this Response corresponds to. The value b’00’ shall indicate a unicast Request (A Request other than a ReadReduce or BlockRead). The value of b’01’ shall be Reserved. The value of b’10’ shall indicate a ReadReduce Command. The value of b’11’ shall indicate a BlockRead Command.
RdRspVC	2	Completer	Read Response Virtual Channel. Specifies the Virtual Channel of the read Response. Matches the Virtual Channel of the originating Request (ReqVC). In UALink 1.0, this field is informational only to the Switch with the exception that this value is returned to the Sender in the Credit Return signals. The Switch has one virtual channel and passes this field through to the source Accelerator unchanged.
RdRspPool	1	Completer	Read Response Pool. Specifies whether the Read Response was issued using a Pool Credit (RdRspPool = 1) or a Virtual Channel Credit (RdRspPool = 0).
Read Response and Data Channel Credit Return signals These signals are not Time Division Multiplexed (TDMed). A Credit or Credits for any port may be returned on any cycle.			
RdRspCreditInitDone	4	Originator	Read Response Credit Initialization Done. When Asserted, no earlier than the cycle after the last Credit is released, indicates to Completer that the initial Credit release for the port associated with the individual signal is complete (RdRspCreditInitDone[2] indicates port 2 is complete). The individual “done” signals can be asserted independently and once asserted remain asserted until the interface is reset or powered off. The Completer shall monitor the RdReqCreditInitDone signal until that signal has been asserted an implementation specific number of cycles greater than 1. Once that has occurred, the Completer shall consider the initial issuance of the Credits to be complete and shall no longer monitor the signal.
RdRspCreditVld	4	Originator	Read Response Credit Valid. Indicates a Credit or Credits is being released to the Completer. Credits are per port and a Station may have 1, 2 or 4 ports. RdRspCreditVld[n] corresponds to Port[n]. Credits can be released on more than one port simultaneously.
RdRspCreditPool	4	Originator	Read Response Credit Pool. Indicates the type of Credit indicated on RdRspPool for the Read Response that consumed the Pool or Virtual Channel Credit or Credits being released. If ‘0’, the Credit or Credits being released is a Virtual Channel Credit and if ‘1’, the Credit or Credits released is a Pool Credit. The RdRspVC value for the Write Response corresponding to the returned Credit or Credits is placed on the appropriate RdRspCreditPort[0,1,2,3]VC signal.
RdRspCreditPort0VC	2	Originator	Read Response Credit Port 0 Virtual Channel. Indicates the Virtual Channel indicated on RdRspVC for the Read Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port0. Valid when RdRspCreditVld[0] is asserted.
RdRspCreditPort1VC	2	Originator	Read Response Credit Port 1 Virtual Channel. Indicates the Virtual Channel indicated on RdRspVC for the Read Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port1. Valid when RdRspCreditVld[1] is asserted.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Size	Driver	Description
RdRspCreditPort2VC	2	Originator	Read Response Credit Port 2 Virtual Channel. Indicates the Virtual Channel indicated on RdRspVC for the Read Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port2. Valid when RdRspCreditVld[2] is asserted.
RdRspCreditPort3VC	2	Originator	Read Response Credit Port 3 Virtual Channel. Indicates the Virtual Channel indicated on RdRspVC for the Read Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port3. Valid when RdRspCreditVld[3] is asserted.
RdRspCreditPort0Num	2	Originator	Read Response Credit Port 0 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, RdRspCreditPort0Num+1). Only valid when RdRspCreditVld[0] is asserted.
RdRspCreditPort1Num	2	Originator	Read Response Credit Port 1 Number of Credits. Indicates the number of Credits being returned for Port1 (up to 4, RdRspCreditPort1Num+1). Only valid when RdRspCreditVld[1] is asserted.
RdRspCreditPort2Num	2	Originator	Read Response Credit Port 2 Number of Credits. Indicates the number of Credits being returned for Port2 (up to 4, RdRspCreditPort2Num+1). Only valid when RdRspCreditVld[2] is asserted.
RdRspCreditPort3Num	2	Originator	Read Response Credit Port 3 Number of Credits. Indicates the number of Credits being returned for Port3 (up to 4, RdRspCreditPort3Num+1). Only valid when RdRspCreditVld[3] is asserted.
Read Response and Data Channel Parity signals			
RdRspVldParity	1	Completer	Read Response Valid Parity. Even parity across RdRspVld. Checked every cycle.
RdRspAuthTagParity	1	Originator	Read Response Authorization Tag Parity. Even Parity across RdRspAuthTag. Checked only when RdRspVld is asserted.
RdRspDataParity	8	Completer	Read Response Data Parity. Each bit in RdRspDataParity provides even parity based error protection for 64 bits of the RdRspData field. The number of set bits in RdRspDataParity[i] and RdRspData[((i+1)*64-1):(i*64)] is always even. Even parity must be provided by the completer, including cases when read data was masked due to the length or the address of the command, and beats where RdRspDataError is asserted. Checked only when RdRspVld is asserted. This field is sent with the same timing as RdRspData.
RdRspParity	1	Completer	Read Response Parity. RdRspParity provides even parity over RdRspTypeInfo, RdRspTag, RdRspStatus, RdRspOffset, RdRspLast, RdRspNumBeats, RdRspVC, RdRspSrcPhysAccID, RdRspDstPhysAccID, RdRspPortID, RdRspDataError, RdRspPool. Checked only when RdRspVld is asserted.
RdRspCreditVldParity	1	Originator	Read Response Credit Valid Parity. Even parity across RdRspCreditVld. Checked every cycle.
RdRspCreditParity	1	Originator	Read Response Credit Parity. Even parity across RdRspCreditPort0VC, RdRspCreditPort1VC, RdRspCreditPort2VC, RdRspCreditPort3VC, RdRspCreditPort0Num, RdRspCreditPort1Num, RdRspCreditPort2Num, RdRspCreditPort3Num, and RdRspCreditPool. Checked in beats where RdRspCreditVld is non-zero.

2.7.5.1 RdRspStatus

The four-bit RdRspStatus field shall provide a success/fail indication of the read Response. The encoding of RdRspStatus[3:0] field shall be encoded as shown in Table 2-16 and Table 2-17. The Switch shall deliver the RdRspStatus to the Initial UPLI Originator of the Request.

Table 2-16 RdRspStatus [3:0] Encoding for Predefined Commands

RdRspStatus[3:0]	Response	Comment
0000b	OKAY (Normal completion)	The transaction completed normally.
0001b	<i>Reserved</i>	
0010b	TARGET ABORT	Indicates that the end-target of the transaction had an error while handling the transaction or is otherwise unable to complete the transaction.
0011b	DECODE ERROR	Address Decode Error
0100b-0101b	<i>Reserved</i>	
0110b	PROTECTION VIOLATION	Protection violation. Indicates that certain security or protection checks caused the transaction to be aborted. This error is normally persistent.
0111b	<i>Reserved</i>	

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

RdRspStatus[3:0]	Response	Comment
1000b	CMPTO (Completion Timeout)	The Completer in the destination Accelerator was unable to complete the Request and timed out.
1001b-1101b	<i>Reserved</i>	
1110b	INC-MixedResponses	The Read Responses for the Block Reads issued by the Switch to the Accelerators in the Group were not all of the same type. This Response is issued only by the Switch.
1111b	<i>Reserved</i>	

Table 2-17 RdRspStatus[3:0] Encoding for Vendor Defined Command

RdRspStatus[3:0]	Response	Comment
0000b	OKAY (Normal completion)	The transaction completed normally.
0001b-1111b	Vendor Defined	

If RdRspStatus shall indicate an error (TARGET ABORT, DECODE ERROR, PROTECTION VIOLATION, CMPTO), the Completer shall transfer all requested data beats including the assertion of RdRspLast, using Manufactured Data if there is no data available due to the error. A Manufactured Data pattern of all ones (0xFF) is recommended.

If the Data for a Read Response would otherwise be cached in some cache in the Accelerator, this shall be prevented for Read Responses with RdRspStatus indicating an error. The Originator shall not cache the returned Data. In general, any transaction that completes with RdRspStatus indicating an error shall not alter the cache state of the requested cache line(s) in the Accelerator that is the Source Accelerator for the Request associated with this Response.

For UPLI Write Message Requests that utilize a Read Response but for which the ReqAddr field is either defined as Reserved or used to convey information other than an address, the DECODE ERROR and PROTECTION VIOLATION Response Status values shall not be legal.

2.7.5.2 RdRspDataError

The RdRspDataError (Read Response Data Error) signal is a data “poison” indicator that shall be asserted at any UALink Protocol Level Interface on a Switch or on an Accelerator that detects a parity error on the RdRsp Data Beat, or by the Switch Core detecting an error on Read Response Data (data only, not control signals) according to the Switch soft error detection scheme. This signal is asserted individually for each Beat according to the error state of that Beat.

2.7.5.3 RdRspOffset

The RdRspOffset (Read Response Offset) shall be used to indicate the position of the current Beat in the Read Data being Returned to the originator in the RdRspData field. Each Beat required for a Response shall be sequentially assigned a number starting at 0 and ending with 3 in ascending address order or as defined by a Vendor Defined Command.

Beat 0 (RdRspOffset = 0) shall always contain the initial (lowest addressed) byte of the requested data payload for non-Vendor Defined Commands. Beats 1-3 (if present) shall provide the data in ascending address order for non-Vendor Defined Commands. For Vendor Defined Commands the ordering of the Data Beats shall be Vendor Defined.

2.7.6 Write Response Channel

The Write Response Channel shall provide feedback to the Initial Originator of a Write Request indicating the outcome and completion of the Request at the Final Completer. Responses are matched with Requests based on the Request tag of the Request. Table 2-18 lists the signals that make up a Write Response Channel.

The column labeled **Driver** indicates whether the Originator or the Completer drives the signal.

Table 2-18 Write Response Channel Signals

Name	Size (bits)	Driver	Description
Write Response Channel Information and Control signals These signals are Time Division Multiplexed (TDMed) according to the WrRspPortID value.			
WrRspVld	1	Completer	Write Response Valid. This signal indicates that the Completer is presenting valid information (a Beat) on this channel.
WrRspPortID	2	Completer	Write Response Port ID. Where the Completer drives the Write Response Channel to a TL, indicates the Port associated with that TL that the Write Response Channel Beat is to be presented on. For all Completers, WrRspPortID indicates the TDM cycle for the Write Response Channel.
WrRspAuthTag	64	Completer	Write Response Authorization Tag. If the WrRspStatus indicates a value other than ISOLATE, this field shall contain the Authorization Tag for the Write Response. See the Chapter on Security for more details about this signal and Authorization. When Authorization is not active for the Response or WrRspVld is de-asserted, this field is driven to zero. If the WrRspStatus field indicates ISOLATE, this field is a don't care and shall be ignored by the Completer and no Authentication failure shall be generated.
WrRspSrcPhysAccID	10	Completer	Write Response Source Physical Accelerator ID. The source Accelerator ID of the Write Response. If the WrRspStatus indicates a value other than ISOLATE, this field shall contain the value of ReqDstPhysAccID for the Request that caused this Write Response (the destination Accelerator of the original Request is the Source Accelerator of the Write Response). This information is not required for functionality but is useful for debug (the Write Response is routed using the WrRspDstPhysAccID). Carrying a correct value in this field is optional (i.e. UALink TL or UALink DL may compress this field out). When the field does not contain an accurate value, the field should be driven to zero. Because this field may be inaccurate, it may not be used for a functional purpose. If the WrRspStatus field indicates ISOLATE, this field is a don't care and shall be ignored by the Completer.
WrRspDstPhysAccID	10	Completer	Write Response Destination Physical Accelerator ID. The destination Accelerator ID of the Write Response. If the WrRspStatus indicates a value other than ISOLATE, this field shall contain the value of ReqSrcPhysAccID for the Request that caused this Write Response (the Source Accelerator of the original Request is the destination Accelerator of the Write Response). This field is used to route the Write Response back to the Accelerator that originally sourced the Request. If the WrRspStatus field indicates ISOLATE, this field is a don't care and shall be ignored by the Completer.
WrRspTag	11	Completer	Write Response Transaction Tag. If the WrRspStatus indicates a value other than ISOLATE, this field shall contain the value of the ReqTag field from the Request that caused this Write Response. If the WrRspStatus field indicates ISOLATE, this field is a don't care and shall be ignored by the Completer.
WrRspStatus	4	Completer	Write Response status. This field shall indicate the status of the Write Request.
WrRspTypeInfo	2	Completer	Write Response Type Info. This signal shall indicate the type of the Request this Response corresponds to. The value of b'00' shall indicate a unicast Request (A Request other than a WriteMulticast, WriteFullMulticast, AtomicNRMulticast, or BlockWriteFull). The value of b'01' shall be Reserved. The value of b'10' shall indicate a

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Name	Size (bits)	Driver	Description
			WriteMulticast, WriteFullMulticast, or AtomicNRMulticast Request. The value of b'11' shall indicate a BlockWriteFull Request.
WrRspVC	2	Completer	Write Response Virtual Channel. If the WrRspStatus indicates a value other than ISOLATE, this field shall specify the virtual channel of the Write Response. This signal shall match the virtual channel of the originating Request (ReqVC). In UALink 1.0, this field is informational only to the Switch with the exception that this value is returned to the Sender in the Credit Return signals. The Switch has one virtual channel and passes this field through to the source Accelerator unchanged. If the WrRspStatus field indicates ISOLATE, this field shall indicate the Virtual Channel Credit used to issue the command if WrRspPool=0.
WrRspPool	1	Completer	Write Response Pool. Specifies whether the Write Response was issued using a Pool Credit (WrRspPool = 1) or a Virtual Channel Credit (WrRspPool = 0).
Write Response Channel Credit Return signals These signals are not Time Division Multiplexed (TDMed). A Credit or Credits for any port may be returned on any cycle.			
WrRspCreditInitDone	4	Originator	Write Response Credit Initialization Done. When Asserted, no earlier than the cycle after the last Credit is released, indicates to Completer that the initial Credit release for the port associated with the individual signal is complete (WrRspCreditInitDone[2] indicates port 2 is complete). The individual "done" signals can be asserted independently and once asserted remain asserted until the interface is reset or powered off. The Completer shall monitor the WrReqCreditInitDone signal until that signal has been asserted an implementation specific number of cycles greater than 1. Once that has occurred, the Completer shall consider the initial issuance of the Credits to be complete and shall no longer monitor the signal.
WrRspCreditVld	4	Originator	Write Response Credit Valid. Indicates a Credit or Credits is being released to the Completer. Credits are per port and a Station may have 1, 2 or 4 ports. WrRspCreditVld[n] corresponds to Port[n]. Credits can be released on more than one port simultaneously.
WrRspCreditPool	4	Originator	Write Response Credit Pool. Indicates the type of Credit indicated on WrRspPool for the Write Response that consumed the Pool or Virtual Channel Credit or Credits being released. If '0', the Credit or Credits being released is a Virtual Channel Credit and if '1', the Credit or Credits released is a Pool Credit. The WrRspVC value for the Write Response corresponding to the returned Credit or Credits is placed on the appropriate WrRspCreditPort[0,1,2,3]VC signal.
WrRspCreditPort0VC	2	Originator	Write Response Credit Port 0 Virtual Channel. Indicates the Virtual Channel indicated on WrRspVC for the Write Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port0. Valid when RdRspCreditVld[0] is asserted.
WrRspCreditPort1VC	2	Originator	Write Response Credit Port 1 Virtual Channel. Indicates the Virtual Channel indicated on WrRspVC for the Write Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port1. Valid when WrRspCreditVld[1] is asserted.
WrRspCreditPort2VC	2	Originator	Write Response Credit Port 2 Virtual Channel. Indicates the Virtual Channel indicated on WrRspVC for the Write Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port2. Valid when WrRspCreditVld[2] is asserted.
WrRspCreditPort3VC	2	Originator	Write Response Credit Port 3 Virtual Channel. Indicates the Virtual Channel indicated on WrRspVC for the Write Response that consumed the Pool or Virtual Channel Credit or Credits being released for Port3. Valid when WrRspCreditVld[3] is asserted.
WrRspCreditPort0Num	2	Originator	Write Response Credit Port 0 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, WrRspCreditPort0Num+1). Only valid when WrRspCreditVld[0] is asserted.
WrRspCreditPort1Num	2	Originator	Write Response Credit Port 1 Number of Credits. Indicates the number of Credits being returned for Port1 (up to 4, WrRspCreditPort1Num+1). Only valid when WrRspCreditVld[1] is asserted.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Name	Size (bits)	Driver	Description
WrRspCreditPort2Num	2	Originator	Write Response Credit Port 2 Number of Credits. Indicates the number of Credits being returned for Port2 (up to 4, WrRspCreditPort2Num+1). Only valid when WrRspCreditVld[2] is asserted.
WrRspCreditPort3Num	2	Originator	Write Response Credit Port 3 Number of Credits. Indicates the number of credits being returned for Port3 (up to 4, WrRspCreditPort3Num+1). Only valid when WrRspCreditVld[3] is asserted.
Write Response Channel Parity signals			
WrRspVldParity	1	Completer	Write Response Valid Parity. Even parity across WrRspVld. Checked every cycle.
WrRspAuthTagParity	1	Originator	Write Response Authorization Tag Parity. Even Parity across WrRspAuthTag. Checked only when WrRspVld is asserted.
WrRspParity	1	Completer	Write Response Parity. WrRspParity provides even parity over WrRspTypeInfo, WrRspTag, WrRspStatus, WrRspSrcPhysAccID, WrRspDstPhysAccID, WrRspPortID, WrRspVC, WrRspPool. Checked only when WrRspVld is asserted.
WrRspCreditVldParity	1	Originator	Write Response Credit Valid Parity. Even parity across WrRspCreditVld. Checked every cycle.4
WrRspCreditParity	1	Originator	Write Response Credit Parity. Even parity across WrRspCreditPort0VC, WrRspCreditPort1VC, WrRspCreditPort2VC, WrRspCreditPort3VC, WrRspCreditPort0Num, WrRspCreditPort1Num, WrRspCreditPort2Num, WrRspCreditPort3Num, and WrRspCreditPool. Checked in beats where WrRspCreditVld is non-zero.

2.7.6.1 WrRspStatus

Table 2-19 WrRspStatus[3:0] Encoding for Predefined Commands

WrRspStatus[3:0]	Response	Comment
0000b	OKAY (Normal Completion)	The transaction completed normally
0001b	<i>Reserved</i>	
0010b	TARGET ABORT	Indicates that the end-target of the transaction had an error while handling the transaction or is otherwise unable to complete the transaction.
0011b	DECODE ERROR	Address Decode Error
0100b-0101b	<i>Reserved</i>	
0110b	PROTECTION VIOLATION	Protection Violation. Indicates that certain security or protection checks caused the transaction to be aborted. This error is normally persistent.
0111b	<i>Reserved</i>	
1000b	CMPTO (Completion Timeout)	The Completer in the destination Accelerator was unable to complete the Request and timed out.
1001b-1101b	<i>Reserved</i>	
1110b	INC-MixedResponses	The Write Responses for the Block Writes issued by the Switch to the Accelerators in the Group were not all of the same type. This Response is issued only by the Switch.
1111b	ISOLATE	Write Response from UALink TL to an Accelerator UPLI Originator to place the UPLI Originator into Isolation Mode. Valid only for UALink TL to UPLI Completer Write Response channel on an Accelerator.

Table 2-20 WrRspStatus[3:0] Encoding for Vendor Defined Commands

WrRspStatus[3:0]	Response	Comment
0000b	OKAY (Normal Completion)	The transaction completed normally
0001b-1110b	Vendor Defined	
1111b	RESERVED	A Vendor defined Command shall not use this WrRspStatus encoding. This encoding is used for ISOLATE responses only which is not a legal status response for a Vendor Defined Command.

2.7.7 Originator Data Channel

The Originator Data Channel shall be utilized to transmit Write Data for Write Requests, Write Data for Vendor Defined Write class Commands, operand data for Atomic Requests, Vendor Defined Atomic class Commands, Message Data for UPLI Write Message Requests and Vendor Defined UPLI Write Requests.

The column labeled **Driver** indicates whether the Originator or the Completer drives the signal.

Table 2-21 Originator Data Channel Signals

Name	Size (bits)	Driver	Description
Originator Data Channel Information and Control signals These signals are Time Division Multiplexed (TDMed) according to the OrigDataPortID value.			
OrigDataVld	1	Originator	Originator Data Valid. This signal indicates that the Originator is presenting valid information (a data beat) on this channel.
OrigDataPortID	2	Originator	Originator Data Port ID. Where the Originator drives the Originator Data Channel to a TL, indicates the Port associated with that TL that the Originator Data Channel Beat is to be presented on. For all Originators, OrigDataPortID indicates the TDM cycle for the Originator Data Channel.
OrigData	512	Originator	Originator Data. This field contains the data for Writes, Vendor Defined Write class Commands, UPLI Write Message Requests, and Vendor Defined UPLI Write Message Requests. This field also contains the Operand Data for Atomics and Vendor Define Atomic class Commands.
OrigDataByteEn	64 (one bit per byte)	Originator	Originator Data Byte Enables. OrigDataByteEn bit for each eight bits of the OrigData field. These signals indicate which byte lanes of OrigData hold valid information.
OrigDataOffset	2	Originator	Originator Data Offset. Indicates the index of the current Data Beat. Data to be written is transferred from the Originator in the OrigData field in one or more Beats. Each beat is sequentially assigned a number starting at 0 and ending with n-1, based on the order of the data in memory; n is the total number of beats required to transfer the data to be written. When the transfer requires more than one Beat, the originator shall begin with beat 0 and continue in Beat order. This field indicates the number of the Beat being presented on this transfer cycle.
OrigDataLast	1	Originator	Originator Data Last. This signal is asserted with the last Data Beat of an Originator Data Transfer.
OrigDataError	1	Originator	Originator Data Error. When asserted this signal indicates that the current Data Beat contains a data error.
OrigDataVC	2	Originator	Originator Data Virtual Channel. Specifies the Virtual Channel of the Originator Data Beat. Matches the Virtual Channel of the original Request (ReqVC). In UALink 1.0, this field is informational only to the Switch with the exception that this value is returned to the Sender in the Credit Return signals. The Switch has one virtual channel and passes this field through to the Destination Accelerator unchanged.
OrigDataPool	1	Originator	Originator Data Pool. Specifies whether the Originator Data Beat was issued using a Pool Credit (OrigDataPool = 1) or a Virtual Channel Credit (OrigDataPool = 0).
Originator Data Channel Credit Return signals These signals are not Time Division Multiplexed (TDMed). A Credit or Credits for any port may be returned on any cycle.			
OrigDataCreditInitDone	4	Completer	Originator Data Credit Initialization Done. When Asserted, no earlier than the cycle after the last Credit is released, indicates to Originator that the initial Credit release for the port associated with the individual signal is complete (OrigDataCreditInitDone[2] indicates port 2 is complete). The individual "done" signals can be asserted independently and once asserted remain asserted until the interface is reset or powered off. The Originator shall monitor the OrigDataCreditInitDone signal until that signal has been asserted an implementation specific number of cycles greater than 1. Once that has occurred, the Originator shall consider the initial issuance of the Credits to be complete and shall no longer monitor the signal.
OrigDataCreditVld	4	Completer	Originator Data Credit Valid. Indicates a Credit or Credits is being released to the Originator. Credits are per port. A Station may have 1, 2 or 4 ports. OrigDataCreditVld[n] corresponds to Port[n]. Credits can be released on more than one port simultaneously.

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Size (bits)	Driver	Description
OrigDataCreditPool	4	Completer	Indicates the type of Credit indicated on OrigDataPool for the Originator Data Beat that consumed the Pool or Virtual Channel Credit of Credits being released. If '0', the Credit or Credits being released is a Virtual Channel Credit and if '1', the Credit or Credits released is a Pool Credit. The OrigDataVC value for the Originator Data Beat corresponding to the returned Credit or Credits is placed on the appropriate OrigDataCreditPort[0,1,2,3]VC signal.
OrigDataCreditPort0VC	2	Completer	Originator Data Credit Port 0 Virtual Channel. Indicates the Virtual Channel indicated on OrigDataVC for the Originator Data Beat that consumed the Pool or Virtual Channel Credit or Credits being released for Port0. Valid when OrigDataCreditVld[0] is asserted.
OrigDataCreditPort1VC	2	Completer	Originator Data Credit Port 1 Virtual Channel. Indicates the Virtual Channel indicated on OrigDataVC for the Originator Data Beat that consumed the Pool or Virtual Channel Credit or Credits being released for Port1. Valid when OrigDataCreditVld[1] is asserted.
OrigDataCreditPort2VC	2	Completer	Originator Data Credit Port 2 Virtual Channel. Indicates the Virtual Channel indicated on OrigDataVC for the Originator Data Beat that consumed the Pool or Virtual Channel Credit or Credits being released for Port2. Valid when OrigDataCreditVld[2] is asserted.
OrigDataCreditPort3VC	2	Completer	Originator Data Credit Port 3 Virtual Channel. Indicates the Virtual Channel indicated on OrigDataVC for the Originator Data Beat that consumed the Pool or Virtual Channel Credit or Credits being released for Port3. Valid when OrigDataCreditVld[3] is asserted.
OrigDataCreditPort0Num	2	Completer	Originator Data Credit Port 0 Number of Credits. Indicates the number of Credits being returned for Port0 (up to 4, OrigDataCreditPort0Num+1). Only valid when OrigDataCreditVld[0] is asserted.
OrigDataCreditPort1Num	2	Completer	Originator Data Credit Port 1 Number of Credits. Indicates the number of Credits being returned for Port1 (up to 4, OrigDataCreditPort1Num+1). Only valid when OrigDataCreditVld[1] is asserted.
OrigDataCreditPort2Num	2	Completer	Originator Data Credit Port 2 Number of Credits. Indicates the number of Credits being returned for Port2 (up to 4, OrigDataCreditPort2Num+1). Only valid when OrigDataCreditVld[2] is asserted.
OrigDataCreditPort3Num	2	Completer	Originator Data Credit Port 3 Number of Credits. Indicates the number of Credits being returned for Port3 (up to 4, OrigDataCreditPort3Num+1). Only valid when OrigDataCreditVld[3] is asserted.
Originator Data Channel Parity signals			
OrigDataVldParity	1	Originator	Originator Data Valid Parity. Even parity across OrigDataVld. Checked every cycle.
OrigDataParity	8	Originator	Originator Data Parity. Originator data parity. Each bit in OrigDataParity provides even-parity based error protection for 64 bits of the OrigData field. The number of set bits in OrigDataParity[i] and OrigData[(((i+1)*64-1):(i*64))] is always even, including data bytes where the OrigDataByteEn masks some or all bytes in that parity group. Checked only when OrigDataVld is asserted.
OrigDataByteEnParity	1	Originator	Originator Data Byte Enables Parity. Even parity across OrigDataByteEnParity. Checked only when OrigDataVld is asserted.
OrigDataFieldsParity	1	Originator	Originator Data Fields Parity. OrigDataFieldsParity provides even parity over OrigDataLast, OrigDataError, OrigDataOffset, OrigDataPortID, OrigDataVC, OrigDataPool. Checked only when OrigDataVld is asserted.
OrigDataCreditVldParity	1	Completer	Originator Data Credit Valid Parity. Even parity across OrigDataCreditVld. Checked every cycle.
OrigDataCreditParity	1	Completer	Originator Data Credit Parity. Even parity across OrigDataCreditPort0VC, OrigDataCreditPort1VC, OrigDataCreditPort2VC, OrigDataCreditPort3VC, OrigDataCreditPort0Num, OrigDataCreditPort1Num, OrigDataCreditPort2Num, OrigDataCreditPort3Num, and OrigDataCreditPool. Checked in beats where OrigDataCreditVld is non-zero.

2.7.7.1 OrigDataError

The OrigDataError (Originator Data Error) signal is a data “poison” indicator that shall be set at any UALink Protocol Level Interface that detects a parity error on the Originator Data, or by the Switch detecting an error on data (data only, not control signals including Byte Enables) according to the Switch soft error detection scheme.

2.7.8 Relationships between UPLI Channels and requirements within the Channels

The following dependencies and rules shall exist between Channel Types and within Channels:

- A UPLI Originator or UPLI Completer shall not transfer any information, including Credits or Request/Response/Data Beats on any channel until it has completed the UALink Protocol Level Interface control handshake.
- A UPLI Originator or UPLI Completer shall not issue a Request Beat, a Read Response/Data Beat, a Write Response Beat, or an Originator Data Beat without having sufficient Credit(s) signaled by the other agent.
- The sender side of a UPLI channel shall be able to unconditionally receive returned Credits for that Channel.
- The first Beat of Originator Data (whether for an Atomic operation, a Write operation, a Write Full, or a UPLI Write Message) for a Request shall be issued on the same cycle as the Request is issued on the Request Channel (i.e. ReqVld and OrigDataVld shall be asserted in the same cycle for all Write and Atomic Requests).
- All Originator Data Beats for any given Write, Write Full, or Atomic Request shall occur as a contiguous series of Data Beats, shall be in ascending address order (i.e. OrigDataOffset starts at 0 and is incremented for each of the up to four Beats in the Originator Data transfer), and the final Beat of an Originator Data transfer shall be indicated by the assertion of OrigDataLast. A consequence of this rule is that an Originator shall obtain Request Channel and Originator Data Channel Credits sufficient to issuing a Write or Atomic Request before issuing the Write or Atomic Request (e.g. a 256-byte Write would require one Request Channel Credit and four Originator Data Channel Credits).
- A consequence of the prior two rules is that Write and Atomic transactions shall not be pipelined on the Request and Originator Data Channels (i.e. when an Originator issues a Write or Atomic Request, the entire set of Originator Data Beats for that Request must be issued before any subsequent Write or Atomic Request may be issued).
- Read Requests may be issued in cycles where Originator Data Beats for a prior Write or Atomic Request are present on the Originator Data Channel (i.e. Read Requests can be pipelined on top of Write and Atomic Request Originator Data Beats).
- To avoid deadlock, once an Originator on an Accelerator has issued a Request, it shall be able to process (or “sink”) the Response Beat(s) associated with that Request without any requirements that the Completer processes any other transaction.
- Read Responses may take two different forms which may be freely chosen on each transaction's Read Response:
 - **Multi-Beat mode:** In this mode, like OrigData transfers, the Read Response/Data Beat(s) shall occur as a contiguous group of Beats, in ascending address order (i.e. RdRspOffset starts with an initial value of 0 and increments for each of the up to four Beats in the transfer), and the last Read Response/Data beat shall be indicated by the assertion of RdRspLast. In addition, the RdRspNumBeats field shall indicate the number of beats -1 involved in the transfer and is the same for all beats (i.e. RdRspNumBeats=0 indicates one beat and RdRspNumBeats=3 indicates four beats).
 - **Single Beat mode:** In this mode, the Read Response/Data beat(s) shall be returned as individual Read Response/Data Beats. These individual Beats can occur in any address order but shall include all values necessary for the size of the transfer. The individual Responses can have arbitrary gaps and may be interleaved with unrelated Data Beats. The final Read Response/Data beat to be transferred shall be

indicated by the assertion of RdRspLast. A Switch shall keep all single beat read Responses from the same ingress port to egress port in order and shall pass along RdRspLast without modifying it. The RdRspNumBeats shall equal 0 (1 Beat transfer) for all the Beats for a given tag in a Single-Beat mode.

- The Read Response/Data Response for a Read transaction that only Requests one Beat of data is the same on the UALink Protocol Level Interface in either mode.
- The Switch is not expected to attempt to “gather” the various beats of a Single Beat mode transfer, but rather to individually forward each beat immediately.
- Write Response beats and Read Response/Data beats shall not be blocked from making progress in returning to the Originator.
- A Completer shall not issue a Write Response any earlier than the cycle following the last beat off Originator Data arriving at the Completer.

2.7.9 UPLI Request, Read Response, and Write Response ordering.

The ordering of UPLI Requests is defined independently for each Source/Destination Accelerator pair in terms of the set of Requests issued on an initial UPLI Interface on the given Source Accelerator that are destined for the final UPLI Interface on the Destination Accelerator.

The ordering of UPLI Responses is defined independently for Read and Write Responses and independently for each Destination/Source Accelerator pair in terms of the set of Responses issued from the final UPLI Interface on the given Destination Accelerator that are destined for the initial UPLI Interface on the given Source Accelerator.

If a Vendor Defined Command or a Vendor Defined UPLI Write Message Request requires an address and has semantics that require the Vendor Defined Request to be ordered with other Requests that have an address, the ReqAddr field shall be used to specify that address in the same manner as is used for the other non-vendor defined Requests with an address.

All Requests, whether they have an address or not, shall be ordered based on the ReqAddr signal value as if it that signal contained an address to memory.

All Requests with an address (non-Vendor Defined Requests with address and Vendor Defined Requests that define the ReqAddr field to be an address and how semantics require they are ordered with other Requests with an address) whose address fall within a given 256-byte region of memory shall egress the Accelerator on the same Port.

In what follows, the term “Strict Ordering Mode” (or “SO”) or “Strict Ordering Lite Mode” (SOL) is defined to mean Authentication and Encryption is enabled or Encryption alone is enabled.

When not in Strict Ordering Mode, the set of all-Requests within each 256-byte aligned region of memory issued on the initial UPLI Interface on the Source Accelerator for a given Virtual Channel shall appear on the final UPLI Interface on the Destination Accelerator in that same order (i.e. remain ordered). Implementations may also choose to keep all Requests ordered when not in Strict Ordering Mode.

In Strict Ordering Mode, all Requests issued on the initial UPLI Interface on the Source Accelerator shall remain ordered.

In Strict Ordering Lite Mode, all Requests issued on the initial UPLI Interface for a given Virtual Channel on the Source Accelerator shall remain ordered. In Strict Ordering Lite Mode, the Requests in different virtual channel may reorder.

The order that the initial Originator issues Requests on the Source Accelerator and the processing of those Requests on the Destination Accelerator after the final Completer are implementation specific characteristics of the Accelerator not specified by the UALink specification. However, if an Accelerator wishes to maintain coherence – the property that all writes to a given memory location are serialized in some order agreed to by all participants – the Accelerator will need to order at least those Requests addressing overlapping memory locations.

When not in Strict Ordering Mode, Read Responses shall have no ordering requirements among themselves and therefore may appear on the initial UPLI Interface on the Source Accelerator in a different order than they were issued by the Destination Accelerator on the final UPLI Interface (i.e. they may be freely reordered). Write Responses shall also have no ordering requirements among themselves and may also be freely reordered. There are no ordering requirements between Read Responses and Write Responses when not in Strict Ordering Mode.

When in Strict Ordering Mode, Read Responses shall remain in order amongst themselves, and Write Responses shall remain in order amongst themselves. There are no ordering requirements between Read Responses and Write Responses in Strict Ordering Mode.

A UPLI Originator on the initial UPLI Interface may issue multiple Read, Write, WriteFull or Atomic Requests to a given 256-byte aligned region of memory location before the Responses for previous Requests to the same 256-byte region have been returned.

No ordering between Requests from different Accelerators is guaranteed even if it is known by other means that these Requests were initiated in some order.

2.7.10 UPLI Request Single-Copy Atomicity

A UPLI access is Single-Copy atomic if it is always performed (i.e., the memory locations are updated for writes, and the values to be returned are bound for reads) in its entirety with no visible fragmentation.

That is, if a given set of locations is accessed only by Single-Copy atomic Reads, Single-Copy atomic Writes, or Single-Copy atomic Atomics, each of which accesses all the locations in the set, the values observed by a given Read or the read portion of the Atomic will be the values written by exactly one of the Writes or the write portion of one Atomic, and never a combination of the values written by more than one of the Writes or Atomics or both.

If an operation is not declared to be Single-Copy atomic, it may be decomposed into and performed as a set of smaller disjoint Single-Copy atomic operations, possibly of different sizes. For example, a 64-byte Write could be decomposed into a set of eight 8-byte Single-Copy atomic Write operations that are performed in some arbitrary order. As another example, a 64-byte Read could be decomposed into a set of four 8-byte Single-Copy atomic Read operations and eight 4-byte Single-Copy atomic Read operations that are performed in some arbitrary order.

Any given Read, Write, or Atomic operation not declared to be Single-Copy atomic can be performed in a Single-Copy atomic fashion or can be decomposed (i.e. the decomposition is not mandatory for a non-Single-Copy atomic operation, just allowed).

An Atomic operation that is not Single-Copy atomic can only be decomposed into some subset of operations having sizes that are multiples of the element size of the Atomic.

While a non-Single-Copy atomic operation may be decomposed into a set of smaller disjoint Single-Copy atomic accesses, if two non-Single-Copy atomic operations are ordered for whatever reason

(for example due to being in the same 256-byte memory region), all of the constituent decomposed operations of the first non-Single-Copy atomic operation shall be performed before the Response for the non-Single-Copy atomic operation is returned.

While the UPLI and TL interfaces are required to deliver a UPLI operation from the Source accelerator to the Destination accelerator without any decomposition, the Single-Copy atomicity of any given UPLI operation, and the decomposition applied, if any, if the operation is not Single-Copy atomic are implementation specific characteristics of the Destination Accelerator and are not specified by the UALink Specification.

2.8 Data/Atomic Operands Transfer

The UALink Protocol Level Interface has an Originator Data Channel that is sourced from the Originator to the Completer. The Originator Data Channel is used to transfer Write data or Atomic operand data to the Completer. The UALink Protocol Level Interface also has a Read Response/Data Channel which is used to transmit Read data or Atomic read data (for AtomicR Requests) from the Completer to the Originator.

Reads

The data transferred for a Read Request consists of a contiguous block of one or more aligned doublewords in a byte addressable system memory address space. The initial address of this block (the address of the least-significant byte of the first doubleword) is equal to the ReqAddr field of the Read Request. Since the address is doubleword aligned, ReqAddr[1:0] is always 2'b00.

The ReqLen field specifies the requested number of doublewords minus one to be transferred (e.g. ReqLen = 0 implies transferring 1 doubleword, the minimum, and ReqLen = 63 implies transferring 64 doublewords, the maximum). The maximum size of a Read Request is 256 bytes (64 doublewords) and a Read Request may not cross a 256-byte boundary.

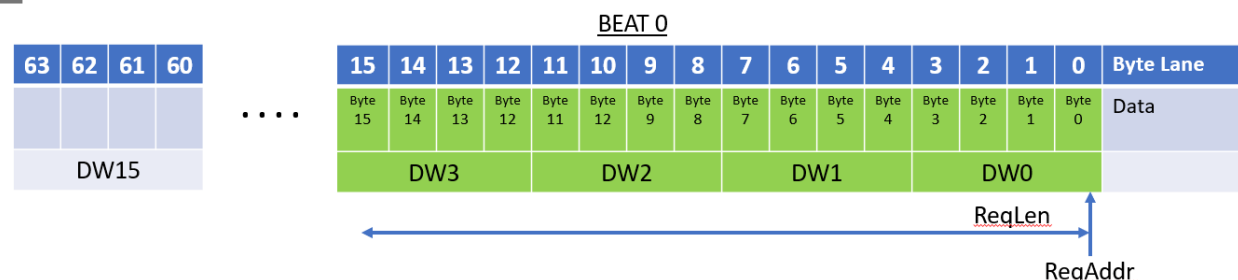
Data is transferred on the Read Response/Data Channel within the 64-byte RdRspData field. The byte lanes within the RdRspData field are numbered 0 to 63 with byte lane 0 being the low order address. Data bytes transferred on the RdRspData field are placed in their naturally aligned byte lane within the 64-byte RdRspData field (e.g. a byte at address X is always placed in byte lane X mod 64 – for example address 0 is placed in byte lane 0 and address 129 is placed in byte lane 1). Transfers that span a 64-byte boundary or boundaries are transferred using multiple 64-byte Read Response beats. For example, an 8-byte read at address 60 will consist of two beats, one transferring addresses 60-63 on the high order byte lanes 60-63 and one transferring addresses 64-67 on the low order byte lanes 0-3). Each beat in such a multi-beat data Response is transferred exactly once. The beats are ordered from low address to high address in multi-beat mode and in single beat mode may come in any order. Beats are labeled by the RdRspOffset field with a value of 0, 1, 2, or 3 indicating the data beat number in ascending address order (RdRspOffset is 0 for the beat containing the initial doubleword).

To allow for arbitrarily sized and arbitrarily aligned Read accesses down to a single byte, byte enables for the first and last doubleword of the Read access are provided in the ReqAttr[7:0] field. ReqAttr[3:0] provides byte enables to specify the bytes(s) being accessed in the first doubleword (with ReqAttr[3] specifying the enable for the high-order byte in the doubleword and ReqAttr[0] specifying the enable for the low-order byte in the doubleword). ReqAttr[7:4] similarly specifies the bytes being accessed in the last doubleword for accesses of two or more doublewords. For an access of one doubleword, ReqAttr[7:4] is ignored. For accesses of three or more doublewords, all

bytes in doublewords that are not the first and last doubleword are assumed to be accessed (i.e. byte enables are provided only for the first and last doublewords).

Bytes within the 64 byte RdRspData field that are masked out by ReqAttr[7:0] or are outside the range called out by ReqAddr/ReqLen are not valid and are ignored by the Originator, however these bytes still participate in the data parity computation.

The following figure illustrates the beat and byte lane assignments for a 16-byte transfer starting at address 0:

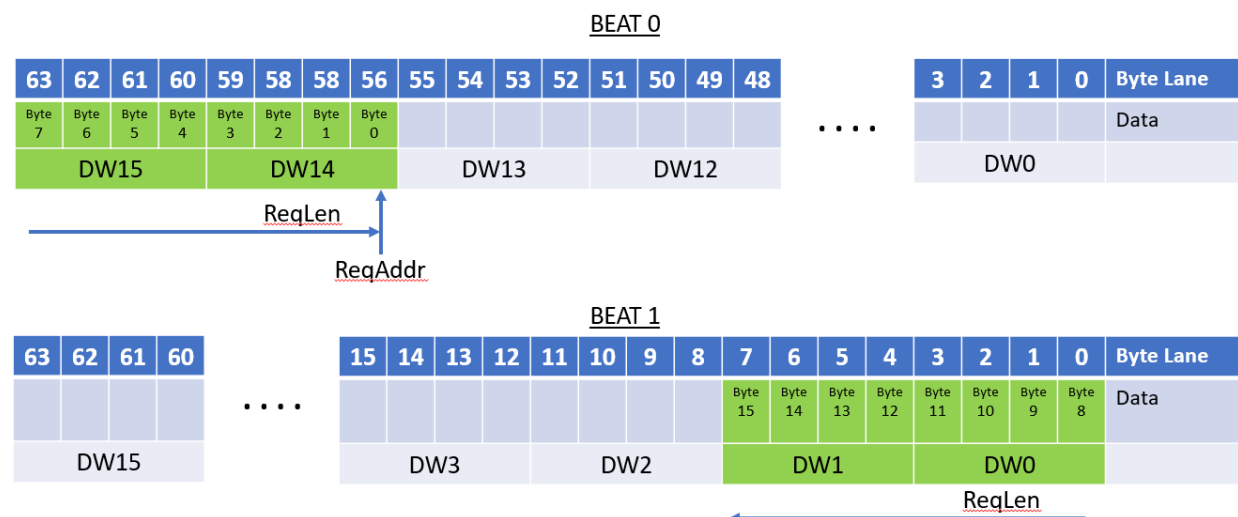


ReqAddr[5:0] = 00_0000b (0); ReqLen = 3 (4 doublewords); ReqAttr[7:4] = 1111b; ReqAttr[3:0] = 1111b

Figure 2-12 Four DoubleWord Read Request Not Straddling a 64-Byte Boundary

Bytes 0 through 15 are transferred in a single beat. The ReqAttr byte enables are all 1's to transfer all bytes in the first and last beat.

The next figure illustrates another 16 byte transfer but starting at byte 56:



ReqAddr[5:0] = 11_1000b (56); ReqLen = 3 (4 doublewords); ReqAttr[7:4] = 1111b; ReqAttr[3:0] = 1111b

Figure 2-13 Four DoubleWord Read Request Straddling a 64-byte boundary

Unlike the previous transfer, this transfer spans a 64-byte boundary and therefore must be transferred using multiple data beats.

The next figure illustrates accessing a single byte (byte 54):

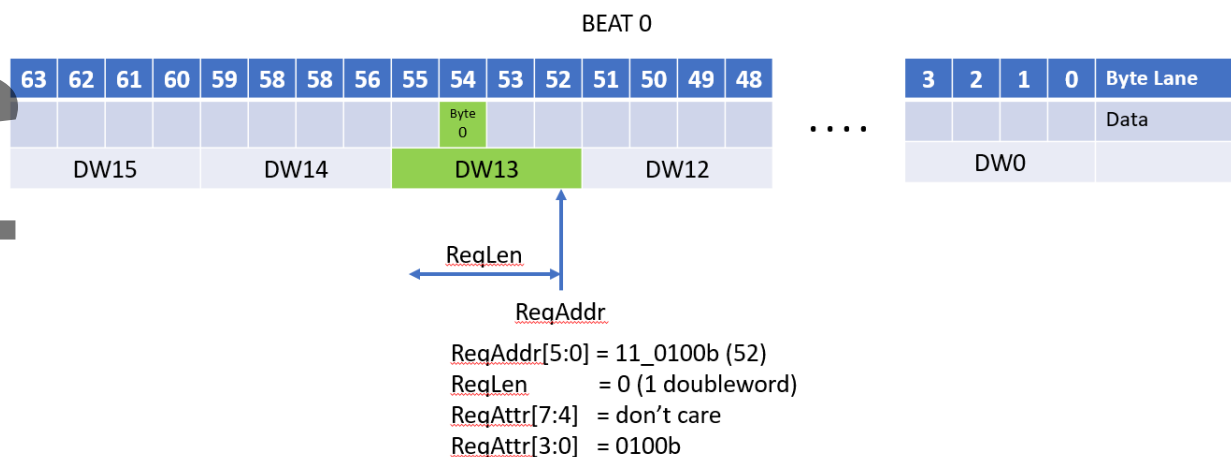


Figure 2-14 Single Byte Read

The ReqAddr/ReqLen fields call out a single double word (DW13 at address 52 in the data field), and ReqAttr[3:0] selects the byte (byte 54) to be accessed. ReqAttr[7:4] is ignored.

The next figure illustrates a six-byte access necessitating the use of both ReqAttr[7:4] and ReqAttr[3:0]:

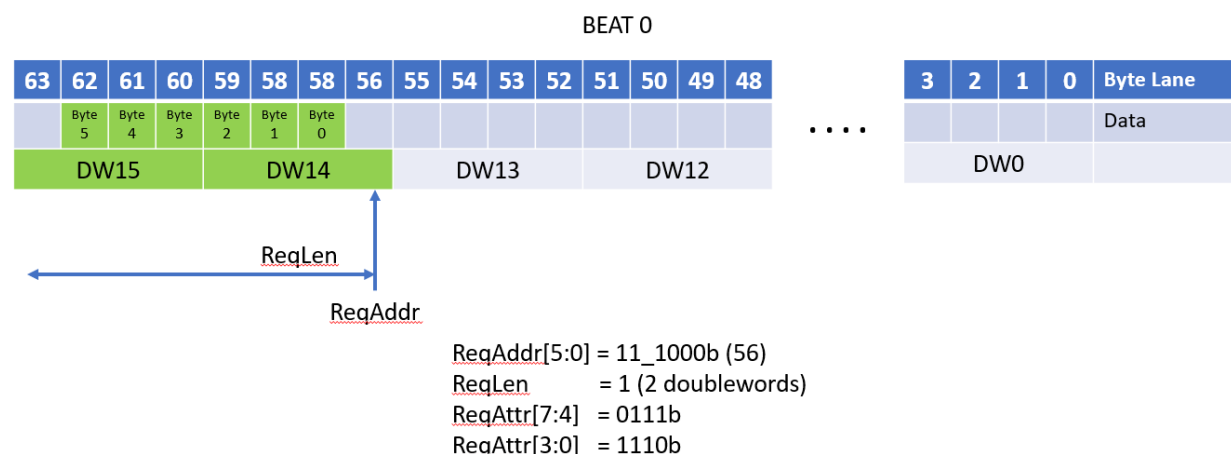


Figure 2-15 Six Byte Read Access Not Straddling a 64-Byte Boundary

In a two doubleword transfer, the ReqAttr[7:0] bit control the bytes within the two doublewords that are valid.

The next figure illustrates another two double word transfer, but one that spans a 64-byte boundary:

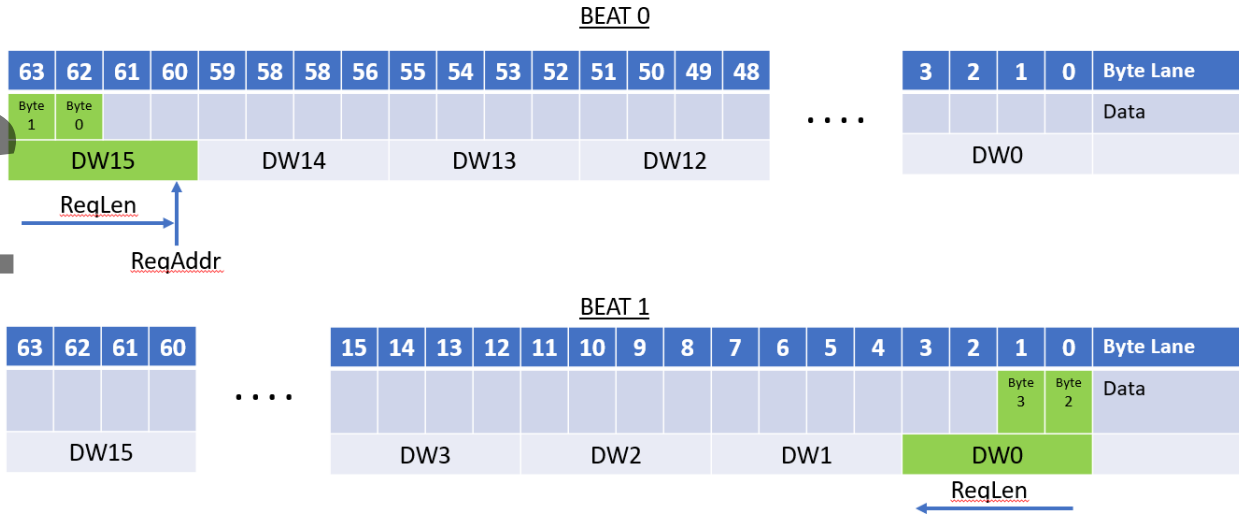


Figure 2-16 Four Byte Read Access Straddling a 64-Byte Boundary

Because the transfer spans a 64-byte boundary, the transfer is broken up into two data beats. The ReqAttr[7:0] field controls which bytes are valid in the two doublewords spread across the two beats.

Writes

Transfers of write data occur on a 64-byte OrigData field in the Originator Data Channel. Like the RdRspData field, the byte lanes of the OrigData field are numbered 0 to 63 from the low order address to the high order address and any given byte of data in a write is transported on the byte lane of the OrigData field that matches the alignment for the byte's address. The ReqAddr/ReqLen fields are defined for Writes as they are for Reads: the address is a doubleword address (ReqAddr[1:0] are 2'b00) and the ReqLen field is the number of doublewords to be transferred minus one. As with Reads, writes can be up to a maximum of 256 bytes and may not cross a 256 byte boundary.

As with Reads, if a Write spans a 64-byte boundary or boundaries (that is not a 256 byte boundary), the transfer utilizes multiple 64 byte beats to transfer the data and each beat is transferred exactly once. The "wrapping" of write data in a beat is not permitted. The first beat of a Write (OrigDataVld asserted) must occur on the same cycle as the Request for that write (ReqVld) and the subsequent data beats (if any) must occur in consecutive cycles in ascending address order. Write data beats are labeled by the OrigDataOffset field with a value of 0, 1, 2, or 3 indicating the data beat number in ascending address order (OrigDataOffset is 0 for the beat containing the initial doubleword).

Write transfers have an additional 64-bit byte enable field – OrigDataByteEn -- that allows for individual bytes within a data beat to be written or not in a write transfer. The OrigDataByteEn[0] signal indicates if byte lane 0 is to be written, OrigDataByteEn[1] indicates that byte lane 1 is to be written, and so on. For Write commands (not WriteFull commands), byte enables for byte lanes outside the region called out by ReqAddr/ReqLen must be set to 0. All other byte enables (within the region called out by ReqAddr/ReqLen) may be '0' or '1'. These other byte enables may be sparse – that is there is no requirement that the byte enables must be consecutively set nor is there a requirement that any byte enables be set.

For WriteFull commands, the transfers are restricted to starting on a 64-byte boundary, being a multiple of 64 bytes in length, and not crossing a 256-byte boundary. For these transfers, all byte enables in the beats must be set to '1'.

The follow figure shows the beat and byte lane assignments and the OrigDataByteEn values for a 16-byte write starting at address 0:

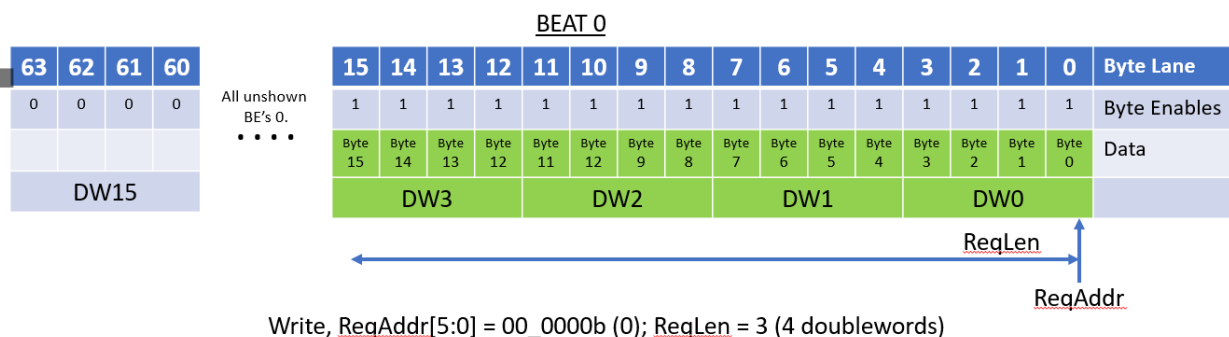


Figure 2-17 Four Doubleword Write Request Not Straddling a 64 Byte Boundary

The sixteen asserted byte enables starting at OrigDataByteEn[0] cause the first 16 bytes to be written.

The following figure shows a sparse write contained within one beat:

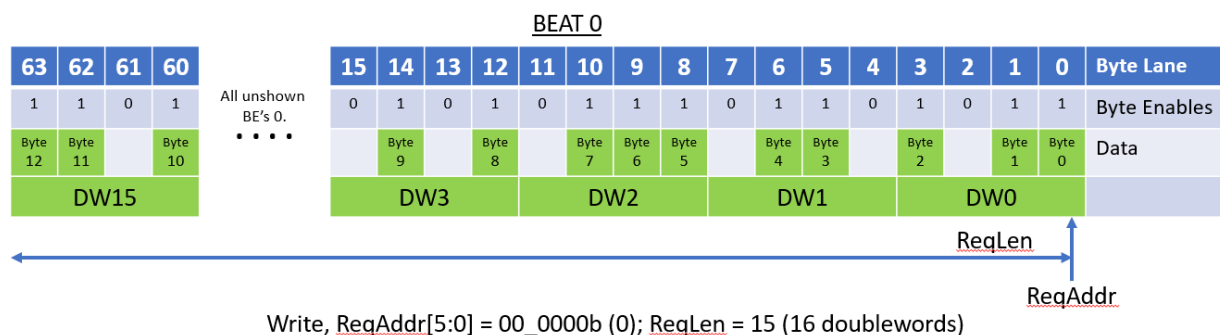


Figure 2-18 Sixteen Doubleword Write Request Not Straddling a 64-Byte Boundary

The various byte enables being on or off produce a sparse write. The ReqAddr/ReqLen are such that the write stays within a single beat.

The following figure shows a sparse write spread across two beats:

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification Rev 2.0

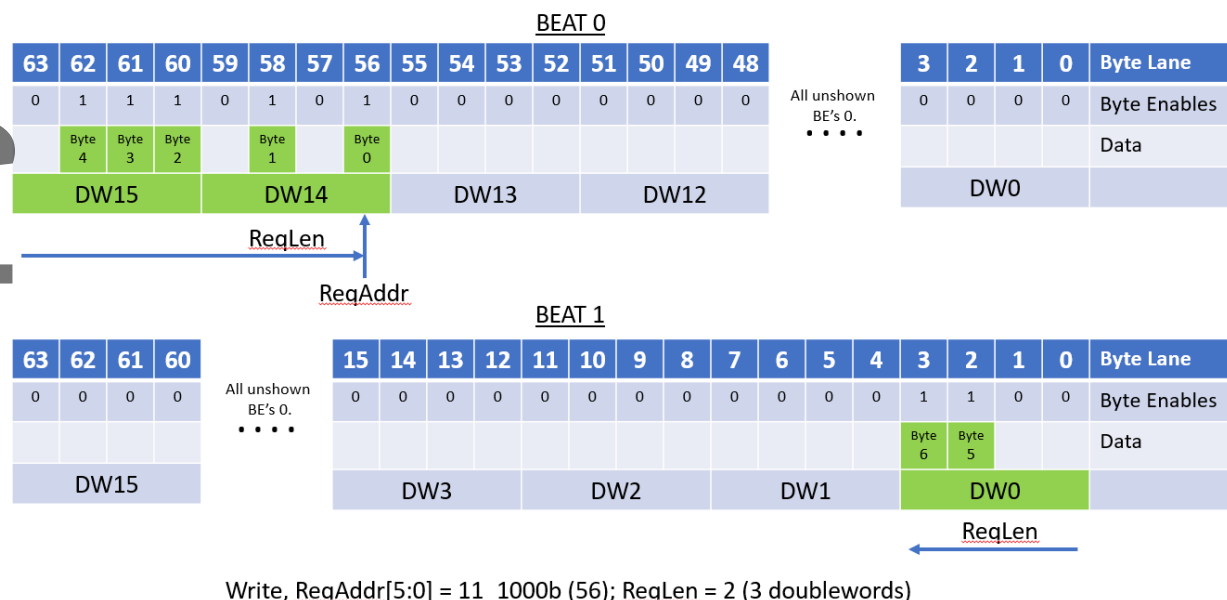


Figure 2-19 Three Doubleword Write Request Straddling a 64 Byte Boundary

The various byte enables being on or off produce a sparse write. The ReqAddr/ReqLen are such that the write spreads across two beats. A sparse write can spread across up to four beats.

Unshown in these figures is the case where no byte enables are asserted. In such a case, the beats are still transferred on the OrigData field, but no updates are made to memory.

The following figure shows a 128-byte WriteFull to address 0:

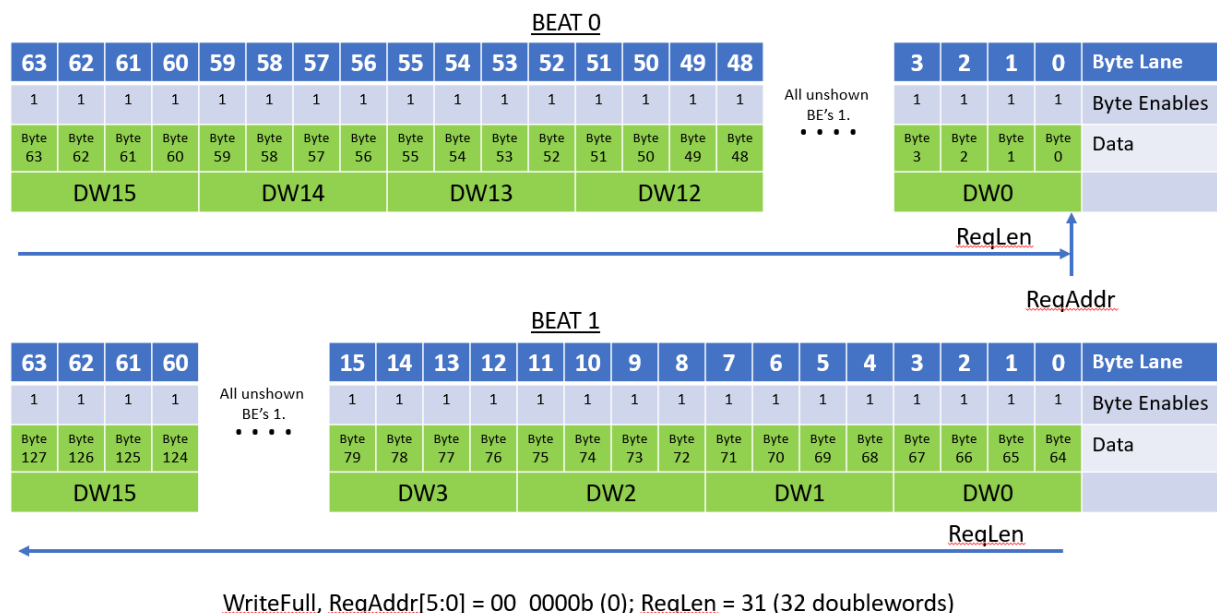


Figure 2-20 A 128 Byte Write Full Request (Requires Two Beats)

The WriteFull command allows the TL and PHY layers the option to compress out the byte enables. The byte enables must still appear on the UALink Protocol Level Interface.

Atomics

Originators issue Atomic commands to perform I/O coherent atomic (at the completer) read-modify-write operations. The AtomicR commands return the value read in the read-modify-write operation and AtomicNR commands do not return data. The semantics of the various AtomicR or AtomicNR operations are called out by the OpType sub-field in ReqAttr field and the size of the element(s) in memory (1-byte, 2-byte, 4-byte or 8-byte) that the Atomic operates on is specified by the OpSize sub-field in the ReqAttr field. The exact semantics for a given OpType field value is implementation specific as is whether the Atomic needs one operand (a single-operand atomic with one “Op1” operand) or two operands (a double-operand atomic with “Op1” and “Op2” operands).

For all Atomics, the operand data is conveyed on the OrigData bus to the completer. For an AtomicR Atomic, the returned data and Response is conveyed on the Read Response/Data Channel. For an AtomicNR command, the status Response (without any data) is conveyed on the Write Response Channel.

For all Atomics, each aligned 64-byte region of memory is divided into 16 aligned 4-byte elements for a 1-byte, 2-byte, or 4-byte atomic or 8 aligned 8-byte elements for an 8-byte atomic. For single operand atomics, ReqAddr must be aligned to the size of the element operated on for 4-byte and 8-byte elements and aligned on a 4-byte boundary for 1-byte and 2-byte elements, ReqLen shall specify a length that is a multiple of 8 bytes for an 8-byte atomic and a multiple of 4 bytes for a 1-byte, 2-byte, or 4-byte atomic, and the Request may not cross a 64-byte boundary (i.e. all single operand atomics transfer their operand data in a single data beat). The operand value in each element in the OrigData field is used to perform the atomic read-modify-write for the corresponding location in memory. Similarly, data returned for a given element by an AtomicR Request is placed in the corresponding locations in the RdRspData field.

Each 1-byte, 2-byte, 4-byte or 8-byte element in the 64 byte region of memory may be updated or not independently by setting or not setting all the byte enables for the element. All byte enables for each element must either be set to ‘1’ or ‘0’. Partial byte enables within an element are not permitted. The elements that are updated may be sparse, that is there is no requirement that the elements that are updated are contiguous, nor is there a requirement that any element be updated at all. For an AtomicR Request, data for the elements read are returned in their naturally aligned positions in the RdRspData field.

The following figure illustrates the byte lane allocation for the Operands and (if present) returned data for an 8-byte single-operand Atomic. The ReqAddr is offset from address ‘0’ and two sparse elements are operated on:

DW 15	DW 14	DW 13	DW 12	DW 11	DW 10	DW 9	DW 8	DW 7	DW 6	DW 5	DW 4	DW 3	DW 2	DW 1	DW 0	Doubleword OrigData
0x00		0xFF		0x00		0xFF		0x00		0x00		0x00		0x00		Byte Enables
		Op1 Element1				Op1 Element0										Operands

ReqLen

 ReqAddr

Single op AtomicR/AtomicNR, ReqAddr[6:0] = 001_0000b (0); ReqLen = 9 (10 doublewords), OPSize = b'01' (8 bytes).

The data elements, for an AtomicR, are returned in their naturally aligned byte lanes.

Operand Data for 4-byte Op1 AtomicR/AtomicNR

DW 15	DW 14	DW 13	DW 12	DW 11	DW 10	DW 9	DW 8	DW 7	DW 6	DW 5	DW 4	DW 3	DW 2	DW 1	DW 0	Doubleword OrigData
0x0	0x0	0x0	0x0	0x0	0x0	0xF	0x0	0xF	0xF	0x0	0xF	0x0	0x0	0x0	0xF	Byte Enables
						Op1 E4		Op1 E3	Op1 E2		Op1 E1				Op1 E0	Operands

ReqLen

ReqAddr

Single op AtomicR/AtomicNR, ReqAddr[6:0] = 000_0000b (0); ReqLen = 10 (11 doublewords), OPSize = b'00' (4 bytes).

The data elements, for an AtomicR, are returned in their naturally aligned byte lanes.

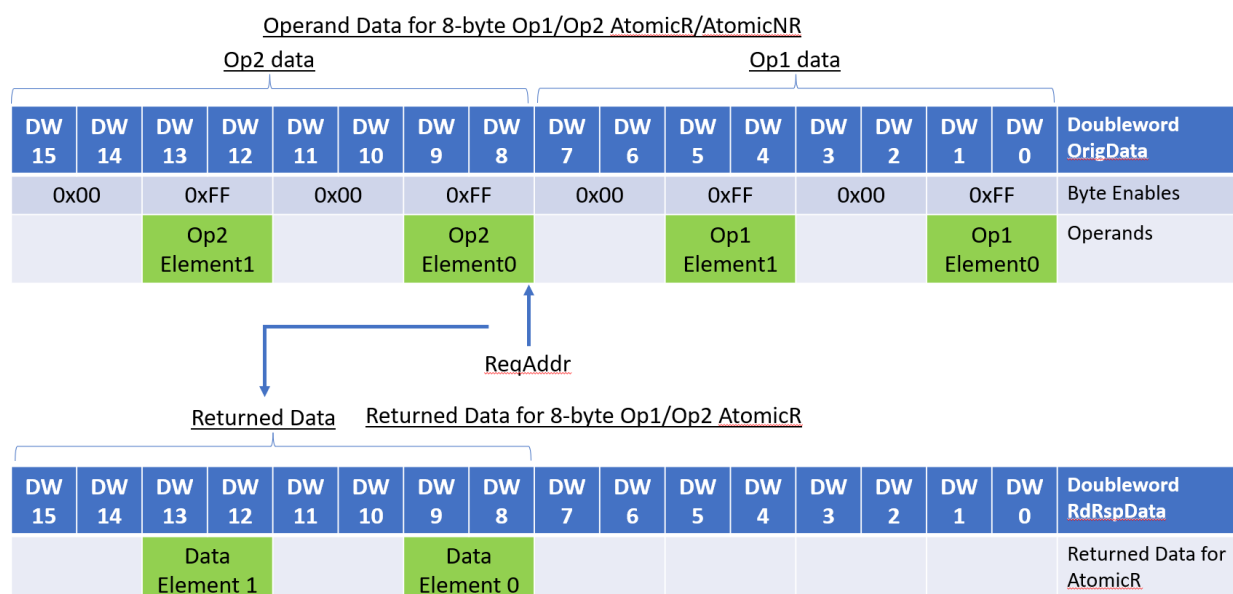
UPLI Interface Definition and Operation Rules

0 to 31 on the OrigData field and the Op2 operand data occurring on bytes lanes 32 to 63 on the OrigData field. The placement of the Op1 and Op2 Operand data is unaffected by ReqAddr.

The 32-byte Operand Data fields Op1 and Op2 are each broken up into 8 aligned 4-byte elements in memory for a 1-byte, 2-byte, 4-byte atomic or 4 aligned 8-byte elements for an 8-byte atomic. Corresponding Op1 and Op2 elements, by address, are utilized by the atomic to update the corresponding element in the 32-byte memory region specified by ReqAddr.

As with single-operand Atomics, byte, the OrigDataByteEn byte enables are used to determine if a given element is to be updated and the byte enables may be sparse (dis-contiguous elements may be updated and no element may updated). Similarly to single-operand Atomics, the byte enables for the Op1 and Op2 operands for each element must all be asserted, or none asserted. No partial enables within an element's operands are permitted (a consequence of this condition is that the Op1 operand data byte enables must have the same values as the Op2 operand data byte enables).

The following figure illustrates the byte lane allocation for the Operands and (if present) returned data for an 8-byte double-operand Atomic. The ReqAddr is offset from address '0x20h' and two sparse elements are operated on:

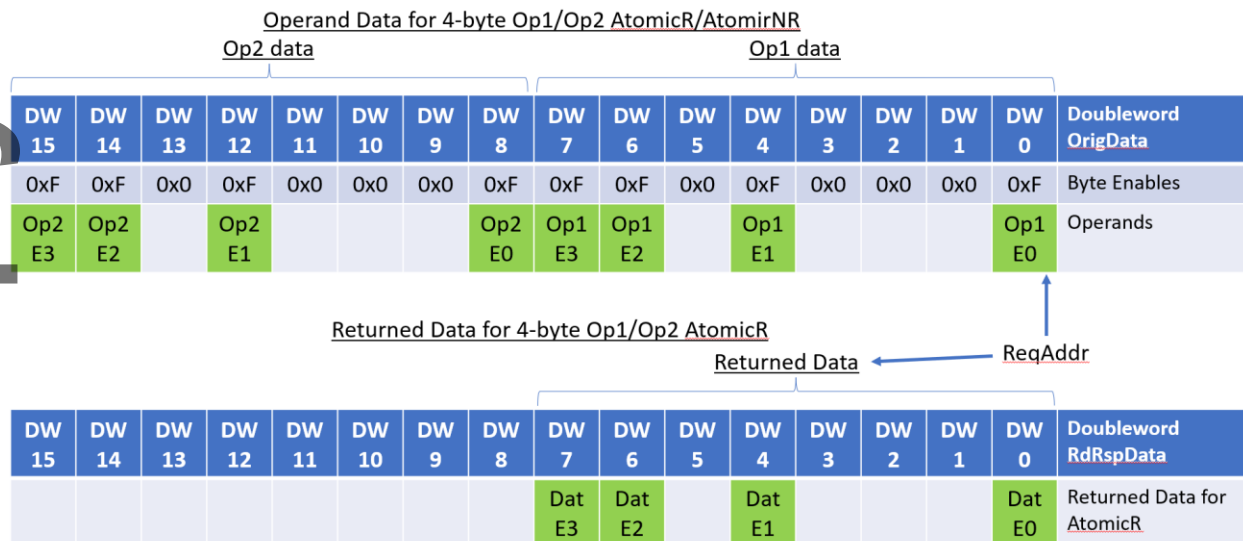


Double op AtomicR/AtomicNR, ReqAddr[6:0] = 010_0000b (0x20h); OPSize = b'01' (8 bytes).

Figure 2-23 Double Operand (Eight-Byte Operands) Atomic, Data Returned in High 32 Bytes.

The data elements, for an AtomicR, are returned in their naturally aligned byte lanes relative to the address specified by ReqAddr.

The following figure illustrates the byte lane allocation for the Operands and (if present) returned data for a 4-byte double-operand Atomic. The ReqAddr is at address '0' and four sparse elements are operated on:



Double op AtomicR/AtomicNR, ReqAddr[6:0] = 000_0000b (0); OPSize = b'00' (4 bytes).

Figure 2-24 Double Operand (Four-Byte Operands) Atomic, Data Returned in Low 32 Bytes.

The data elements, for an AtomicR, are returned in their naturally aligned byte lanes relative to the address specified by ReqAddr.

While the examples above do not explicitly illustrate one-byte and two-byte Elements in Atomic operations, the obvious substitutions for byte enables, operands and data, and their alignments for one and two-byte elements apply.

A Completer Device that does not support a given Atomic Request returns a TARGET ABORT Response on the appropriate Read Response Channel (for AtomicR Requests) or Write Response Channel (for AtomicNR Requests).

3 Reliability, Availability, and Serviceability (RAS)

3.1 RAS Requirements

Accelerator Pods require a robust fault management system to identify hardware failures and restore normal operations after errors with minimal disruption. This specification defines the Reliability, Availability and Serviceability requirements for Accelerators and Switches that use Ultra Accelerator Link technology.

This specification considers three broad classes of errors,

- Accelerator errors
- Link Down errors
- Switch errors.

and a set of RAS requirements and assumptions:

- A System Node is recommended as the smallest unit of allocation for a Virtual Pod. This recommendation is for simplicity, but implementations may vary. An Accelerator shall be the smallest allowable unit of allocation for a Virtual Pod.
- System Node errors (an error in one of the components in a System Node, e.g. CPU, GPU, PCIe, and other components) should not 'spread' beyond a virtual pod, however in the case where a System Node spans more than one Virtual Pod, the error may spread to all Virtual Pods associated with the System Node.
- Link Down errors shall not 'spread' beyond a Virtual Pod.
- An Accelerator attempts to handle Link Down errors and Switch errors without requiring a reboot of the associated System Node(s), Accelerator(s), or the associated Switch.
- For Request that are not associated with In Network Collectives (INC), the Switches shall be stateless (i.e., Switches do not track Requests nor Responses once sent) and shall not be expected to implement any timeout detection. Accelerators shall implement timers, and timeout detection for these Requests
- Switches shall be stateful (i.e. issue and track Requests and Responses) for INC Requests replicated by the Switch for Collective Primitives (ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMulticast) and for INC Requests initiated by the Switch for Block Collectives (BlockRead and BlockWriteFull). Switches shall implement timers and timeout detection for these Requests.
- Since UALink is a network that uses load-store operations (memory-semantic operations), it shall be acceptable for applications to be terminated and restarted when there is a Link Down error, a Switch error or an Accelerator error.
- UALink Data payloads shall be protected with parity or ECC and shall detect data errors that are uncorrectable. On such a detection, that data shall be marked as "poisoned".
- The UALink control path shall be protected and errors shall be isolated to a given port where required (such as Link Down errors – to avoid propagating an error outside the virtual Pod), but are otherwise handled at a coarser grain (i.e. at a given station or possibly all links on a Switch or an Accelerator).
- Component errors on a Switch Platform which may contain one or more Switches, CPU, FPGA, etc., may impact all the Switches in the Switch Platform and require a reset of the Switch Platform. However, Accelerators and other Switches in the Pod shall be able to recover/restart and continue to operate.

- The recovery process from a RAS error shall be controlled by fault management software and firmware running on BMC and host processors. Details of the fault management software may be found in the separate *Ultra Accelerator Link Manageability Specification*.

This specification attempts to simplify the hardware requirements for Switches while placing more responsibility for RAS error handling on Accelerators.

3.1.1 End to End Data Protection

End to End protection of control and data buses shall be provided in UALink from a Source Accelerator to the Destination Accelerator. The protection mechanisms used by various components along this path does not need to be the same. However, an explicit overlap on the protection mechanisms at abutment points between components where the protection mechanism changes shall be required. In other words, the protection mechanism in a prior component along the path shall continue to be checked until the protection mechanism in the subsequent component is established without error on the prior component. This is conceptually shown in Figure 3-1.

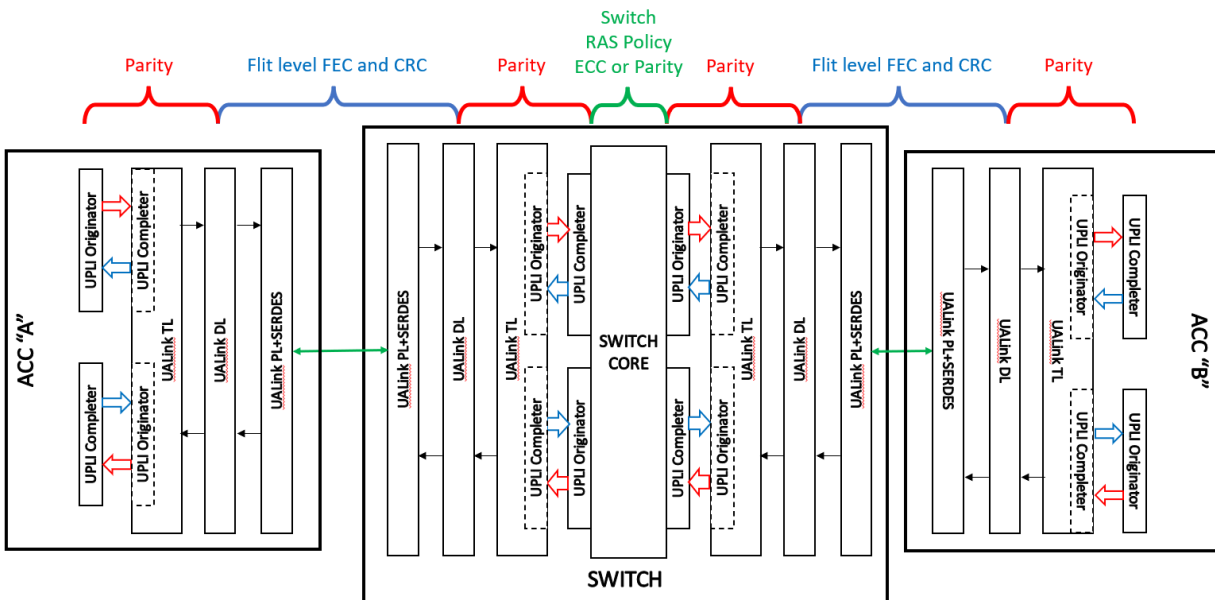


Figure 3-1 UALink End to End Data Protection

The UALink Protocol Level Interfaces shall be protected by even parity bits (the number of set bits in a protected group including the parity bit itself is even). Each channel within a UALink Protocol Level Interface shall have a separate set of parity signals for the individual channel. Each channel that has a *Vld signal shall protect the *Vld signal with a single parity bit. These parity bits shall be checked every cycle to detect extraneous or missing *Vld assertions.

All other parity bits in the channel (not including the Credit Return Interface) shall be checked in cycles where *Vld is asserted. For channels with data or byte enables, those fields (OrigData OrigDataByteEn, RdRspData) shall be protected with distinct parity bits to allow for the detection of data errors and poisoning of Beats by setting RdRspDataError or OrigDataError fields in the respective channels.

Each channel shall have one parity bit for the control information, except the Request Channel which shall have a separate bit for the Request address and one parity bit for the other control fields.

Each channel shall have a Credit Return Interface consisting of four *CreditVld bits and some control fields. The *CreditVld bits shall be protected by a single *CreditVldParity signal that shall be checked every cycle to detect extraneous and missing *CreditVld assertions. The control fields in the Credit Return Interface shall have a single parity bit that shall be checked in cycles that *CreditVld is asserted.

The DL and PHY layers shall protect DL and TL Flits through FEC (Forward Error Correction) and CRC (Cyclic Redundancy Checks). The UALink Protocol Level Interfaces shall be protected by Parity. However, within the Switch Core, a distinct Error Protection mechanism (possibly utilizing ECC) may be used. If the Switch Core error protection scheme is unable to distinguish between errors involving the data and control fields, the error shall be assumed to have occurred in a control field.

3.1.2 RAS Error Types

UALink RAS errors are divided into several categories:

1. UPLI Control Errors
2. UPLI Data Errors
3. UPLI Protocol Errors
4. Switch Core Control Errors
5. Switch Core Data Errors
6. Link Down Error.

The UALink Protocol Level Interface errors (Control, Data, Protocol) shall be defined as errors that occur at the UALink Protocol Level Interfaces on the Accelerators or on the UALink Protocol Level Interfaces on a Switch. The Link Down error shall be defined as the error that occurs when a UALink Link is disconnected or takes some other failure that renders it inoperable.

A UPLI Control Error shall be defined as a parity error involving the ReqVldParity, RdRspVldParity, WrRspVldParity, or OrigDataVldParity signals (i.e. the parity check signals for the one bit *Vld signal in each channel), a parity error involving the ReqParity, ReqAddrParity, RdRspParity, WrRspParity, or OrigDataFieldsParity signals (i.e. the parity check signals for the “control” information in each channel), a parity error involving the ReqCreditVldParity, RdRspCreditVldParity, WrRspCreditVldParity, OrigDataCreditVldParity signals (i.e. the parity check signals for the four bit *CreditVld signals), or a parity error involving the ReqCreditParity, RdRspCreditParity, WrRspCreditParity, OrigDataCreditParity signals (i.e. the parity check signals for the “information” fields in the Credit return interface).

A UPLI Data Error shall be defined as a parity error involving the OrigDataParity, OrigDataByteEnParity, RdRspDataParity signals (i.e. the parity bits protecting the Read Response Channel data field, the Originator Data Channel data field, or the byte enables for the Originator Data Channel).

A UPLI Protocol Error shall be defined as an error where the UPLI protocol is not followed and no UPLI Interface Control or UPLI Data Error occurs. As an example, a Request Channel Beat with a ReqAddr and ReqLen field that calls for a transaction that crosses a 256-byte boundary is a UPLI Protocol Error. Many other possible UPLI Protocol Errors exist and the Switch is shall not be required to check for any given protocol error.

UPLI errors (Control/Data/Protocol) shall be checked at the receiving end of a given UPLI Channel.

A Switch Core Control error shall be defined as an error occurring on control fields in a beat or beats or an error occurring on the data fields (OrigData, RdRspData, OrigDataByteEn) when the Switch error protection mechanism cannot isolate that error to the data fields (e.g. an error on a

data field where a Switch ECC protection scheme that mixes data and control fields in a way that an error in the data field is indistinguishable from an error in a control field).

A Switch Core Data error shall be defined as an error occurring on data fields that the Switch can isolate to the data fields (OrigData, RdRspData, OrigDataByteEn) in one or more Beats.

3.1.3 RAS Error Handling Mechanisms

There are three primary RAS error handling mechanisms:

1. A TL Drop Mode implemented at the TL's on both the Switch and the Accelerator.
2. An Originator Drop Mode implemented at the UPLI Originators and a Completer Drop Mode implemented at the UPLI Completers in the Accelerator and UPLI Completers in the Switch that are not in the TLs.
3. An Isolation Mode implemented at the UPLI Originators in the Accelerators.

An Isolation Mode implemented at the Switch Egress Port Originators in the Switch. Generally speaking, any Drop Mode not in the TL shall cause all traffic for all channels for all ports at the Originator or Completer to be dropped (implementations may choose to limit drop mode to the Channel with the error and/or the port with the error). Drop mode for the TL shall cause all traffic on all channels for either a given port or all ports at both the Completer and the Originator in the TL to be dropped. Whether all ports are dropped or a single port is dropped shall depend on whether the error is a Control Error (generally all ports, but an implementation may choose to only drop the affected port) or a Link Down Error (single port).

Isolation Mode shall be a distinct mechanism from Drop Mode that shall be implemented at the UPLI Originators in the Accelerators and at the Egress UPLI Originators in the Switch. These UPLI Originators on the Accelerators shall be stateful and maintain a history of all the Requests that have been issued by the Originator or that are queued to be issued and these UPLI Originators on the Switch Egress Port Originators for the INC Related Requests (BlockRead, BlockWriteFull, ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMulticast) that have been issued or that are queued to be issued. This state shall be maintained for each Request until the Request has received all Response Beats associated with the Request. Within this state, each Request shall have a Watch Dog Timer that monitors the amount of time the Request has been outstanding.

When a programmable time limit is reached, the Watch Dog Timer expires and shall cause the UPLI Originator to enter Isolation Mode. Isolation Mode shall block all outbound Request and Originator data traffic for an appropriate set of ports in the Originator and shall discard any inbound Response traffic on those Ports. The appropriate set of Ports shall be all ports for an Originator on the Station and on the Switch, the appropriate set of Ports shall be the Port associated with Request that timed out (on a Switch, if multiple Requests time out on different Ports, each Port is placed in Isolation in succession as the timeouts occur). In addition, the Originator shall provide "dummy" Completion Timeout Responses for all outstanding Requests to the appropriate Port(s). On the Accelerator, because the "dummy" Completion Time Out Responses will quickly lead to the termination of processing on the Accelerator, there is no meaningful advantage to perform Isolation Mode to anything less than all ports in the station. On the Switch, the Isolation Mode shall be implemented on a per-Port basis in order to prevent a timeout in one virtual Pod from spreading to a different Virtual Pod.

Isolation Mode and Drop Mode at the UPLI Originator on the Accelerator and at a Switch Egress Port shall be distinct mechanisms though both modes do drop traffic. The UPLI Originator on the Accelerator or at the Switch Egress Port can be in neither mode, either mode, or both modes.

The following actions occur in Isolation Mode:

1. The UPLI Originator shall stop issuing new Requests to the TL (either outstanding Requests in the UPLI Originator or newly arriving Requests at the UPLI Originator) for all appropriate ports.
2. The UPLI Originator shall provide “dummy” Completion Timeout Responses to all outstanding or newly arriving Requests for all appropriate ports in the Originator which shall complete those Requests in the Completer.
3. The UPLI Originator shall discard any Responses, for all appropriate ports, received from the UALink TL (the “dummy” Completion Timeout Responses replace the Responses from the Completers).
4. The UPLI Originator shall continue to accept returned Credits and shall return any outstanding Credits as normal.
5. Implementations may terminate unfinished bursts (Read Responses or Originator Data) due to entering Isolation mode.

The “dummy” Completion Time Out Response to the Originator Devices shall cause that Originator Device to transition to an “idle” state. This allows the Originator Device to be able to be used again once management software takes appropriate clean up actions and begins executing programs after the error, as specified in the *Ultra Accelerator Link Manageability Specification*.

In contrast to Isolation Mode at an Accelerator, the Drop Mode implemented at a UALink TL (whether at the Switch or Accelerator) can apply to an individual port, a subset of the ports, or all the ports in the station depending on the error that invoked UALink TL Drop Mode and which UALink TL (Switch or Accelerator) the error occurred at. UALink TL Drop Mode, whether for a given port or all ports in the UALink TL, shall always apply to all channels for the port or ports affected.

For example, for a Link Down error detected on a Switch, the UALink TL is only allowed to enter UALink TL Drop Mode for the port that whose Link entered a Link Down state. This is to prevent a Link Down error from propagating to other Virtual Pods (if a single Link Down at a station on the Switch were to put all the ports on that Station into UALink TL Drop Mode, that could impact other Virtual Pods associated with the other links in the Station). More than one Link taking a Link Down error in a single station results in a subset of Links in a Station being in UALink TL Drop Mode.

If, however, instead of a Link Down, a UPLI Control error is detected at a UALink TL on the Switch, the UALink TL shall be placed in Drop Mode for all ports. This is because it is impossible, in general, to determine which port is involved in a UPLI Control Error from the signals present in the Beat that took an error: either the parity signal protecting the *Vld is corrupt and the entire beat including port information is suspect or the *Vld is correct, but the parity field for the other control fields took an error and the port information is again suspect (with the sole exception of the case of a parity error on the ReqAddr field which is not worth creating an exception for).

While it is conceptually possible to infer the port that took an error from the TDM phase in some cases, this technique will not work for errors on the Credit Return Interfaces that are not managed by TDM. These techniques are not prohibited, but they are not recommended because UPLI Control Errors will typically be handled at a coarser grain by the recovery management software (i.e. UPLI Control Errors on the Switch will typically be handled at at least a Station level if not by resetting the entire Switch Platform, therefore isolating the Drop Mode to a port for these errors provides no meaningful advantage).

When a TL enters into TL Drop Mode at a given port, the following actions occur:

1. All traffic (inbound or outbound) for that port between the TL and DL shall be dropped.
2. All UPLI traffic inbound to the TL -- the Read Response and Write Response Channels received at the Originator in the TL and the Request and OrigData Channels received at the Completer in the TL-- shall be dropped.
3. All UPLI traffic outbound from the UALink TL -- the Request and Originator Data Channels driven at the TL Originator and the Read Response and Write Response Channels driven at the TL Completer-- shall be dropped.
4. The UPLI Originator and the UPLI Completer in the TL may continue to accept returned Credits and may return any outstanding Credits as normal where possible. Certain errors that cause drop mode corrupt information necessary to return credits accurately.
5. Implementations may terminate outbound unfinished bursts (Read Responses or Originator Data) due to entering TL Drop mode.

If entering TL Drop Mode due to a UPLI Control Error, the TL shall ensure that corruption does not occur in the system by preventing subsequent Beats for the given channel and port from being delivered.

UPLI Originator Drop Mode and UPLI Completer Drop Mode are similar to TL Drop Mode but shall occur at UPLI Completers and Originators that are not part of a UALink TL and shall be entered as a result of a UPLI Control Error. The UPLI Originator/UPLI Completer Drop Modes can also be limited to a specific port (implementations can also choose to limit the drop mode to the specific Channel that took the error). However, as explained above, it is not possible to always determine the port that took the UPLI Control Error and therefore these drop modes are typically implemented to affect all ports associated with the Originator or Completer (in contrast, the Link Down events that cause the TL Drop Mode to be invoked for a specific port shall be limited to the specific port because the involved port can be trivially determined by which port took the error).

When a UPLI Originator or UPLI Completer not on a UALink TL enters Originator Drop Mode or Completer Drop Mode, the following actions shall occur:

1. All subsequent traffic Beats for the Completer or Originator for the channel and port that detected the UPLI Control Error shall be dropped (if the channel is in the middle of an incomplete burst, that burst shall be terminated).
2. All subsequent traffic Beats for the other channels and all other ports for the Completer or the Originator shall also be dropped (unless the implementation chooses to limit the drop mode to the port and/or channel detecting the error), however implementations are permitted to complete outstanding bursts on these channels.
3. The UPLI Originator or the UPLI Completer may continue to accept returned Credits and may return any outstanding Credits as normal where possible. Certain errors that cause drop mode corrupt information necessary to return credits accurately.

3.1.4 UALink RAS Error Handling

The following sections provides more detailed descriptions of the error handling sequences for the various classes of UALink RAS errors. The table below provides an overview or errors and actions:

	Errors and Actions		
	Link Down	Data Error	Control & Protocol Errors
Switch UALink	- The Switch shall enter Drop Mode at the affected Switch TL for the affected port only. - The TL shall drop traffic in both directions on all channels for the affected port. - The Switch shall notify firmware which will log the error. - All ports on the Station except the port attached to the dropped link shall continue to function. - Returning the Link to an active state shall not have any impact to other ports on the Station. - All UPLI channels attached to the Switch TL shall remain active and maintain their credits throughout the link going down and being returned to an active state.	- The Data Error signal shall be asserted to indicate poisoned or corrupted data for the corrupted beat. - Good parity shall be generated for the Beat and the data shall be forwarded with the asserted Data Error Signal. - The Switch Core shall Notify firmware which will log the error.	- Protocol and Control errors shall be handled similarly at the Switch. - Drop Mode shall be entered at the Completer, Originator, or TL that detected the error for all ports. - Traffic shall be dropped in both directions on all channels for the affected ports. - The Switch shall notify firmware which will log the error. - Recovery may require a full reset of the Station or possibly the full Switch to resume operation.
Switch Core	- No action needed in the switch core. - Handled by the Switch TL, Accelerator TL, and the Accelerator Originator.		- The Switch shall enter Drop Mode at the egress Originator for all ports. - The Originator shall drop traffic in both directions on all channels for the affected ports. - Log error and notify firmware. - Protocol and Control Errors detected within a switch Core may impact the entire Switch and all virtual Pods, however implementations may choose to reduce the impact, where possible, to a given Station or Stations. - The Switch shall go through a full Reset to resume operation.
Accelerator	- The affected Accelerator TL shall enter Drop Mode for all ports in the TL (station). - Drop traffic in both directions on all channels for the affected ports. - The affected TL shall issue an ISOLATE Write Response to Originator to cause Originator to enter Isolation Mode. - The affected TL shall notify firmware which will log the error. - All UPLI channels attached to the Accelerator TL shall remain active and maintain their credits throughout the link going down and being returned to an active state. - Firmware may choose to reduce the number of active Links before resuming the applications on the Accelerator.		- Protocol and Control Errors at the Accelerator should be handled similarly. - The Originator or Completer that detected the Error shall Enter Drop Mode at the Originator or Completer. - Traffic shall be dropped in both directions on all channels for the affected ports. - The Switch shall notify firmware which will log the error. - Protocol and Control Errors are low probability event and implementations are expected to impact the entire Accelerator for these errors. However, implementations may be able to isolate and recover errors more precisely. - The level of Reset required to resume operation is implementation specific.

Firmware handling for error log notifications and software interfaces for Station, Accelerator, and Switch resets are specified in the separate *Ultra Accelerator Link Manageability Specification*.

3.1.4.1 UPLI Data Error Processing

A UPLI Data Error (a parity error in the OrigData, RdRspData, or OrigDataByteEn fields) is handled by setting the OrigDataError or RdRspDataError field to indicate the beat has "poisoned" data, replacing the bad parity with good parity, and letting the beat continue through the system to deliver it to the destination Originator Device (Reads) or Completer Device (Writes). Implementations may choose to implement a mode where Data Errors are treated as Control Errors.

3.1.4.2 UPLI Control Error Processing

Accelerator UPLI Control Error processing

The processing of a UPLI Control Error shall depend on whether the error is taken on the Accelerator or on the Switch and at what interface boundary. The next figure illustrates the processing involved in handling a UPLI Control Error that occurs on an Accelerator UALink Protocol Level Interface on an Accelerator.

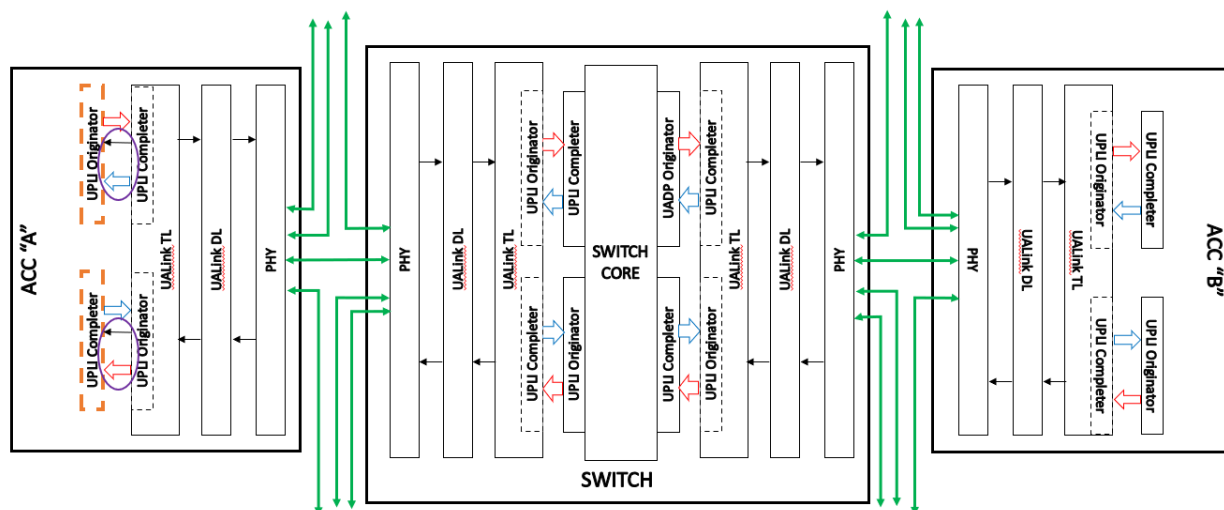


Figure 3-2 UPLI Control Error detected at an Originator or Completer not on a UALink TL on an Accelerators

In Figure 3-2 above, the processing for a UPLI Control Error detected at a UPLI Originator or UPLI Completer in an Accelerator is shown (errors are detected on both the received Channels and the received Credit Return Interfaces). Processing these Control Errors involves dropping the bad Beat or returned Credit, placing the detecting UPLI Originator or UPLI Completer into Drop Mode for all channels for all ports (implementations can chose to limit Drop Mode to the specific channel with the error), and signaling firmware by an implementation specific means that the error has occurred. Implementations may choose, either through implementation specific hardware means or in the RAS recovery sequence in firmware to place other TLs, Originators, or Completers in this or other Stations into Drop Mode and to possibly take various links into a link down state (up to and including all Stations and links on the Accelerator). It is acceptable to place all links on an Accelerator into a link down state on a Control Error because these links can only communicate with other Accelerators in the same Virtual Pod.

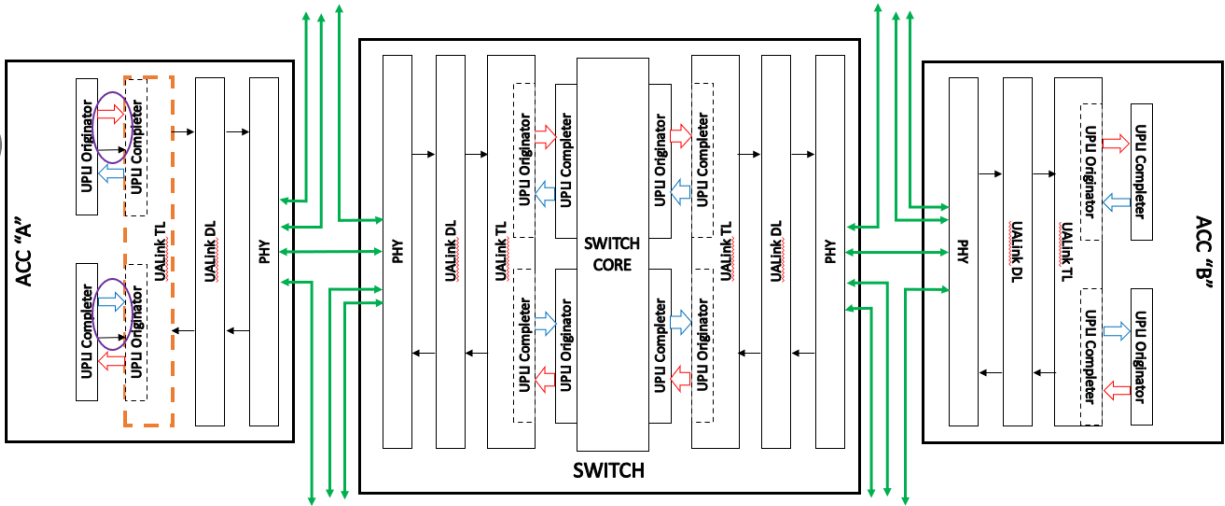


Figure 3-3 UPLI Control Error detected at a UALink TL on an Accelerator

As shown in Figure 3-3 above, the processing of a UPLI Control Error detected at the UALink TL (either at the Originator or Completer) on an Accelerator involves dropping the bad Beat or returned Credit, placing the TL into Drop Mode for all ports and all Channels on both UPLI interfaces (implementations can choose to apply Drop Mode to only one port) and signaling the firmware by an implementation specific means of the error. Implementations may choose, either through implementation specific hardware means or in the RAS recovery sequence in firmware, to place other TLs, Originators, or Completers in this or other Stations into Drop Mode and to possibly take various links into a link down state (up to and including all Stations and links on the Accelerator). Like Control Errors detected on the Accelerator Completers and Originators, it is acceptable to place all links on the Accelerator into a link down state because these Accelerator links can only communicate with Accelerators in the same Virtual Pod. The TL being in Drop Mode will block Responses to the UPLI Originators in the Accelerators in the Virtual Pod causing them to eventually enter Isolation Mode.

Switch UPLI Control Errors on the Switch UALink Protocol Level Interfaces

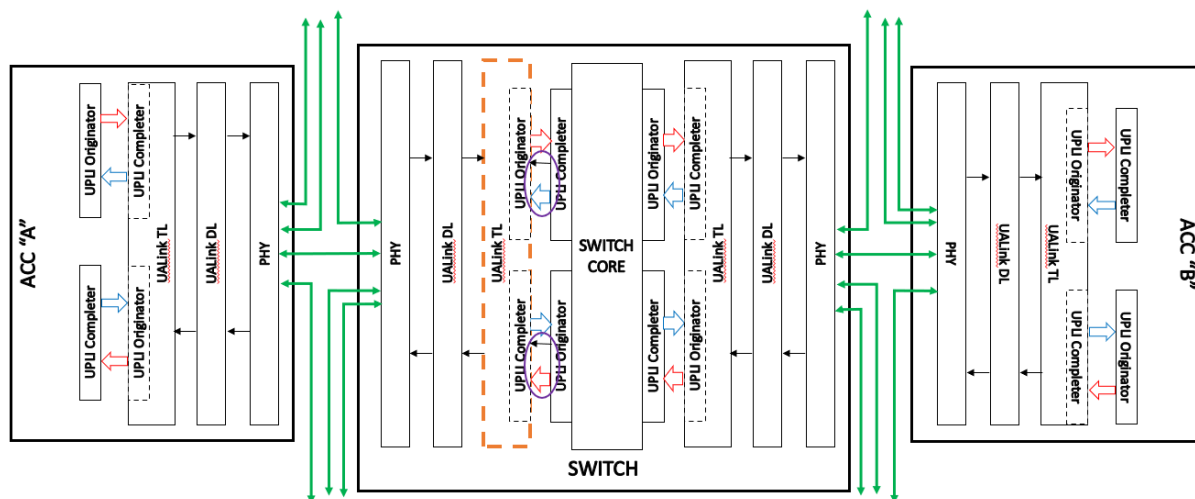


Figure 3-4 UPLI Interface Control Error detected at the UALink TL on the Switch

As shown in Figure 3-4 above, the processing of a UPLI Control Error detected at the UALink TL (either at the Originator or Completer) on the Switch is the same as processing the same error on the Accelerator: the bad Beat is dropped and the TL detecting the error enters Drop Mode for all channels, on all ports, for both UPLI interfaces (implementations may limit this to a port on both interfaces). Implementations may also, either through implementation specific hardware means or in the RAS recovery sequence in firmware, place other TLs, Originators, or Completers in this or other Stations in the Switch into Drop Mode, and to possibly take various links into a link down state (up to and including all Stations and links on the Switch). This is possible because this is a UPLI Control Error and not a Link Down error. For Link Down errors, only the involved port may be affected. The TL being in Drop Mode will block Responses to the UPLI Originators in the Accelerators in the Virtual Pod causing them to eventually enter Isolation Mode.

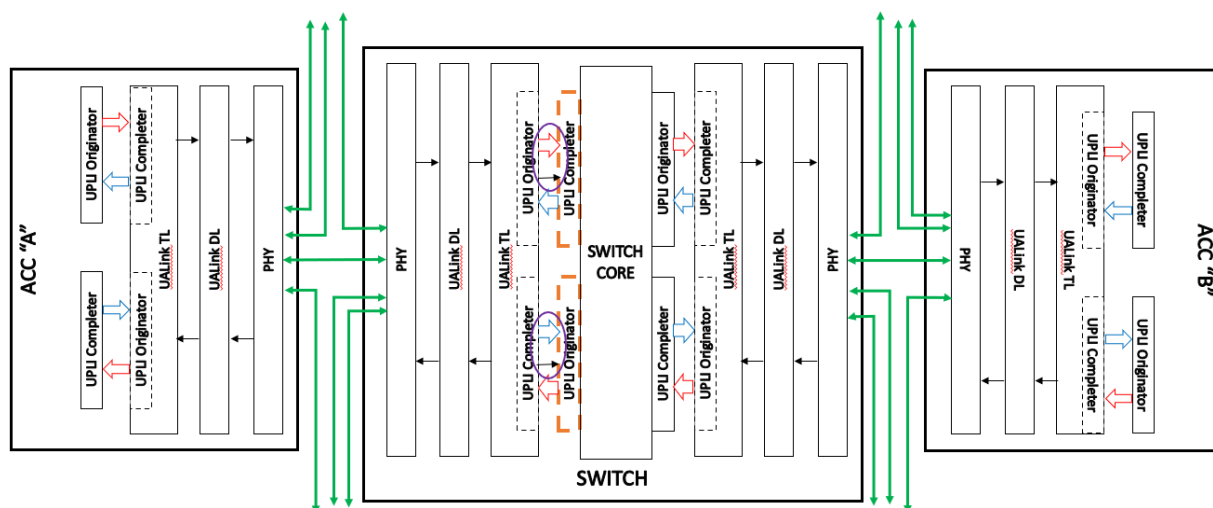


Figure 3-5 UPLI Control Error detected at the UPLI Completer on a Switch

Figure 3-5 above illustrates the processing for a UPLI Control Error detected at the UPLI Completer or UPLI Originator at the Switch Core (errors are detected on both the received Channels and the

received Credit Return Interfaces). The processing involves dropping the bad Beat or returned Credit, placing the detecting UPLI Completer or UPLI Originator into Drop Mode for all ports (implementations can choose to limit Drop Mode to the specific channel with the error), and signaling firmware by an implementation specific means that the error has occurred.

Implementations may choose, either through implementation specific hardware means or in the RAS recovery sequence in firmware to place other TLs, Originators, or Completers in this or other Stations into Drop Mode and to possibly take various links into a link down state (up to and including all Stations and links on the Switch). It is acceptable to place all links on a Switch into a link down state on a Control Error because the Switch is treated as a single point of failure for these errors.

Switch UPLI Control Errors within the Switch Core

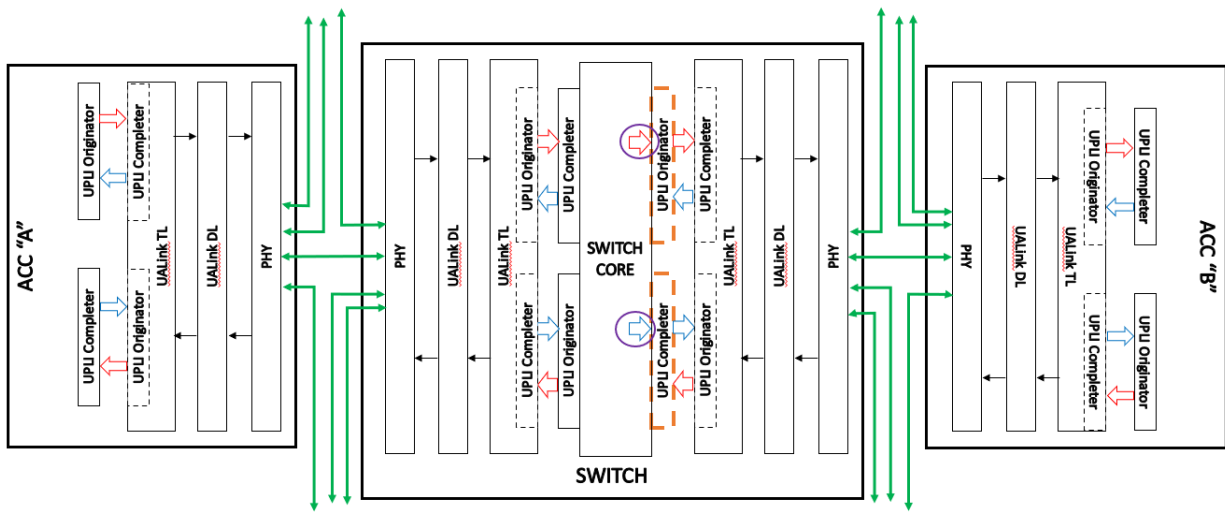


Figure 3-6 UPLI Control Error detected within the Switch Core at the UPLI Originator

As shown in Figure 3-6 above, the processing for a UALink Interface Control Error detected at Switch egress at the UPLI Originator drops the bad Beat, causes the UPLI Originator to enter Drop Mode for at least the channel on which the error was detected, and signals firmware by an implementation specific means.

3.1.4.3 Link Down Error Processing

The following section describes the sequencing to process a Link Down Error. In UALink, a Link Down Error is intended to be recoverable without resetting the Accelerator or Switch and shall not impact Virtual Pods not utilizing the affected Link.

The various applications running on the Accelerator shall be terminated (as UALink is a load/store architecture, application termination on a Link Down Error is unavoidable), rolled back to a checkpoint, and restarted.

The sequencing for a Link Down Error is described below:

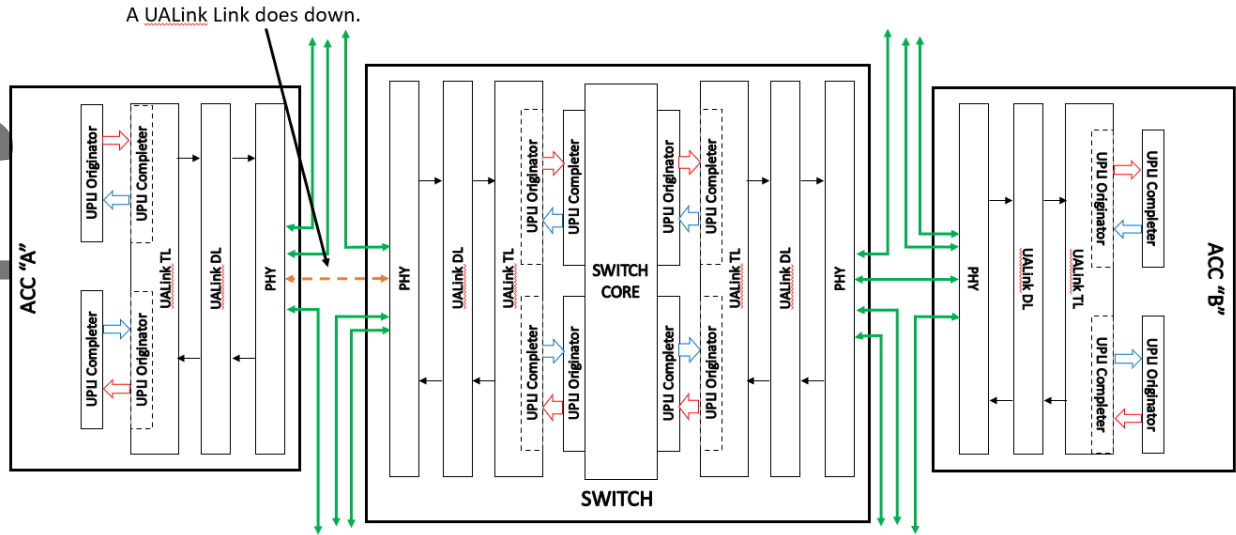


Figure 3-7 UALink Link goes down

As shown in Figure 3-7 above, the process begins with a UALink Link going down.

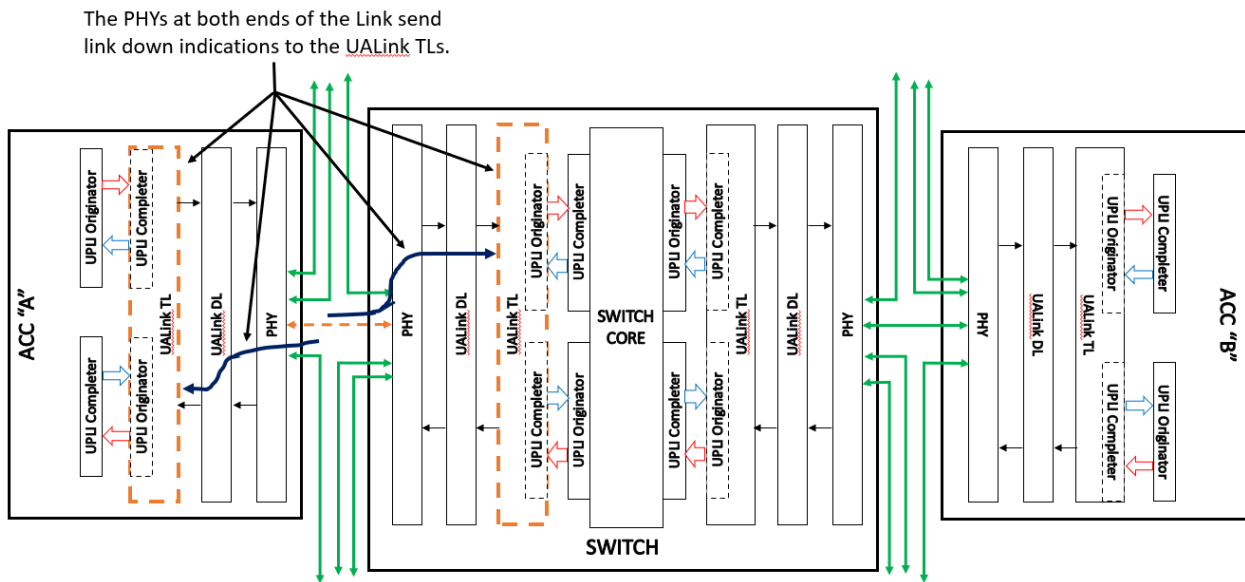


Figure 3-8 PHYs Inform UALink TMs in Accelerator and Switch Which Enter Drop Mode

As shown in Figure 3-8 above, the PHYs at both ends of the down UALink Link shall determine the link is down and communicate to the two UALink TMs indicating which port has gone into a Link Down state.

The UALink TL in the Accelerator shall go in Drop Mode for all ports. The UPLI Originator will be going into Isolation Mode in the next step, and therefore it is not useful in this case to enter Drop Mode in the UALink TL on the Accelerator only for the port associated with the dropped link.

On the Switch, however, the UALink TL shall go into Drop Mode only for the affected port. Going into Drop Mode for any other ports could impact Accelerators not in the Virtual Pods which shall not be permitted for a Link Down Error.

A credited Write Response is issued from UALink TL to the UADP Originator to put it into Isolation Mode.

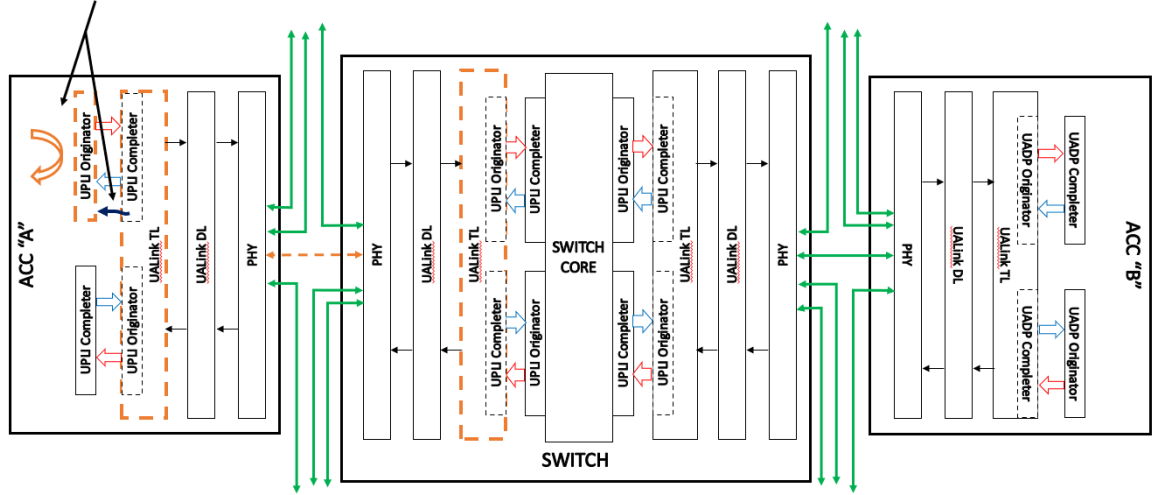
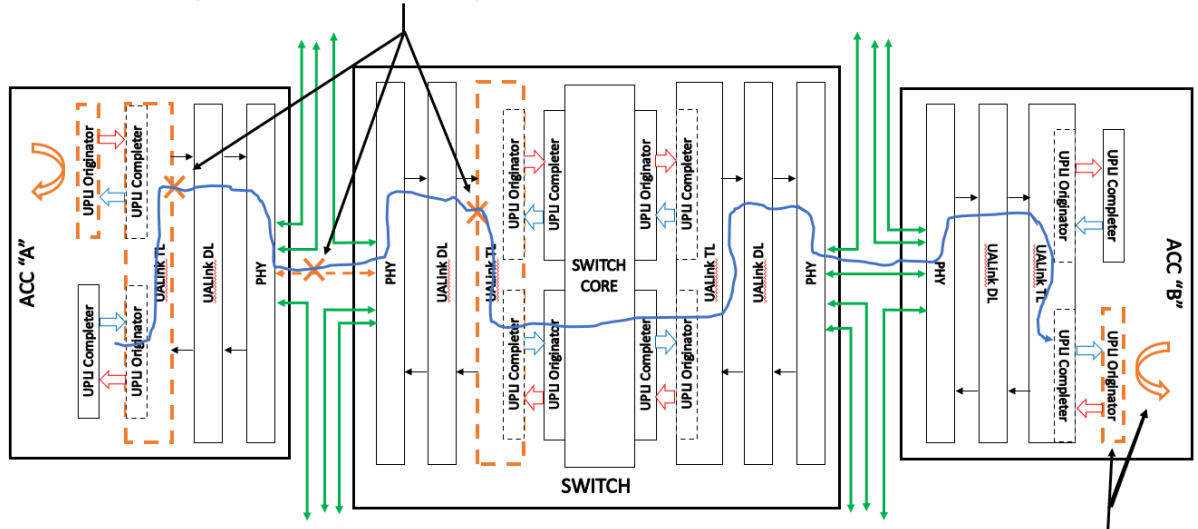


Figure 3-9 Placing the Accelerator UPLI Originator Into Isolation Mode

As shown above in Figure 3-9, the UALink TL shall issue a credited Write Response of "Isolate" to the UPLI Originator which shall cause the UPLI Originator to enter Isolation Mode. This shall prevent the Accelerator from creating new traffic for the dropped link (an all links in the Station).

Responses from ACC A to ACC B requests are blocked.



Watch Dog Timers in UADP Originator expire and place the UADP Originator into Isolation Mode.

Figure 3-10 Other Accelerators Time Out

As shown in Figure 3-10 above, once the Accelerator UPLI Originator is in Isolation Mode, the Accelerator UALink TL is in Drop Mode, the link is down, and the UALink TL on the Switch is in Drop Mode for the affected ports, Responses from Accelerator A to Accelerator B's Requests may be blocked. This shall cause the Watch Dog Timers in Accelerator B's UPLI Originator(s) to time out and enter Isolation Mode.

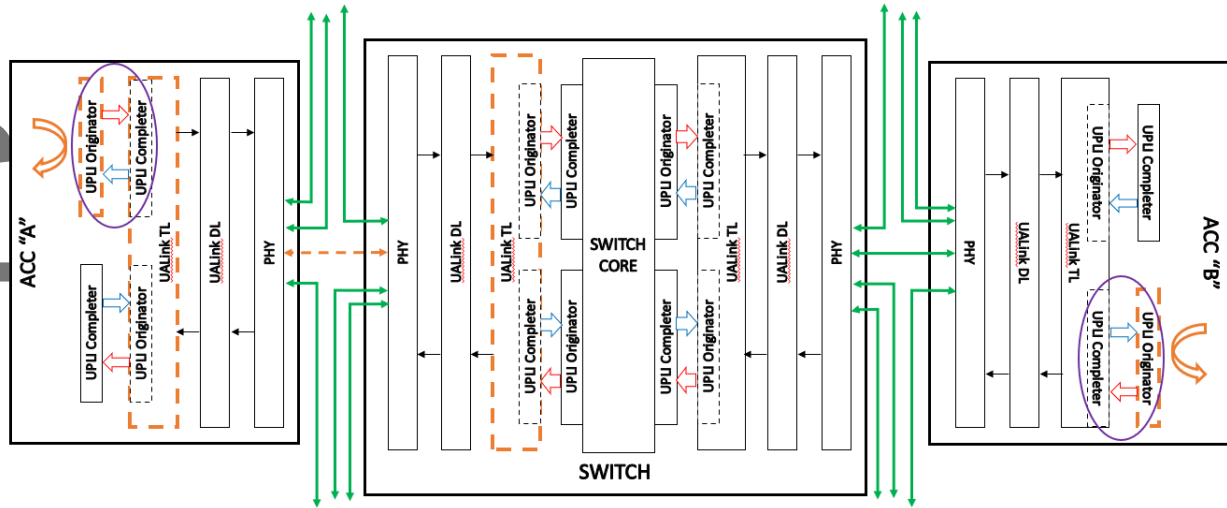


Figure 3-11 Link Down Error Processing Complete

The final state of the system for a Link Down Error is shown above in Figure 3-11. The UPLI Originators in both Accelerators shall be in a “safe” state (both in Isolation Mode) and no new traffic shall be generated by these Accelerators (including the other Accelerators in the Virtual Pod, not shown in the figure, that have also timed out). At this point, system management software can gain control of the Accelerators and recover the system, and redispach applications on the Accelerators. The relevant interfaces for system management software are documented in the separate *Ultra Accelerator Link Manageability Specification*.

3.1.4.4 Isolation Mode Handling at Switch Originators for INC Requests

The following section describes the sequencing for Isolation Mode occurring within a Switch Originator due to an INC Request not receiving a Response. The Sequence below begins (in the

following figure) with the INC Engines (either the Collective Primitive Engine or the Block Collective Engine) issuing a INC Request:

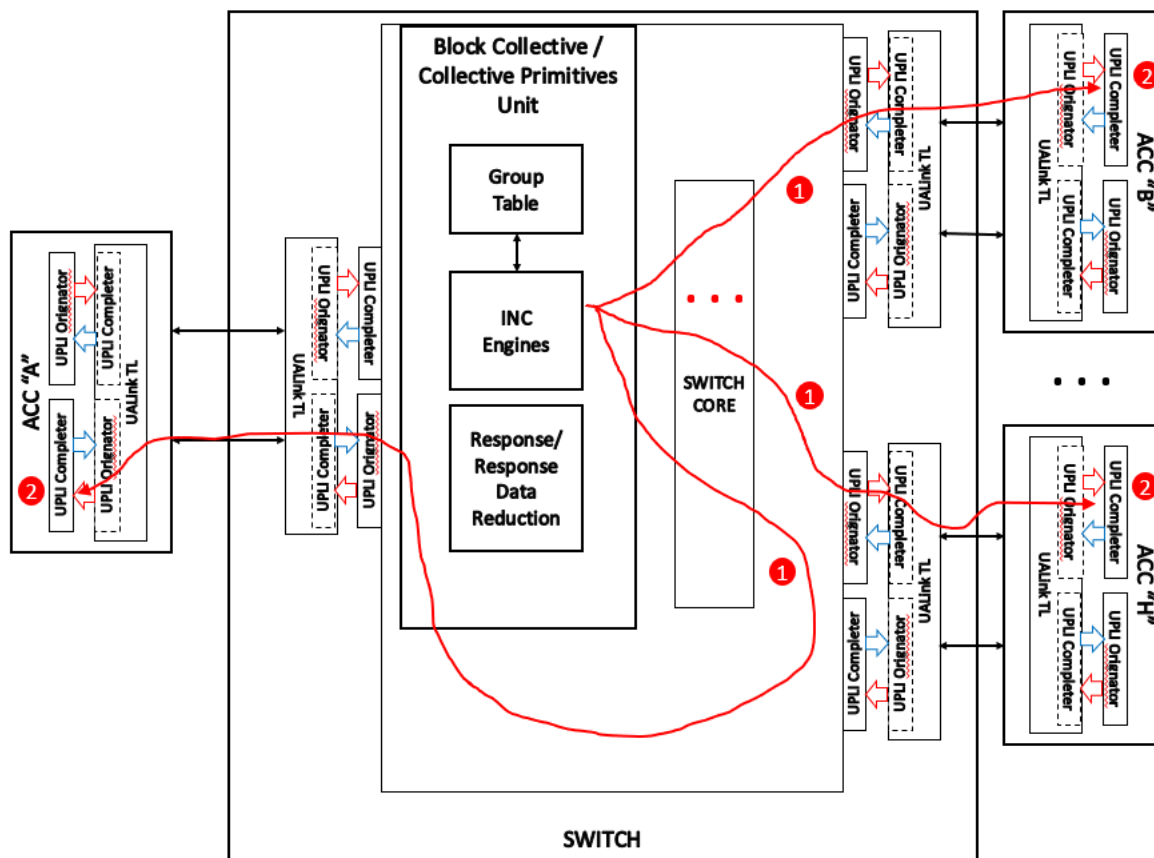


Figure 3-12 Inc Engines issuing INC Requests

As shown above in Figure 3-12, the INC engines issues the INC Requests ① which are received by the UPLI Completers at the Accelerators involved in the Collective ②.

The following figure illustrates the various accelerators in the Collective returning a Response to the Collective Primitive Request with one of the Accelerators taking an error and failing to return a Response.

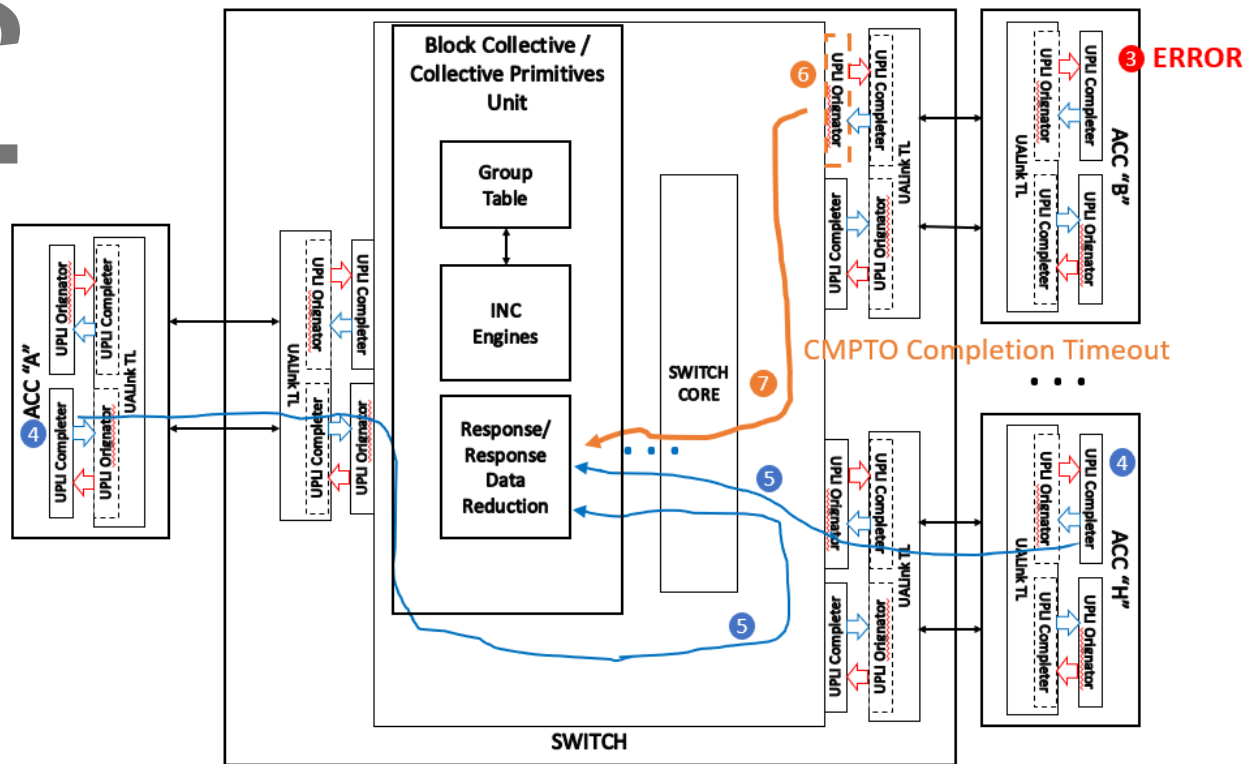


Figure 3-13 Accelerators Responding to INC Request With An Error on One Accelerator

As shown above in Figure 3-13, the Accelerators that do not take an error **4** issue Responses to the Switch which are received at the Response/Response Data Reduction logic **5**. Accelerator “B” takes an error in this example and produces no Response to the Request **3**.

The UPLI Originator in the Switch eventually times out and enters ISOLATION mode ⑥ and then returns a “dummy” Completion Timeout (CMPTO) Response to the Response/Response Data Reduction logic ⑦ to indicate the error and allow all the Responses to return for the INC operation so the tracking structures allocated to this Request will be released.

The following Figure 3-14 illustrates the Switch returning a status indication to the Accelerator after all the Responses are received.

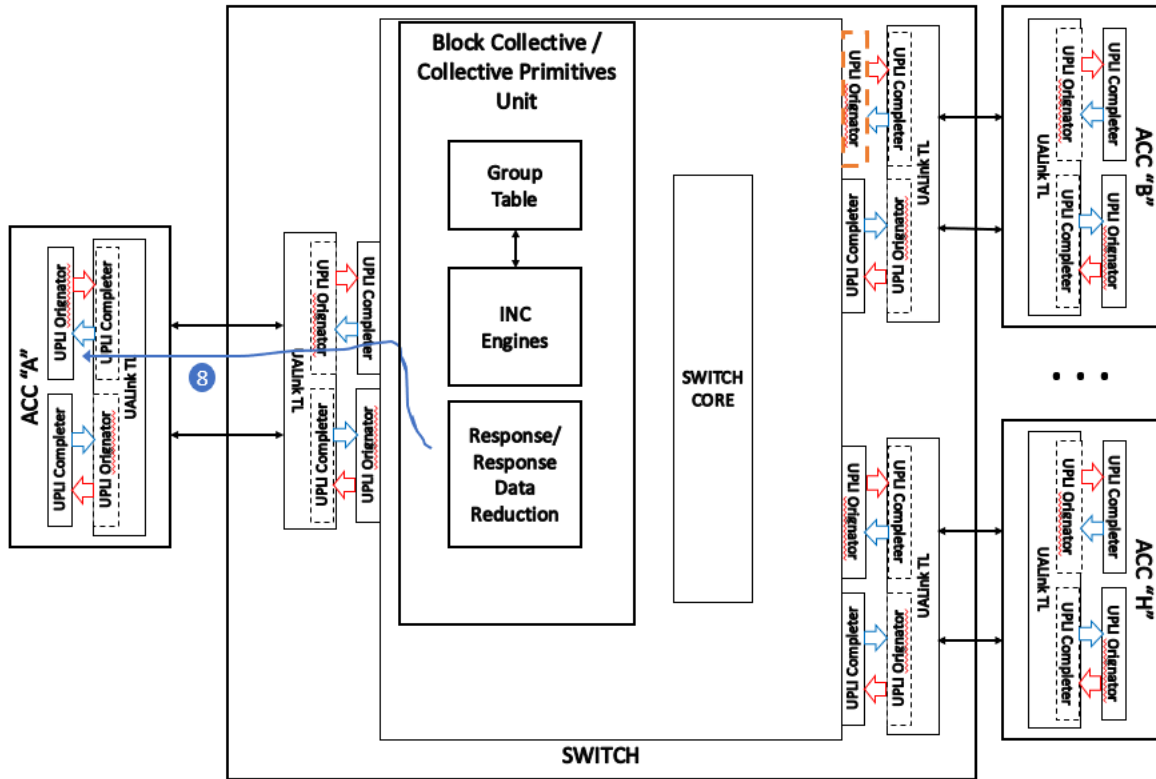


Figure 3-14 Switch Returning Status for INC Operation.

When all the Responses for a Collective Primitive have been received or when all of the Responses for Requests associated with a Block Collective that are going to be issued (the Block Collective may terminate early based on a bad Response) have been received, the Switch will indicate the status of the INC Operation back to the Requesting Accelerator ⑧. For a Collective Primitive the status is returned by a Response, and for a Block Collective, the status is returned by a write to the status buffer.

4 UPLI Interface Reset, Signaling, and Connection

4.1 UPLI Interface Reset

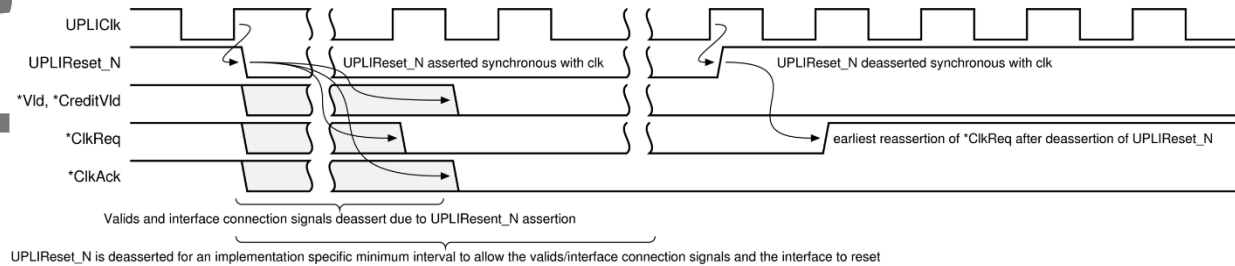


Figure 4-1 UPLI Interface Reset Requirements.

The system reset signal UPLIReset_N shall be a negative-active signal that shall be used to initialize the interface logic. No state information shall be retained across a reset event (an assertion followed by a de-assertion of UPLIReset_N). Any established Credits shall be reset and any outstanding transactions shall be terminated and shall not receive a Response.

The assertion (negative-going edge) of UPLIReset_N and the de-assertion (rising edge) of UPLIReset_N shall be synchronous to the clock: UPLIClk. When the UPLIReset_N signal is first asserted, the *Vld, *CreditVLD and interface control signals *ClkReq and *ClkAck signals shall transition to their inactive values in an implementation specific number of cycles which may vary for each signal, but shall be before the UPLIReset_N signal is de-asserted. The UPLIReset_N signal shall remain asserted for an implementation specific minimum interval to allow the interface to reset and the valid signals and interface connection signals to transition to their de-asserted values before UPLIReset_N is de-asserted.

Any attempt to reconnect the interface by asserting either of the *ClkReq signals (described below) shall not occur until at least one cycle after the de-assertion of UPLIReset_N signal as shown in Figure 4-1 UPLI Interface Reset Requirements.

When a UPLI Interface is reset, the Tx Address Compression Cache associated with the UPLI Completer in the TL associated with that UPLI interface, if present, shall be cleared.

4.2 UPLI Interface Signaling Requirements

When the Source Originator or Completer has not asserted its *Vld signal for a given channel, all of the command and data information on that channel is not valid and shall be ignored. For example, there is no requirement that ReqCmd has a valid encoding when ReqVld is de-asserted or that the parity signals on the OrigData or RdRsp Channels are consistent with the data on those channels when OrigDatVld or RdRspVld, respectively, is de-asserted. The validity of the *CreditVld signals and associated Credit management signals (driven from the receiver Completer or Originator side of the UPLI interface) shall not be impacted by the *Vld signal.

All signals on the UPLI interface shall be synchronous to a common clock (UPLIClk) supplied by system pervasive logic. Source logic switching and signal propagation times shall be expected to be such that there is sufficient time margin to the next rising clock edge to allow simple combinatorial logic on the receiving side of the interface.

4.3 UPLI Interface Control

A UALink Protocol Level Interface shall provide signals that allow the interface to be connected (brought into an active state), either at power up or after a reset of the interface, in an ordered sequence (the Connection Handshake Protocol) that eliminates race conditions that might corrupt the transfer of information across the interface. The Connection Handshake Protocol may be orchestrated by state machines on both sides of the interface or coordinated directly by SOC-level logic and/or management firmware. Both the Originator and Completer shall be powered and actively clocked for the Connection Handshake Protocol to proceed.

In the following diagrams and discussion, "Originator" shall refer to the Originator for the UALink Protocol Level Interface and "Completer" shall refer to the Completer for the UALink Protocol Level Interface. The interface control signals used in the Connection Handshake Protocol shall be the OrigClkReq, OrigClkAck, CompClkReq, and CompClkAck signals.

An Originator shall use the OrigClkReq signal to request the connection of the Originator to Completer Channels and Credit Return Interfaces (the Request Channel, Originator Data Channel, Read Response/Data Credit Return Interface, and Write Response Credit Return Interface). The Completer shall respond with the CompClkAck signal to acknowledge the Originator's request and indicate the Completer's ability to accept information on those Channels and Credit Return Interfaces. Once the Originator is connected to the Completer, the Originator may transfer Credits to the Completer, but the Completer shall not send Beats to the Originator until the Completer to Originator connection is also completed. Finally, the Originator shall not send Beats to the Completer until the Originator has Credits to do so. The Completer to Originator connection shall be completed before the Credits from the Completer can be transferred to the Originator.

The Completer shall use the CompClkReq signal to request the connection of the Completer to Originator Channels and Credit Return Interfaces (the Read Response/Data Channel, Write Response Channel, Request Credit Return Interface, Originator Data Credit Return Interface). The Originator shall respond with the OrigClkAck signal to acknowledge the Completer's request and indicate the Originator's ability to accept information on those Channels and Credit Return Interfaces. Once the Completer is connected to the Originator, the Completer may transfer Credits to the Originator, but the Originator may not send Beats to the Completer until the Originator to Completer connection is also completed. Finally, the Completer shall not send Beats to the Originator until the Completer has Credits to do so. The Originator to Completer connection shall be completed before the Credits from the Originator can be transferred to the Completer.

In order to transfer Beats, the Originator shall be connected to the Completer and the Completer shall be connected to the Originator (and appropriate Credits transferred) implying all four interface control signals: *ClkReq and *ClkAck shall be asserted before Beats can be transferred across the interface.

Either the Completer or the Originator may hold off the completion of the connection by delaying its *ClkAck response or deferring its assertion of *ClkReq. Prior to asserting *ClkAck the acknowledging Completer or Originator shall be prepared to latch valid information presented by the requesting agent on any of the requesting agent's outbound channels. Once an Originator or Completer asserts any of *ClkReq or *ClkAck, it shall not de-assert that signal until UPLIReset_N is asserted. The Completer may wait for the Originator to establish the connection first, or both sides may independently connect. It shall not be legal for the Originator to wait for a Completer to establish the connection first.

The following figures show the UPLI Connection Handshake Protocol with the Originator connecting first (Figure 4-2), the Completer connecting first (Figure 4-3), and finally, both Originator and Completer connecting concurrently (Figure 4-4).

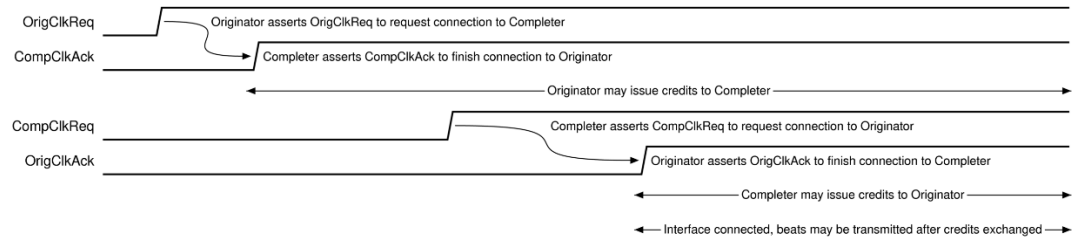


Figure 4-2 UPLI Connection Handshake Protocol – Originator connects first

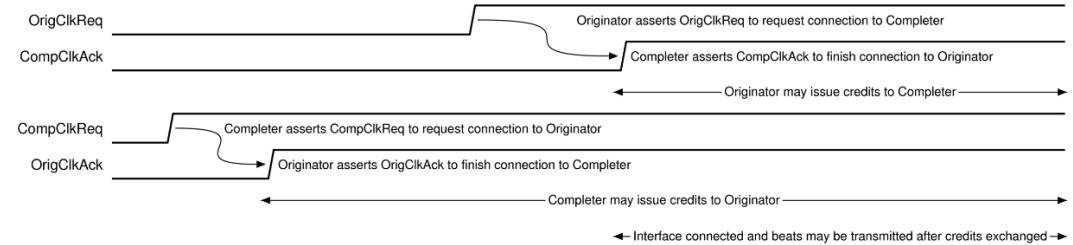


Figure 4-3 UPLI Connection Handshake Protocol – Completer connects first

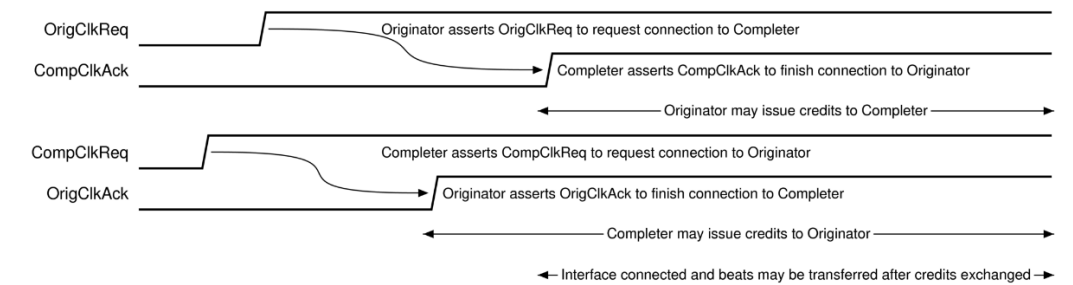


Figure 4-4 UPLI Connection Handshake Protocol – Originator and Completer connecting concurrently

5 Transaction Layer (TL)

The UALink Transaction Layer (TL Layer) is responsible for converting UPLI beats from the inbound (to the TL) channels from the two UPLI interfaces connected to the UALink TL into TL Flits on the outbound or Transmit (Tx) TL Flit Channel. The TL also converts TL Flits received from the inbound or Receive (Rx) TL Flit Channel into UPLI beats for the outbound (from the TL) UPLI channels on the two UPLI interfaces connected to the UALink TL.

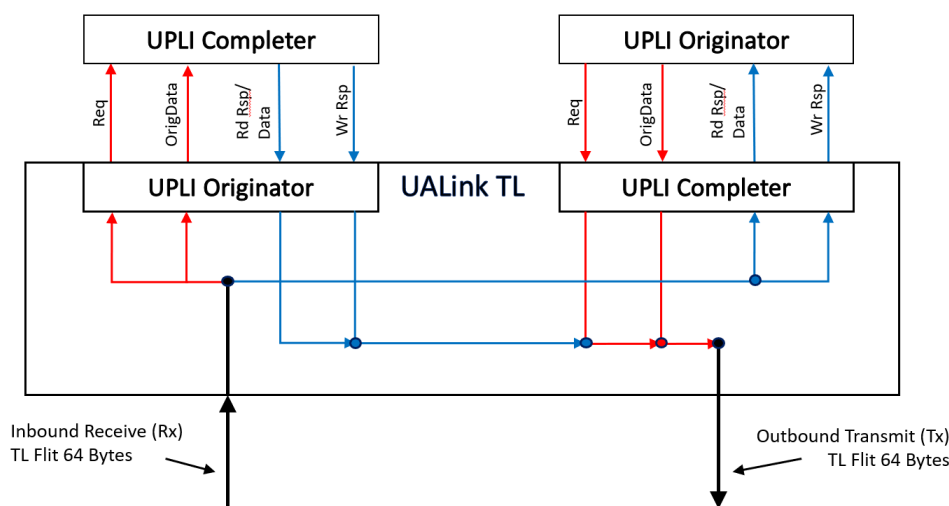


Figure 5-1: TL Flit connections to UPLI interfaces

The figure above, TL Flit connections to UPLI interfaces, schematically illustrates the connections between the various channels for the two UPLI interfaces attached to a UALink TL and their relationship to the Tx and Rx TL Flit Channels. Because of the symmetry of the interfaces, the format for both the Receive and Transmit Flits are the same. The 64-byte Transmit Flit and Receive Flit Channels each encode the information for a UPLI Request Channel, Originator Data Channel, Read Response/Data Channel, and Write Response Channel.

5.1 TL Flit and Half Flit formats

5.1.1 TL Flit and TL Control and Data Half-Flit formats and Sequencing

Each 64-byte TL Flit is divided into an Upper and a Lower 32-byte Half-Flit and the 64-byte TL Flit is also divided into sixteen 4-byte Sectors numbered from the least-significant 4-byte sector in the TL Flit as shown below in Table 5-1: TL Flit organization:

64-byte TL Flit															
Upper TL Half-Flit								Lower TL Half-Flit							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Table 5-1: TL Flit organization

A TL Half-Flit may be a Control Half-Flit, a Data Half-Flit, a Message Half-Flit, or an Authentication Tags (AuthTags) Half-Flit.

The Control Half-Flit shall be used to encode the following information:

- Requests (Reads, Writes, AtomicR, AtomicNR)
- Read Responses (but not the data associated with them – note that an AtomicR Request causes a Read Response)
- Write Responses (Note that an AtomicNR Request causes a Write Response).
- Flow Control/NOP information.

In the Control Half-Flit, Requests can be encoded in 4 or 2 sector fields, the Responses can be encoded in 2 or 1 sector Fields, and the Flow Control (FC) information and the NOP indications are each encoded using one sector. A special control Half-Flit called the NOP Half Flit consists of eight NOP indications. The alignment of Control Half-Flit fields shall be according to the table below:

Field Type	Legal Sector Footprint
4 sector Request	7654 or 3210
2 sector Compressed Request	76 or 54 or 32 or 10
2 sector Response	76 or 54 or 32 or 10
1 sector Compressed Response	7 or 6 or 5 or 4 or 3 or 2 or 1 or 0
Flow Control/NOP Information	7 or 6 or 5 or 4 or 3 or 2 or 1 or 0

Table 5-2: Control Half Flit Field Footprints and Sizes

Within the Control Half-Flit, the various field types, subject to the footprints and sizes indicated above, may be freely intermingled. Any unused sector in the Control Half-Flit shall be a NOP field.

The Flow Control Field shall consist of four multi-bit signals:

- Request CMD: indicates Credits for Requests in TL Control Half-Flits.
- Response CMD: indicates Credits for Read Responses (not the associated data) or Writes Responses.
- Request Data: indicates Credits for 64-byte data buffers to hold data for TL Requests for Write, WriteFull, AtomicR/AtomicNR Operand Data, and UPLI Write Message UPLI Requests.
- Response Data: indicates Credits for 64-byte data buffers to hold data for TL Read Responses.

Each of these signals independently indicates a number of credits being returned and furthermore each signal independently indicates if the credits are Pool Credits or Credits associated with a specific Virtual Channel. While a Control Half-Flit may contain more than one Flow Control (FC) Field, only one Flow Control Field in the Control Half-Flit shall contain a non-zero number of Credits value for any Virtual Channel or Pool Credits for each of the above signals. This is intended to allow the logic updating credit values to logically OR the Credit count values in the various signals from all the FC sectors together when forming an update value for a specific Pool or Virtual Channel Credit type rather than having to logically ADD the differing count values together.

A Data Half-Flit is used to encode the following information:

- Read Response Data (Read and AtomicR operations)
- Write Data (Write, WriteFull, UPLI Write Message, and AtomicNR operations)
- Byte Enables (Write and AtomicR/AtomicNR operations).
- Atomic Operand Data (AtomicR and AtomicNR operations),

A series of 32-byte Data Half-Flits shall be used to encode the data for a Read Response or a Write Request. Read Response Data and Write Data on the UPLI interface are 1, 2, 3 or 4 64-byte beats (64, 128, 196, 256 bytes in total) which are transferred in 2, 4, 6, or 8 32-byte TL Data Half-Flits that convey data in the same order as the UPLI beats, respectively, as shown in the examples below. For Write and Vendor Defined Commands that issue data beats on the UPLI OrigData Channel, a 32-byte Write Data Half-Flit containing the Byte Enables shall be appended to the end of the Data Half-Flits for the Request. For WriteFulls, no such Data Half-Flit shall be appended. A full 32-byte Half-Flit, capable of transferring byte enables for up to 256-byte transfers, shall be used regardless of the data transfer width. The Byte Enables shall be aligned within the 32-byte Half-Flit according to the bytes within the 256-byte memory block that the Write is altering (e.g. the byte enables for a 64-byte Write at address 128 would occupy bytes 16 through 23 in the Data Half-Flit). No effort is made to optimize the Byte Enable overhead for Writes or Vendor Defined Commands. WriteFulls, that do not require Byte Enables, are the overwhelmingly common use case.

Atomic Operands and Byte Enables for both AtomicR and AtomicNR operations are transferred on the UPLI interface in a single 64-byte data beat on the UPLI OrigData Channel which shall then be encoded into three consecutive 32-byte TL Data Half-Flits. The first two 32-byte TL Data Half-Flits shall be the Operand Data and the third TL Data Half Flit shall contain the Byte Enables for the Atomic. The Operands for an Atomic shall be aligned in the two 32-byte TL Data Half-Flits in the same manner as the Operand Data is aligned on the UPLI OrigData Channel. The Byte Enables shall be aligned within the 32-byte Half-Flit according to the bytes within the 256-byte memory block that the Atomic is altering (e.g. the byte enables for a 64-byte Atomic at address 128 would occupy bytes 16 through 23 in the Data Half-Flit). The Read Response Data for an AtomicR shall be transferred as two ascending 32-byte TL Data Half-Flits for both OP1 and OP1/OP2 AtomicR operations.

An Authentication Tags (AuthTags) Half-Flit shall be used to convey up to four 8-byte Authentication Tags, one for each of the, up to total of four, Requests, Read Responses, or Write Responses in a Control Half-Flit when the TL channel is operating in a security mode that enables Authentication or Encryption. When an Authentication Tag is unused in an AuthTags Half-Flit it shall be set to a value of zero. See the Security Section for details on how Authentication and/or Encryption is enabled and how Authentication Tags are generated. A given TL Channel shall either have Authentication enabled for all the traffic within the TL Channel or none of the traffic in the TL Channel.

The determination of whether a given Half Flit is a Control Half-Flit, a Data Half-Flit, or an AuthTags Half-Flit is not indicated within the Half-Flit itself but instead shall be inferred from the sequencing of the Half-Flits on the interface.

In the mode where Authentication is disabled, the first Half-Flit (in the Lower TL Half-Flit) shall be interpreted as a Control Half-Flit. If this Control Half Flit calls for Data Half-Flits (contains a Read Response Field, Write Request Field, UPLI Write Message, Vendor Defined Command that issues data beats on the UPLI OrigData Channel or AtomicR/AtomicNR Request Field: these will be referred to hereafter as "Data Request Fields"), the subsequent Half-Flits shall be interpreted as Data Half-Flits until the appropriate number of Half-Flits have occurred to satisfy the Data Request Fields in the Control Half-Flit (unless the final Data Half-Flit called for occurs in the lower Half-Flit).

The order of the data Half-Flits shall correspond to the order for the Data Request Fields in the Control Half-Flit starting with the low order (lowest numbered sector) Data Request Field and then following though the remaining Data Request Field(s) in ascending order.

If the final Data Half-Flit implied by a Control Half-Flit would occur in a lower Half-Flit in the TL Flit, that final Data Half-Flit shall be “swapped” into the upper Half-Flit in the TL Flit and the lower Half-Flit shall be interpreted as the next Control Half-Flit. This “swapping” causes the non-NOP Control Half-Flit, if present, to always be in the lower Half-Flit of the overall TL Flit.

More generally, a TL Flit may only contain one non-NOP Control Half-Flit and that non-NOP Control Half-Flit shall be in the lower TL Half-Flit. Therefore, if the lower Half-Flit of a TL Flit is a non-NOP Control Half-Flit that does not imply subsequent Data Half-Flits, the upper Flit shall be a NOP Control Half-Flit (when Authentication is disabled). When a NOP Control Half-Flit is required by the TL protocol, as in this case, the NOP Half-Flit is referred to as a MANDATORY NOP Half-Flit. These restrictions on the number and placement of non-NOP Control Half-Flits significantly reduce complexity in the decoder logic and the catch buffer logic (described below in 5.7).

If Authentication is enabled, the TL Half-Flit immediately following a TL Control Half-Flit that contains Requests or Responses shall be an AuthTags Half-Flit containing the Authentication Tags for the Requests and Responses in the TL Control Half-Flit. The Control Half-Flit and its corresponding AuthTag Half-Flit shall always occur in the same TL Flit.

However, in Authentication mode if the final Data Half-Flit implied by the most recent Control Half-Flit is swapped into the upper Half-Flit as described above, the Control Half-Flit in the lower Half-Flit shall only contain Flow Control or NOPs Fields. The upper Half-Flit of this “swapped” Flit is already occupied by the trailing Data Half-Flit and therefore the AuthTags Flit cannot be put there. Any pending Requests or Responses may be placed in the Control Half-Flit in the next (or a subsequent) TL Flit.

Because an AuthTags Half-Flit cannot contain more than four Authentication Tags, in Authentication mode, the non-NOP Control Half-Flit shall be limited to having a total of at most four Requests, Read Responses, and/or Write Responses.

The order of the Authentication Tags in the AuthTags Half-Flit shall follow the order of the Requests and Responses in the control Half-Flit. That is, the low-order AuthTag corresponds to the lower-order Request or Response and so on through the Control Half-Flit and AuthTags Half-Flit. If any Authorization Tags in the AuthTags Half-Flit are not used, they shall be set to zero.

5.1.2 TL Message Flit Format and Sequencing

In addition to the two 32-byte Half-Flits in a TL Flit, a one-bit Message Indicator shall be included for each of the two Half-Flits as shown below (M1 for the Upper TL Half-Flit and M0 for the Lower TL Half-Flit):

64-byte TL Flit																	
Msg	Upper TL Half-Flit 32bytes								Msg	Lower TL Half-Flit 32 bytes							
M1	15	14	13	12	11	10	9	8	M0	7	6	5	4	3	2	1	0

Table 5-3: TL Flit with Message Indicator Bits

When the Message Indicator is b'1', a TL Half-Flit is interpreted as a Message TL Half-Flit as shown below in Table 5-4: Message TL Half-Flit:

Msg	32-byte TL Half-Flit	
Msg Indicator Bit (M1/M0)	31-byte Message Specific Payload	1-byte Msg Type

Table 5-4: Message TL Half-Flit

If the Msg Indication Bit is set to b'1', the low order byte (not sector) of the 32-byte TL Half-Flit shall be interpreted as a Msg Type indication allowing for 256 distinct message types. The remaining 31 bytes in the TL Half Flit are the Message Specific Payload.

When a TL Message Half-Flit occurs in the stream of TL Flits the sequencing of the Half-Flits shall be delayed for any inserted TL Message Half-Flits with two exceptions.

The first exception is when an implementation chooses to have a TL Message Flit in the Lower Half-Flit of the TL Flit that would have otherwise been a Control Half-Flit. In this case, the Control Half-Flit that would otherwise have been in that lower Half-Flit cannot be delayed into the upper Half-Flit. The Upper Half-Flit will either be another TL Message Flit, a MANDATORY NOP TL Half-Flit, or be the final Data-Half Flit from the immediately preceding Control Half-Flit that implied Data Half-Flits that is being swapped into the upper Half-Flit.

The second exception is a "Data Poisoned" TL Message Half-Flit that is used to indicate corrupt Data Half-Flits. As explained below in 5.3, rather than delaying the TL Half-Flits, the Data Poisoned TL Message Half-Flit instead indicates the poisoned Half-Flit by replacing the Data Half-Flit that is corrupted. Data Poisoned TL Message Half-Flit's are only legal in TL Half-Flits that would otherwise have been Data Half-Flits delivering data or Atomic Operands (but not Byte Enables).

When Authentication is enabled, a TL Message Half-Flit shall not replace an AuthTags TL Half-Flit.

5.2 TL Flit Sequencing and Packing Examples

In the Flit Sequence examples shown below, the examples are constructed in a way that the lower TL Half Flit in the TL Flit immediately after the last illustrated TL Flit will be interpreted as a Control Half-Flit (i.e. the examples stop when the next Control Half-Flit would occur).

The following example, Table 5-5, illustrates a single 4-sector Read Request Followed by the Mandatory NOP Half Flit (a Control Half-Flit that consists of only NOPs and is required to be present in the given TL Half-Flit for protocol reasons, as opposed to a Control Half-Flit that also consists only of NOPs, but could have contained Requests, Responses, or Flow Control Fields).

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	MANDATORY NOP								Req0 (Read)				NOP	NOP	NOP	NOP

Table 5-5: An 8-sector Request (Read) followed by a Mandatory NOP Half Flit.

The following example, Table 5-6, illustrates a 4-sector 256-byte WriteFull request. As a WriteFull Request, no byte enables need to be appended to the Data Half-Flits. The last Data Half-Flit is swapped to the upper TL Half-Flit and the next Control Half Flit is placed in the lower TL Half-Flit of the final TL Flit. The Control Half-Flit is shown (arbitrarily) containing a Flow Control (FC) indicator in sector 3.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req0 (WriteFull)				NOP	NOP	NOP	NOP
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req0.Data.7								NOP	NOP	NOP	NO	FC	NOP	NOP	NOP

Table 5-6: A 4 sector Request (WriteFull 256 bytes) followed by a Control Half Flit with Flow Control.

The following example, Table 5-7, illustrates a 4-sector 192-byte Write request. As a Write Request, a byte enable Data Half-Flit is appended to the end of the Data Half-Flits conveying the data for the Write.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								NOP	NOP	NOP	NOP	Req0(Write)			
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.ByteEnables								Req0.Data.5							

Table 5-7: A 4 sector Request (Write 192 bytes) followed by a Control Half Flit with Flow Control

The following example, Table 5-8, illustrates a 2-sector 64-byte Write request (Req0) which will require byte enables, a NOP (sector 2), a Flow Control (FC) indication (sector 3), and finally a 4-sector AtomicR request. The first two Half-Flits control the Req0 data, followed by the Byte Enables for Req0. The next Half-Flits contain the Operands for the Req1 Atomic. The Byte Enables for the Req1 Atomic are swapped into the upper TL Half-Flit and the Control Half-Flit in the lower TL Half Flit illustrates multiple Flow Control Fields in a single Control Half-Flit.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req1 (AtomicR)				FC	NOP	Req0	
1	Req0.ByteEnables								Req0.Data.1							
2	Req1.AtomicOperands.1								Req1.AtomicOperands.0							
3	Req1.AtomicByteEnables								NOP	NOP	FC	NOP	FC	FC	NOP	NOP

Table 5-8: A 2-sector Request (64 byte write), NOP, FC, and 4-sector AtomicR request

The following example, Table 5-9, illustrates a 4-sector AtomicNR Req0, and a 4-sector AtomicR request. Req1. Req0 and Req1 each requires two TL Half-Flits to convey the Operands and one TL Half-Flit to convey the Byte Enables. The Req1 Atomic Byte Enables are swapped into the upper TL Half-Flit.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.AtomicOperands.0								Req1 (AtomicR)				Req0 (AtomicNR)			
1	Req0.AtomicByteEnables								Req0.AtomicOperands.1							
2	Req1.AtomicOperands.1								Req1.AtomicOperands.0							
3	Req1.AtomicByteEnables								NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP

Table 5-9: A 2-sector Request (64 byte write), NOP, FC, and 4-sector AtomicR request

The following example, Table 5-10, illustrates a 2-sector 256-byte Write request (Req0 which will require byte enables), a 256-byte WriteFull request (Req1 which will not require byte enables), and finally, a 4-sector AtomicNR Request (Req2). The last Data Half-Flits called for by the Control Half-Flit is swapped into the upper Half-Flit in the final TL Flit and the lower Half-Flit is shown containing a Flow Control Field.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req2 (AtomicNR)				Req1		Req0	
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req0.ByteEnables								Req0.Data.7							
5	Req1.Data.1								Req1.Data.0							
6	Req1.Data.3								Req1.Data.2							
7	Req1.Data.5								Req1.Data.4							
8	Req1.Data.7								Req1.Data.6							
9	Req2.AtomicOperands.1								Req2.AtomicOperands.0							
10	Req2.AtomicByteEnables								FC	NOP	NOP	NOP	NOP	NOP	NOP	NOP

Table 5-10: A 2-sector Request (256 byte write), 2-sector Request (256-byte WriteFull), and a 4-sector AtomicNR

The following example, Table 5-11, illustrates an example with Authentication enabled. The AuthTags Half-Flit is shown with Authentication Tags (AuthTag0, AuthTag1, AuthTag2) for the three Requests (two WriteFulls and an AtomicNR) in the Lower TL Half-Flit and the final AuthTag

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

has a value of '0'. The final Data Half-Flit called out by the Control Half-Flit is swapped into the upper Half-Flit in Flit 10.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	0		AuthTag2		AuthTag1		AuthTag0		Req2 (AtomicNR)				Req1		Req0	
1	Req0.Data.1								Req0.Data.0							
2	Req0.Data.3								Req0.Data.2							
3	Req0.Data.5								Req0.Data.4							
4	Req0.Data.7								Req0.Data.6							
5	Req1.Data.1								Req1.Data.0							
6	Req1.Data.3								Req1.Data.2							
7	Req1.Data.5								Req1.Data.4							
8	Req1.Data.7								Req1.Data.6							
9	Req2.AtomicOperands.1								Req2.AtomicOperands.0							
10	Req2.AtomicByteEnables								NOP	NOP	FC	NOP	FC	FC	NOP	NOP

Table 5-11: Authentication mode example matching the prior example.

The following example, Table 5-12, illustrates a single 4-sector Read Request and two 2-sector Compressed Read Requests each accessing 128 bytes on the Tx TL Channel and the subsequent Mandatory NOP Half Flit followed by the corresponding Read Responses (CRR0 is a compressed Read Response) and data on the Rx Channel for the TL which occur later in time.

	(Tx Channel) 64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	MANDATORY NOP								Req2 (Read)				Req1		Req0	

	(Rx Channel) 64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Rsp0.Data.0								RdRsp1	RdRsp2	NOP	CRR 0	NOP	NOP		
1	Rsp0.Data.2								Rsp0.Data.1							
2	Rsp2.Data.0								Rsp0.Data.3							
3	Rsp2.Data.2								Rsp2.Data.1							
4	Rsp1.Data.0								Rsp2.Data.3							
5	Rsp1.Data.2								Rsp1.Data.1							
6	Rsp1.Data.3								NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP

Table 5-12: Read Requests and Associated Read Responses.

5.3 Indicating Data Corruption in TL Data Half-Flits.

When an error occurs on a UPLI Data Beat (either a Read Response Data Beat or an Originator Data Beat as indicated by the UPLI RdRspDataError or UPLI OrigDataError signal) any subsequent TL Data Half-Flit carrying that data shall indicate that the Data Half-Flit is corrupted. An Accelerator may mark any received corrupted data as “poisoned” in any caching structures or take whatever actions are required at the Accelerator for corrupted data. The corrupt data indication shall be carried through the various subsequent TL interfaces and UPLI interfaces (via the UPLI RdRspDataError signal or the UPLI OrigDataError signal) until the data reaches the Accelerator ultimately receiving the data.

To avoid significant buffering complexity and an acknowledgement protocol loop, this indication shall be carried coincident with the data Half-Flit that is corrupted on the TL Flits (the UPLI RdRspDataError signal and the UPLI OrigDataError signal in the UPLI Interfaces are already coincident with the corrupted data beat).

To indicate corrupt data in the TL Flits, the Data Half-Flit that would be carrying the corrupted data shall be replaced with a TL Flit Message with Message Type 0x20 (Poisoned Data). The remaining bytes aside from the low-order byte indicating the Message Type in the Corrupted Data-Half Flit are undefined and may, for example, be whatever the current value of the data was for those bytes or may be set to any value. The TL Flit Message with Message Type 0x20 (Poisoned Data) shall only be used in TL Half-Flits that would have been Data Half-Flits conveying data. Because the UPLI Interface indications for corrupted data (RdRspDataError or OrigDataError) cover a 64-byte UPLI data beat, pairs of TL Data Half-Flits (each with 32 bytes of data) shall be replaced with TL Message Flits indicating Poisoned data when an error occurs.

Any errors occurring on the TL Rx interface (detected by an implementation specific error detection scheme) are uncorrectable and constitute a fatal error that cannot be addressed by poisoning data. The Rx TL interface on which the error occurs stops forwarding all received TL Flits after an error is detected. Errors detected on the TL Tx interface are managed by the UALink DL.

The example below, Table 5-13, illustrates a 256-byte WriteFull whose second 64-byte UPLI beat has been corrupted. The Req0.Data.2 and Req0.Data.3 TL Half Flits are replaced with TL Message Flits to indicate the corrupted Half-Flits (Data Half-Flits for Write/WriteFull/Read Response Data are always corrupted in pairs corresponding to the Half-Flits for the corrupted 64-byte UPLI Data Beat). When the Data arrives at the Accelerator that is consuming the data, the OrigDataError signal for the second UPLI beat will be set based on the TL Messages.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req0 (WriteFull)				NOP	NOP	NOP	NOP
1	TL Msg: Data Poisoned 0x20								Req0.Data.1							
2	Req0.Data.4								TL Msg: Data Poisoned 0x20							
3	Req0.Data.6								Req0.Data.5							
4	Req0.Data.7								NOP	NOP	NOP	NOP	FC	NOP	NOP	NOP

Table 5-13: 256-byte WriteFull with corrupt second 64-byte beat.

The following example, Table 5-14, illustrates a corrupted Operand Data for an Atomic. The Req1.AtomicOperands.1 and Req1.AtomicOperands.0 Half-Flits are both replaced with TL Msgs to indicated Poisoned Beats (all Operand Data for Atomic is 64 bytes on a single UPLI Data beat, so both are corrupted or neither are corrupted). Byte Enables for Atomics are considered part of information conveyed on the UPLI Request Channel and therefore are not subject to poisoning. Errors on Control signals cause the various UPLI Interfaces to enter Drop Mode instead.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req1 (AtomicR)				FC	NOP	Req0	
1	Req0.ByteEnables								Req0.Data.1							
2	TL Msg: Data Poisoned 0x20								TL Msg: Data Poisoned 0x20							
	Req1.AtomicByteEnables								NOP	NOP	FC	NOP	FC	FC	NOP	NOP

Table 5-14: An example illustrating corrupted Atomic Operand data.

5.4 TL Write Flit Sequence Encoding Efficiency Examples.

This section shows some examples of WriteFull requests and response with various packing efficiencies. These examples assume symmetric write traffic between Accelerators where both Accelerators send an equal number of WriteFull requests to the other accelerator. Consequently,

each Accelerator will return a Write Response for each WriteFull Request received and therefore the number of Write Responses received at each Accelerator in the pair will match. This symmetric WriteFull pattern is the most common use case for UALink.

The efficiency of the link is defined as the number data bytes transferred divided by the number of data bytes transferred plus all other bytes transferred.

Note that in all the write efficiency examples below, the Write Responses shown in the example are not related to the WriteFull Requests showing in the example (hence the Requests being labeled with numbers 0, 1, 2 ... and the Responses being labeled with letters A, B, C....). The Responses correspond to Requests that were sent earlier in time and are not shown in the examples.

5.4.1 Single WriteFull Request and Single WriteFull Response

The following example, Table 5-15, illustrates a single 4-sector 256-byte WriteFull Request, Req0, and a single 2-sector Write Response, WrRspA, in the first Control Half Flit along with a NOP sector and a Flow Control Indicator sector and similar Control Half-Flit with a WriteFull Request, Req1, and a Write Response, WrRspB. The transfer for both Writes and the Write Responses and Flow Control/NOPs takes nine flits for $9 \times 64 = 576$ total bytes transferred with $8 \times 64 = 512$ bytes of data transferred for an efficiency of $512/576 = 88.89\%$.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req0 (WriteFull)				WrRspA		NOP	FC
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req0.Data.7								Req1 (Write Full)				WrRspB		NOP	FC
5	Req1.Data.1								Req1.Data.0							
6	Req1.Data.3								Req1.Data.2							
7	Req1.Data.5								Req1.Data.4							
8	Req1.Data.7								Req1.Data.6							

Table 5-15 A single 4-sector Request (WriteFull 256 bytes) and a 2-sector Write Response.

5.4.2 WriteFull Requests and Compressed WriteFull Responses

The following example, Table 5-16, illustrates three 4-sector 256-byte WriteFull Requests (Req0, Req1, Req2) and three unrelated single-sector Write Responses (CWR A, CWR B, CWR C) for previous (unshown) Write Requests, and a Flow Control Sector in two Control Half Flits. This packing allows for three 256-byte WriteFull Requests to be controlled by two Control Half Flits. The transfer for all three Writes and the Write Responses and Flow Control takes thirteen flits for

13*64= 832 total bytes transferred with 12*64=768 bytes of data transferred for an efficiency of 768/832 = 92.31%.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								Req1 (WriteFull)				Req0 (WriteFull)			
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req1.Data.0								Req0.Data.7							
5	Req1.Data.2								Req1.Data.1							
6	Req1.Data.4								Req1.Data.3							
7	Req1.Data.6								Req1.Data.5							
8	Req1.Data.7								Req2 (Write Full)				CWR C	CWR B	CWR A	FC
9	Req2.Data.1								Req2.Data.0							
10	Req2.Data.3								Req2.Data.2							
11	Req2.Data.5								Req2.Data.4							
12	Req2.Data.7								Req2.Data.6							

Table 5-16 Three 4 sector WriteFull 256 byte Requests and three Compressed Write Responses.

5.4.3 Compressed WriteFull Requests and Compressed WriteFull Responses

The efficiency can be further increased by resorting to Compressed (2-sector) WriteFull Requests and Compressed (1-sector) Write Responses. The following example, Table 5-17 illustrates four 2-sector 256-byte Compressed WriteFull Requests (CWReq3, CWReq2, CWReq1, CWReq0) and four single-sector Compressed Write Responses (CWR A, CWR B, CWR C, CWR D) and two Flow Control Sectors in two Control Half Flits. This packing allows for four 256-byte WriteFull Requests to be controlled by two Control Half Flits. The transfer for all four Writes and the Write Responses and Flow Control takes seventeen flits for 17*64= 1088 total bytes transferred with 16*64=1024 bytes of data transferred for an efficiency of 1024/1088 = 94.12%.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								CWReq1	CWReq0		CWR _B	CWR _A	NOP	FC	
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req1.Data.0								Req0.Data.7							
5	Req1.Data.2								Req1.Data.1							
6	Req1.Data.4								Req1.Data.3							
7	Req1.Data.6								Req1.Data.5							
8	Req1.Data.7								CWReq3	CWReq2		CWR _D	CWR _C	NOP	FC	
9	Req2.Data.1								Req2.Data.0							
10	Req2.Data.3								Req2.Data.2							
11	Req2.Data.5								Req2.Data.4							
12	Req2.Data.7								Req2.Data.6							
13	Req3.Data.1								Req3.Data.0							
14	Req3.Data.3								Req2.Data.2							
15	Req3.Data.5								Req3.Data.4							
16	Req3.Data.7								Req3.Data.6							

Table 5-17 Four 2 sector WriteFull 256-byte Requests and four Compressed Write Responses.

5.4.4 Maximum Efficiency WriteFulls.

This example, Table 5-18, removes both NOP Sectors and one Flow Control Indicator Sector from the prior example and replaces them with another Compressed (2-sector) WriteFull Request and a Compressed (1-sector) Write Response. This allows the two Control Half Flits to effect five 256-byte writes in 20 Flits for $21 \times 64 = 1344$ total bytes transferred with $5 \times 256 = 1280$ data bytes transferred for an efficiency of $1280/1344 = 95.24\%$. This is the maximum efficiency for WriteFull requests.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Req0.Data.0								CWReq2	CWReq1		CWR _B	CWR _A	CWReq0		
1	Req0.Data.2								Req0.Data.1							
2	Req0.Data.4								Req0.Data.3							
3	Req0.Data.6								Req0.Data.5							
4	Req1.Data.0								Req0.Data.7							
5	Req1.Data.2								Req1.Data.1							
6	Req1.Data.4								Req1.Data.3							
7	Req1.Data.6								Req1.Data.5							
8	Req2.Data.0								Req1.Data.7							
9	Req2.Data.2								Req2.Data.1							
10	Req2.Data.4								Req2.Data.3							
11	Req2.Data.6								Req2.Data.5							
12	Req2.Data.7								CWReq4	CWReq3		CWR _E	CWR _D	CWR _C	FC	
13	Req3.Data.1								Req3.Data.0							
14	Req3.Data.3								Req3.Data.2							
15	Req3.Data.5								Req3.Data.4							
16	Req3.Data.7								Req3.Data.6							
17	Req4.Data.1								Req4.Data.0							
18	Req4.Data.3								Req4.Data.2							
19	Req4.Data.5								Req4.Data.4							
20	Req4.Data.7								Req4.Data.6							

Table 5-18: Five 2 sector WriteFull 256-byte Requests and Five Compressed Write Responses.

5.4.5 Maximum Efficiency WriteFulls with Authentication.

This example, Table 5-19, illustrates the maximum efficiency possible for Write Full Requests with Authentication Enabled. Two Compressed Write Requests and two Compressed Write Responses with a NOP and a Flow Control Field make up the Control Half-Flit. Another sixteen Half-Flits deliver the Write Data for both Write Requests allowing for the transfer of 512 bytes of data in nine 64-byte TL Flits ($9 \times 64 = 576$ total bytes transferred) for an efficiency of $512/576 = 88.89\%$, or a net loss of $(95.24 - 88.89) = 6.35\%$ over the max efficiency case without Authentication enabled.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	AuthTag1		AuthTag0		AuthTagB		AuthTagA		CWReq1		CWReq0		CWR B	CWR A	NOP	FC
1	Req0.Data.1								Req0.Data.0							
2	Req0.Data.3								Req0.Data.2							
3	Req0.Data.5								Req0.Data.4							
4	Req0.Data.7								Req0.Data.6							
5	Req1.Data.1								Req1.Data.0							
6	Req1.Data.3								Req1.Data.2							
7	Req1.Data.5								Req1.Data.4							
8	Req1.Data.7								Req1.Data.6							

Table 5-19: Two 2 sector WriteFull 256-byte Requests and Two Compressed Write Responses.

5.4.6 Maximum Efficiency Reads.

This example, Table 5-20, shows the maximum efficiency achievable with all Reads, which matches the efficiency achievable with WriteFulls. Only one TL Flit stream is shown and the five Compressed Read Responses (CRR0, CRR1, CRR2, CRR3, CRR4) are for Requests issued on the Tx Flit stream for this TL and are returned on this Rx Flit Stream. The Compressed Read Requests (CRReq0, CRReq1, CRReq2, CRReq3) are Read Requests issued by this TL on the Tx Flit stream and will have Responses (unshown) later in time. The total number of bytes transferred is 1344 bytes and the

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

useful data transferred is 1280 bytes for an efficiency of $1280/1344 = 95.24\%$ as it was for WriteFull case.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	Rsp0.Data.0								CRReqB	CRReqA	CRR 2	CRR 1	CRR 0	FC		
1	Rsp0.Data.2								Rsp0.Data.1							
2	Rsp0.Data.4								Rsp0.Data.3							
3	Rsp0.Data.6								Rsp0.Data.5							
4	Rsp1.Data.0								Rsp0.Data.7							
5	Rsp1.Data.2								Rsp1.Data.1							
6	Rsp1.Data.4								Rsp1.Data.3							
7	Rsp1.Data.6								Rsp1.Data.5							
8	Rsp2.Data.0								Rsp1.Data.7							
9	Rsp2.Data.2								Rsp2.Data.1							
10	Rsp2.Data.4								Rsp2.Data.3							
11	Rsp2.Data.6								Rsp2.Data.5							
12	Rsp2.Data.7								CRReqE	CRReqD	CRR 4	CRR 3	CRReqC			
13	Rsp3.Data.1								Rsp3.Data.0							
14	Rsp3.Data.3								Rsp3.Data.2							
15	Rsp3.Data.5								Rsp3.Data.4							
16	Rsp3.Data.7								Rsp3.Data.6							
17	Rsp4.Data.1								Rsp4.Data.0							
18	Rsp4.Data.3								Rsp4.Data.2							
19	Rsp4.Data.5								Rsp4.Data.4							
20	Rsp4.Data.7								Rsp4.Data.6							

Table 5-20: Five 2 sector Read 256-byte Requests and Five Compressed Read Responses.

5.4.7 Maximum Efficiency Reads with Authentication.

The following example, Table 5-21, illustrates the maximum efficiency possible for Read Requests with Authentication Enabled (using 256- byte Read Requests). Two Compressed Read Requests and two Compressed Read Responses with a NOP and a Flow Control Field make up the Control Half-Flit. Another sixteen Half-Flits deliver the Read Data for both the Write Requests allowing for the transfer of 512 bytes of data in nine 64-byte TL Flits ($9 \times 64 = 576$ total bytes transferred) for an

Transaction Layer (TL)

efficiency of $512/576 = 88.89\%$, or a net loss of $(95.24 - 88.89) = 6.35\%$ over the max efficiency case without Authentication enabled.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	AuthTag1		AuthTag0		AuthTagB		AuthTagA		CRReqB		CRReqA		CRR 1	CRR 0	NOP	FC
1	Rsp0.Data.1								Rsp0.Data.0							
2	Rsp0.Data.3								Rsp0.Data.2							
3	Rsp0.Data.5								Rsp0.Data.4							
4	Rsp0.Data.7								Rsp0.Data.6							
5	Rsp1.Data.1								Rsp1.Data.0							
6	Rsp1.Data.3								Rsp1.Data.2							
7	Rsp1.Data.5								Rsp1.Data.4							
8	Rsp1.Data.7								Rsp1.Data.6							

Table 5-21: Two 2-sector Read 256-byte Requests and Two Compressed Read Responses.

5.4.8 Maximum Efficiency With Mixed Reads and Writes.

The following example, Table 5-22, shows a maximum efficiency achievable with a mixture of Reads and WriteFulls, which matches the efficiency achievable with WriteFulls. Only one TL Flit stream is shown with Write Requests 1, 2, and 4, Read Requests 0 and 3, Write Responses B, D, and E, and Read Responses A and C. The total number of bytes transferred is 1344 bytes and the useful data transferred is 1280 bytes for an efficiency of $1280/1344 = 95.24\%$ as it was for WriteFull case.

Table 5-22 Three Write, Two Read Maximum Efficiency Example

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	RspA.Data.0								CWrReq2	CWrReq1		CWr Rsp B	CRd Rsp A	CRdReq0		
1	RspA.Data.2								RspA.Data.1							
2	RspA.Data.4								RspA.Data.3							
3	RspA.Data.6								RspA.Data.5							
4	Req1.Data.0								RspA.Data.7							
5	Req1.Data.2								Req1.Data.1							
6	Req1.Data.4								Req1.Data.3							
7	Req1.Data.6								Req1.Data.5							
8	Req2.Data.0								Req1.Data.7							
9	Req2.Data.2								Req2.Data.1							
10	Req2.Data.4								Req2.Data.3							
11	Req2.Data.6								Req2.Data.5							
12	Req2.Data.7								CWReq4	CRdReq3		CW Rsp E	CW Rsp D	CRd Rsp C	FC	
13	RspC.Data.1								RspC.Data.0							
14	RspC.Data.3								RspC.Data.2							
15	RspC.Data.5								RspC.Data.4							
16	RspC.Data.7								RspC.Data.6							
17	Req4.Data.1								Req4.Data.0							
18	Req4.Data.3								Req4.Data.2							
19	Req4.Data.5								Req4.Data.4							
20	Req4.Data.7								Req4.Data.6							

5.5 TL Tx and Rx Dataflow and Tx and Rx Compression Caches

The following figure gives a more detailed schematic description of a representative datapath in the UALink TL for the Outbound Tx Flit interface.

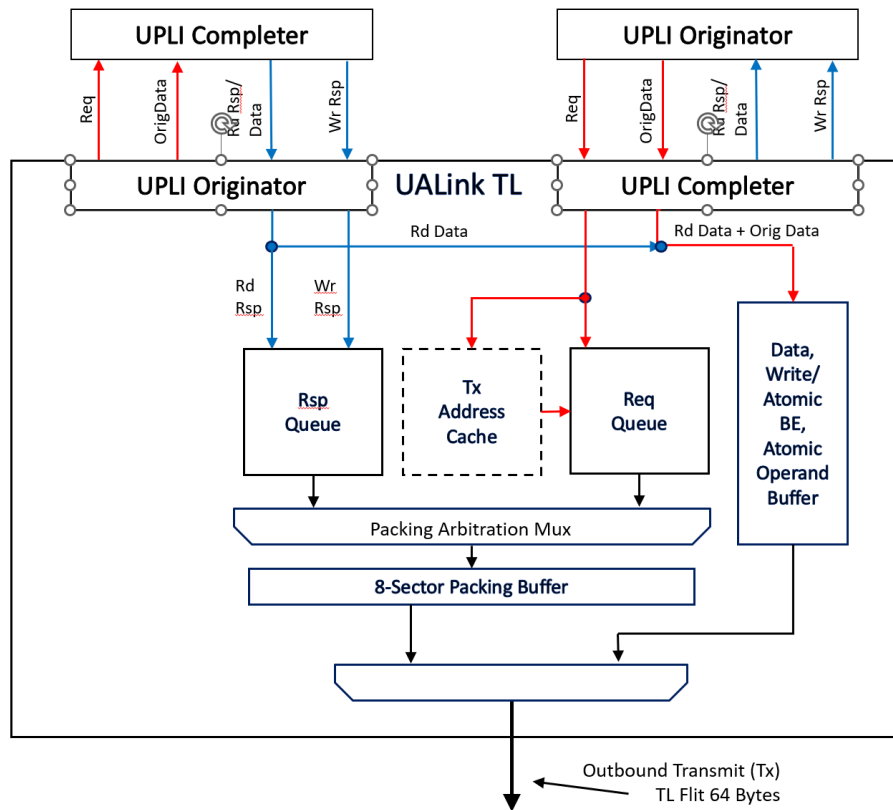


Figure 5-2: UALink TL Tx Dataflow.

Outbound Requests are routed to a Request Queue and the Tx Address Cache Request is referenced to determine if the address of the Request has been sent previously and is cached in the Receiver Rx cache in the UALink TL on the other end of the UALink (if present – the Tx and Rx Address Caches are optional and if they are not implemented, the interface only uses Uncompressed Requests that are marked to not load the absent Rx Address Cache). If the Tx Address Cache hits, a Compressed Request that omits many of the address bits and some other information may be used for the Request. If not, and this Request address can be cached, the Request address is typically, but not always, loaded in the Tx Address Cache and the uncompressed Request will indicate whether or not to load the cache at the receiving end of the UALink. If the Tx Address Cache is loaded, future Requests may then be issued as Compressed Requests and the Receive Address Cache will reconstitute the omitted address bits. If, however, for implementation specific reasons, the Transmit cache cannot be loaded or the implementations chooses not to load the Transmit cache for this specific Request, an Uncompressed Request indicating to not load the Rx Address Cache shall be sent.

The Receive cache shall be controlled by the Transmit cache and shall be kept in lockstep with the Transmit cache (the Transmit cache may be cleared at any time and no Compressed Requests will be sent until the Transmit cache is reloaded which will also reload the Receive Cache; Individual entries in the Transmit cache may be invalidated at any time).

Requests are kept in order between the Transmit cache and the Receive cache by the intervening UALink DL and PHY layers, therefore guaranteeing that Requests arrive at the Receive Cache in the order they were processed at the Transmit cache.

The Read Responses (not data) and Write Responses are queued in a Response queue. These can be drained opportunistically and out of order to fill in slots in the Control Half Flit.

The Responses (Read and Write) as well as the Requests are selected by Picking Logic and passed through the Packing Arbitration Mux into an 8-sector Packing Buffer to produce the next Control Half-Flit.

The Read Response Data, Write Data, Byte Enables for Atomics and Writes, and Atomic Operands are held in a Data Buffer and drained in the order required by the sequence of the Data Request Fields in the Control Half Flit. The data buffer consists of 64-byte buffers holding two TL Half-Flits. The Write and Atomic Byte Enables may be held in a set of parallel 8-byte buffers per 64-byte data buffer (in the limit, all the data could be for 64-byte Writes that each need byte enables). An alternative implementation could place the Byte Enables in the Request queue instead of in the data queues.

A final mux stage selects between the 8-Sector Packing Buffer and the Data Buffer to emit Control Half Flits or Data Half Flits onto the interface.

The following figure, Figure 5-3: UALink TL Rx Dataflow, gives a more detailed schematic description of a representative datapath in the UALink TL for the Inbound Rx Flit interface.

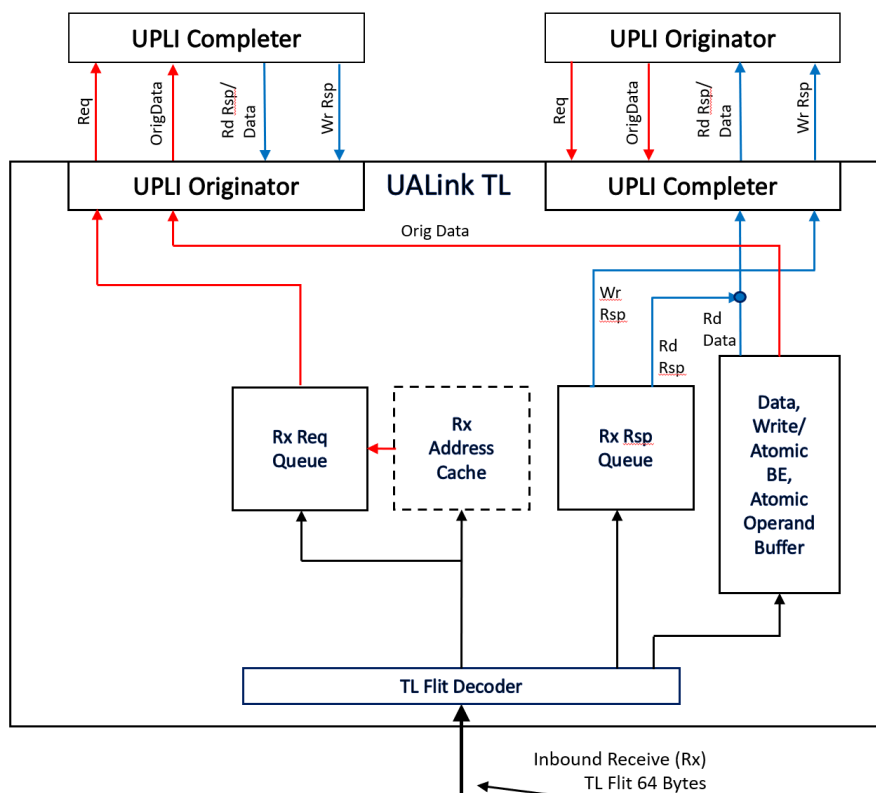


Figure 5-3: UALink TL Rx Dataflow

The Inbound TL Rx Flit interface is first processed by a TL Flit Decoder which decodes the Flit into Read Response and Write Data, Write and Atomic Byte Enables, Atomic Operand Data, Requests,

and Responses. The Requests (and their Authorization Tags if present) are passed to the Rx Address Cache and Request (Req) Queue for processing. If the Request is an uncompressed Request the Request is placed directly into the Req Queue and if the uncompressed Request is marked to load the Rx Cache for future accesses, the appropriate Rx Address Cache Entry is loaded and validated. If the Request is a compressed request, the Rx Address Cache provides the untransmitted address bits which are loaded with the Request into the Req Queue. The Read Responses (not Data) and the Write Responses are placed in a Response (Rsp) Queue. The Read Response and WriteData and Atomic Operand Data are loaded into 64-byte buffers holding two TL Half-Flits (the Atomic Operand Data only occupies half of the 64-byte buffer). The Write and Atomic Byte Enables are held in a set of parallel 8-byte buffers per 64-byte data buffer (in the limit, all the data could be for 64-byte Writes that each need byte enables).

Requests are loaded into the Request queue in the order they are received from the TL Flit Decoder. The Requests are unloaded from the Rx Request Queue onto the UPLI Request interface according to the current UPLI Ordering mode (either strictly in order or in at least 256-byte region order). The Responses are unloaded into the Rx Rsp Queue and then onto the appropriate Read or Write Response interface in any order (the Read Response and Write Data as well as Write and Atomic Byte Enables and Atomic Operand Data are unloaded as needed with their corresponding Requests or Responses).

The Tx and Rx Address Caches shall conform to the following characteristics which allow for four concurrently active streams between any two Accelerators:

- The caches are up to (and are typically) 4-way set associative.
- The caches control up to 1024 congruence classes (cache rows), one per supported Accelerator. The caches on a Switch shall be sized to match the maximum number of ports the Switch may accommodate (the maximum number of ports a switch may accommodate sets the limit on the maximum number of Accelerators in the Pod). The caches on an Accelerator shall be sized to accommodate the maximum number of Accelerators expected in the Pod in which the Accelerator may occur.
- The caches can process at least one lookup or update of a cache entry per cycle. The lookup result is available an implementation defined fixed number of cycles later. The result of an update is available to a subsequent lookup a defined fixed number of cycles later (that may be different than the cache read latency).
- The Tx and Rx caches are indexed by the SRCACCID and DSTACCID field values in the UPLI Request Channel.
- The indexing of a given Tx cache/Rx cache pair is controlled by the Tx cache.
- Address Caches must meet resiliency requirements (Bit Error Rate) of the product. Any errors detected shall initiate a link down and/or Drop Mode for the TL containing the Address Cache.

The figure below, Figure 5-4: Indexing of Accelerator Tx Address Caches/Switch Rx Address Caches , illustrates the indexing of the Accelerator Tx Address Caches and Switch Rx Address Caches:

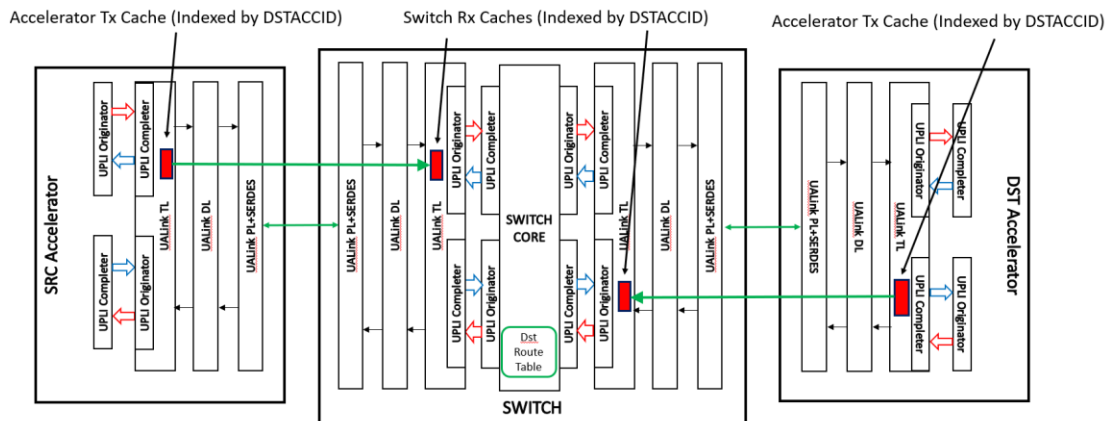


Figure 5-4: Indexing of Accelerator Tx Address Caches/Switch Rx Address Caches

All Requests issued by a given Accelerator will have only one value for the ReqSrcPhysAccID[9:0] signal: the identifier of the given Accelerator. This renders the ReqSrcPhysAccID field unsuitable to index the Accelerator Tx Address Cache. Instead, the Tx Address Caches on the Accelerators (and therefore the Rx Address Caches on the Switch) shall be indexed by the ReqDstPhysAccID[9:0] signal value in the UPLI Request Interface in order to fully populate the caches.

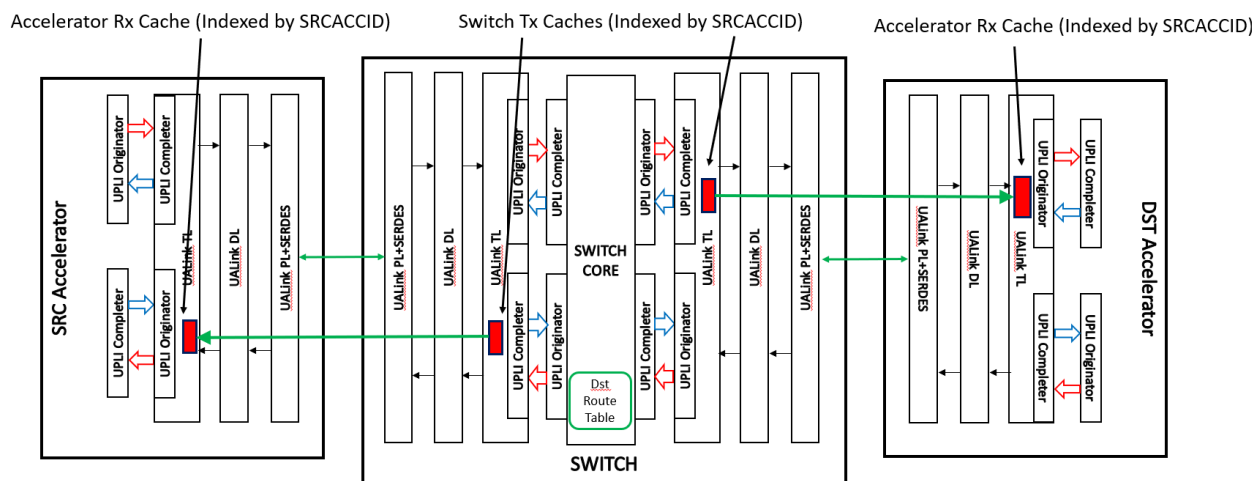


Figure 5-5: Indexing of Switch Tx Address Caches/Accelerator Rx Address Caches

The figure above, Figure 5-5: Indexing of Switch Tx Address Caches/Accelerator Rx Address Caches, illustrates the indexing of the Switch Tx Address Caches and Accelerator Rx Address Caches. The Requests issued by a Switch TL will have at most four values for ReqDstPhysAccID[9:0] – depending on bifurcation, for the up to four Accelerators attached to the given UALink TL instance. This renders the ReqDstPhysAccID signal unsuitable to index the Switch Tx Address Cache. Instead, the Tx Caches on the Switch (and therefore the Rx Caches on the Accelerators) shall be indexed by the ReqSrcPhysAccID[9:0] signal value in the UPLI Request Interface in order to fully populate the caches.

5.6 TL Tx and Rx Address Cache Synchronization.

This section describes a set of rules in the TL Flit protocol that ensure that the Tx and Rx Address Caches remain synchronized.

5.6.1 CLOAD and CWAY control signals

Uncompressed Requests have a 1-bit signal called CLOAD (or Cache Load) that indicates to the Rx Address Cache that the address in the Uncompressed Request shall be loaded into the Rx Cache (when CLOAD=1). An additional 2-bit signal CWAY (Cache Way) shall indicate what way in the Congruence Class (row) in the Rx Cache shall be loaded with the address.

The CLOAD signal shall only be asserted for the Request types listed below in Table 5-33 Compressed Request Command Encoding. This is intended to prevent Requests (such as AtomicR) that can never compress from polluting the compression caches.

The Congruence Class shall be selected by the SRCACCID[9:0] signal value (for Tx caches in the Switch and their corresponding Rx Address Caches in the Accelerators) or by the DSTACCID[9:0] signal value (for Tx caches in the Accelerators and their corresponding Rx Address Caches in the Switches) as explained above.

The CWAY signal value in an uncompressed Request shall only have meaning if the CLOAD signal is b'1'.

Compressed Requests also have a 2-bit CWAY signal that indicates the way in the Congruence Class indexed by SRCACCID[9:0] or DSTACCID[9:0] that should be utilized to reconstitute the rest of the address bits not transmitted in a Compressed Request.

As shown below in Table 5-23 Address Cache Loading Request Availability, a Request that loads the RX Address Cache shall be available to all subsequent Compressed Requests even in the same Control Half-Flit (i.e. Req0's loaded value of the cache must be available to Compressed Requests Req1/Req2/Req3) until that entry is overwritten in the Address Rx Cache. This does not mean the address cache itself must immediately hit on a write as described below with respect to the optional Address CAM logic in Figure 5-6 TL Receive Catch Buffer Dataflow.

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	MANDATORY NOP								Compressed Req2 (Rd A)	Compressed Req1 (Rd A)		Req0(Read A) CLOAD=1				
1	MANDATORY NOP								NOP	NOP	NOP	NOP	NOP	NOP	Compressed Req3 (Rd A)	

Table 5-23 Address Cache Loading Request Availability

5.6.2 Address Cache sequencing at the Tx Address Cache and Rx Address Cache

The Tx and Rx Address Caches shall follow a set of rules to ensure the Address Caches are properly synchronized:

- A TL Tx cache may always issue an Uncompressed Request (with CLOAD=0) regardless of whether the address of the Request is valid or invalid in the TL Tx cache.
- If a TL Tx cache issues an Uncompressed Request with CLOAD=1, the entry at the appropriate congruence class and way (CWAY) shall be loaded (or invalidated) in the Tx cache and not be overwritten by another Request before the Uncompressed Request is issued in the TL Tx Channel.

- A TL Tx shall not issue a Compressed Request unless the address for that Request hits in the Tx cache and the matching entry in the Tx Cache will not be overwritten by another Request before the Compressed Request is issued in the TL Tx Channel.
- Requests are ordered within a Control Half-Flit from the low order byte to the high order byte.
- A Request is considered “issued” when it is placed in the Control Half-Flit in the position in which it will appear on the TL Tx Channel.
- The Rx cache shall update the entry specified by CWAY and SRCACCID[9:0] or DSTACCID[9:0] for any Uncompressed Request that has CLOAD=1.
- The results of a CLOAD = 1 Request that updates the TL Rx Cache shall be available to any subsequent matching Compressed Request within the TL Control Half-Flit or any later TL Control Half-Flit until that entry is overwritten in the TL Rx cache (this requires that writes to the Rx Address Cache are available in the next cycle or external Address CAM logic, as described above, provide the newly written value until the Address Cache can directly provide the written value).
- Once an entry is loaded into the TL Rx Cache, it may not be invalidated, but instead it may only be replaced by a subsequent CLOAD=1 Uncompressed Request.

5.7 TL Control Half-Flit Request/Response Field packing limits.

This section describes TL Flit protocol rules that limit the number of Requests and Responses in the TL Control Half-Flits generated at the Transmit TL to prevent the Transmit TL from overrunning the Request/Response queuing structures in the Receive TL. In particular, the number of Requests and Responses the Transmit TL can issue in any given TL Flit shall be source rate limited to match the capacity of the Receive TL to absorb those Requests and Responses.

At the Transmit TL, Requests that can be issued in any Flit shall be limited to a maximum of four including outstanding Requests issued in the previous TL Flits that have not been retired plus any Requests to be issued in the current TL Flit. Requests shall be retired at the rate of one Request per TL Flit at the Receive TL. Therefore, in any given Flit, at least one Flit may be issued).

Responses shall be similarly rate limited with a limit of eight including outstanding Responses issued in the previous TL Flits that have not been retired plus any Responses to be issued in the current TL Flit. Responses shall be retired at a rate of one Response per TL Flit at the Receive TL. Therefore, in any given Flit, at least one Flit may be issued.

For example, as shown below in Table 5-24 Request Packing without Data Half-Flits, if four Requests are legally issued in a given TL Flit, at most one Request may be issued in the following TL Flit. One of the four Requests issued in the first TL Flit will have been retired when the second TL Flit arrives at the Receiver TL. Similarly, a third TL Flit in such a sequence could again issue one Request. For any TL Flit that does not issue a Request, the number of allowable Requests increases by one Request (up to a maximum of four). The “Req Cnt” column below indicates the maximum allowable number of Requests that may be issued in that TL Flit:

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

	64-byte TL Flit																
	Upper TL Half-Flit								Lower TL Half-Flit								
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Flit																	Req Cnt
0	MANDATORY NOP								Req3 (Rd)		Req2 (Rd)		Req1 (Rd)		Req0 (Rd)		4
1	MANDATORY NOP								NOP	NOP	Req4 (Rd)		NOP	NOP	NOP	NOP	1
2	MANDATORY NOP								NOP	NOP	NOP	NOP	NOP	NOP	Req5 (Rd)		1
3	MANDATORY NOP								NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP	1
4	MANDATORY NOP								Req7 (Rd)				Req6 (Rd)		NOP	NOP	2
5	MANDATORY NOP								Req8 (Rd)				NOP	NOP	NOP	NOP	1

Table 5-24 Request Packing without Data Half-Flits

The following table, Table 5-25 Request Pacing with Data Half-Flits, illustrates the behavior of the allowable number of Requests in a given TL Flit when data tenures are present. The Control Half-Flit consists of two 64-byte WriteFull Requests (Req0/Req1) followed by two 64-byte Write Requests (which each need ByteEnables). Even in this extreme case of only 64 bytes being transferred per Requests, the Data Half-Flits necessary to transfer the 64 bytes provide time for the Receiver TL to retire the Requests and be able to receive fully populated Control Half-Flits on the next Control Half-Flit (the Read Example above does not directly illustrate this pattern because the Read Data Responses occur later in time, but in the aggregate, each Request Requires at least 64 bytes of Data which requires one TL Flit which allows for the Receive TL to retire the Request). With larger transfer sizes (128 to 256 Bytes) per request, the Control Half-Flits are naturally spaced out farther allowing the Receive TL more than adequate time to unpack Control Half-Flits.

	64-byte TL Flit																
	Upper TL Half-Flit								Lower TL Half-Flit								
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Flit																	Req Cnt
0	Req0.Data.0								Req3		Req2		Req1		Req0		4
1	Req1.Data.0								Req0.Data.1								1
2	Req2.Data.0								Req1.Data.1								2
3	Req2.ByteEnalbes								Req2.Data.1								3
4	Req3.Data.1								Req3.Data.0								4
5	Req3.ByteEnables								NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP	4

Table 5-25 Request Pacing with Data Half-Flits

The source rate limitations for Responses shall follow the same behavior as described above for Requests with the exception that the maximum allowable number of Responses is eight instead of fourC.

The following figure, Figure 5-6 TL Receive Catch Buffer Dataflow, provides a more detailed schematic representation of the “catch buffers” for processing Inbound Receive (Rx) TL Flits in the Receive TL.

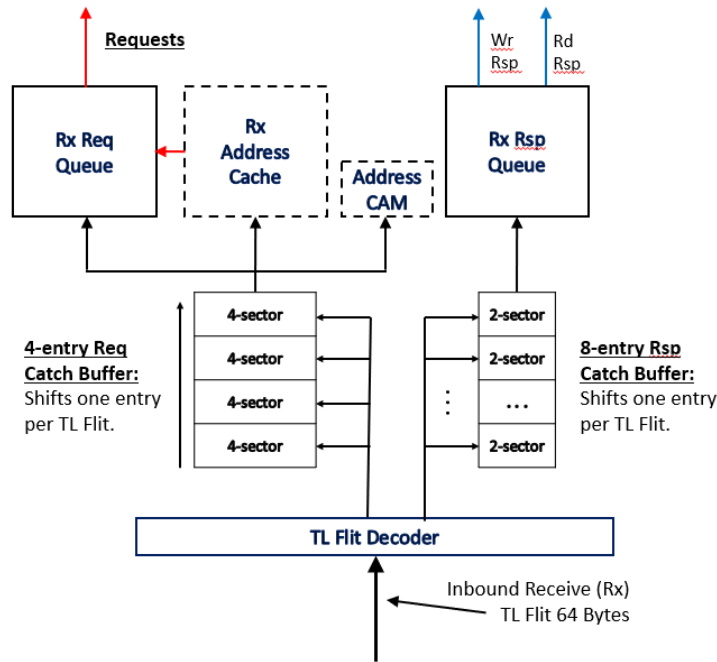


Figure 5-6 TL Receive Catch Buffer Dataflow

The TL Flit Decoder processes the non-NOP Control Half-Flit to produce an ordered list of the Responses and Requests that are present. The Catch Buffers shift one entry into the Rx Req and Rx Rsp Queues per incoming TL Flit.

The Requests are loaded in order into the Request Catch Buffer starting at the first non-empty entry in the buffer (accounting for the buffer shift per TL Flit) and Responses are similarly loaded into the Response Catch Buffer. Each entry in the catch buffer is sized to accommodate the largest Request or Response to allow for the case where the given Catch Buffer is full, but a series of TL Flits containing maximum sized Requests or Responses (or both) occur. This will fill the Catch Buffer with maximum sized entries.

The Address CAM is an optional address pipeline of the last “n” addresses that have been entered into the Rx Address Cache but are not yet available to a subsequent read of the Rx Address Cache (if the value written to the cache can be read in the cycle immediately after the write, the Address CAM is not necessary). Reads of the address cache preferentially take the value from the Address CAM. As writes become available from the Rx Address Cache, they fall out of the Address CAM, if implemented.

The Transmitter TL source rate limitations allow a TL Control Half-Flit to be fully populated with either the maximum number of Requests (four) or Responses (eight) provided there are no outstanding Requests or Responses, respectively. The source rate limitations shall be independent of the limitations placed on the Transmitter TL by Flow Credits (described in 5.8). The source rate limitations prevent the Transmitter TL from overrunning the catch buffers in the Receiver TL. The Flow Control limitations prevent the Transmitter TL from overrunning the Req/Rsp/Data queues that are after the catch buffers in the Receiver TL.

A one-sector Flow Control Field (described in more detail below in 5.9.6) in a Control Half-Flit contains indications for 4 classes of Credits and each class of Credit may independently return Credits for a given one of four Virtual Channel or a Pool Credit (usable for a transfer for any Virtual Credit). A TL Control Half-Flit may be fully populated with up to eight Flow Control Fields. A Receiving TL shall process all Credit counter updates at the rate of the incoming TL Flits (i.e. all updates for a given TL Flit shall be processed in a manner that places no restrictions on the number of Flow Control Fields in the next TL Flit).

To facilitate this per TL-Flit processing rate, no more than one of the Credit Class indications across all the Flow Control Fields present in a given Control Half-Flit may have a non-zero value for a given Virtual Channel or Pool. This allows the Credit Class values for all Flow Control Fields to be OR reduced into a set of Pool and Virtual Channel update values per Credit Class which can then be applied simultaneously to the Credit counters (i.e. the Flow Control Fields have no catch buffer structures like the Requests and Responses).

5.8 TL Flow Control

The Flow Control between UALink TLs shall be managed by means of Credits for the Request and Response Fields in TL Control Half-Flits, and Credits for pairs of TL Data Half-Flits carrying data (Read Response Data, Write Data, and Atomic Operands). Byte Enables for Writes, while delivered in a TL Data Half-Flit, are presumed to be held in dedicated side buffers associated with either the data or the Request and therefore do not require separate Credits. A Data Credit shall reserve a 64-byte data Buffer.

A Credit is either a Pool Credit that can be used with a Request, Response, or Data on any Virtual Channel or a Virtual Channel Credit that shall only be used with a Request, Response, or Data specifically associated with one of the four Virtual Channels.

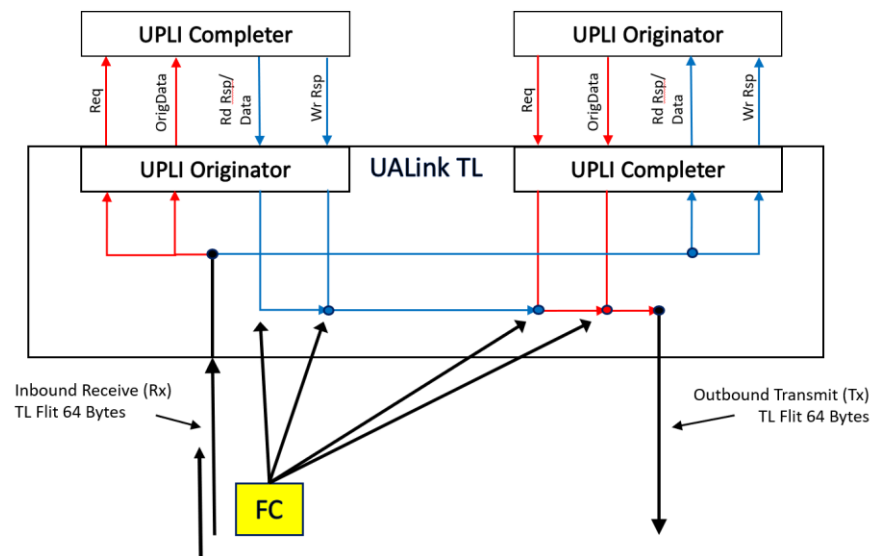


Figure 5-7: Flow Control Field Relation to Credit Channels.

The Figure above, Figure 5-7: Flow Control Field Relation to Credit Channels., illustrates the relationship between a Flow Control Field received on the Inbound Receive (Rx) TL Flit Channel and the information on the outgoing UPLI Channels this Flow Control Field manages.

The Flow Control Field consists of signals for four classes of Credits: Request CMD, Response CMD, Request Data, Response Data as shown in the following table:

Flow Control Field Signals	
Request CMD	Credits for Uncompressed or Compressed Requests in TL Control Half-Flits.
Response CMD	Credits for Uncompressed or Compressed Read Responses (not the Data for the Read Responses) or Uncompressed or Compressed Write Responses in the TL Control Half-Flits.
Request Data	Credits for 64-byte data buffers used to hold data for Write and WriteFull Requests, Operand Data for AtomicR/AtomicNR Requests, Vendor Defined Commands with Data, and UPLI Write Message Requests. Byte Enables are held in dedicated buffers associated with the Data or Request and therefore do not require explicit Credits.
Response Data	Credits for 64-byte data buffers used to hold data for Read Responses (which have no associated Byte Enables).

Table 5-26: Flow Control Field Signals

To allow for the worst case Write Example (see 5.4.4) a single Flow Control Field must be able to return at least 20 Data Credits each and 5 CMD Credits each. Therefore, the Flow Control Field signals returning the Request Data Credits and Response Data Credits are 8 bits each (5 bit to allow returning 20 credits, 2 bits to identify the virtual channel, and 1 bit to identify if these are Pool Credits or Virtual Channel Credits). The Flow Control Field signals returning the Request CMD and Response CMD Credits are 6 bits each (3 bits to return 5 credits, 2 bits to indicate the virtual channel, and 1 bit to identify if these are Pool Credits or Virtual Channel Credits). All four signals together consume 28 bits (the upper 4 bits of the Field are used to indicate the Field Type – FTYPE, explained below), filling the 32-bit single Sector Flow Control Field.

In summary, a single Flow Control Field can return up to 31 Credits for each of the Request Data Credits and Response Data Credits and up to 7 Credits for each of the Request CMD Credits and Response CMD Credits and each of these signals may independently choose to indicate Pool or Virtual Channel Credits.

At initialization, both UALink TLs connected together through DL and PHY blocks shall be programmed by an implementation specific means with the number of Credits and their types (Pool or Virtual Credit) for the Requests, Responses, and Data that the Rx portion of that TL can hold. Both UALink TLs shall have no Credits to issue TL Control Half-Flit Fields or Data Half-Flits. Each UALink TL shall issue one or more Flow Control Fields on their Outbound Transmit (Tx) Flit Channel to indicate the available buffering that the issuing TL has to receive Control Half-Flit Fields and Data Half-Flits for each class (Request CMD, Response CMD, Request Data, Response Data) of TL Credits. The receiving TL shall record the number of Credits, their class, virtual channel if necessary, and type (Pool or Virtual Channel). When the initial issuance of the Credits has been completed, an Initial Credit Release Complete TL Message Half-Flit indicating that the initial Credit release is complete shall be issued by the TL.

The set of Credits issued for each class may be Pool Credits, Virtual Channel Credits, or a combination of Pool and Virtual Channel Credits. A Pool Credit may be used to issue a Request, Response, or data associated with any Virtual Channel according to an allocation of pool credits to Virtual Channels controlled by the transmitting TL. The transmitting TL may vary the allocation of

Pool Credits to Virtual Channels over time but shall never have more Pool Credits outstanding than were initially issued. A Virtual Channel Credit shall be used to issue a Request, Response, or data only for the Virtual Channel associated with the Credit.

Once the initial Credits are completely issued, the TL may start issuing Requests, Responses, or Data. Each Request or Response has a “Pool” signal that indicates, when set to b’1’, that the Request or Response was issued using a Pool Credit and when set to b’0’ and indicates a Virtual Channel Credit associated with the Virtual Channel (VCHAN) field in that Request or Response when set to b’1’. The TL Receiving the Request or the Response shall record the value of the Virtual Channel and Pool signals on the Request or Response and shall use those values to form the values in the Flow Control Field to return the Credit back to the originator of the Request or Response.

The Data Credits for any Data or Atomic Operand Data shall inherit the Pool and Virtual Channel values from their associated Request or Rd Response. The Virtual Channel and Pool values are not indicated directly in the Data Half-Flits.

The UALink TL may optionally support a Shared Data Buffer mode. In Shared Data Buffer mode, the Pool Credits for the Request Data and Response Data buffers may be shared. In other words, the Pool Credits initially issued for Request Data may be used to issue TL Data Half-Flits for Response Data and vice-versa. In Shared Data Buffer mode, the initial issuance of the Data Credits may be all for one class (Request Data or Response Data) or the other (or Credits may be released in both classes). The TL issuing Credits indicates to the TL receiving the Credits whether Shared Data Buffer mode is supported or not by an indication in the Initial Credit Release Complete TL Message Half-Flit. The low-order bit of the 31-byte Message Specific Payload for the TL message shall be a flag indicating that Shared Data Buffer mode is supported when the bit is set to b’1’ and not supported when the bit is set to b’0’. The transmit side of a TL shall accommodate being issued zero credits for one class of Data Credits or another if the Shared Data Buffer mode is enabled. If Shared Data Buffer mode is not enabled, the TL initially issuing Credits shall issue at least some credits for both classes of Data Credits.

The Flow Control Field that returns zero pool credits for all 4 classes of Credits has a value of x’0000_0000’ and is the NOP Field.

5.9 TL Control Field Bit Assignments and Legal TL Message Flit Types

The Fields within a TL Control Half-Flit may be any of the following Field Types:

- Uncompressed Requests 4 sectors
- Uncompressed Responses 2 sectors.
- Compressed Request 2 sectors
- Compressed Response for Single-Beat Read 1 sector
- Compressed Response for Write or Multi-Beat Read 1 sector
- Flow Control/NOP Indication 1 sector

The high-order 4 bits (providing for 16 possible Field Types) of a field indicate the Field Type and implicitly the number of sectors contained within the Field.

The following table lists the legal values for the Field Type high-order 4 bit of the Control Field:

TL Control Half-Flit Field legal Field Type Encodings (upper 4 bits of Control Field).	
Value	Message Type
0x0	Flow Control Pool/NOP Indication.
0x1	Uncompressed Request
0x2	Uncompressed Response
0x3	Compressed Request
0x4	Compressed Response for Single-Beat Read Response
0x5	Compressed Response for Write or Multi-Beat Read Response

Table 5-27: TL Control Half-Flit Message Type Values

The following table lists the legal TL Message Half-Flits:

Defined TL Message Half-Flits	
Value	Message Type
0x00	NOP TL Message Half-Flit
0x20	Poisoned Data TL Message Half-Flit
0x01	Initial Credit Release Complete TL Message Half-Flit.

Table 5-28: Legal TL Message Half-Flits

5.9.1 Uncompressed Request Field

An uncompressed Request can be used to indicate any of the legal UPLI Requests without conditions (Compressed Requests can be utilized for only a subset of the legal UPLI Requests). An Uncompressed Requests consists of 4 sectors.

Table 5-29 Uncompressed Request Field Signals

Uncompressed Request Field (FTYPE = 0x01) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[127:124]	Field Type. This signal, with a value of 0x01, indicates the Field is an Uncompressed Request Field.
CMD	6	[123:118]	Command. This signal carries the UPLI ReqCmd signal value for this Request.
VCHAN	2	[117:116]	Virtual Channel. This signal carries the UPLI ReqVC signal value for this Request.
ASI	2	[115:114]	Address Space Identifier. This signal carries the UPLI ReqASI signal value for this Request.
TAG	11	[113:103]	Tag. This signal carries the UPLI ReqTag signal value for this Request.
POOL	1	[102]	Pool. This signal indicates whether the TL is utilizing a Pool Credit to issue this Request or a Virtual Channel Credit. The value on this signal is independent of the ReqPool signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
ATTR	8	[101:94]	Attribute. This signal carries the UPLI ReqAttr signal for this Request.
LEN	6	[93:88]	Length. This signal carries the UPLI ReqLen signal value for this Request.
METADATA	8	[87:80]	Metadata. This signal carries the UPLI ReqMetadata signal value for this Request.
ADDR	55	[79:25]	Address. This signal carries the UPLI ReqAddr[56:2] signal value for this Request. This is a doubleword aligned address.
SRCACCID	10	[24:15]	Source Accelerator ID. This signal carries the UPLI ReqSrcPhysAccID signal value for this Request.
DSTACCID	10	[14:5]	Destination Accelerator ID. This signal carries the UPLI ReqDstPhysAccId signal value for this Request.
CLOAD	1	[4]	Cache Load. This signal, when equal to '1', indicates that the Receiver TL should load ReqAddr[56:20] (ADDR[54:18]) into the Rx Address Cache at the way indicated by CWAY and at the row indicated by either SRCACCID[9:0] (for Rx Caches on Accelerators) or DSTACCID[9:0] (for Rx Address Caches on Switches). When this signal equals '0', the Rx Address Cache is not loaded.
CWAY	2	[3:2]	Cache Way. Designates which way of the Address Cache to load the UPLI signal ReqAddr[56:20] (ADDR[54:18]) into when CLOAD='1'. This signal is only valid when CLOAD='1'.
NUMBEATS	2	[1:0]	Number of Beats. This signal indicates the number of Data Beats that will be transferred for this Request if CMD[5] = b'1' either to transport Atomic Operands, Data for a Write or Write Full, on any Vendor Defined Commands.
TOTAL BITS:	128	4 Sectors	

5.9.2 Uncompressed Response Field

An uncompressed Response can be used to indicate any of the legal UPLI Responses without conditions (Compressed Responses can only be utilized for a subset of the legal UPLI Responses). An Uncompressed Requests consists of 2 sectors.

Table 5-30 Uncompressed Response Field Signals

Uncompressed Response Field (FTYPE = 0x2) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[63:60]	Field Type. This signal, with a value of 0x2, indicates the Field is an Uncompressed Response Field.
VCHAN	2	[59:58]	Virtual Channel. This field carries the UPLI RdRspVC or WrRspVC signal value (determined by the RD/WR signal value).
TAG	11	[57:47]	Tag. This field carries the UPLI RdRspTag or WrRspTag signal value (determined by the RD/WR signal value).
POOL	1	[46]	Pool. This signal indicates whether the TL is utilizing a Pool Credit or a Virtual Channel Credit to issue this Response. The value on this signal is independent of the RdRspCreditPool or WrRspCreditVC signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
LEN	2	[45:44]	Length. This signal carries the value of the UPLI RdRspNumBeats signal for Multi-Beat Read Responses. The LEN field is only meaningful for Multi-Beat Read Responses. Single Beat Read Responses and all Write Responses are always one beat, and therefore the LEN field is not valid for these operations. For these operations this field shall be set to b'00'.
OFFSET	2	[43:42]	Offset. This signal carries the value of the UPLI RdRspOffset signal for Single-Beat Read Responses. For Write Responses (no data is returned with a Write Response) and Multi-Beat Read Responses (the TL is responsible for recreating the RdRspOffset values in the burst transfer), this signal is not valid and should be set to b'00'.
STATUS	4	[41:38]	Status. This signal carries the value of the UPLI RdRspStatus or WrRspStatus signals (determined by the RD/WR signal value).
RD/WR	1	[37]	Read/Write Indicator. This signal indicates whether the Response is a Read or a Write Response (note: AtomicNR Requests cause a Write Response, and AtomicR Requests cause a Read Response).
LAST	1	[36]	Last. This signal carries the value of the UPLI RdRspLast signal. This signal is valid only for Read Responses and should be set to b'0' for Write Responses.
SRCACCID	10	[35:26]	Source Accelerator ID. This field carries the UPLI RdRspSrcPhysAccID or WrRspSrcPhysAccID signal value for this Response depending on whether the Response is for a Read, Write/WriteFull, or an Atomic/AtomicNR Request (identified by the TAG value for this Response). (Note: the SRCACCID signal value, if carried, on the Response is equal to the DSTACCID signal value on the corresponding Request). The SRCACCID value is not required functionally but is useful for debug. Implementations may choose to not carry the SRCACCID value through the TL and/or Switch. When the signal does not contain an accurate value, it should be drive to zero on corresponding UPLI interfaces.
DSTACCID	10	[25:16]	Destination Accelerator ID. This field carries the UPLI RdRspDstPhysAccID or WrRspDstPhysAccID signal depending on whether the Response is for a Read, Write/WriteFull, or an AtomicR/AtomicNR (identified by the TAG value). (Note: the DSTACCID signal value on the Response is equal to the SRCACCID signal value on the

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Uncompressed Response Field (FTYPE = 0x2) signals			
Name	Size (bits)	Position	Description
			corresponding Request). This signal is used to route the Response back to the Accelerator that originally sourced the Request.
RSPTYPE	2	[15:14]	Based on the value of the RD/WR field, this field contains the UPLI RdRspTypeInfo[1:0] (RD/WR = 1) signal value or the UPLI WrRspTypeInfo[1:0] (RD/WR = 0) signal value.
SPARE(S)	14	[13:0]	Fourteen bits are unassigned in this field format.
TOTAL BITS:	64	2 Sectors	

5.9.3 Compressed Request Field

A Compressed Request may be used for a specific subset of UPLI Requests. If the Request is not in this specific subset (see Table 5-33 Compressed Request Command Encoding below), an Uncompressed Request shall be used. To issue a compressed Request, the Request shall hit in the Tx Address Cache and the entry that hit shall not be removed or invalidated before the Request is issued to the Tx TL Flit Channel. A Compressed Request is 2 sectors as opposed to the 4 sectors for an Uncompressed Request. This is achieved by encoding the UPLI ReqCmd signal (by only supporting a subset of the Commands) and ReqLen signal (by insisting that the address of the Request is aligned on a 64 byte boundary and the request is a multiple of 64 bytes in length, and does not cross a 256 byte boundary) into smaller signals and omitting the ReqAttr, ReqAddr[56:20], and ReqMetaData[7:3] signals. The TL shall not be required to issue a Compressed Request and may choose to issue an Uncompressed Request even if conditions would otherwise have permitted the issuance of a Compressed Request.

Table 5-31 Compressed Request Field Signals

Compressed Request Field (FTYPE = 0x3) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[63:60]	Field Type. This signal, with a value of 0x3, indicates the Field is a Compressed Request Field.
CMD	3	[59:57]	Command. This signal carries a compressed version of the UPLI ReqCmd signal value for this Request. See Table 5-33 Compressed Request Command Encoding below for details.
VCHAN	2	[56:55]	Virtual Channel. This signal carries the UPLI ReqVC signal value for this Request.
ASI	2	[54:53]	Address Space Identifier. This signal carries the UPLI ReqASI signal value for this Request.
TAG	11	[52:42]	Tag. This signal carries the UPLI ReqTag signal value for this Request.
POOL	1	[41]	Pool. This signal indicates whether the TL is utilizing a Pool Credit to issue this Request or a Virtual Channel Credit. The value on this signal is independent of the ReqPool signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
LEN	2	[40:39]	Length. This signal carries a compressed version of the UPLI ReqLen signal value for this Request. A value of '0' indicates a 64-byte transfer, a value of '1' indicates a 128-byte transfer, a value of '2' indicates a 192 byte transfer, and a value of '3' indicates a 256 byte transfer.
METADATA	3	[38:36]	Metadata. This signal carries the UPLI signal ReqMetaData[2:0]. The remaining bits of Metadata are not carried and must have a value of b'00000'.
ADDR	14	[35:22]	Address. This signal carries the UPLI ReqAddr[19:6] signal value indicating the aligned 64-byte block within the previously cached 1MB region for this Request that is being addressed. UPLI signal ReqAddr[56:20] are reconstituted from the Rx Address Cache.
SRCACCID	10	[21:12]	Source Accelerator ID. This field carries the UPLI ReqSrcPhysAccID signal value for this Request.
DSTACCID	10	[11:2]	Destination Accelerator ID. This field carries the UPLI ReqDstPhysAccID signal value for this Request.
CWAY	2	[1:0]	Cache Way. Designates which way of the Address Cache to use to reconstitute UPLI signal ReqAddr[56:20] for this Request.
TOTAL BITS:	64	2 Sectors	

The following table lists the set of conditions that shall be met in order to issue a Compressed Request:

Table 5-32 Compressed Request Usage Restrictions.

Compressed Request Usage Restrictions	
CMD	The command requested shall be compressible (i.e. must be one of the command types listed in the table)
ATTR	Reads: The UPLI ReqAttr field shall have a value 0xFF (i.e. all bytes are enabled). Writes: The ReqAttr field shall have a value of 0x00. Atomics are not supported in Compressed Requests.
LEN	The access must be 64, 128, 192, or 256 bytes.
ADDR	The address of the Request must be on a 64 byte alignment and the transfer must not cross a 256 byte boundary.
METADATA	ReqMetadata[7:3] shall be b'00000'.
Address Cache	The Address of the Request shall hit in the Address Cache and the entry in the Address Cache shall not be overwritten (or invalidated) before the Compressed Request is issued.

The following table lists the encodings and legal Commands for the Compressed Request CMD signal. Commands outside of those listed below shall not be supported in Compressed Requests (including Vender Defined Commands which are never Compressed).

Table 5-33 Compressed Request Command Encoding

Compressed Request Command Encoding	
000	Read
001	ReadReduce
010	BlockRead
011	WriteMulticast
100	Write
101	WriteFullMulticast
110	WriteFull
111	BlockWriteFull

5.9.4 Compressed Response Field for Single Beat Read Response

The following tables illustrate the signals within a Compressed Response for a Single Beat Read Response:

Table 5-34 Compressed Response for Single Beat Read, AtomicR, and ReadReduce Field Signals

Compressed Response Field (FTYPE = 0x4) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[31:28]	Field Type. This signal, with a value of 0x4, indicates the Field is a Compressed Response for a Single Beat Read Response for a Read, AtomicR, or ReadReduce Request.
VCHAN	2	[27:26]	Virtual Channel. This signal carries the UPLI RdRspVC signal value.
TAG	11	[25:15]	Tag. This signal carries the UPLI RdRspTag signal value.
POOL	1	[14]	Pool. This signal indicates whether the TL is utilizing a Pool Credit or a Virtual Channel Credit to issue this Response. The value on this signal is independent of the RdRspCreditPool or WrRspCreditVC signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
DSTACCID	10	[13:4]	Destination Accelerator ID. This signal carries the UPLI RdRspDstPhysAccID signal value (Note: the DSTACCID signal value on the Response is equal to the SRCACCID signal value on the corresponding Request). This signal is used to route the Response back to the Accelerator that originally sourced the Request.
OFFSET	2	[3:2]	Offset. This signal carries the value of the UPLI RdRspOffset signal.
LAST	1	[1]	Last. This signal carries the value of the UPLI RdRspLast signal.
INC	1	[0]	In-Network-Collective. This signal indicates whether the Response is associated with a Read or an AtomicR Request (INC = b'0') or with a ReadReduce Request (INC = b'1').
TOTAL BITS:	32	1 Sector	

Table 5-35 Compressed Response for Single Beat BlockRead Field Signals

Compressed Response Field (FTYPE = 0x6) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[31:28]	Field Type. This signal, with a value of 0x4, indicates the Field is a Compressed Response for a Single Beat Read Response for a BlockRead.
VCHAN	2	[27:26]	Virtual Channel. This signal carries the UPLI RdRspVC signal value.
TAG	11	[25:15]	Tag. This signal carries the UPLI RdRspTag signal value.
POOL	1	[14]	Pool. This signal indicates whether the TL is utilizing a Pool Credit or a Virtual Channel Credit to issue this Response. The value on this signal is independent of the RdRspCreditPool or WrRspCreditVC signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
DSTACCID	10	[13:4]	Destination Accelerator ID. This signal carries the UPLI RdRspDstPhysAccID signal value (Note: the DSTACCID signal value on the Response is equal to the SRCACCID signal value on the corresponding Request). This signal is used to route the Response back to the Accelerator that originally sourced the Request.
OFFSET	2	[3:2]	Offset. This signal carries the value of the UPLI RdRspOffset signal.
LAST	1	[1]	Last. This signal carries the value of the UPLI RdRspLast signal.
SPARE(S)	1	[0]	One bit is unassigned in this field format
TOTAL BITS:	32	1 Sector	

Transaction Layer (TL)

142

The TL shall not be required to issue a Compressed Response. Even if the conditions to issue a Compressed Response are met, the TL may issue an Uncompressed Response. The following table lists the set of conditions that shall be met to issue a Compressed Request for a Single-Beat Read Response:

Table 5-36 Compressed Response for Single-Beat Read Response Usage Restrictions.

Compressed Response for Single-Beat Read Response Usage Restrictions	
STATUS	The STATUS field of the response must have a value of b'0000': "OKAY (Normal Completion)".

5.9.5 Compressed Response Field for a Write or Multi-Beat Read Response

The following tables illustrate the signals within a Compressed Response for a Write Response or a Multi-Beat Read Response:

Table 5-37 Compressed Response for a Write, WriteFull, AtomicNR, WriteMulticast, WriteFullMulticast, AtomicNRMulticast, or for a Multi-Beat Read, AtomicR, or ReadReduce Request Field Signals

Compressed Response Field (FTYPE = 0x5) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[31:28]	Field Type. This signal, with a value of 0x5, indicates the Field is a Compressed Response for a Write, WriteFull, AtomicNR, WriteMulticast, WriteFullMulticast, or AtomicNRMulticast Request or is for a Multi-Beat Read, AtomicR, or ReadReduce Request.
VCHAN	2	[27:26]	Virtual Channel. This signal carries the UPLI RdRspVC or WrRspVC signal value.
TAG	11	[25:15]	Tag. This signal carries the UPLI RdRspTag or WrRspTag signal value.
POOL	1	[14]	Pool. This signal indicates whether the TL is utilizing a Pool Credit or a Virtual Channel Credit to issue this Response. The value on this signal is independent of the RdRspCreditPool or WrRspCreditVC signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
DSTACCID	10	[13:4]	Destination Accelerator ID. This signal carries the UPLI RdRspDstPhysAccID or WrRspDstPhysAccID signal value (Note: the DSTACCID signal value on the Response is equal to the SRCACCID signal value on the corresponding Request). This signal is used to route the Response back to the Accelerator that originally sourced the Request.
LEN	2	[3:2]	Length. This signal carries the values for the UPLI RdRspNumBeats signal for a Read Response and b'00' for a Write Response. This signal is only valid for a Read Response and indicates the number of contiguous 64-byte data beats to be returned.
RD/WR	1	[1]	This signal indicates whether the Response is a Read Response or a Write Response (b'1' is a Read, b'0' is a Write)
INC	1	[0]	In-Network-Collective. This signal indicates whether the Response is associated with a Write, WriteFull, AtomicNR Request or is associated with a Multi-Beat Read or AtomicR Request for INC = b'0'. For INC = b'1' the Response is associated with a WriteMulticast, WriteFullMulticast or AtomicNRMulticast Request or is associated with a Multi-Beat ReadReduce Request (INC = b'1')
TOTAL BITS:	32	1 Sector	

Table 5-38 Compressed Response for a BlockWriteFull or Multi-Beat BlockRead Field Signals

Compressed Response Field (FTYPE = 0x7) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[31:28]	Field Type. This signal, with a value of 0x7, indicates the Field is a Compressed Response for a Write, WriteFull, AtomicNR, WriteMulticast, WriteFullMulticast, or AtomicNRMulticast Request or is for a Multi-Beat Read, AtomicR, or ReadReduce Request.
VCHAN	2	[27:26]	Virtual Channel. This signal carries the UPLI RdRspVC or WrRspVC signal value.
TAG	11	[25:15]	Tag. This signal carries the UPLI RdRspTag or WrRspTag signal value.
POOL	1	[14]	Pool. This signal indicates whether the TL is utilizing a Pool Credit or a Virtual Channel Credit to issue this Response. The value on this signal is independent of the RdRspCreditPool or WrRspCreditVC signal on the UPLI interface (TL Credit management is independent of UPLI Credit Management).
DSTACCID	10	[13:4]	Destination Accelerator ID. This signal carries the UPLI RdRspDstPhysAccID or WrRspDstPhysAccID signal value (Note: the DSTACCID signal value on the Response is equal to the SRCACCID signal value on the corresponding Request). This signal is used to route the Response back to the Accelerator that originally sourced the Request.
LEN	2	[3:2]	Length. This signal carries the values for the UPLI RdRspNumBeats signal for a Read Response and b'00' for a Write Response. This signal is only valid for a Read Response and indicates the number of contiguous 64-byte data beats to be returned.
RD/WR	1	[1]	This signal indicates whether the Response is a Read Response or a Write Response (b'1' is a Read, b'0' is a Write)
SPARE(S)	1	[0]	One bit is unassigned in this field format
TOTAL BITS:	32	1 Sector	

The TL shall not be required to issue a Compressed Response. Even if the conditions to issue a Compressed Response are met, the TL may issue an Uncompressed Response. The following table lists the set of conditions that shall be met to issue a Compressed Request for a Single-Beat Read Response:

Table 5-39 Compressed Response for Write or Multi-Beat Read Response Usage Restrictions.

Compressed Response for Single-Beat Read Response Usage Restrictions	
STATUS	The STATUS field of the response must have a value of b'0000': "OKAY (Normal Completion)".

5.9.6 Flow Control/NOP Field.

The following table illustrates the signals within a Flow Control/NOP Field.

Table 5-40 Flow Control/NOP Field

Compressed Response Field (FTYPE = 0) signals			
Name	Size (bits)	Position	Description
FTYPE	4	[31:28]	Field Type. This signal, with a value of 0, indicates the Field is a Flow Control/NOP field.
ReqCmd	6	[27:22]	Request Cmd. Returns the Request Command Credits (i.e. Credits for Requests from the UPLI Request Channel). The field format is 'tvvccc' where 't' is a type field (0=pool, 1=VC) of the Credits, 'vv' is the Virtual Channel of the Credits (valid only if t = 1), and 'ccc' is the number of Credits being returned (0 to 7).
RspCmd	6	[21:16]	Response Cmd. Returns the Response Command Credits (i.e. Credits for Rd Responses (not data) and the Wr Responses from the Requests from the UPLI Rd Response/Data and Write Response Channels). The field format is 'tvvccc' where 't' is a type field (0=pool, 1=VC) of the Credits, 'vv' is the Virtual Channel of the Credits (valid only if t = 1), and 'ccc' is the number of Credits being returned (0 to 7).
ReqData	8	[15:8]	Request Data. Returns the Request Data Credits (i.e. Credits for data from the UPLI OrigData Channel). The field format is 'tvvccccc' where 't' is a type field (0=pool, 1=VC) of the Credits, 'vv' is the Virtual Channel of the Credits (valid only if t = 1), and 'ccccc' is the number of Credits being returned (0 to 31).
RspData	8	[7:0]	Response Data. Returns the Response Data Credits (i.e. Credits for data from the UPLI Read Response/Data Channel). The field format is 'tvvccccc' where 't' is a type field (0=pool, 1=VC) of the Credits, 'vv' is the Virtual Channel of the Credits (valid only if t = 1), and 'ccccc' is the number of Credits being returned (0 to 31).
TOTAL BITS:	32	1 Sector	

5.10 Recommended TL backoff modes.

The following is a list of backoff modes that are recommended to be implemented in a UALink TL:

- A mode to limit the number of Uncompressed Requests that can be packed into a Control Half-Flit to be 1 (at most 2 Uncompressed Requests can be packed into a Control Half-Flit).
- A mode to limit the number of Compressed Requests that can be packed into a Control Half-Flit to be 1, 2, or 3 (at most 4 Compressed Requests can be packed into a Control Half-Flit).
- A mode to limit the number of Uncompressed Responses that can be packed into a Control Half-Flit to be 1, 2, or 3 (at most 4 Uncompressed Responses can be packed in a Control Half-Flit).
- A mode to control the number of Compressed Responses that can be packed into a Control Half-Flit to be 1, 2, 3, 4, 5, 6, or 7 (at most 8 Compressed Responses can be packed into a Control Half-Flit).
- A mode to control the number of non-NOP Flow Control/NOP indications that can be packed into a Control Half-Flit to be 1, 2, 3, 4, 5, 6, or 7 (at most 8 non-NOP Flow Control/NOP Indications can be packed into a Control Half-Flit). It must always be possible to pack 8 NOP Flow Control/NOP indications in a Control Half-Flit to indicate NOP.

The above modes are only meant to limit the number of the type of item that may be packed into a Control Half-Flit. The allowed placements of the items within a Control Half-Flit are not affected.

- A mode to disable using the Compressed Address Caches (logically the same as disabling both the TL Tx and TL Rx Address Caches)

6 UALink In-Network Collectives

6.1 Introduction

The workloads on scale-up networks typically do not fit within a single Accelerator and therefore must be partitioned across multiple Accelerators. To facilitate the computation and communication between Accelerators processing these cross-Accelerator workloads, various Collective Communication Libraries such as NCCL, RCCL, MSCCL, MSCCL++ have been created to facilitate communication and computation between Accelerators using a concept known as “Collectives”.

In UALink, a Collective shall be defined as an operation that involves a set of Accelerators, also known as a “Group”. Each Accelerator in the Group is also referred to as a “Rank”. These Ranks shall have input data buffers and output data buffers (for certain Collectives, the input data buffer or the output data buffer may only be present on one “Root Rank”). The input and output buffers may be at the same locations in memory but shall not partially overlap (i.e. the input and output buffers may be at the same memory locations, or they shall have no overlap).

A Collective operation reads from the input buffer or buffers in memory on the Accelerators, processes the data read through a reduction function if data is read from multiple ranks, and then writes to the output buffer or buffers in memory on the Accelerators.

6.1.1 BROADCAST Collective

The BROADCAST Collective is shown below in Figure 6-1:

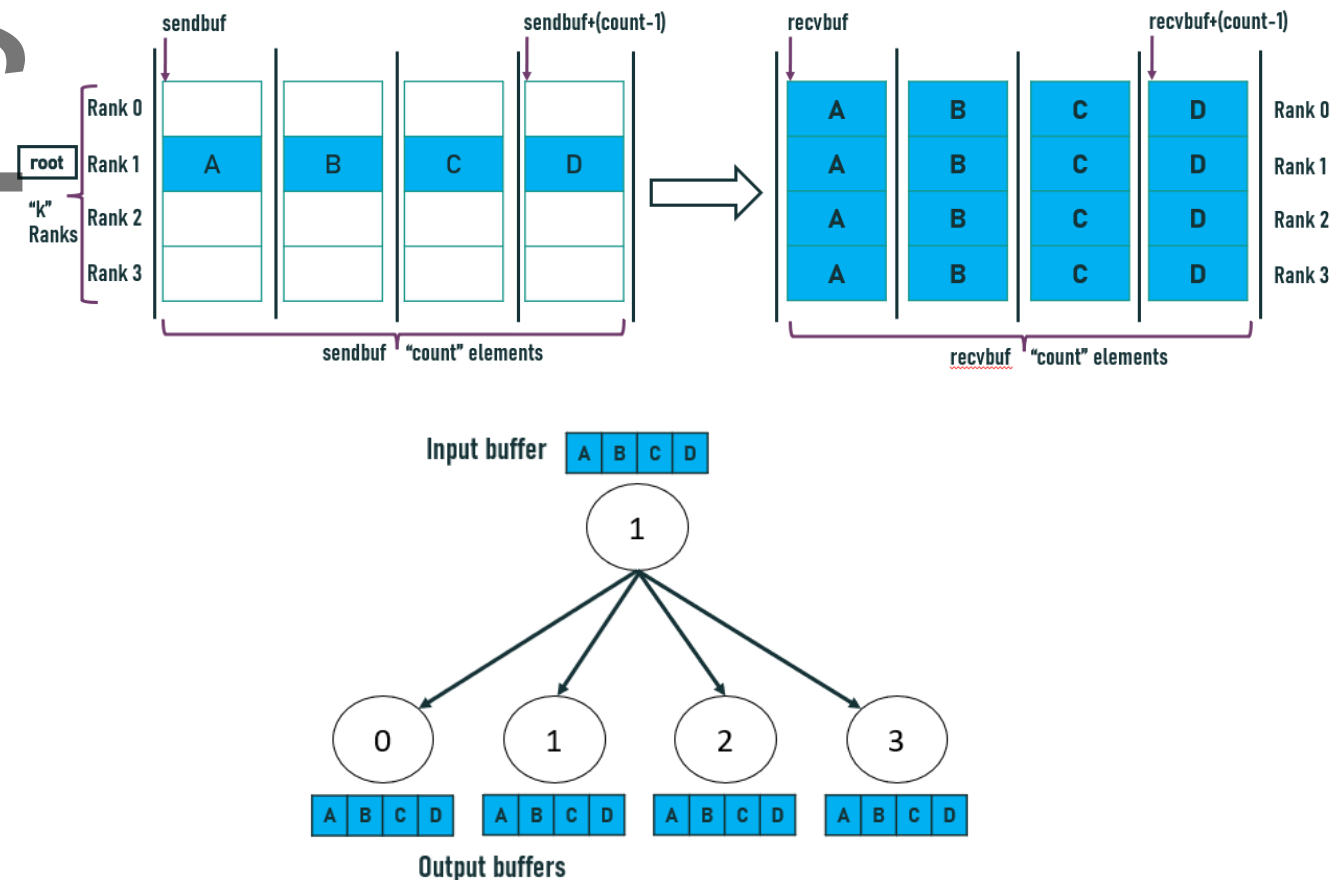


Figure 6-1 BROADCAST Collective

The BROADCAST Collective copies an array of “count” elements located at a UALink address “sendbuf” from the Root Rank to buffers of “count” elements on all k Ranks in the Collective at a UALink address “recvbuf”. If the sendbuf and recvbuf addresses are equal, an “in-place” Collective occurs. If the sendbuf and recvbuf addresses are not equal and the input buffer and output buffer overlap, the results of the Collective are indeterminate.

6.1.2 REDUCE Collective

The REDUCE Collective is shown below in Figure 6-2:

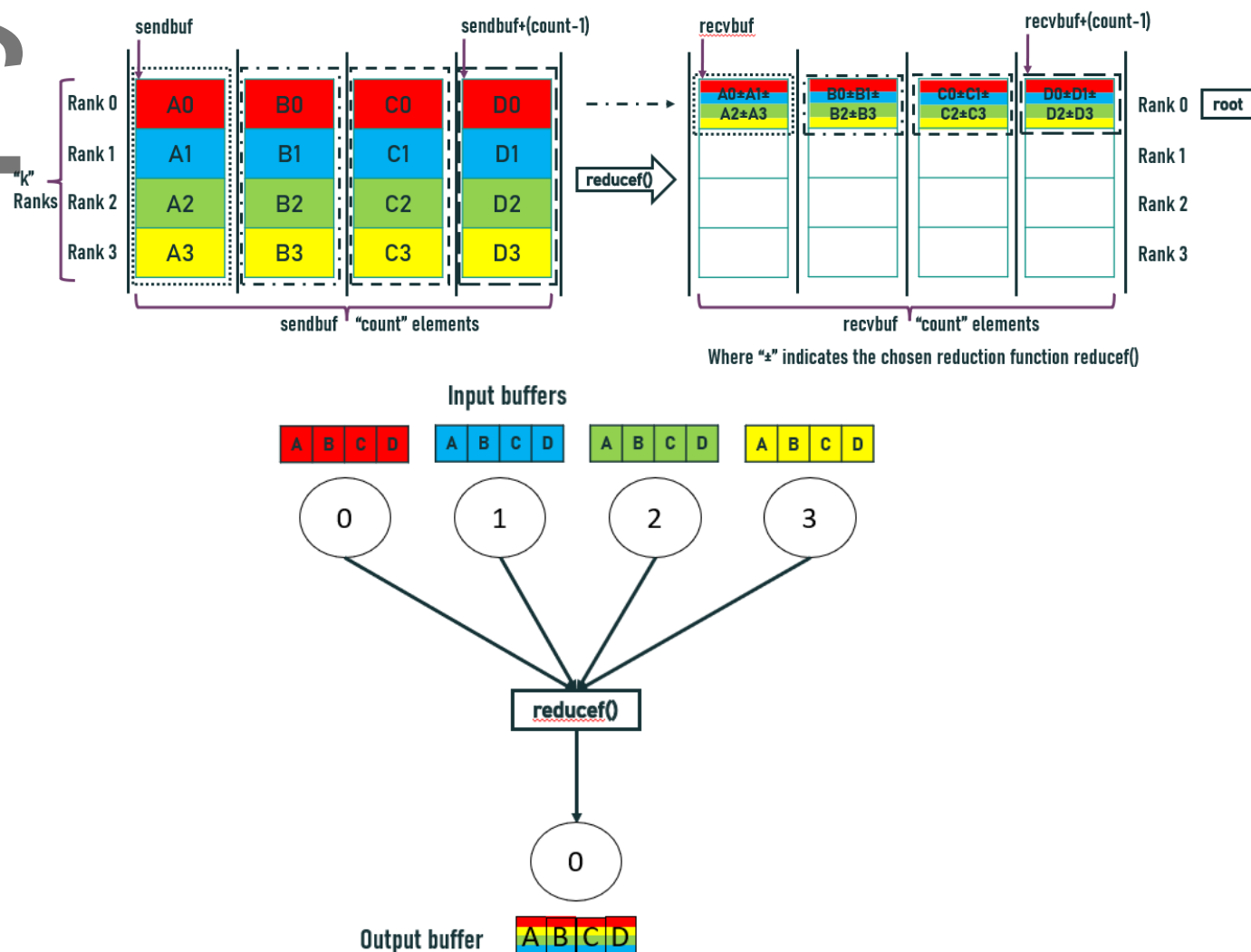


Figure 6-2 REDUCE Collective

The REDUCE Collective reduces each of the corresponding elements in a given location in sendbuf (e.g. A0/A1/A2/A3, B0/B1/B2/B3, etc.) in sendbuf on all the Ranks in the Group using a specified Reduction Function “`reducef()`” to produce a single output value which is placed at the corresponding position in the Receive Buffer “`recvbuf`” on the Root Rank. If the sendbuf and recvbuf addresses are equal, an “in-place” Collective occurs. If the sendbuf and recvbuf addresses are not equal and the input buffer and output buffer overlap, the results of the Collective are indeterminate.

6.1.3 ALL-REDUCE Collective

The ALL-REDUCE Collective is shown below in Figure 6-3 :

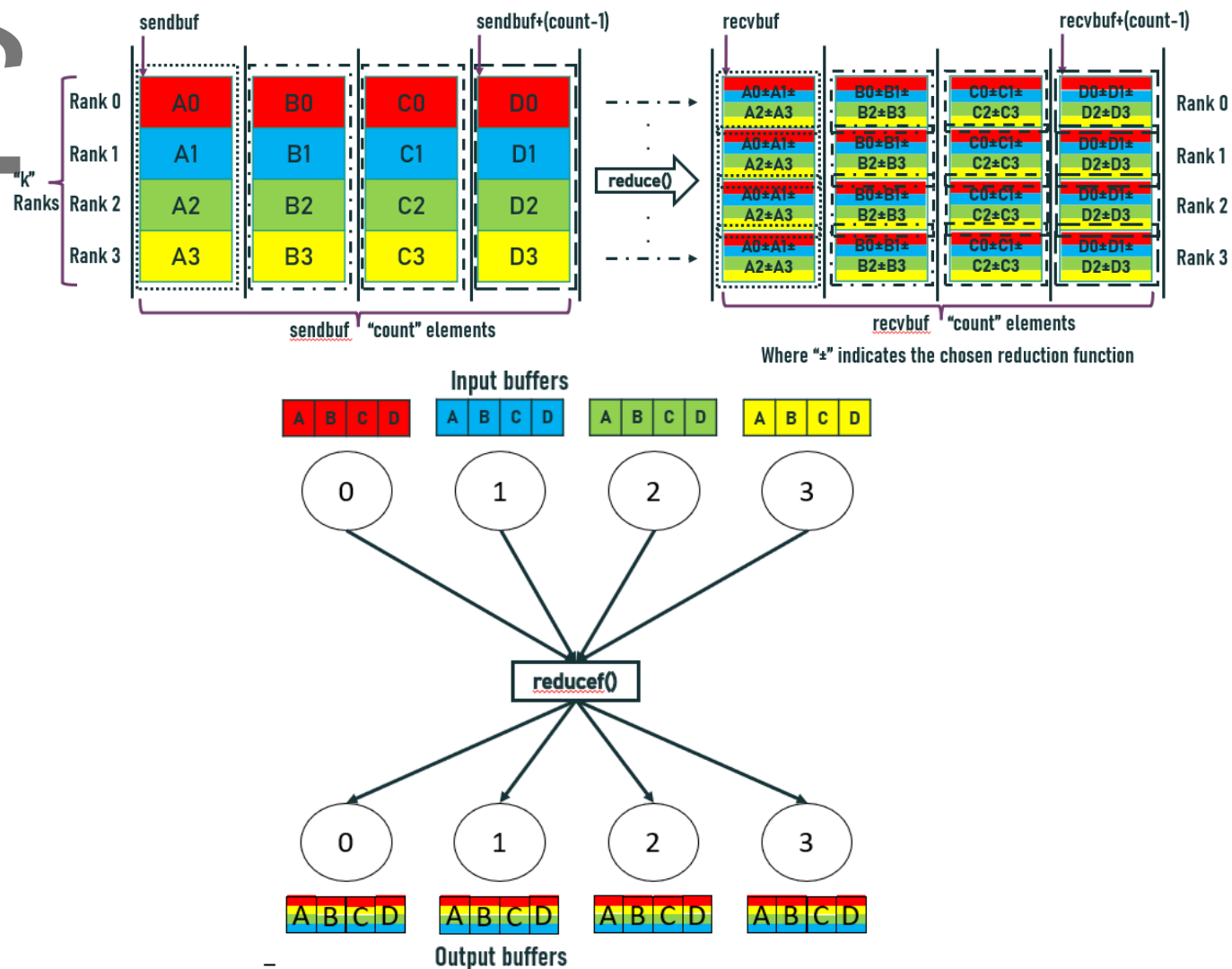


Figure 6-3 ALL-REDUCE Collective

6.1.4 REDUCE-SCATTER Collective

The REDUCE-SCATTER Collective is shown below in Figure 6-4:

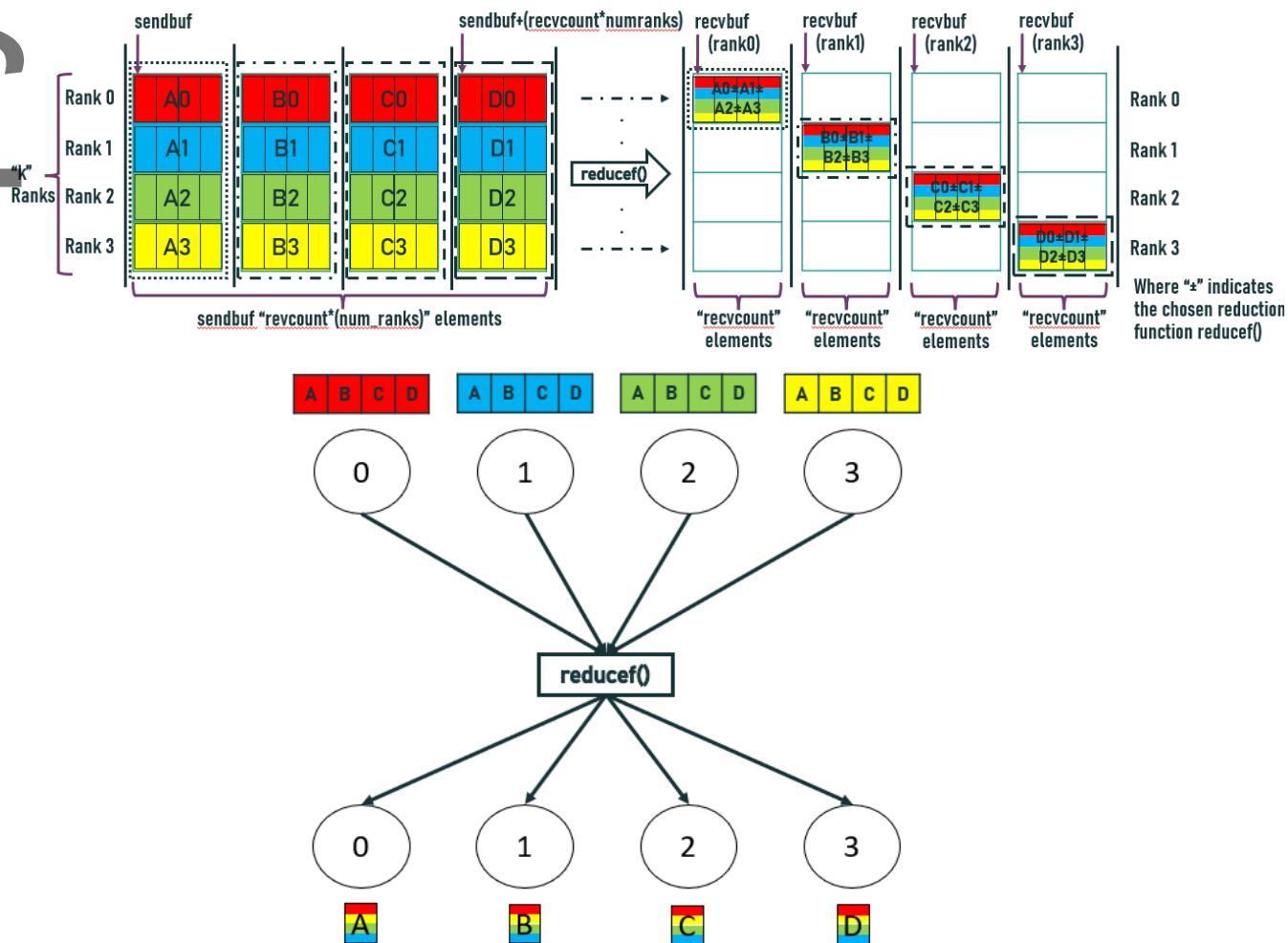


Figure 6-4 REDUCE-SCATTER Collective

The REDUCE-SCATTER Collective reduces repeated arrays of “`recvcount*(num_ranks)`” elements from each input Rank through a reduction function (`reducef()`) to produce a set of `num_ranks` arrays of size `recvcount` (each labeled box, A0-A3, B0-B3, C0-C3, D0-D3, or A, B, C, and D, above represents an array of elements rather than a single element).

The 1st array produced (A0-A3 above) is placed in Rank0’s memory at the `recvbuf` address for Rank0, the 2nd array produced (B0-B3 above) is placed in Rank1’s memory at the specified `recvbuf` address and so on for the remaining Ranks.

If `recvbuf = sendbuf+(rank*recvcount)` for all Ranks, an “in-place” Collective occurs. If the `sendbuf` and `recvbuf` addresses do not follow this rule, and the `sendbuf` buffer and the `recvbuf` buffers overlap, the results of the Collective are indeterminate.

A series of repeated REDUCE Collectives can be utilized to implement REDUCE-SCATTER as shown below in Figure 6-5:

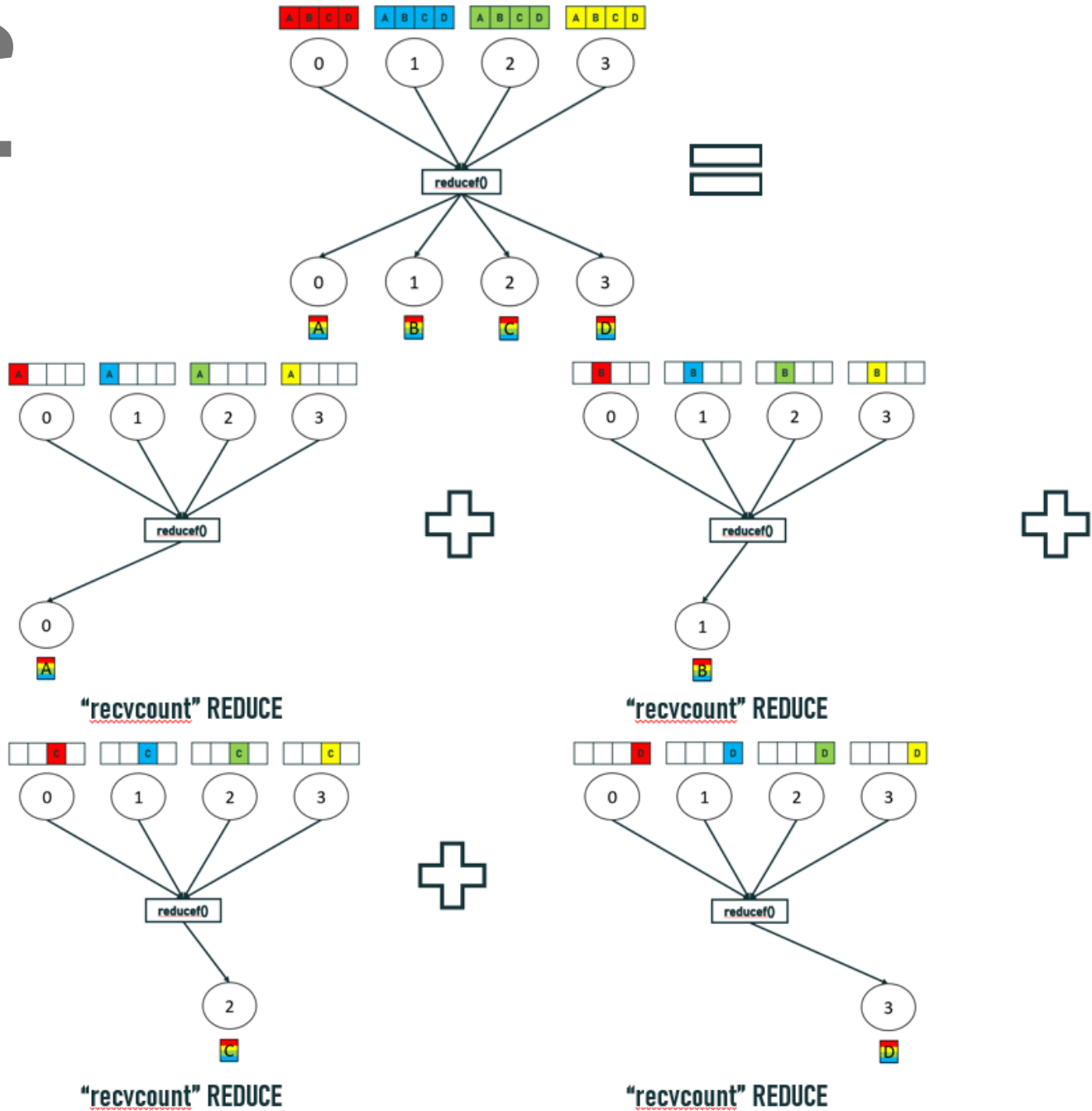


Figure 6-5 REDUCE-SCATTER implemented with repeated REDUCE Collective

6.2 Collective Primitives

UALink provides Collective Primitives as a mechanism to help implement Collectives. Collective Primitive Requests, ReadReduce, WriteMulticast, WriteFullMulticast, and AtomicNRMulticast, are variants of the Read, Write, WriteFull, and AtomicNR Requests, respectively, that UALink uses for unicast Requests. Sequences of read and write accesses to local memory on the Accelerators along with Collective Primitive Requests can be used to implement Collectives.

Collective Primitive Requests are issued by an Accelerator, replicated at the Switch, and then issued to the Accelerators in the Collective Group. When issued from the Requesting Accelerator, each Collective Primitive Request shall have an associated "GroupID" value (which shall be conveyed in ReqDstPhysAccID signal) that shall be used to index a Group Table on the Switch that shall indicate the members of the Collective Group. This Group Table shall be pre-loaded when a Collective Group is created and before any Collective Primitive Requests with that GroupID are issued.

A ReadReduce Request issued from an Accelerator also has additional indications of the Data Type of the data being read, the Reduction Operation to be performed on the data being read, the precision of the reduction if applicable and the Rounding Mode to be used if applicable (all of which shall be conveyed in the ReqSrcPhysAccID signal). The Switch shall look up the Group members and shall issue a ReadReduce operation to each Accelerator in the Group.

The Destination Accelerators shall separately return a Read Response Status indication and Response Data for the individual ReadReduce Requests. The Switch shall reduce the individual copies of the Response Data into a single copy of data according to the specified Reduction Operation, Reduction Precision (if applicable), DataType, and Rounding mode (if applicable). The Switch shall also reduce the Read Response Status indications received from each of the Accelerators in the Group. The Switch shall then return the reduced single copy of the Response Data and Response Status to the Requesting Accelerator that initially issued the ReadReduce Request.

WriteMulticast, WriteFullMulticast, and AtomicNRMulticast Requests shall follow a similar progression. The Requesting Accelerator shall issue the WriteMulticast, WriteFullMulticast, or AtomicNRMulticast Request with write data or operand data, respectively, to the Switch. The Switch shall look up the Group members and shall issue a WriteMulticast, WriteFullMulticast, or AtomicNRMulticast Request, respectively, to each Accelerator in the Group. Each destination Accelerator shall update memory and shall return a Write Response status indication. The WriteMulticast, WriteFullMulticast, and AtomicNRMulticast Collective Primitives shall update memory in the same manner as a Write, WriteFull, or AtomicNR Request, respectively. The Switch shall combine the returned Write Response Status indications into a single overall Write Response Status indication and shall return this Write Response Status indication to the Requesting Accelerator.

Collective Primitives issued from a given Accelerator typically use the same mechanism to select an egress port on the Accelerator as unicast Requests to spread the Collective Primitives Requests across the Pod Switches in the same manner as unicast Requests.

As with the AtomicNR operations in UALink, the exact nature of the atomic read-modify-write operation performed by an AtomicNRMulticast is implementation specific.

6.2.1 Switch Structures to support Collective Primitives

To properly support the replication of the initial Collective Primitive sent by the Accelerator to the Switch, the Switch shall implement various additional tables and structures.

1. Each Port shall contain a Group Table consisting of up to 1024 entries. Each entry in this table shall consist of a valid bit and a bitmask of a width equal to the radix of the Switch. The index of each bit in the Group Table entry shall correspond to a Physical Accelerator ID for an Accelerator within the Collective Group (i.e. Accelerator ID numbers shall be assigned within the range 0 to the radix of the switch minus 1). The Group Table shall be programmable by privileged system software.
2. Each Switch Port shall contain a 10-bit register to identify the Accelerator ID of the Accelerator attached to the Port. These registers shall be programmed by the Pod Controller.

For Collective Primitive Requests, the ReqDstPhysAccID field shall contain a GroupID[9:0] value that specifies the entry in the Group Table associated with the Collective Primitive. This GroupID value shall be present on both the Request issued by the Requesting Accelerator and on the duplicated Requests issued by the Switch to the Destination Accelerators.

The ReqSrcPhysAccID field shall be overloaded between the Requesting Accelerator and the Switch with a ReduceOpDataType[6:0] value in bits [6:0] and by a RoundingMode[2:0] value in bits [9:7]. The ReduceOpDataType value indicates the Reduction Operation, the DataType of the data, the Reduction precision if necessary, and the write back Data Type if necessary.

For a ReadReduce Collective Primitive, the Switch shall set the ReqSrcPhysAccID field on the replicated ReadReduce Requests to the Requesting Accelerator ID. This shall be accomplished by referencing a 10-bit register at the given Ingress Port that indicates the Accelerator ID for the Accelerator attached to that Port. This register shall be programmed by the Pod Controller.

If the Collective Primitive is not a ReadReduce, The ReqSrcPhysAccID field shall contain the Accelerator ID of the Requesting Accelerator on both the initial Request from the Requesting Accelerator and on the replicated Requests to the Destination Accelerators in the Group.

The structures to Support a Collective Primitive and the progression of a Collective Primitive through these structures are illustrated below in Figure 6-6:

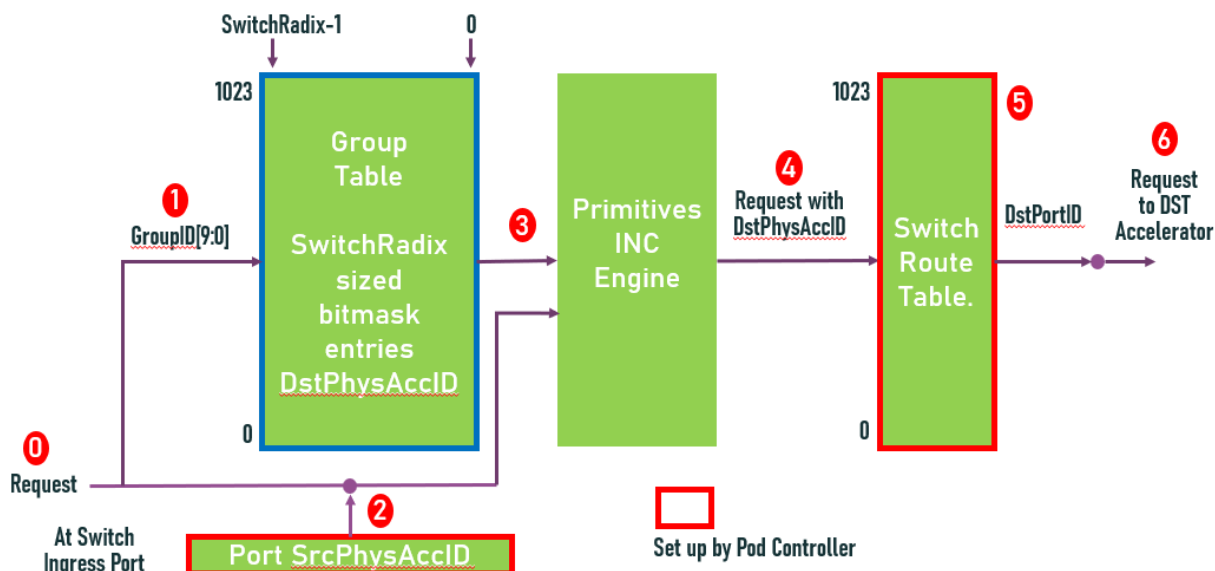


Figure 6-6 Collective Primitives Support Structures

A Collective Primitive Request is Received by the Switch at 0. The GroupID[9:0] value in ReqDstPhysAccID shall be used to index the GroupTable to obtain the bitmask entry in the GroupTable (the Group Accelerator Mask) and the Valid bit. This bitmask shall indicate the Accelerators in the Group in terms of Accelerator Physical IDs (i.e. bit [5] = b'1' implies the Accelerator with Physical ID 5 is in the group) 1.

If the Valid bit is not set, or the bit corresponding to the Requesting Accelerator is not set in the Group Accelerator Mask, the Switch shall drop the Collective Primitive Request and shall issue a TARGET ABORT Response to the Switch for the Collective Primitive and return a fixed data pattern for ReadReduce. If the Group Accelerator Mask has less than two bits set, the Accelerator shall drop the Collective Primitive Request and shall issue a TARGET ABORT Response to the Switch. The Switch may begin processing the Collective Primitive before recognizing this error condition. If the ReqLen, ReqAddr, and ReqAttr fields for a Read Reduce Collective Primitive do not indicate a region of memory that is aligned to start at and is a non-zero integer multiple of the size of the output data elements being returned to the Switch, the Switch shall drop the Collective Primitive Request and shall issue a TARGET ABORT response to the Switch. If the values for either ReduceOpDataType[6:0] or RoundingMode[2:0] conveyed in ReqSrcPhysAccID are not one of the legal values, the Accelerator shall drop the Collective Primitive Request and shall issue a TARGET ABORT Response to the Switch.

Otherwise, the Group Accelerator Mask, shall be passed on to the Primitives INC engine 3 along with the Request and the Physical Accelerator ID of the Requesting Accelerator 2 for processing.

The Primitives INC Engine shall issue a Request to Switch Route Table for each Physical Accelerator ID indicated in the Group Accelerator Mask 4. The Switch Route Table (which is also used for non-INC Requests) 5 shall provide the Destination Port Identifier for the replicated Request instance and the replicated Request instance shall be routed to the Destination Port and issued to the Destination Accelerator 6.

Isolation between Virtual Pods for Collective Primitives shall be provided by the Switch Route Table in the same manner as for non-INC Requests.

6.2.2 Collective Primitive Request/Response Flows

The following figure, Figure 6-7, illustrates the flow of the Collective Primitive Requests:

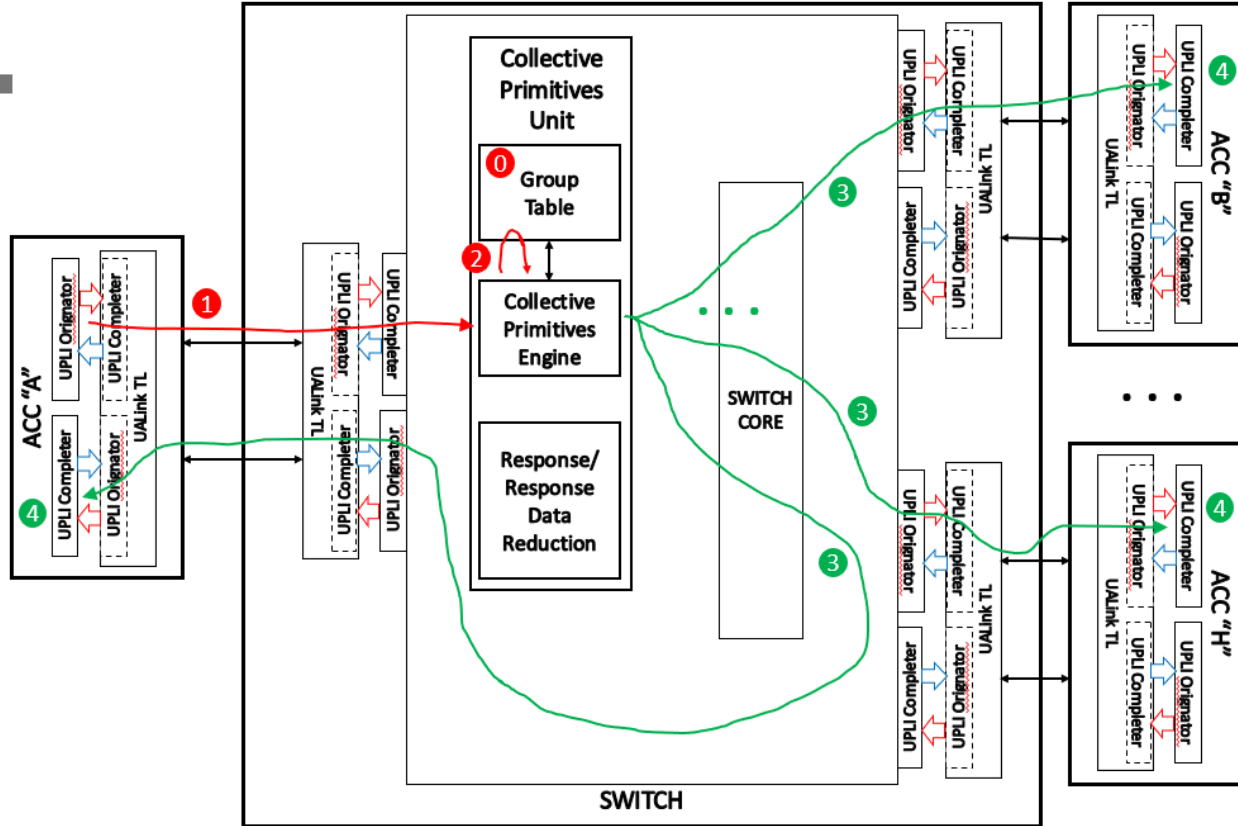


Figure 6-7 Collective Primitive Request Flow

The Group Table entry for the Block Collective shall be loaded prior to the initiation of the Collective Primitive ①. A Collective Primitive Request is initiated by Accelerator "A" ① and is transmitted to the Collectives Primitive Engine. The ReqDstPhysAccID field shall be overlaid with the GroupID[9:0] value for the Collective Primitive. The ReqSrcPhysAccID field shall be overlaid with the ReduceOpDataType[6:0] in ReqSrcPhysAccID[2:0] and with the RoundMode[2:0] value in ReqSrcPhysAccID[9:7] for a ReadReduce Collective Primitive and shall indicate the Requesting Accelerator Physical Accelerator ID for all other Collective Primitives. For a ReadReduce Request, without regard to the RoundMode value, the ReqAttr field shall contain an 8-bit Stochastic Seed value that shall be used when the Rounding Mode is Stochastic.

The Collectives Primitives Engine shall look up the Group Table on the Switch utilizing the GroupID[9:0] value to obtain the Group Accelerator Mask ②. The Collectives Primitive Engine shall issue a series of replicated Collective Primitives Requests to the various destination Accelerators with the ReqSrcPhysAccID field set to indicate the Physical Accelerator ID of the Requesting Accelerator ③. The following signals in the Request Channel shall be passed through the Switch unchanged from the initial Request to each of the duplicated Requests: ReqASI, ReqTag, ReqNumBeats, ReqAddr, ReqCmd, ReqLen, ReqMetaData, ReqVC. If OrigDataError is asserted for

any beat of Originator Data for a WriteMutlicast, WriteFullMulticast, or an AtomicNRMulticast, the Switch shall mark that beat with an OrigDataError in all the replicated Requests from the Switch to the Destination Accelerators.

The Collectives Primitive Engine shall maintain tracking logic to track the outstanding Requests for each Collective Primitive, to reduce the Responses for the outstanding Requests for each Collective Primitive, and to return a final response to the Requesting Accelerator when all the Responses for the Collective Primitive have been received and reduced.

When an individual Request is received at the Destination Accelerator Completer, the Completer shall retain indications for the type of Request received ④ in order to properly drive the RdRspTypeInfo[1:0] and WrRspTypeInfo[1:0] signals.

The Destination Accelerator shall distinguish tags according to a combination of the ReqTag value, the ReqSrcPhysAccID value, the ReqPortID value, and whether the Request is a BlockWrite, BlockWriteFull, or BlockRead Request or the Request is any other Request Type ④. This tracking scheme provides Block* Requests for Block Collectives (whose tags are assigned by the Switch issuing the Requests) an independent tag space from the tag space for the Collective Primitives and normal Requests (whose tags are assigned by the Accelerator issuing the Requests) preventing tag collisions between the two groups of Requests.

The following figure, Figure 6-8,, illustrates the flow of the Collective Primitive Responses:

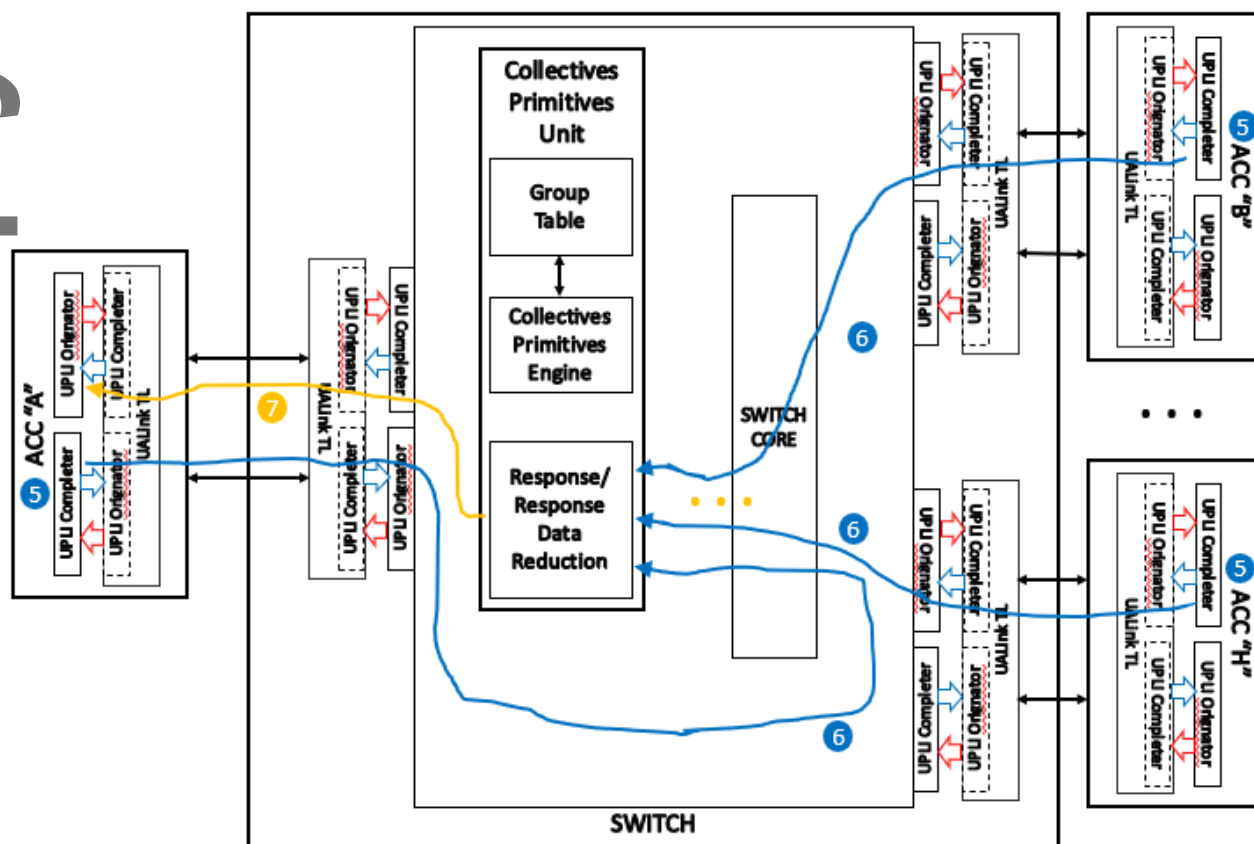


Figure 6-8 Collective Primitive Response Flow

After the Destination Accelerator processes the Collective Primitive ⑤, the UPLI Completer shall return a Read or Write Response to the Switch ⑥ with the RdRspInfo or WrRspInfo signals driven to the appropriate values. The Tracking Logic in the Collectives Primitive unit shall reduce Response Statuses and any Data and provide a final Response to the Requesting Accelerator ⑦. If a RdRspDataError is asserted on a beat of data from any of the Ranks for a given location, the Switch shall mark the beat corresponding to that location with a RdRspDataError in the reduced single Read Response for that location.

The following table, Table 6-1, shows how Read Response Statuses and Write Response Statuses are reduced into a single status. In general, if all the non-OKAY Response Statuses are the same, the reduced Response Status matches the given non-OKAY Response Status. If multiple conflicting

Response Statues are received, the reduced Response Status is “Mixed”. If all the Response Statues are OKAY, an OKAY Response Status is returned.

Table 6-1 Read/Write Response Reduction

Received --> Current V	OKAY	TARGET ABORT	DECODE ERROR	PROTECTION VIOLATION	CMPTO (Completion Timeout)
OKAY	OKAY	TARGET ABORT	DECODE ERROR	PROTECTION VIOLATION	CMPTO (Completion Timeout)
TARGET ABORT	TARGET ABORT	TARGET ABORT	Mixed	Mixed	Mixed
DECODE ERROR	DECODE ERROR	Mixed	DECODE ERROR	Mixed	Mixed
PROTECTION VIOLATION	PROTECTION VIOLATION	Mixed	Mixed	PROTECTION VIOLATION	Mixed
CMPTO (Completion Timeout)	CMPTO (Completion Timeout)	Mixed	Mixed	Mixed	CMPTO (Completion Timeout)
Mixed	Mixed	Mixed	Mixed	Mixed	Mixed

6.3 Block Collectives

UALink provides Block Collectives as an alternative means to completely offload BROADCAST, REDUCE, and ALL-REDUCE Collectives to the Switch for processing. In a Block Collective, all accesses to memory to perform the Collective are instigated by the Switch. Block Collectives do not include the ALL-GATHER and REDUCE-SCATTER Collectives. For ALL-GATHER the Input Buffer at each Rank may have a unique address and similarly for the Output Buffer for each Rank for REDUCE-SCATTER. It is impractical to implement these Collectives as a Block Collective because a list of addresses for each rank would have to be conveyed to the Switch for each Block Collective (up to 1024 57-bit addresses).

The Input and Output Buffer addresses for Block Collectives shall start on a 256-byte address boundary and have a transfer length that is a multiple of 256 bytes. To initiate a Block Collective, the Requesting Accelerator shall issue a BlockCollectiveInvoke Request which is a write operation that delivers a 64-byte data payload to the Switch. The 64-byte data payload shall contain a Block Collective Control Block. The Block Collective Control Block shall contain the parameters necessary to specify the Collective.

When a BlockCollectiveInvoke Request arrives at a Switch, the Switch shall always return an "OKAY" Write Response Status indication without undue delay. The Switch shall then, subject to various error checks (described below Table 6-4), enqueue the Block Collective in a Block Collective Control Block Queue (Figure 6-9) at the Port that received the BlockCollectiveInvoke. If an error occurred when enqueueing the Block Collective, the Switch shall drop the Block Collective and indicate an error by updating the Block Collective Status Buffer (Figure 6-16).

Once a Block Collective has been enqueued in the Switch, it shall eventually be dequeued, according to an implementation specific policy, to be processed. To distinguish between Requests for Block Collectives and Requests for unicast Requests and Collective Primitives, the Switch shall issue a series of BlockRead and BlockWriteFull Requests for all reads to memory and all updates to memory (including updates to the Status Buffer), respectively, due to a Block Collective. The Tag values for these BlockRead and BlockWriteFull Requests shall be generated by the Switch and shall be independent of (i.e. the values may overlap) the tag values used for unicast Requests and Collective Primitive Requests.

For a given 256-byte block in a Collective, the Switch shall perform Block Read(s) to the Input Buffer(s), shall then perform the reduction on the read data if applicable, and then shall perform BlockWriteFull(s) to the Output Buffer(s). The order of the Block Read(s) for a given 256-byte block to the various Ranks in the Collective shall be implementation dependent. The order of the Block Write(s) for a given 256-byte block to the various Rank(s) in the Collective shall be implementation dependent.

For any reduction operation that is not associative, the reduction for a given 256-byte block shall occur in an implementation specific order that shall remain the same for every Block Collective instance and shall be determined solely by the set of Ranks involved in the Collective. The Block Read Requests and Block Write Requests for a given Block Collective across multiple Collectives may be freely interleaved in an implementation specific fashion subject to the above constraints for Requests for a given 256-byte block within the Collective.

If a BlockWriteFull or a BlockRead that accesses the input buffer(s) or output buffer(s) receives a Response Status indication other than "OKAY", the Switch shall cease issuing new BlockWriteFull and BlockRead Requests for the Block Collective and shall wait for any outstanding BlockWriteFull or BlockRead Requests to return their Responses. The Block Collective shall then be dropped and

the Switch shall write the Status Buffer using a BlockWriteFull to indicate the failure of the Block Collective.

If the Block Collective completes successfully, the Switch shall update the Status Buffer using a BlockWriteFull to indicate the successful completion of the Block Collective,

While Block Collectives directly implement the full semantics of a Collective and therefore repeated invocations of a Block Collective are not required to perform a Collective, it may still be useful to shard (divide) the input and output arrays in the Collective into distinct subsets that are processed by different Accelerators in the Collective.

Furthermore, it may be useful to further divide the subset of the Collective assigned to a given Accelerator into smaller distinct subsets that are processed by different Block Collectives that are issued to the different Switches in the Pod. The Switch in the Pod that processes a Block Collective invocation is determined by the Egress Port the Accelerator issues the BlockCollectiveInvoke Request on.

Unlike Collective Primitives, which are typically issued on an Egress Port derived from the address of the Collective Primitive, the Egress Port of a BlockCollectiveInvoke Request is typically controlled by an Accelerator specific means, usually software driven, that allows the spreading of the Block Collective invocations for a given Accelerator to the various Switches in the Pod.

6.3.1 Switch Block Collective Queues

A Switch shall contain a per-Port Control Block Queue with an implementation specific number of entries. Each entry in the Control Block Queue shall contain a Valid bit, the Submission Queue Number from the InvokeBlockCollective Request, the fields associated with the InvokeBlockCollective Request as needed (e.g. ReqAddr, ReqSrcPhysAccID, ReqDstPhysAccID, ReqNumBeats, ReqTag, ReqVC, etc.), and the parameters that were contained in the data delivered with the InvokeBlockCollective Request. Each Port shall also contain a set of Control Logic for managing the Control Block Queue entries as shown below in Figure 6-9:

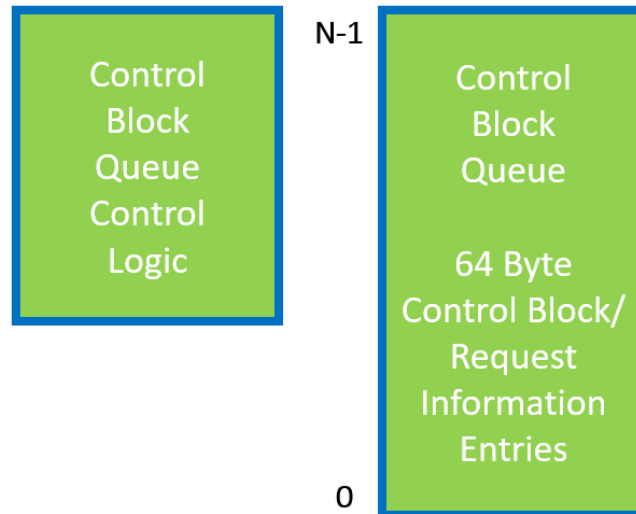


Figure 6-9 Control Block Queue and Control Logic

The Control Block Queue entries shall be dynamically utilized by various numbered logical “Submission Queues” consisting of one or more of the Control Block Queue entries. The set of

Control Block Queue entries assigned to a given Submission Queue may change within the lifetime of a given Submission Queue being active.

Submission Queue numbers shall fall in a range between 0 to 63 inclusive. For example, a Control Block Queue containing 64 entries could have 16 entries to Submission Queue 0, 8 entries to Submission Queue 1, 8 entries to Submission Queue 2, and 32 entries to submission Queue 15 with the rest being reassigned or any of several other combinations.

The logical Submission Queues within the Control Block Queue at a Port are utilized to provide flow control amongst Block Collectives received at a Port

6.3.2 BlockCollectiveAllocate Request

To allocate a set of Control Block Queue entries to a given Submission Queue, the Accelerator shall issue a BlockCollectiveAllocate Request that specifies in the ReqAddr field the Submission Queue and the number of entries being requested for allocation as shown in the following figure (Figure 6-10):

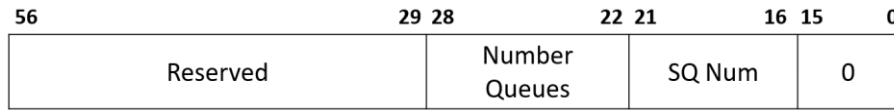


Figure 6-10 ReqAddr field for BlockCollectiveAllocate Request

The Submission Queue number that the entries in the Collective Block Control Queue shall be allocated to shall be indicated in bits [21:16] of ReqAddr allowing for Submission Queue numbers of 0 to 63 inclusive. The number of Queue entries being requested for the given Submission Queue shall be indicated in bits [28:22] of ReqAddr, allowing for 1 to 128 queues being requested inclusive (bits [28:22] shall indicate the number of queues being requested minus 1).

The low order eight bytes of the 64-bytes of data delivered with the AllocateBlockCollective Request shall indicate the 256-byte aligned UALink Address of the Status Buffer for the BlockCollectiveAllocate Request as shown in the following figure (Figure 6-11):

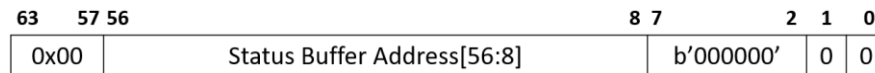


Figure 6-11 Low Order 8 bytes of Data Conveyed with BlockCollectiveAllocate

The remaining bytes within the 64 bytes of data delivered with the AllocateBlockCollective Request are Reserved and shall be set to zero. The Status Buffer Address shall be StatusBufferAddress[56:8] concatenated with eight bits of b'0' to produce a 256-byte aligned address that shall point to a 256-byte buffer that the Switch shall update using a BlockWriteFull Request to indicate the status of the BlockCollectiveAllocate Request. The first byte at this 256-byte buffer address shall indicate the status.

The high-order bit of the status byte shall be a Valid indication that indicates the Switch has updated the status. Software shall ensure that the Valid bit is cleared in the status byte before issuing the associated BlockCollectiveAllocate Request and shall poll the status byte after issuing the BlockCollectiveAllocate Request until the Valid is set by the Switch after the BlockCollectiveAllocate Request is issued.

The status response encoding for bits [6:0] for a BlockCollectiveAllocate command shall be as shown in the following Table (Table 6-2):

Table 6-2 BlockCollectiveAllocate Status Field Indications

Status[6:0]	Meaning
x'00'	Successful Allocation
x'01'	Invalid Allocation
x'02'	Duplicate Allocation
x'03'-x'3F'	Reserved.

If a Switch receives a BlockCollectiveAllocate Request for a Submission Queue that is currently allocated, the Switch will drop the BlockCollectiveAllocate Request and update the status buffer to indicate "Duplicate Allocation" (x'02'). If a Switch receives a BlockCollectiveAllocate Request for a Submission Queue that is currently unallocated, but is also requesting more entries than remain unallocated in the Control Block Queue or a Submission Queue that is currently being allocated or deallocated, the Switch will drop the BlockCollectiveAllocate Request and update the status buffer to indicate "Invalid Allocation" (x'01').

If, however, the Switch receives a BlockCollectiveAllocate Request for an unallocated Submission Queue and there are sufficient unallocated entries to fulfill the Request, the Switch shall allocate the entries to the requested Submission Queue and update the status buffer to indicate "Successful Allocation" (x'00').

6.3.3 BlockCollectiveDeallocate Request

To deallocate the Control Block Queue entries currently allocated to a given Submission Queue or to simply ensure that the operations associated with the Submission Queue are completed, the Accelerator shall issue a BlockCollectiveDeallocate Request that specifies the Submission Queue being deallocated or drained in the ReqAddr field as shown in the following figure (Figure 6-12):



Figure 6-12 ReqAddr field for BlockCollectiveDeallocate Request

The Submission Queue being deallocated shall be indicated in bits [21:16] of ReqAddr allowing for Submission Queue numbers of 0 to 63 inclusive. The "DO" (drain only) bit [22] shall indicate if entries associated with the SQ are to be deallocated after the outstanding operations complete or are terminated (DO = b'0') or if the entries are to remain allocated (DO = b'1').

The low order eight bytes of the 64-bytes of data delivered with the DeallocateBlockCollective Request shall indicate the 256-byte aligned UALink Address of the 256-byte Status Buffer for the BlockCollectiveAllocate Request as shown in the following figure (Figure 6-13):

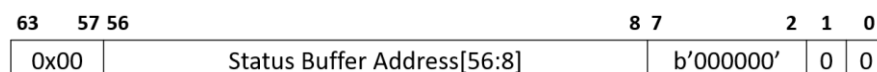


Figure 6-13 Low Order 8 bytes of Data Conveyed With CollectiveBlockDeallocate

The remaining bytes within the 64 bytes of data delivered with the DeallocateBlockCollective Request are Reserved and shall be set to zero. The Status Buffer Address shall be

StatusBufferAddress[56:8] concatenated with eight bits of b'0' to produce a 256-byte aligned address that points to a 256-byte buffer that the Switch shall update using a BlockWriteFull Request. The first byte at this 256-byte buffer address shall indicate the status. The remaining bytes in the buffer are Reserved and shall be written to zero.

The high-order bit of the status byte shall be a Valid indication that indicates the Switch has updated the status. Software shall ensure that the Valid bit is cleared in the status byte before issuing the associated BlockCollectiveDeallocate Request and shall poll the status byte after issuing the BlockCollectiveDeallocate Request until the Valid is set by the Switch after the BlockCollectiveDeallocate Request is issued.

The status response encoding for bits [6:0] for the status byte for a BlockCollectiveAllocate command are shown in the following Table (Table 6-3):

Table 6-3 BlockCollectiveDeallocate Status Field Indications

Status[6:0]	Meaning
x'00'	Successful Deallocation
x'01'	Invalid Deallocation
x'02'-x'FF'	Reserved.

If a Switch receives a BlockCollectiveDeallocate Request for a Submission Queue that is not currently allocated or is currently being allocated or deallocated, the Switch shall drop the BlockCollectiveDeallocate Request and update the status buffer to indicate "Invalid Deallocation" (x'01').

If a Switch receives a BlockCollectiveDeallocate Request for a Submission Queue that is currently allocated and active, the Switch shall stop accepting any new BlockCollectiveInvoke Requests for that Submission Queue. The Switch shall stop issuing new BlockRead and BlockWriteFull Requests for any outstanding Block Collectives in the Submission Queue and shall allow the outstanding BlockRead and BlockWrite Requests to complete. When the BlockRead and BlockWrite Requests for a given Block Collective have Completed or the given Block Collective was terminated before starting, the Switch shall update the status buffer for the Block Collective. When the status buffer(s) have been updated for all outstanding Block Collectives, the Switch shall update the status byte for the BlockCollectiveDeallocate to indicate "Successful Deallocation" (x'00') and once that update is complete, the Submission Queue shall be considered unallocated and is available to be reallocated.

6.3.4 BlockCollectiveInvoke Request

To invoke a Block Collective, the Accelerator shall issue a BlockCollectiveInvoke Request that specifies the Submission Queue to process the Request in bits [21:16] of ReqAddr and a "Base Address" in bits [56:22] of ReqAddr as shown in the following figure (Figure 6-14):

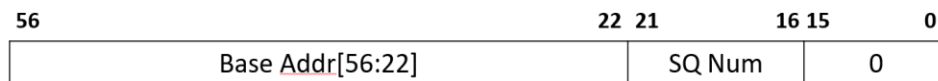


Figure 6-14 ReqAddr field for BlockCollectiveInvoke Request

The "Base Address" of the Block Collective shall be the address given by the ReqAddr field of the BlockCollectiveInvoke Request with the low-order 22 bits zeroed as show in the following figure (Figure 6-15):



Figure 6-15 Base Address of the Block Collective

The 64 bytes of data delivered with the BlockCollectiveInvoke shall contain a set of parameters to fully specify the Collective to be performed by the Switch (Section 6.3.7). One of these parameters shall be a 48-bit offset value to which eight bits of zero are concatenated to form an offset that is added to the Base Address to compute the 256-byte aligned address of the 256-byte Status Buffer. The Switch shall use a BlockWriteFull Request to update the 256-byte status buffer to indicate the completion status of the Block Collective.

The low-order 16 bytes of the 256-byte Status Buffer shall contain a status field for the Block Collective. The remaining bytes in the 256-byte Status Buffer are reserved and shall be written to zero. The lower-order 16 bytes of the Status Buffer are illustrated in the following figure (Figure 6-16):

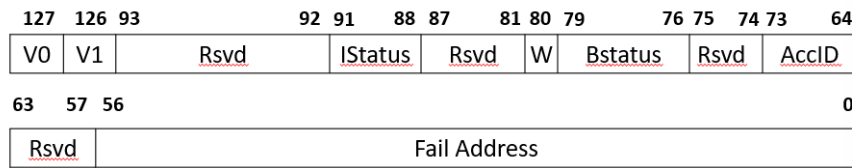


Figure 6-16 Low Order 16 bytes of BlockCollectiveInvoke Status Buffer

The fields in the Status Buffer are described in the following table (Table 6-4):

Table 6-4 Block Collective Invoke Status Buffer Field Definitions

Name	Size	Description
V0	1	Valid0: This field indicates that the IStatus field is valid. The V0 and V1 bits are set independently and both may be active.
V1	1	Valid1: This field indicates that the W, BStatus, AccID, and Fail Address fields are valid. The V0 and V1 bits are set independently and both may be active.
IStatus	4	Invocation/Termination Status: This field indicates the nature of the Invocation Error for the Block Collective or that the Block Collective was terminated for a BlockCollectiveDeallocate.
W	1	Block Read/Block Write: This field indicates the error occurred on a BLockRead (W=0) or a BlockWrite (W=1).
BStatus	10	Block Read/Block Write Status: This field indicates the RdRspStatus field value for the Response (if W=0) for the error and indicates the WrRspStatus field values for the Write Response (if W=1) for the error.
AccID	11	Acccelerator Physical ID: This field indicates the Physical Acceelrator ID of the Accelerator that the error occurred on.
FailAddress	57	Fail Address: This field indicates the 256-byte aligned address that the failure occurred at.

The V0 and V1 bits in the Status Buffer shall each be a Valid indication that indicates the Switch has updated the status. Software shall ensure that both bits are cleared before issuing the BlockCollectiveInvoke Request and shall poll these bits until at least one is set by the Switch after the BlockCollectiveInvoke Request is issued.

The information fields (W, Bstatus, AccID and Fail Address) should be set on a best-effort basis to ensure that the Bstatus, AccID, W, and Fail Address fields are consistent. For example, if a first error is detected on a given BlockWrite for a given Block Collective for some 256-byte region of memory, the Switch may present only the information associated with that initial error, even if the IStatus for

the Block Collective later changes to INC-MixedResponses as subsequent outstanding Requests are received (or the Switch may choose to present the INC-MixedResponses). If an initial error occurs for a given address, the Switch may report that first address or some later Request address.

If a Block Collective is terminated by a BlockCollectiveDeallocate Operation while completing the Block Collective after an error is detected, both V0 and V1 may be set or the Switch may choose to only track and report one of the errors.

The status response encoding for Istatus[3:0] field is shown in the following Table (Table 6-5):

Table 6-5 BlockCollectiveAllocate IStatus Field Indications

Status[3:0]	Meaning
x'0'	Successful Invocation
x'1'	Invalid Invocation
x'2'	Insufficient Resources
x'3'	Terminated
x'4'-x'F'	Reserved.

If the Switch receives a BlockCollectiveInvoke Request and the Valid bit for the corresponding Group Table entry is not set or the bit in the Group Table Mask corresponding to the Requesting Accelerator is not set, the Switch shall drop the BlockCollectiveInvoke Request and update the Status Buffer to indicate "Invalid Invocation" (x'1').

If the Group Accelerator Mask has less than two bits set, the Accelerator shall drop the BlockCollectiveInvoke Request and shall update the Status Buffer to indicate "Invalid Invocation" (x'1'). The Switch may begin processing the Block Collective before recognizing this error condition. If the values in any of the BTYPE, CTYPE, ROPDT, RMODE fields in Control Block delivered with the BlockCollectiveInvoke contain an illegal value, the Accelerator shall drop the BlockCollectiveInvoke Request and shall update the Status Buffer to indicate "Invalid Invocation" (x'1'). If the ReqLen and ReqAddr fields for the Block Collective do not indicate a 64-byte aligned 64 byte region of memory, the Switch shall drop the BlockCollectiveInvoke Request and shall update the Status Buffer to indicate "Invalid Invocation" (x'1').

If the Switch receives a BlockCollectiveInvoke Request and the indicated Submission Queue is not active or is being deallocated, the Switch shall drop the BlockCollectiveInvoke Request and update the Status Buffer to indicate "Invalid Invocation" (x'1'). If the Switch receives a BlockCollectiveAllocate Request and the indicated Submission Queue is active, but full, the Switch shall drop the BlockCollectiveRequest and update the Status Buffer to indicate "Insufficient Resources" (x'2').

If the Switch receives a BlockCollectiveInvoke Request for an active and not full Submission Queue and the Group Table entry is valid and the bit in the Group Table Mays corresponding to the Requesting Accelerator is set, the Switch shall enqueue the Block Collective for processing and shall set the Istatus field to "Successful Invocation" (x'0').

6.3.5 Block Collective Normal and Strided Accesses/Address Masking

For Block Collectives, an Offset parameter (be it for the Input or Output Buffer(s) at the Ranks) shall be concatenated with eight binary 0's and then added to the Base Address for the Block Collective to determine the starting 256-byte aligned address for the Buffer or Tile to be accessed.

For non-strided accesses Block Collective, a Length parameter indicates the number of 256-byte blocks of memory at the starting address that should be processed (Length+1 blocs) for the Block Collective as shown in the following figure (Figure 6-17):

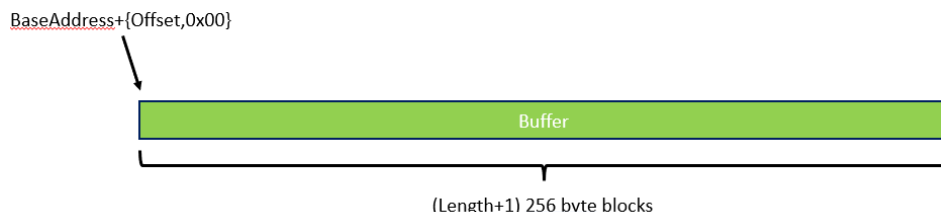


Figure 6-17 Non-Strided Block Collective Memory Access Pattern

For a strided access Block Collective, an Xsize, Ysize, and a Skipsize parameter shall be used to indicate how many 256-byte blocks (Xsize+1 blocks) shall be processed in a row of a Tile before skipping Skipsize+1 256-byte blocks to reach the beginning of the next row of the Tile. The process for a strided Block Collective shall repeat for Ysize+1 rows as shown in the following figure (Figure 6-18):

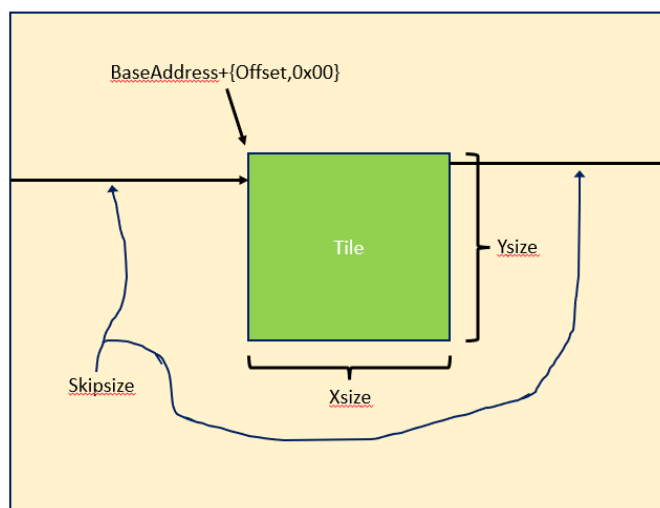


Figure 6-18 Strided Block Collective Memory Access Pattern

For non-strided access Block Collectives, the Xsize, Ysize, and Skipsize parameters are invalid and shall be ignored. For strided access Block Collectives, the Length parameter is invalid and shall be ignored.

6.3.6 Address Masking

The Switch shall support one logical Block Collective Address Mask Register that may be used to provide memory isolation for Block Collective memory accesses. The Register shall consist of bits corresponding to ReqAddr[56:24] as shown in the following figure (Figure 6-19):

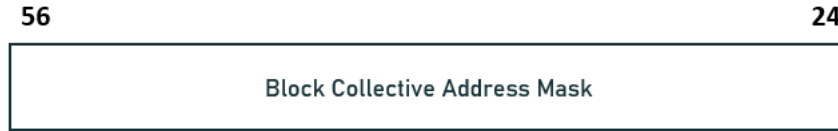


Figure 6-19 Block Collective Address Mask Register

When used, the Block Collective Address Mask Register shall be loaded with a mask with a contiguous set of one or more b'1' values starting at bit 56.

When the Switch generates an address for a BlockWrite or BlockRead, the Switch shall invert the mask and logically AND the inverted mask with the generated BlockWrite or BlockRead address to produce an Address with the high-order bits set to zero according to the mask. The Switch shall also bitwise AND the un-inverted mask with the Base Address for the Block Collective producing an address containing only the high-order bits of the Base Address. The Switch shall then logically OR these two addresses together to form the address that shall be used for the Block Collective.

This process has the effect preventing the Switch from generating a BlockRead or BlockWrite address outside a region bounded by the low-order bits of ReqAddr that are not masked off and is identified by the high-order bits of ReqAddr that are masked off by the Block Collective Address Mask.

6.3.7 Block Collective Control Block Parameters and Format

The BlockCollectiveInvoke Request shall deliver a 64-byte block of data containing the Block Collective Control Block. The Block Collective Control Block shall contain the set of parameters

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

necessary to specify the Block Collective to be implemented as shown in the following table (Table 6-6):

Table 6-6 Block Collective Control Block Fields

Name	Size (bits)	Abbreviation	Description
Block Collective Control Block Fields			
Control Block Type	8	BTYPE	Control Block Type. Specifies the format of the Collective Block Type. Initially only value 0x'00' is valid.
Collective Type	8	CTYPE	Collective Type: Specifies the type of Collective and the address access pattern: strided or non-strided.
Length	24	LEN	Collective Length. Number of 256-byte Blocks to be processed for a non-strided Block Collective minus one (i.e. Length = 0 implies processing one 256-byte block). For a REDUCE or ALLREDUCE Block Collective that has an output datatype that is larger than the input datatype this field indicates the size of the input buffer. The output buffer length is two or four times this length based on the difference in size between the input buffer datatype and the output buffer datatype. Ignored for strided Block Collectives.
Reduce OP/Data Type	8	ROPDT	Reduce Op/DataType: Specifies the Reduction Operator, Input DataType, Intermediate Precision (if applicable), and the output DataType (if applicable) for the reduction. This field consists of a binary '0' concatenated to a field in the same format as the ReduceOp/DataType[6:0] field specified for Collective Primitives in the ReqSrcPhysAccID field from the Requesting Accelerator to the Switch.
Round Mode	8	RMODE	Rounding Mode: This field specifies the rounding mode (if applicable) for the Block Collective. This field consists of b'00000' concatenated to a field in the same format as the RoundMode[2:0] field specified for Collective Primitives in the ReqSrcPhysAccID field from the Requesting Accelerator to the Switch.
Input Buffer Offset	48	I_OFF	Input Buffer Offset. The offset value that after concatenation with 0x00 is to be added to the Base Address to determine the 256-byte aligned UALink Address for the start of the Input Buffer(s) for the Collective.
Output Buffer Offset	48	O_OFF	Output Buffer Offset. The offset value that after concatenation with 0x00 is to be added to the Base Address to determine the 256-byte aligned UALink Address for the start of the Output Buffer(s) for the Collective.
Status Buffer Offset	48	S_OFF	Status Buffer Offset. The offset value that after concatenation with 0x00 is to be added to the Base Address to determine the 256-byte aligned UALink Address for the start of the 256-byte Status Buffer for the Collective.
X-Size	32	XSIZE	X-Size: Number of 256-byte Blocks in a Tile row minus 1 (i.e. X-Size = 0 implies a 256-byte row) for the Strided Block Collective. For a REDUCE or ALLREDUCE Block Collective that has an output datatype that is larger than the inputdata type this field indicated the size of the input row-size. The output buffer row size is two or four times this length based on the difference in size between the input buffer datatype and the output buffer datatype. Ignored for non-strided Block Collectives.
Y-Size	32	YSIZE	Y-Size: Number of rows in a Tile minus 1 (i.e. y-Size = 3 implies a four-row tile). Ignored for non-strided Block Collectives.
Skip-Size	32	SSIZE	Skip-Size: Number of 256-byte blocks minus 1 (i.e. Skip-Size = 10 implies eleven 256-byte blocks) that should be skipped at the end of a row in the Tile to reach the beginning of the next row. Ignored for non-strided Block Collectives
Skip-Size2	32	SSIZE2	Skip-Size2: Number of 256-byte blocks minus 1 (i.e. Skip-Size = 10 implies eleven 256-byte blocks) that should be skipped at the end of a row in the Tile for the Output Buffer only to reach the beginning of the next row. Ignored for non-strided Block Collectives
Stochastic Seed	32	SSEED	Stochastic Seed: 32-bit Stochastic Seed for Add REDCUE Collectives.

The only currently legal value for the Control Block Type field shall be x'00'. As new features are added to Block Collective, additional legal values will be added with new legal Control Block Formats.

The Collective Type parameter values shall be those shown in the following table (Table 6-7):

Table 6-7 Collective Type Parameter

Value	Collective
x'00'	BROADCAST (Non-Strided)
x'01'	REDUCE (Non-Strided)
x'02'	ALL-REDUCE (Non-Strided)
x'03'-x'6F'	Reserved
x'70'-x'7F'	Vendor Defined Block Collective
x'80'	BROADCAST (Strided)
x'81'	REDUCE (Strided)
x'82'	ALL-REDUCE (Strided)
x'83'-x'EF'	Reserved
x'F0'-x'FF'	Vendor Defined Block Collective

The Block Collective Control Block Parameters shall be arranged in the 64-byte data payload delivered with a BlockCollectiveInvoke according to the following table (Table 6-8):

Table 6-8 Block Collective Parameter Arrangement

7	6	5	4	3	2	1	0	<-Byte
Rsvd								Quadword7
Rsvd				SSIZE2				Quadword6
SSEED				SSIZE				Quadword5
YSIZE				XSIZE				Quadword4
Rsvd	Rsvd	S_OFF						Quadword3
Rsvd	Rsvd	O_OFF						Quadword2
Rsvd	Rsvd	I_OFF						Quadword1
Rsvd	RMODE	RDOPT	LEN			CTYPE	BTYPE	Quadword0

6.3.8 Switch Structures to support Block Collectives

The Switch shall use the same Group Table and Switch Route Table structures used for Collective Primitives to determine the Accelerators involved in a Block Collective and to route Block* Requests that effect the Block Collective as illustrated below in Figure 6-20.

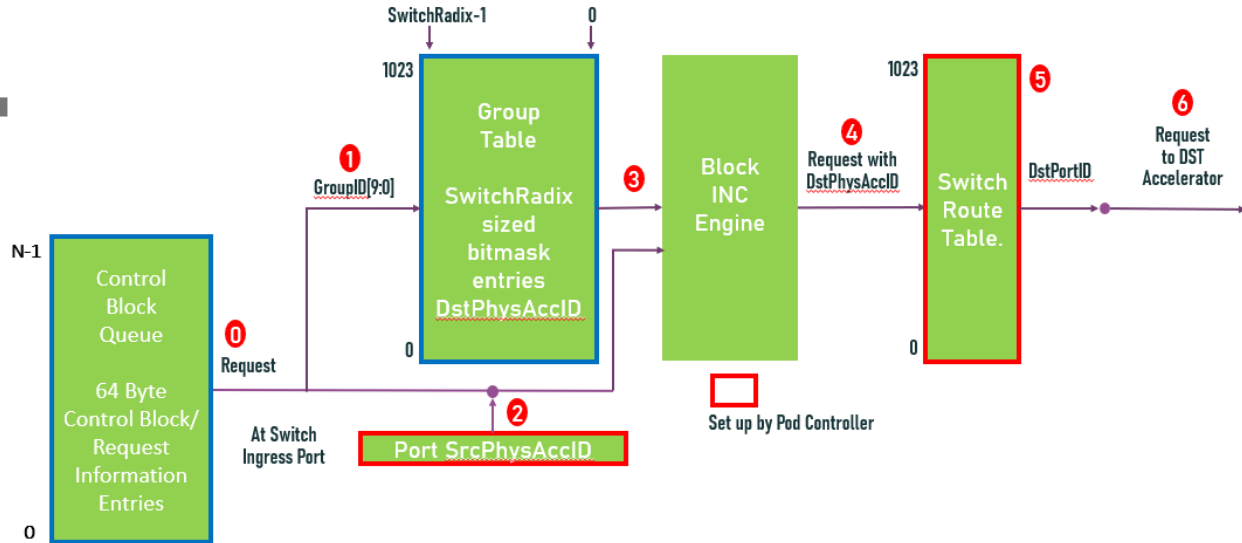


Figure 6-20 Block Collectives Support Structures

The Block Collective Request is Received at the Group Table from the Control Block Queue at 0. The ReqDstPhysAccID signal value stored in the Block Collective shall contain the GroupID[9:0] value which shall be used to index the GroupTable to obtain the Group Accelerator Mask corresponding to the Block Collective 1.

The Group Accelerator Mask shall be passed on to the Block INC engine 3 unless the entry is Group Table entry is not valid or the bit in the Group Table mask corresponding to the Requesting Accelerator for the Block Collective is not set, in which case the Switch shall terminate the Block Collective and write the Status Buffer Istatus field with an Invalid Invocation indication. The ReqSrcPhysAccID field in the Request shall be reconstituted by referencing the 10-bit register containing the Accelerator ID for the Accelerator connected to the Port and the Request is then passed to the Block INC Engine for processing 2.

The Block INC Engine shall issue a set of Block* Requests to the Switch Route Table to effect the Block Collective 4. The Switch Route Table (which is also used for non-INC Requests) 5 shall provide the Destination Port Identifier for the various Block* Requests that shall be routed to the appropriate Destination Port and issued to the Destination Accelerator. 6.

Isolation between Virtual Pods for Block Collective Block* Requests shall be provided by the Switch Route Table in the same manner as for non-INC Requests.

6.3.9 Block Collective Request/Response Flows

The following figure, Figure 6-21, illustrates the invocation of a Block Collective utilizing an BlockCollectiveInvoke Request:

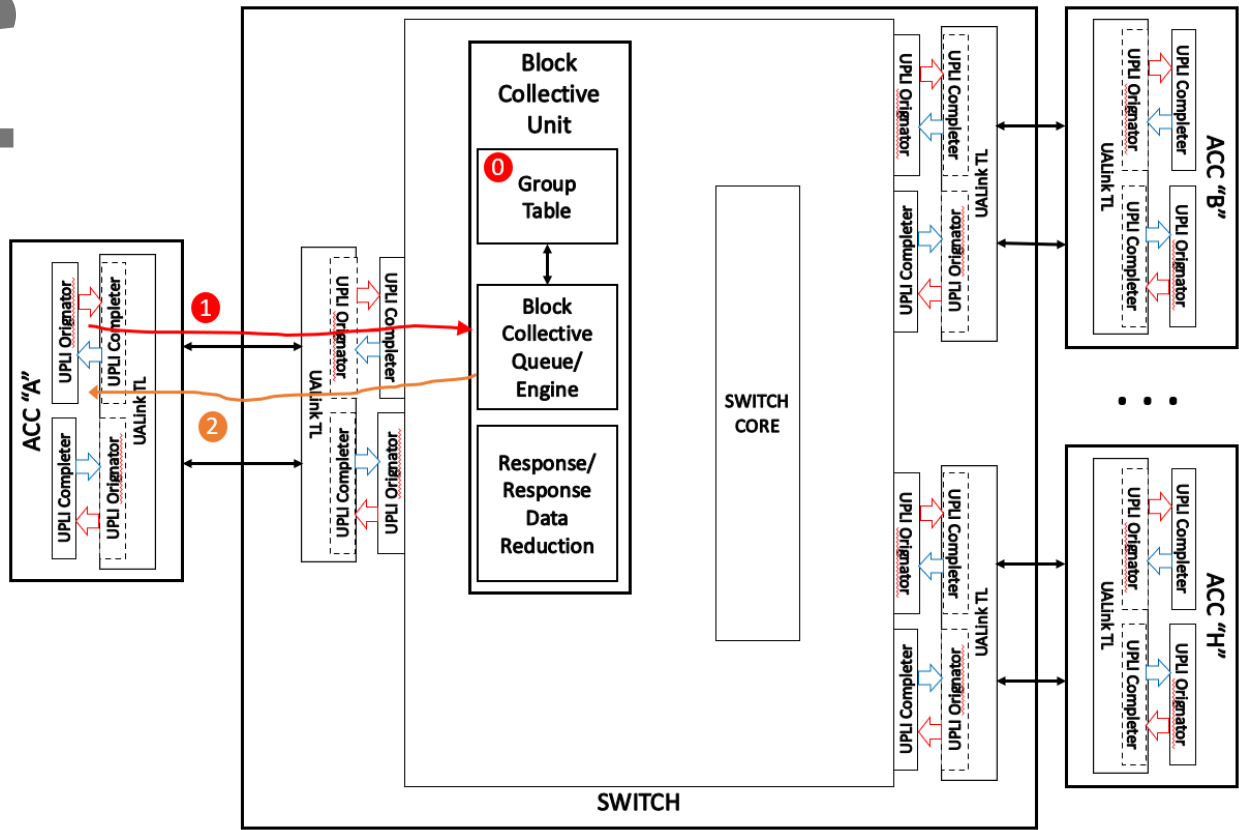


Figure 6-21 Block Collective Invocation.

The Group Table entry for the Block Collective shall be loaded prior to the initiation of the Block Collective ❶. A BlockCollectiveInvoke Request is issued by Accelerator "A" to invoke the Block Collective ❶. The Block Collective Queue/Engine returns an OKAY Write Response without undue delay ❷.

The following figure, Figure 6-22 illustrates the initiation of BlockRead Requests to read the input buffer or buffers for a BROADCAST/REDUCE/ALL-REDUCE Collective.

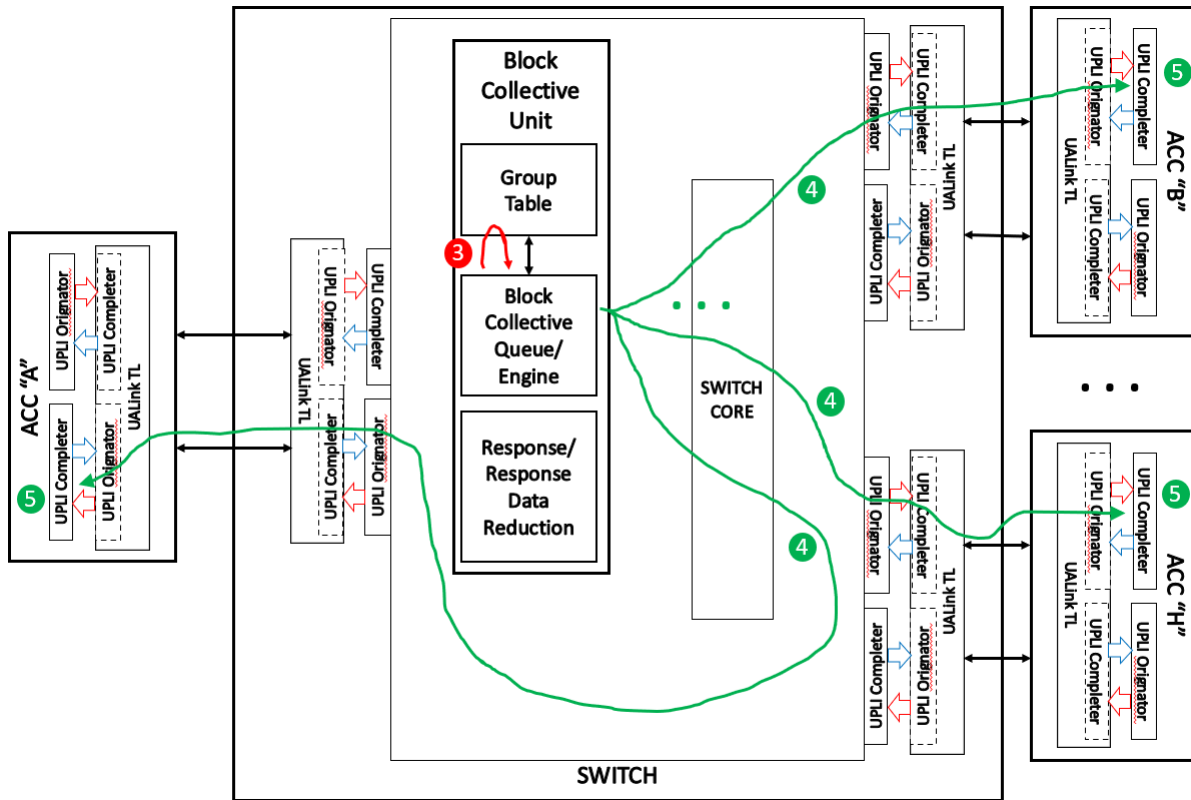


Figure 6-22 Block Collective Input Buffer Read(s)

The Block Collective Engine shall first read the Group Table on the Switch to obtain the Group Accelerator Mask ③ and then shall issue one BlockRead Request to the Requesting Accelerator (BROADCAST Collective) or BlockRead Requests to all Accelerators in the Group (REDUCE/ALL-REDUCE Collective) to the input buffer(s) on the Rank(s) for each 256-byte block in the Collective Input Buffer ④. The ReqSrcPhysAccID signal on the BlockRead Requests shall be set to indicate the Accelerator ID of the Requesting Accelerator. The ReqTag field for the BlockRead Request shall be set to a Tag value assigned by the Switch. The following signals in the Request Channel shall be passed through the switch unchanged from the initial BlockCollectiveInvoke: ReqASI, ReqAuthTag, ReqMetaData, ReqVC.

The Block Collective Engine shall maintain Block Collective Tracking Logic to track the outstanding Block Read Requests for the Block Collective, reduce the Responses for the outstanding Requests, data and Responses returned, and to determine when all the Block Reads for a given 256-byte region of memory have completed.

When an individual Request is received at the Destination Accelerator Completer, the Completer shall retain indications for whether the Request was a Collective Primitive (ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMulticast), a Block Collective (BlockWriteFull, BlockRead), or neither of the above ⑤.

The Destination Accelerator shall distinguish tags according to a combination of the ReqTag value, the ReqSrcPhysAccID value, the ReqPortID value, and whether the Request is a BlockWriteFull or

BlockRead Request or the Request is any other Request Type ⑤. This tracking scheme provides Block* Requests for Block Collectives (whose tags are assigned by the Switch issuing the Requests) an independent tag space from the tag space for the Collective Primitives and normal Requests (whose tags are assigned by the Accelerator issuing the Requests) preventing tag collisions between the two groups of Requests.

The following figure, Figure 6-23, illustrates the flow of the Read Responses for the BlockRead Requests for a given 256-byte region of memory.

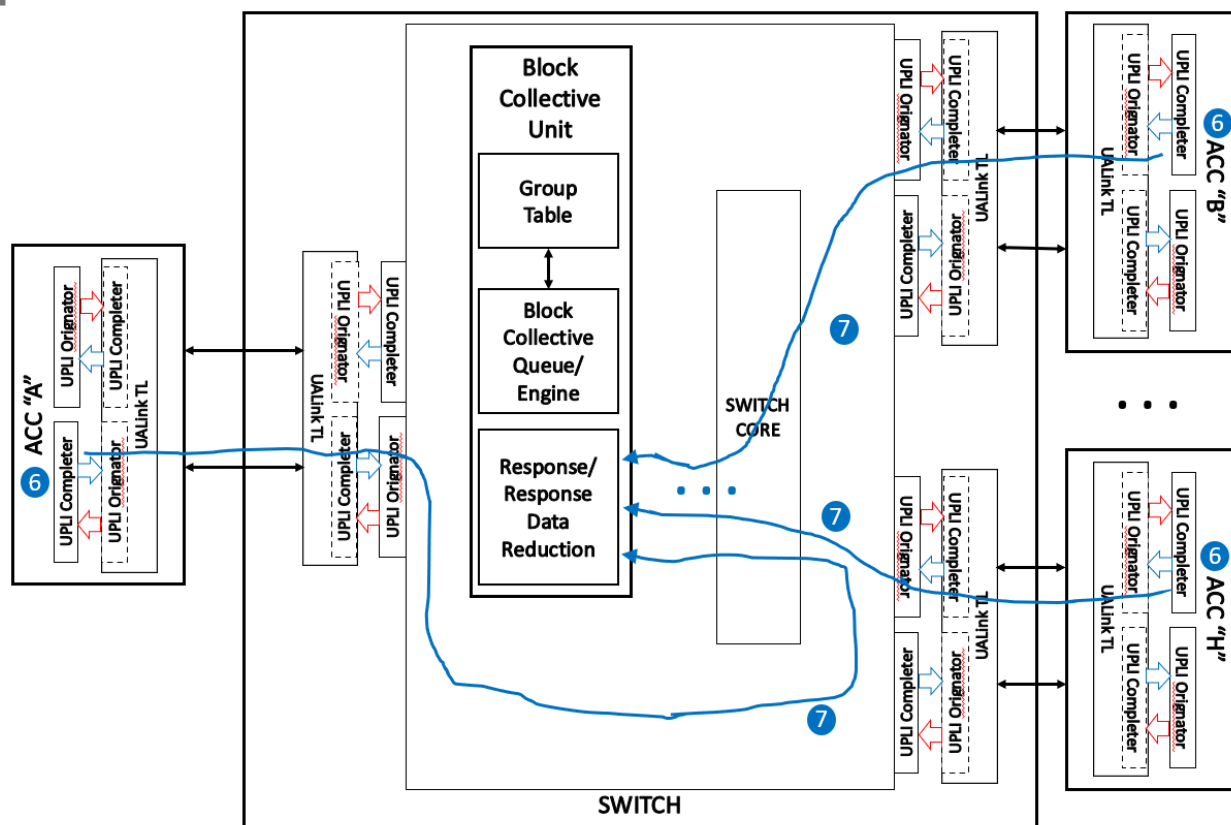


Figure 6-23 Block Collective Response Flow

The Destination Accelerator shall process a BlockRead Request in the same manner as a Read Request is processed. When the BlockRead is completed ⑥, the UPLI Completer shall issue a Read Response to the Switch and the Response/Response Data Reduction logic in the Switch shall reduce the data and the responses ⑦. If any Response has a non-OKAY Status Response the Block Collective shall be terminated.

If a RdRspDataError is asserted on a beat of data from any of the Ranks for a given location, the Switch shall mark the beat corresponding to that location with a RdRspDataError in the ensuing BlockWrite(s) for the location for the Block Collective.

The following figure, Figure 6-24, illustrates the initiation of Write Request(s) to write the output buffer (for a REDUCE Collective) or output buffers (for BROADCAST/ALL-REDUCE Collectives) for a given 256-byte region of the Block Collective.

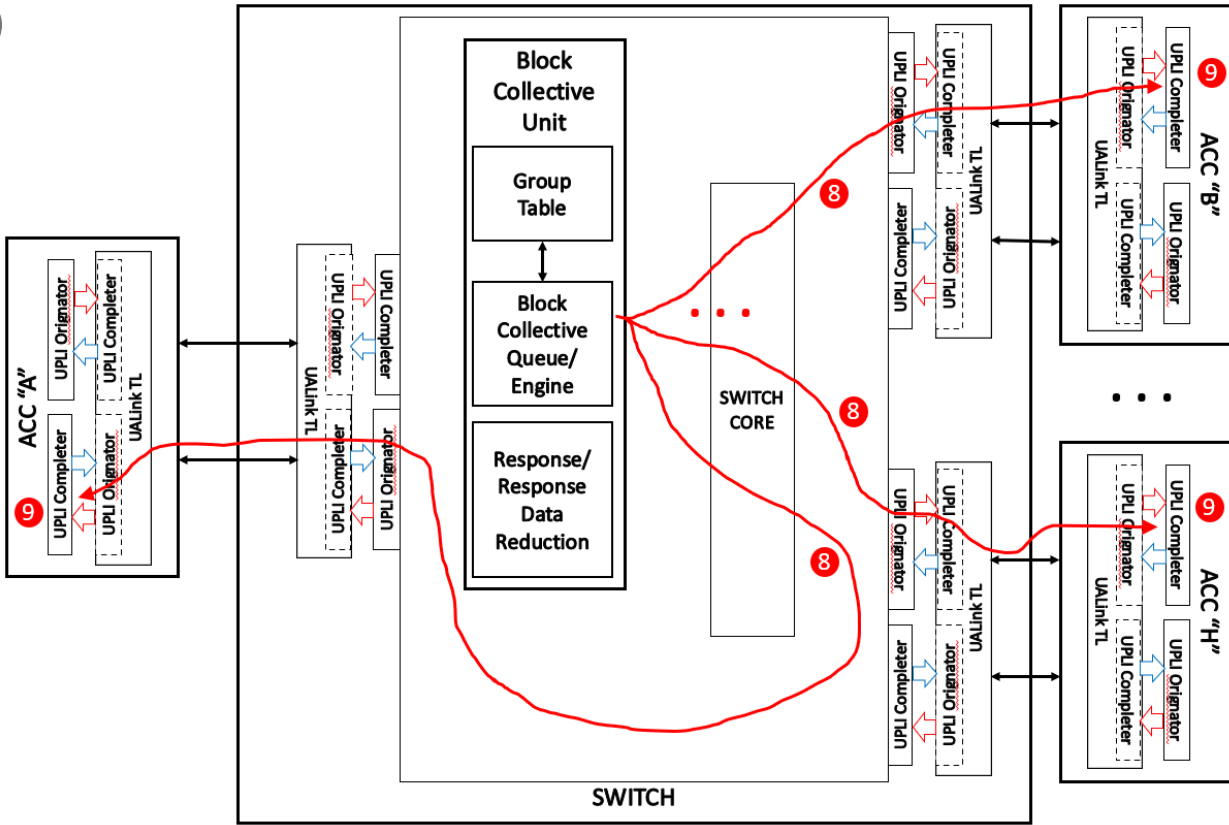


Figure 6-24 Block Collective Output Buffer Write(s)

Once the BlockReads for a given 256-byte region of memory in the prior figure, Figure 6-24, are complete, the Block Collective Engine shall issue one BlockWrite Request to the Requesting Accelerator (REDUCE Collective) or BlockWrite Requests to all Accelerators in the Group (BROADCAST/ALL-REDUCE Collective) to the output buffer(s) on the Accelerator(s) ⑧. The ReqSrcPhysAccID signal on the BlockRead Requests shall be set to indicate the Accelerator ID of the Requesting Accelerator. The ReqTag field for the BlockRead Request shall be set to a Tag value assigned by the Switch. The following signals in the Request Channel shall be passed through the switch unchanged from the initial BlockCollectiveInvoke: ReqASI, ReqAuthTag, ReqMetaData, ReqVC.

The Destination Accelerator(s) shall process a BlockWriteFull in the same manner as a WriteFull Request ⑨.

The following figure, Figure 6-25, illustrates the return of the Write Responses from the Destination Accelerators for the writes to the buffer or buffers (depending on the Collective being processed) for a given 256-byte region of the block Collective.

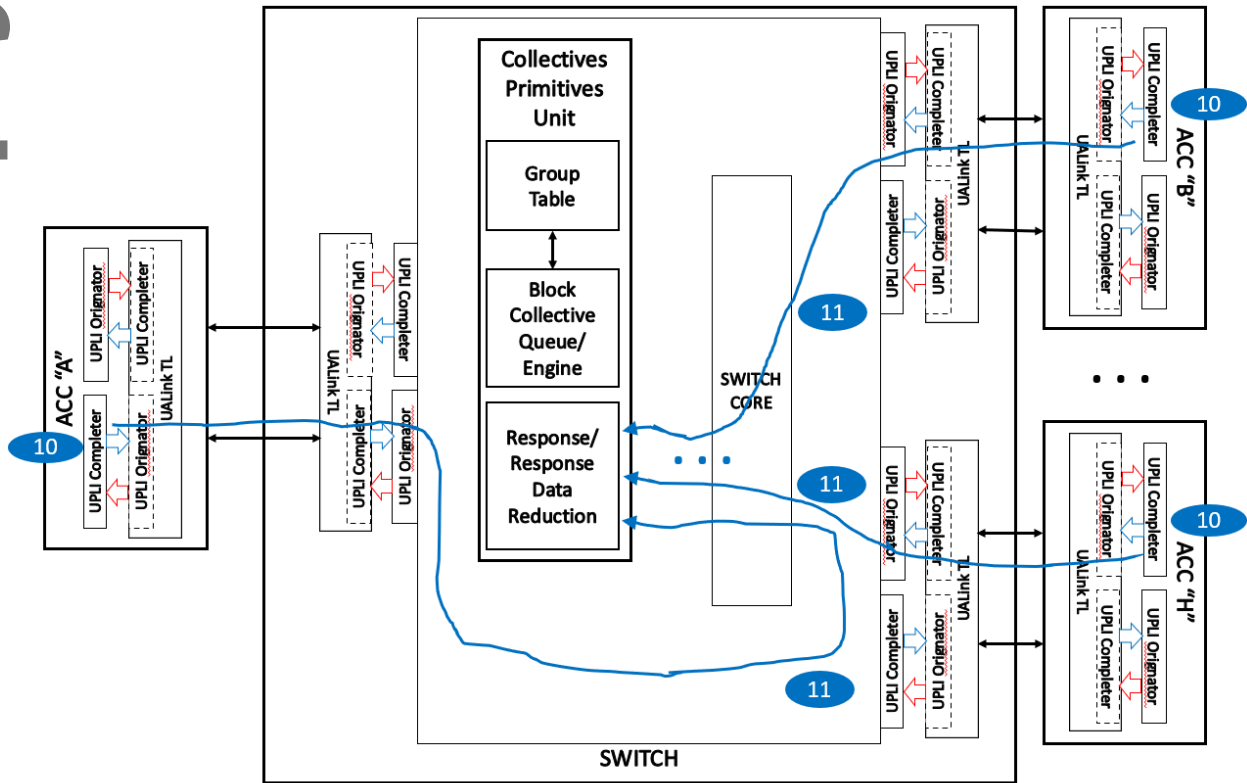


Figure 6-25 Block Collective Output Buffer Write Response(s)

The Destination Accelerator shall process a BlockWriteFull Request in the same manner as a Write Request is processed. When the BlockWriteFull Request is completed ⑩, the UPLI Completer shall issue a Write Response to the Switch ⑪. If any Response is an error the Block Collective shall be terminated.

The following figure, Figure 6-26, illustrates the Switch updating the Status Buffer at the completion of the Block Collective.

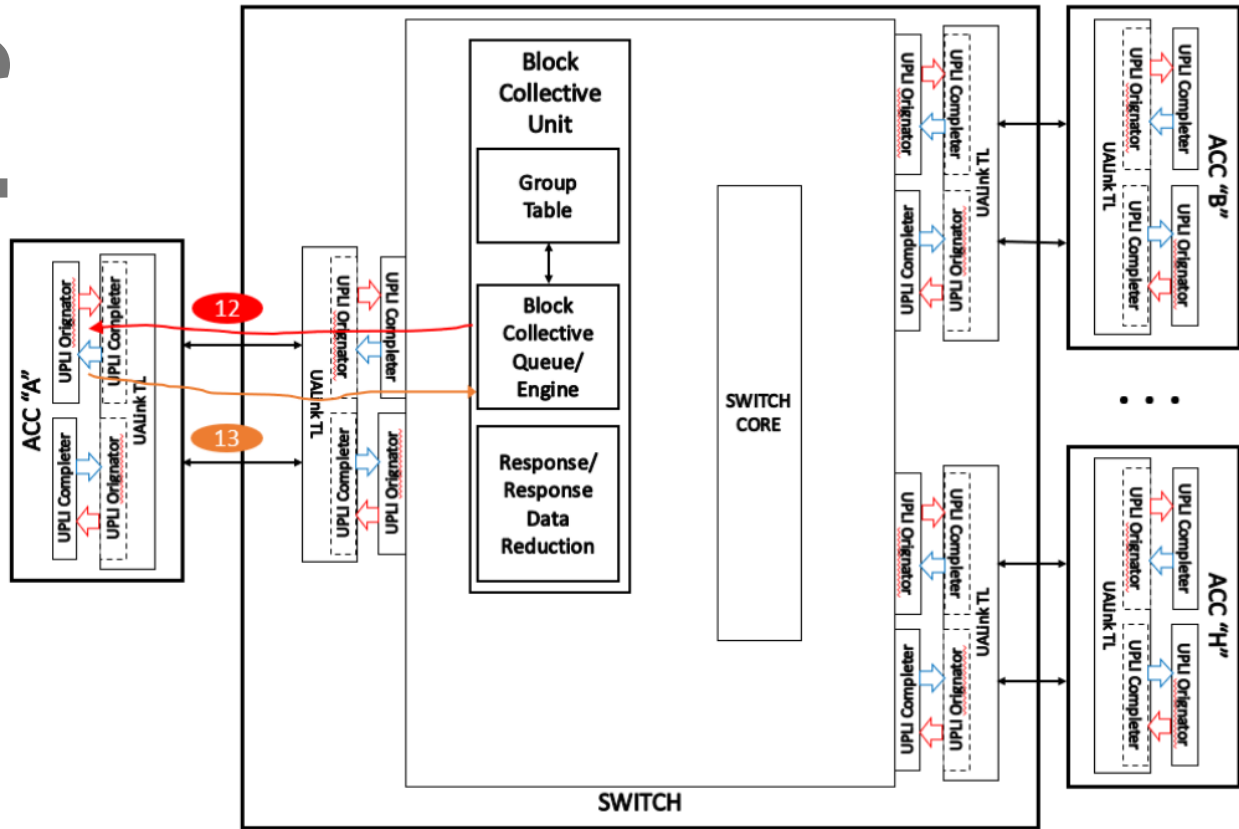


Figure 6-26 Block Collective Status Buffer Update

When steps ④ through ⑪ have been completed for each of the 256-byte regions of memory in the Block Collective or the Block Collective has been terminated due to an error, the Switch shall perform a BlockWrite in order to update the Status Buffer ⑫ with the Accelerator returning a Response for the BlockWrite ⑬.

6.4 Relationships between the Collectives, Collective Primitives, and Block Collectives

There is a hierarchical relationship between the Collective Primitives and the Block Collectives. Specifically, a properly structured sequence of Collective Primitives can be used to implement all the Block Collectives. A sequence of WriteFullMulticasts (that either read from local memory on the root rank or read from registers loaded from local memory on the root rank) can be used to implement a BROADCAST Collective and a sequence of ReadReduce Collective Primitives (that either update local memory on the Root Rank or update registers that are then written back to local memory on the Root Rank by store instructions) can be used to implement a REDUCE Collective. The BROADCAST and REDUCE Collectives thus implemented can then be used to implement ALL-REDUCE, ALL-GATHER, REDUCE-SCATTER.

UALink supports Block Collectives that directly implement BROADCAST, REDUCE, and ALL-REDUCE. The ALL-GATHER and REDUCE-SCATTER Collectives can be implemented using repeated BROADCAST and REDUCE Block Collectives Respectively. In addition, the ALL-REDUCE Collective, can be implemented by a REDUCE Block Collective followed by a BROADCAST Block Collective.

The AtomicNR-Multicast Collective Primitive is not typically intended to implement a Collective. Rather, the AtomicNR-Multicast is intended to provide Accelerators with a straightforward means of updating synchronization variables in the local memory of each Rank. For example, each Rank could issue an AtomicNRMulticast to increment a copy of a counter at each Rank when that Rank has completed its work on the Collective (presuming that the implementation AtomicNRMulticast has an increment or add function). The individual Ranks can then poll the local copy of the synchronization variable to determine when all Ranks have completed their processing.

These relationships between the Collective Primitives, Block Collectives, and the various Collectives are illustrated in the following (Figure 6-27):

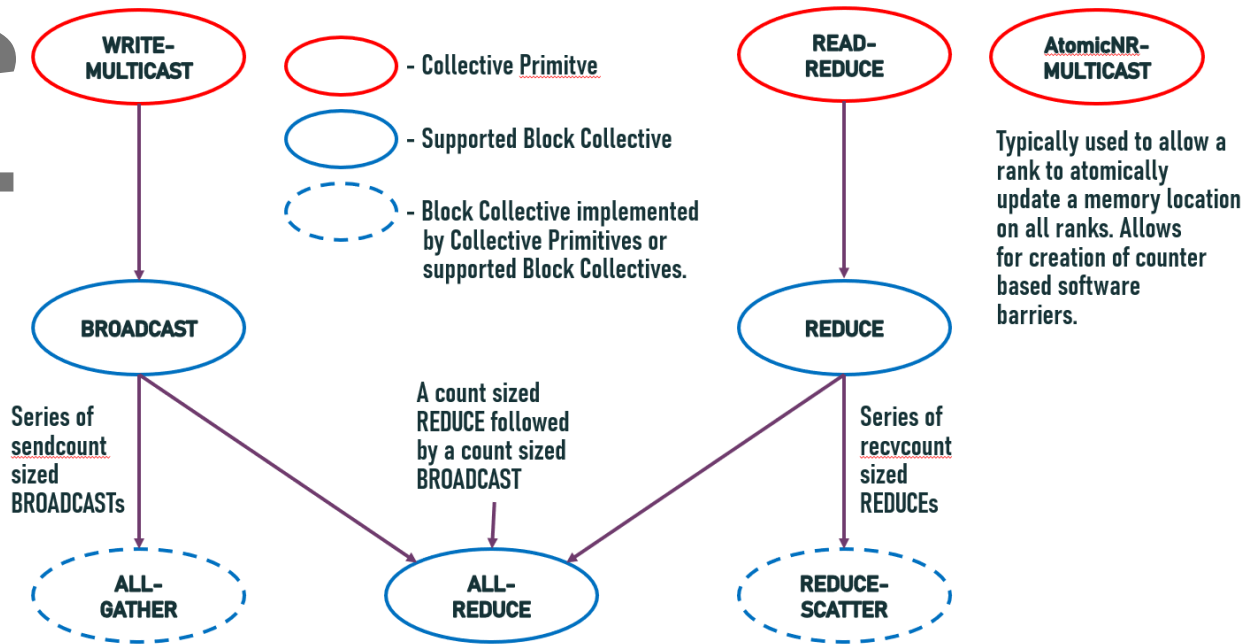


Figure 6-27 Collective Primitives and Block Collectives Relationships

6.5 Reproducibility

To produce repeatable results for Block Collectives and Collective Primitives, the same data elements from the Ranks in the Collective must be processed in the same order from one run to the next (this includes utilizing the same Stochastic Seeds for the different Collective Primitives or Block Collectives where Stochastic Rounding is used). This is necessary because some of the reduction operations for Collectives are not associative (e.g., for floating point addition, $(a+b)+c$ is not necessarily equal to $a+(b+c)$).

Achieving Reproducibility in UALink is a cooperative property of at least the application program design, the placement of the threads for the workload on various Accelerators, the interconnection between the Accelerators and Switches, and the design of the Switches and the Accelerators.

UALink systems shall produce repeatable results from run to run on the exact same system for the same program whose threads of execution are allocated to the same Accelerators and are presented with the same initial input data if the workload does not contain any inherent non-determinism.

One set of conditions to cause the Collective Primitives and Block Collectives to be processed against the same Rank data in the same order is listed below:

- The program cannot contain any inherent non-determinism that would alter the run-to-run behavior of the workload when presented with the same initial data with the threads of execution executed in the same manner on the same Accelerators assuming the In Network Collectives inherently maintain the same reduction order across the software ranks.

- All Accelerators should issue Requests for the same set of Address to the same Egress Ports, and this allocation should be fixed.
- All Switches should perform reductions for ReadReduce, REDUCE, or ALL-REDUCE in the same fixed order. This order may be a linear order or partial order, and it may be in terms of Accelerator IDs or Ports on the Switch.
- All Switches should connect to a given Accelerator on the same port.

Other sets of conditions may be possible if they ensure that the Collectives are processed against the same data in the same rank order given the same workload thread placement and initial data.

Replication of a workload on a different system or configuration is not guaranteed. To replicate the execution of a program on a different system, steps must be taken to ensure that reduction order of the Ranks on the new system is consistent with the reduction order of the Ranks on the old system.

To provide Reproducibility, the system should provide an implementation specific means to determine the reduction order or partial order for Collectives executed on a given set of Accelerators in the system. This means should be employed on the first run on the initial system to determine the reduction order of the Accelerators on that system and by implication the reduction order of the workload threads as assigned to those Accelerators on the initial system.

To reproduce the run on a subsequent new system on a newly assigned set of Accelerators, that same system specific means should be applied to the new set of assigned Accelerators to determine the reduction order on the new system. The workload threads should then be assigned to the Accelerators on the new system in a manner that ensures the reduction order is the same on the new system as it was on the prior system.

If the reduction order is a linear order, it will always be possible to permute the workload threads in a manner that achieves the same reduction order. However, if the reduction order is partial (say due to a hierarchical reduction of operands in the Switch), it may not be possible to find an assignment of workload threads on the new system that has the same reduction order as the old system.

If an Accelerator should fail during an execution, to continue to be able to achieve Reproducibility, the threads of the workload should be reallocated to the set of Accelerators the program continues execution on in a way that preserves the reduction order for the Collectives (i.e. the Accelerator removal case is simply a special case of migrating the application to a new system).

6.6 Rounding Modes and Stochastic Rounding

UALink supports the following IEEE Rounding Modes:

- (RNE) Round to nearest, ties to even
- (RNA) Round to nearest, ties away from zero
- (RZ) Round towards zero (truncation)
- (RU) Round towards +infinity (rounding up or ceiling)
- (RD) Round towards -infinity (round down or floor)

With RNE being required and the other modes being optional. If the other modes are not implemented, they shall instead Round to Nearest, ties to even.

In addition to the IEEE Rounding modes, UALink supports an implementation defined Stochastic Rounding Mode. Stochastic Rounding generally consists of a random number source (typically a Linear Feedback Shift Register) that generates a sequence of Random Numbers based on a Seed value. Stochastic Rounding is typically only applied when converting a higher precision floating point format down to a lower precision floating point format (and typically only applied when converting from a higher precision floating point format back to an FP8 format – E5M2 or E4M3).

Stochastic Rounding typically involves taking a generated Random Number and adding that value to a set of Mantissa bits that are to be dropped in the down conversion and using the carry into the remaining retained mantissa bits to produce a “random” rounding.

A Block Collective or Collective Primitive when presented with the same input data in the same reduction order for the Collective and the same Stochastic Seed shall produce consistent answers and rounding results.

6.7 In Network Collective Optionality

A Switch shall implement neither Collective Primitives nor Block Collectives, Collective Primitives only, or both Block Collectives and Collective Primitives. If Collective Primitives are implemented, the Switch shall implement all Collective Primitive Request types (ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMulticast) and shall implement all reduction operations/datatypes for ReadReduce. If Block Collectives are implemented, the Switch shall implement all Block Collectives (REDUCE, ALL-REDUCE, BROADCAST) and shall implement all reduction operations/datatypes for REDUCE and ALL-REDUCE.

7 Manageability Requirements

A UALink Pod is comprised of one or more UALink Accelerators connected via the UALink fabric, and managed by a UALink Pod Controller. Each Accelerator is hosted on a UALink System Node, and Accelerator traffic may be routed through the UALink fabric via UALink Switches. Multiple UALink Pods may be connected to create even larger Accelerator clusters; however, inter-Pod communication is outside the scope of this specification. While a brief overview of these components and their management interfaces is provided below, the full solution is documented in the separate *Ultra Accelerator Link Manageability Specification*.

7.1 UALink Accelerators and System Nodes

UALink System Nodes shall contain at least one UALink Accelerator, at least one high-performance host CPU to schedule and launch compute jobs, and at least one network interface. Commonly, a System Node should also include a baseboard management controller (BMC) for chassis management tasks and an interface for out-of-band management. The System Node acts as an OS Domain in that a single instance of an operating system or hypervisor may utilize all Accelerators in the System Node, or assign them to tenant Virtual Machines (VMs).

Each System Node shall host one or more Node Management Agents that communicate with the Pod Controller as specified in the *Ultra Accelerator Link Manageability Specification*. Node Management Agents perform fabric management functions on the System Node such as adding Accelerator(s) on the System Node to a Virtual Pod by configuring the UALink ports. In the case of SR-IOV enabled Accelerators, the Node Management Agents shall only communicate with the Physical Function (PF) drivers that run in the host OS domain and are outside the trust boundary of any confidential VM.

In aggregate across the Pod, the System Nodes may host up to 1024 Accelerators. Accelerators targeted for a Confidential Compute¹ use case must support a minimum baseline of security features.

The following figure illustrates one possible UALink System Node with a Node Management agent managing four UALink Accelerators (each with three ports). Also present are two host CPUs, a NIC, and a BMC.

¹ <https://confidentialcomputing.io/>

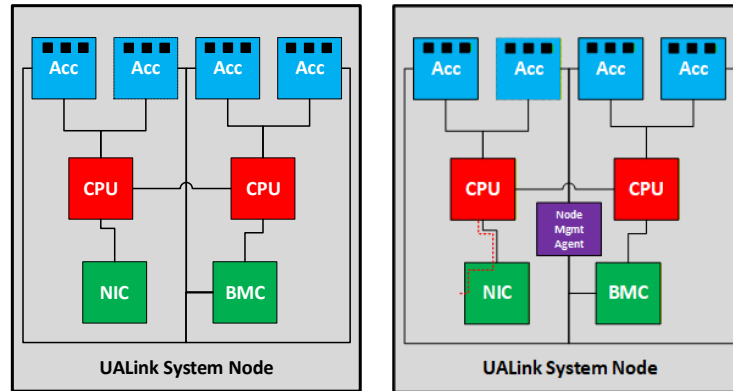


Figure 7-1 UALink System Node

7.2 UALink Switches and Switch Platforms

UALink Switches are responsible for routing traffic between Accelerators, and are managed by software/firmware called a Switch Management Agent. Physical Switches are implemented in hardware, but may be partitioned into multiple (Logical) Switches for purposes of bandwidth aggregation.

Often, Physical Switches are hosted on Switch Platforms that run the Switch Management Agent on a processor (such as an x86 CPU or a Baseboard Management Controller). The processor is attached to each Physical Switch via a high-speed interface such as PCIe, as well as to a network interface for communication with the Pod Controller. Commonly, for security and identity purposes, a Switch Platform contains one or more e/iRoTs (Root of Trust) and/or a trusted platform module (TPM). A Switch Platform may contain a separate BMC for platform management.

The Switch Management Agent shall interface with the Switches as specified in the *Ultra Accelerator Link Manageability Specification*. Detailed requirements for UALink Switches are included in the *Switch Requirements* chapter of this specification.

The following figure illustrates one possible UALink Switch Platform with a Switch Management Agent running on a processor, managing two Physical Switches each with six ports. In this implementation, the processor and Physical Switches are separate devices. This Switch Platform also contains a NIC, a TPM, and a BMC separate from the processor.

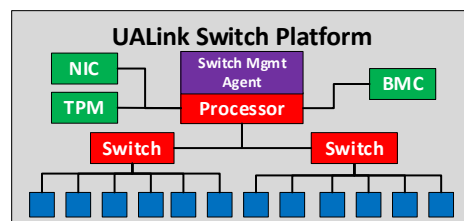


Figure 7-2. A UALink Switch Platform

7.3 UALink Pod Controller

The UALink Pod Controller software shall act as the centralized manager of the Pod's resources, and should run external to the System Nodes and Switch Platforms under its management. Its responsibilities include:

- Pod resource configuration
- Validation of Pod wiring against UALink rules and user-defined policies
- Allocation and assignment of Accelerator IDs
- Setup and teardown for Switch routing
- Managing Pod RAS including error recovery and log collection
- Pod health management
- Pod telemetry collection

The Pod Controller interacts with several interfaces relating to the Pod:

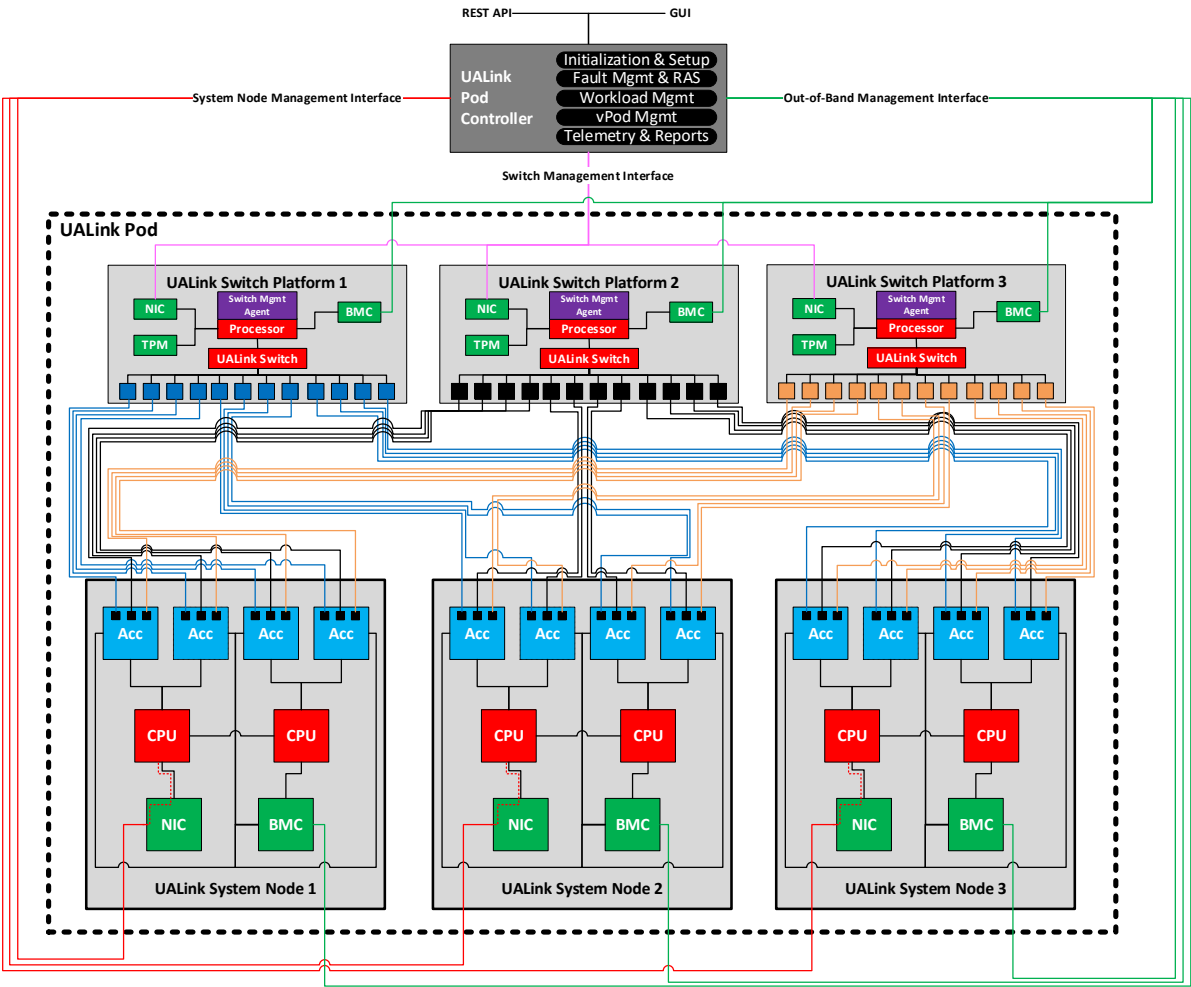
- It connects with each Node Management Agent to assign Accelerator IDs.
- It connects with each Switch Management Agent to configure ports and set up routing information for the Pod.
- It may connect to each Switch Platform or System Node's BMC to do out-of-band management such as via Redfish².
- It may present one or more Pod-external interfaces such as a REST API, GUI, or command terminal.

A singular Pod Controller manages the Pod at any given time. However, implementations should consider Pod Controller architectures that permit high availability/redundancy/failover.

The following figure illustrates a UALink Pod containing a Pod Controller, three Switch Platforms, and three System Nodes. Each Switch Platform features a single Physical Switch with twelve ports, managed by a Switch Management Agent. Each System Node features four Accelerators, each with three ports. The Pod Controller utilizes multiple logical interfaces for both managing the Pod as well as for interaction outside of the Pod.

² <https://www.dmtf.org/standards/redfish>

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0



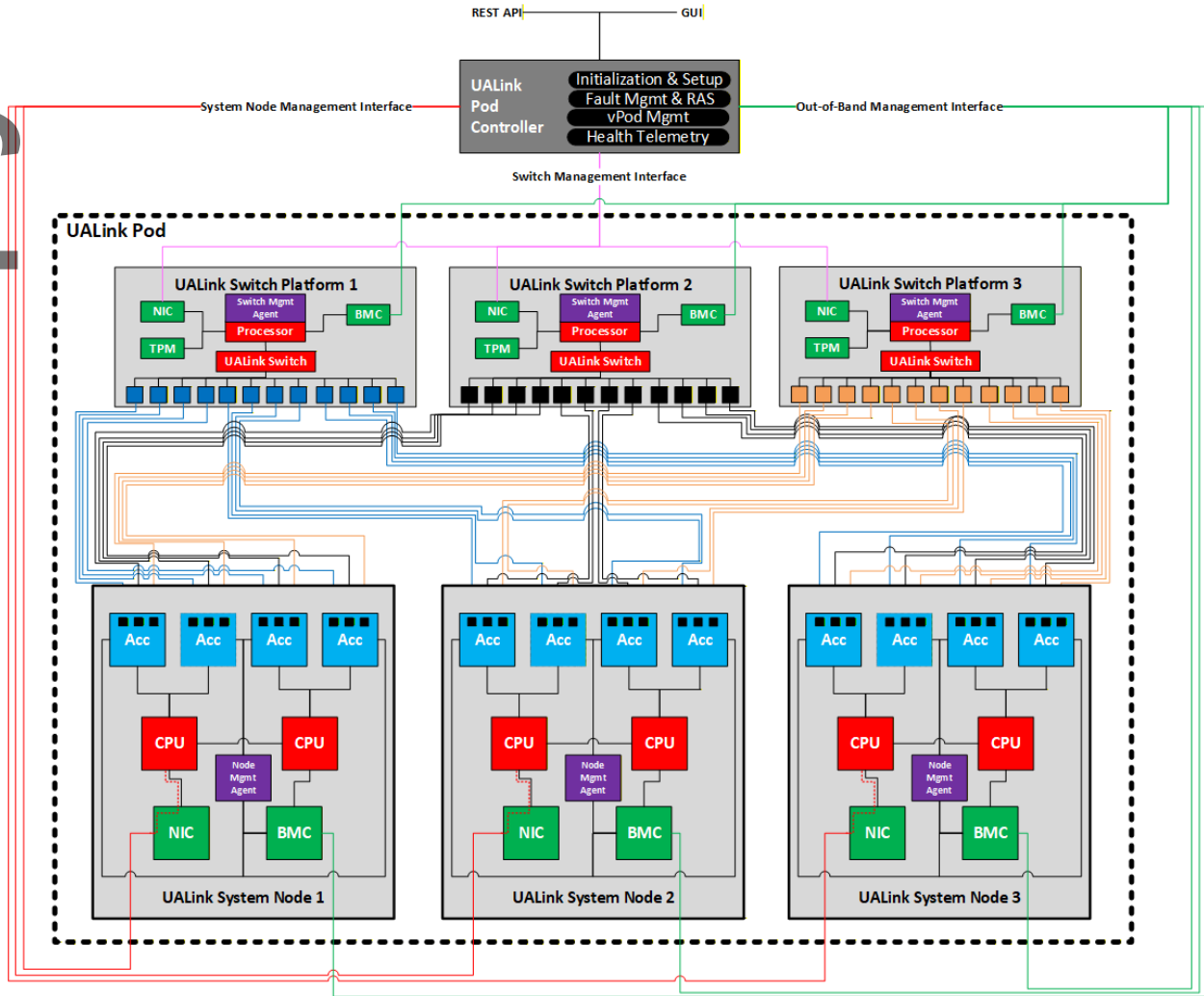


Figure 7-3. A UALink Pod managed by a Pod Controller

7.4 UALink Virtual Pods

A workload may require all the Accelerators in a Pod to meet its compute needs or it may require only a subset of the Accelerators. To use the Pod resources efficiently, the Pod Controller may permit partitioning the Pod into Virtual Pods. The Virtual Pods are assignable to individual tenants, allowing multiple tenants to share the Pod and run their workload in their respective partitions concurrently. These Virtual Pods may be restricted to a single Accelerator within a System Node, may encompass multiple Accelerators within the same System Node, or may span multiple System Nodes (up to all the Accelerators in the Pod) via the Switches. Virtual Pods also require compute and network resources (i.e., for use by virtual machines that run on the System Node's host CPUs).

An Accelerator shall not belong to more than a single Virtual Pod at a given time. Further, Accelerators that are members of a Virtual Pod shall be assigned as a whole; fractional assignment of Accelerators is not permitted in a Virtual Pod. If SR-IOV is utilized on an SR-IOV capable Accelerator, the SR-IOV Virtual Functions are not isolated from each other with respect to traffic on the UALink fabric and therefore must be assigned to the same tenant security domain.

Each System Node participating in a Virtual Pod hosts the tenant VM, and the appropriate Accelerators on the System Node are assigned to that VM. The VM hosts the device driver, e.g. VF drivers for SR-IOV supported Accelerators, that send control messages to the Accelerator for configuring the Accelerator including the UALink ports (VM/VF specific configuration) and programming the device for data transfers between VM and Accelerator and between peer Accelerators in a Virtual Pod.

The following figure illustrates a UALink Pod that has been partitioned by the Pod Controller into three Virtual Pods. Virtual Pod 1 contains a subset of the Accelerators in System Node 1. Virtual Pod 2 contains a subset of the Accelerators in both System Nodes 1 and 2. Virtual Pod 3 contains all the accelerators in System Node 3. One Accelerator on System Node 2 is not part of any Virtual Pod and is excluded from routing tables on the Switches.

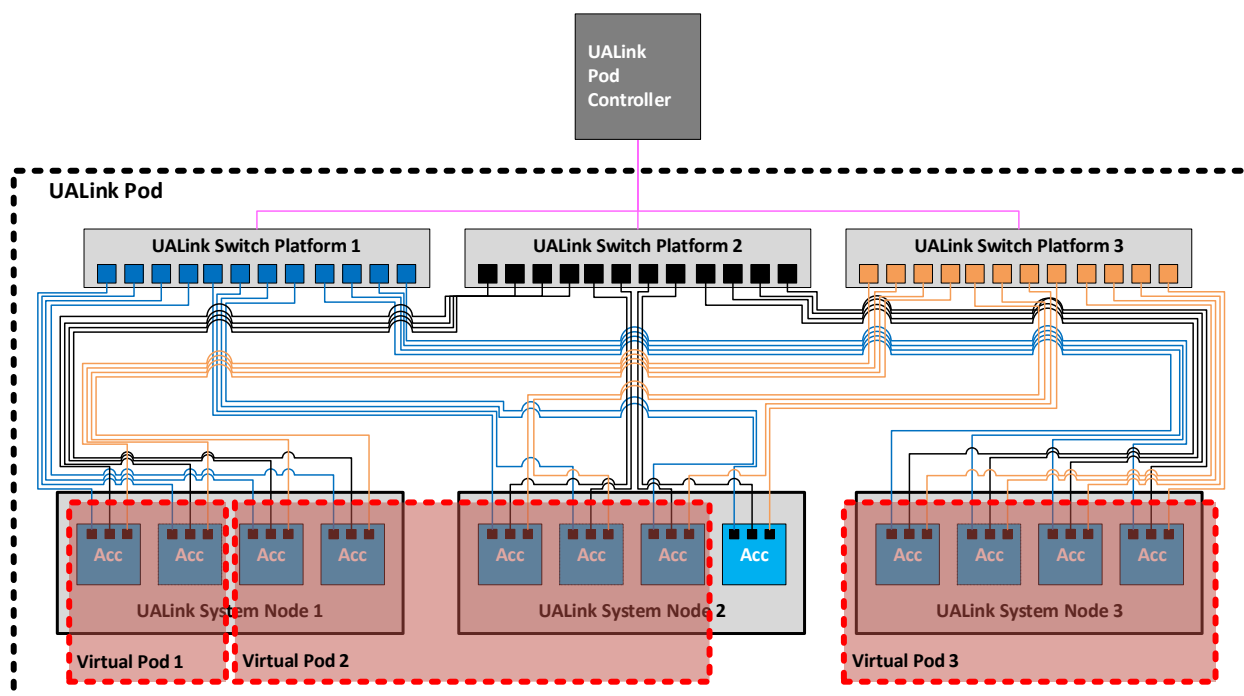


Figure 7-4. A UALink Pod partitioned into three Virtual Pods

7.5 Manageability Workflows

The requirements for managing a UALink Pod are documented in the *Ultra Accelerator Link Manageability Specification*, including workflows for:

- Accelerator and Switch admission to the Pod
- Pod topology discovery and validation
- Switch configuration including partitioning and routing table setup
- Virtual Pod creation
- Link, Accelerator, and Switch failure and recovery
- Error reporting such as drop counters and link state changes
- Accelerator telemetry such as transmit/receive statistics
- Switch telemetry such as port-level statistics and Switch health reporting

8 Security

The UALink link protection feature, referred to as UALinkSec, is intended to protect traffic on a UALink network from a physically present adversary; the adversary might be present at the time of the attack or may have placed a device (e.g., an interposer) to snoop or tamper with the UALink traffic. Additionally, in platforms that support Confidential Computing (CC), UALinkSec protects the Tenant data on a UALink network from the infrastructure provider and from other Tenants potentially collocated on the same UALink cluster. CC implies that a Trusted Execution Environment (e.g., Intel TDX, AMD SEV and ARM CCA) exists on the platform; the TEE under the control of the Tenant is responsible for UALinkSec configuration. When enabled, UALinkSec minimally provides data confidentiality and optionally data integrity (including replay protection) to UALink network traffic. The next section provides an example of system used to illustrate UALinkSec principles followed by the Security Model in Section 8.3.

8.1 References

SPDM: Security Protocol and Data Model (SPDM) over MCTP Binding Specification:

<https://www.dmtf.org/dsp/DSP0275>

AES-GCM: NIST Special Publication 800-38D Recommendation for Block Cipher Modes of Operation- Galois/Counter Mode (GCM) and GMAC:

<https://csrc.nist.gov/pubs/sp/800/38/d/final>

TDISP: Refer to PCIe Express specification for TDISP:

<https://pcisig.com/specifications>

Key Derivation Function: NIST SP 800-108 Rev. 1, "Recommendation for Key Derivation Using Pseudorandom Functions", August 2022 (includes updates as of February 2, 2024), URL:

<https://csrc.nist.gov/pubs/sp/800/108/r1/upd1/final>

KMAC256: NIST SP 800-185, "SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash", December 2016, URL:

<https://csrc.nist.gov/pubs/sp/800/185/final>

8.2 System Overview

An UALink-Pod is comprised of a set of System Nodes each of which may have one or more accelerators. Each accelerator will have one or more UALink ports. The accelerators within the Pod communicate with each other via a switch-based UALink fabric. It is assumed that every accelerator in a Pod can communicate with every other accelerator in the Pod. The Pod shall be managed by a central Pod Controller software, running on platform(s) outside of the Pod, that works in concert with Node Management Agents on the System Node. The system software such as host OS and hypervisor is responsible for performing actual resource management and scheduling on each System Node as directed by the Pod Controller.

The Pod Controller creates a Virtual Pod that contains the number of accelerators needed by the tenant and then configures the UALink ports on the accelerators and the UALink switches for communication within that Virtual Pod. Each System Node shall run tenant software, such as a tenant VM, and the accelerators (including the UALink ports) that are part of tenant's Virtual Pod shall be assigned to the tenant VM, with the actual assignment tasks performed by the OS or hypervisor on the System Nodes.

The tenant VMs on each System Node shall configure the accelerators including programming of tenant specific keys. The tenant VMs belonging to different System Nodes may communicate using frontside networking regarding configuration and programming of accelerators in the Virtual Pod..

The tenant that intends to use collectives shall also configure the switches for collective operations such as programming the group table in the switch for collective operations. The communication between tenant SW on the host and switch SW/FW for group table and other collective configuration may occur over UALink via Load/Store operation.

There are two distinct role-based software execution domains; one is the infrastructure owner controlled remote and local management software (and firmware) that manage compute and networking resources at Pod, System Node and switch level and the other is the tenant software that run on the compute resources assigned to it and use the UALink network to allow its accelerators to access each other's data. A tenant that uses collectives will also have resources assigned on the switch for collective operations. The UALink security for confidential computing use requires HW enforced isolation of these two execution domains to provide strong security assurance to the tenant application in presence of potential SW and HW exploits in the datacenter.

For more details on the manageability aspects of an UALink Pod including Virtual Pod creation and its lifecycle management, refer to the manageability chapter in this specification.

8.3 Security model

This section describes the security objectives for the UALink Link Protection feature, the corresponding TCB (Trusted Computing Base) and the adversary profile and capabilities.

One of the use cases for the UALink protection feature is Confidential Computing (CC). Confidential Computing mandates the most stringent threat model for the use cases envisioned, where the Infrastructure provider is not trusted. As such, the security model presented in this section is defined with CC in mind; UALinkSec aims at providing the owner of a Virtual Pod (called the Tenant hereafter) control over the security of its data, without reliance on the Infrastructure provider to meet the security objectives stated below.

8.3.1 Security objectives:

The UALink link protection feature aims at providing confidentiality and optional integrity (including replay protection) for data exchanged between accelerators belonging to the same Virtual Pod.

If the tenant uses collectives, the UALink link protection feature also aims to provide confidentiality and optional integrity of data exchanged between the accelerator and switch for collective operations.

These security objectives are qualified by the adversary profile and capabilities described in Section 8.3.3.

Note: Availability is not in scope for the UALink link protection feature, and it is considered the responsibility of the infrastructure provider.

8.3.2 Trusted Computing Base (TCB)

The TCB describes the trusted elements that UALinkSec relies on to meet its security objectives:

- The *Tenant* is at the heart of the TCB as it deploys TVM (aka Trusted OS domains) that control the accelerators in the Virtual Pod assigned by the Pod controller
- All the accelerators (hardware and firmware) belonging to the Virtual Pod assigned to the Tenant are trusted, after they are brought in TCB. The accelerators in an OS domain are brought in TCB by the TVM through standard secure mechanisms for authentication and attestation (e.g., SPDM, DICE) that are out of scope for the UALinkSec specifications. For instance, if the accelerators are attached through PCIe to the Host CPUs running the TVM, the PCI SIG TDISP standard should be leveraged by the TVM to bring the accelerators in TCB.
- The switches that perform collectives for a tenant are trusted after they are brought into the TCB. The switches are brought into TCB by the TVM through similar standard mechanisms for authentication and attestation as the ones used to bring accelerators into the TCB.

For clarity, the untrusted agents are captured below. However, the list is not exhaustive, and any agents not listed as part of the TCB shall be considered untrusted:

- The hardware and firmware of UALink switch not used for collectives are not trusted and are not brought into tenant's TCB.
- All control and management firmware and software provided by the infrastructure provider are untrusted, including the hypervisor running on the Host CPU as well as the Pod controller software and firmware running on a BMC or potentially as a service process on Host CPUs.
 - Note: Our threat model addresses the scenario where a host SW arbitrarily adds or removes a Virtual Pod member with malicious intent.

8.3.3 Adversary profile and capabilities

Table 8-1 below describes the types of attack the adversary can attempt to mount an exploit.

Table 8-1 Possible attack types

Adversary tools	Description	Adversary goal
Transaction snooping	The adversary extracts UALink packets at any accessible interface to read, analyze UALink traffic	Read cleartext. Retrieve ciphertext to observe and analyze patterns.
Transaction corruption	The adversary either replaces genuine packets on UALink link with chosen ones or flip specific bits of packets in transit.	The purpose of the adversary is to get the corrupted packets accepted and consumed at destination.
Transaction replay	The adversary records packets and injects them later on.	The purpose of the adversary is to get the replayed packets accepted and consumed at destination.
Transaction deletion	The adversary intercepts and drops a packet sent by a source in order for the destination to never receive it.	The purpose of the adversary is to get the destination to consume stale data, for instance by preventing a memory location from being updated.
Transaction injection	The adversary crafts a packet it injects toward an accelerator of its choice.	The purpose of the adversary is to get the injected packets accepted and consumed at destination.
Endpoint spoofing	The adversary assumes the identity of a valid endpoint	The purpose of the adversary is to steal sensitive data by sending read request or receiving read response.

UALinkSec aims at countering two types of adversaries: physically present and remote adversaries.

The *Physically present adversary* may be an Infrastructure provider employee, a system debugger or an intruder getting physical access to the UALink Network to perform the malicious operations specified in Table 8-1 and build a successful exploit. Such an adversary may:

- Perform a live attack while being physically present, e.g., by probing the UALink links
- Place an interposer to snoop or inject packets in live traffic at a later time
- Replace accelerators with vulnerable ones or from malicious manufacturers, untrusted by the Tenant

The *Remote adversary* typically leverages firmware or software vulnerabilities in the system to gain access to the UALink traffic and perform the malicious operations specified in Table 8-1 and build a successful exploit.

Note: An adversary may be a hybrid of the two profiles above, where a physically present adversary (e.g., system debugger) leverages a vulnerable firmware or software to build a successful exploit.

8.3.4 Security Assumptions

UALinkSec has been defined with the following security assumptions:

- It is expected that the platform security solution and the accelerator design provide a mechanism for the Tenant to securely configure the UALink endpoint (e.g. configure encryption settings) and program the Tenant specific master key(s). The exact mechanism is out of scope for the UALink specification and is implementation specific.
- The UALink fabric is ordered, i.e., packets are processed cryptographically in the same order at the source and at the destination.
- Packet headers for both request and response shall be encrypted to meet the UALinkSec security objectives, except for the fields required for routing, request/response compression and flow control. When integrity protection is enabled, fields required for routing, request/response compression and flow control are integrity protected but shall remain in plaintext.
- When an accelerator or a switch is accepted by the Tenant as part of its Virtual Pod and UALinkSec is enabled, all the traffic is protected with UALinkSec, i.e., the standard does not support mixing protected and unprotected traffic.
- Key materials shall be adequately protected in the accelerator to minimally meet the threat model defined in this specification. How this is achieved is outside the scope of this specification and is implementation specific.
- The firmware and software ingredients in TCB are free from vulnerabilities and are not exploitable by the adversary.
- The hardware ingredients in TCB are free from defects and their capabilities (e.g., privilege debug capability) cannot be exploited by the adversary.
- When security mode is enabled, it is enabled for all the ports on an accelerator including bifurcated 2 lane ports and 4 lane port.

8.3.5 Threat model

Threat model is given in Table 8-2 below:

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification Rev 2.0

Table 8-2 Threat model

Threat description	Mitigation
1. Adversary gets access to UALinkSec secrets (e.g., keys or encrypted traffic)	UALink traffic is encrypted on interfaces accessible to the adversary. Keys are kept inside the accelerators. It is the responsibility of the accelerator manufacturer to properly protect key storage and programming from Adversaries in scope for UALinkSec. The following requirements shall be met: Secrets shall not be included in unencrypted headers A monotonic counter is used to generate the Invocation field of the IV Follow the recommended number of key invocation limit before refreshing the key.
2. Adversary injects chosen UALink packets or corrupts packets between two UALink ports	When integrity protection is enabled, an integrity tag is calculated at the destination port and verified against the integrity tag carried in the received packet. Since the adversary does not have the key, an injected or corrupted packet is detected by failure of the verification of the integrity tag. <i>Note:</i> when integrity protection is not enabled, the decryption of the injected or corrupted packet will likely result in unexpected behavior as decryption will use a different counter value than the one used for encryption of the packet.
3. Adversary replays UALink packets	When integrity protection is enabled, an integrity tag is verified at the destination port. The integrity tag computation is based on a counter (nonce) that is incremented for each packet received. When the adversary replays a packet, the counter used by the destination port for integrity tag calculation is different than the counter that was used for the tag carried with the replayed packet, resulting in a tag mismatch and replay detection. <i>Note:</i> when integrity protection is not enabled, the decryption of the replayed packet will likely result in unexpected behavior as decryption will use a different counter value than the one used for encryption of the packet.
4. Adversary intercepts and drops packet(s) between two UALink ports	When integrity protection is enabled, an integrity tag is verified at the destination port. The integrity tag computation is based on a counter (nonce) that is incremented for each packet received. The dropped packet is detected as soon as the next packet is received, as a counter mismatch triggers a failure of the integrity tag verification.
5. Adversary leverages a vulnerability in the switch to snoop on traffic or inject traffic	The switch is outside the TCB and only sees encrypted and integrity protected traffic. It does not have the key required to properly encrypt and integrity protect UALink packets. As a result, any attack from the switch is mitigated in the same way as for Threat 1, 2, 3 and 4 above.
6. Adversary (i.e., infrastructure provider) includes a vulnerable accelerator (e.g., running known vulnerable firmware) in the Virtual Pod offered to the Tenant.	UALinkSec requires the Tenant to authenticate and attest the accelerators in the Virtual Pod before distributing the secret keys necessary to send and receive encrypted UALink traffic. Through authentication and attestation, the Tenant verifies the accelerators offered by the infrastructure provider meet its policy requirements.
7. Adversary swaps an accelerator with malicious one after initial authentication and attestation	UALinkSec provides runtime binding. The initial accelerator authentication and attestation at Virtual Pod creation/acceptance by the Tenant, is extended through the life of the Virtual Pod by establishing the shared secret used for traffic encryption. If the adversary swaps a genuine device with a malicious one, the malicious device cannot participate in the Virtual Pod traffic as it does not know the necessary secret key to send and receive traffic.
8. Brute force attack by observing cipher text on UALink link and knowledge of plaintext from workload analysis. Goal of the adversary is to retrieve the key.	UALinkSec uses AES-GCM with 256b following NIST recommendations – to prevent adversary from retrieving the secret key or plaintext from ciphertext.

8.4 UALink System Security

The confidential computing capable UALink system is assumed to have baseline TEE capabilities on each system node.

Following assumptions are made about the UALink system

- A Pod may have a mix of confidential and non-confidential Virtual Pods running concurrently
- An accelerator that is part of a multi-accelerator virtual Pod shall be assigned to a single tenant e.g. single VM if virtualized. An accelerator that is partitioned into multiple virtual functions (i.e., multiple virtual accelerator instances) and assigned to multiple VMs shall not be added to a Virtual Pod and shall not be allowed to access the UALink fabric.

- UALink subsystem shall be disabled and isolated from all other accelerator subsystems if the accelerator is not configured for communicating via UALink fabric. The accelerators that are not configured for scale-up shall not allow access to or from the UALink fabric to prevent network attacks

The following security features are assumed on the CPU:

- Shall be TEE capable allowing Tenant data and control on the host to be protected in a TVM.
- Shall support TEE-IO to allow bringing TEE capable accelerators into TVM's TCB. While UALinkSec does not mandate it, it is recommended that CPU has TEE-IO support (for example, AMD SEV-TIO and Intel TDX Connect) for devices that are TEE Device Interface Security Protocol (TDISP) compliant.
- Shall support accelerator authentication and link protection on the link between host and device.

The accelerator shall be TEE capable with secure interface to the host to enable TEE-IO with TVM. The accelerator is assumed to have support for:

- TDISP version 1.2 or above or equivalent security protocol. It shall allow locking the device configuration, providing device report and status and enabling secure MMIO and DMA.
- A unique cryptographic device identifier and device attestation enabling establishment of authenticated secure session between host and the accelerator. It is recommended that the accelerator support DMTF defined SPDm 1.3 or equivalent protocol for device attestation and secure session setup between TEE security manager (TSM) on the host and Device Security Manager (DSM) on the device.
- If virtualization is supported, device shall provide HW enforced isolation of PF and VF.

The INC capable switch is assumed to be TEE capable and provide a secure interface to the node SW for programming the encryption key material and for secure configuration of collective settings.

Switch is assumed to support:

- A unique cryptographic device identifier
- SPDm 1.3 or equivalent protocol for switch attestation and establishment of secure session with the switch.
- A switch security manager (SSM), responsible for managing the security of the Virtual Pod Partition in the switch. This is equivalent to the TDISP DSM in a PCIe device. The SSM is required to expose a security interface, equivalent to the TDISP interface, to the trusted SW on the host. The trusted SW is expected to use this interface to lock the switch configuration, such as the routing table, associated with the tenant's Virtual Pod partition, to transition the Virtual Pod Partition to run state and to read Virtual Pod Partition settings. If the switch detects a security error such as an integrity failure in the crypto engine, the SSM will transition the Virtual Pod Partition to stop state and stop processing any incoming data from Virtual Pod members and stop sending any outgoing messages to the Virtual Pod members.
- Secure storage and management of cryptographic keys used for link encryption and SPDm sessions.
- Isolation of resources associated with a Virtual Pod partition, such as group table, memory, and other assets from local and remote adversaries.

The system level security architecture described below assumes accelerator to be TDISP compliant and describes the security workflow using TDISP context. If implementation uses some other device security protocol, the system level security architecture shall be adapted accordingly. The choice of device security protocol does not affect UALinkSec directly but may affect security workflow such as UALink key programming.

Figure 8-1 shows a system level view of confidential computing in a pod. The components that are not in Tenant's TCB include Pod controllers, Node Management Agents, host OS, PF drivers and hypervisors. These perform resource management, allocation and scheduling as they would for a non-confidential computing use case. The TCB includes TVM, TSM and the accelerator, including the UALink ports. The figure shows switches to be outside the TCB, however, they are part of the TCB if the tenant workload performs collective operations.

The TVM shall verify the attestation of Virtual Pod's compute elements and verify the correctness of Virtual Pod configuration before running its application. This may be done by a Confidential Cluster Manager (CCM) within the TVM as shown in Figure 8-1.

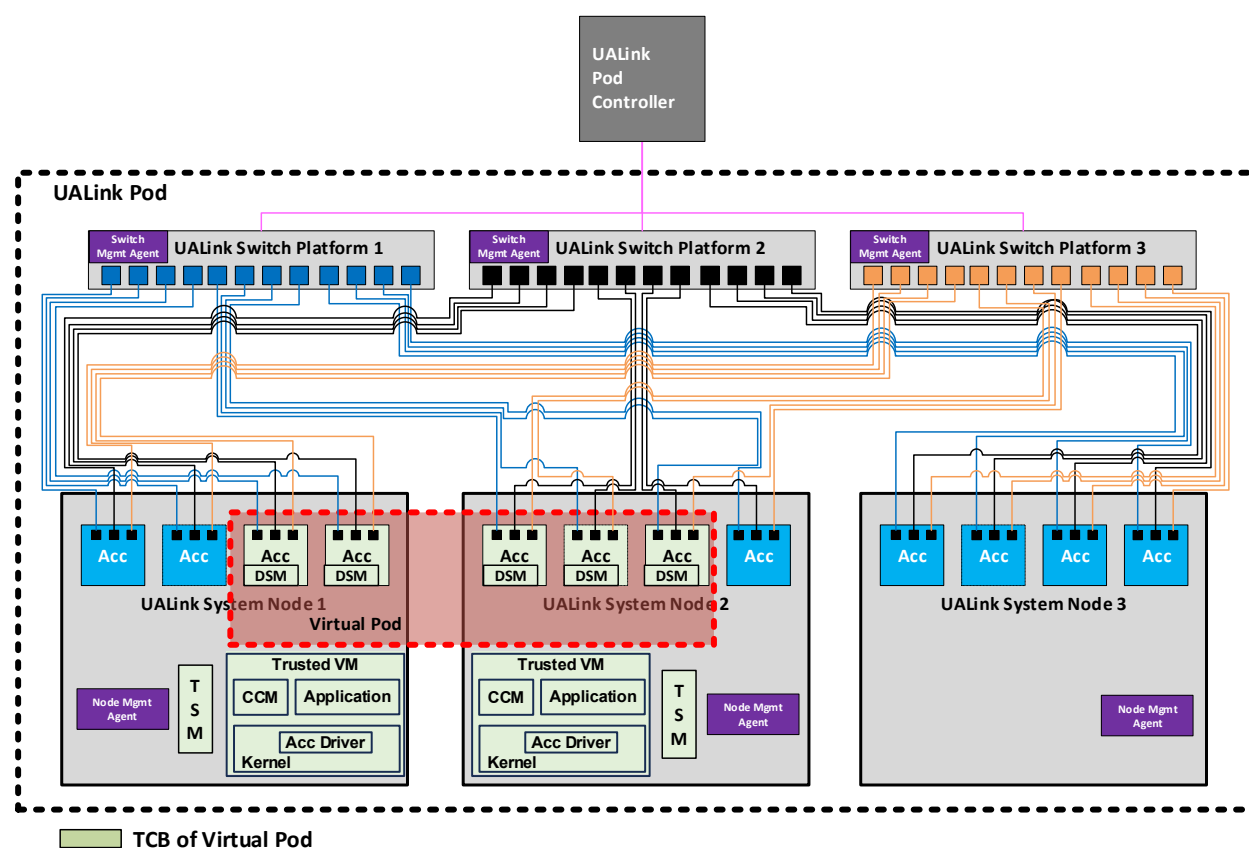


Figure 8-1 System level view of confidential computing in a pod

A Tenant running a Virtual Pod spanning several System nodes shall have a TVM on each of its System nodes. The Pod Controller shall provide a list of tenant IP addresses for all TVMs to every TVM in the Virtual Pod which allows the TVMs to attest each other and establish cryptographically protected sessions with each other. All communication between TVMs shall occur via the front-end network and shall be encrypted and integrity protected.

When creating a confidential computing Virtual Pod, the initial steps for creating a Virtual Pod maybe similar to a non-CC Virtual Pod creation. TVMs shall perform a set of security steps to verify attestation of all devices in the TCB and verify the correct construction and configuration of the Virtual Pod. If verification succeeds, TVM shall lock the configuration of the accelerators (including the UALink port configurations) and program UALink keys in the UALink ports before starting the workload. A lead TVM, typically the one running on the System node where the workload was first initiated, is expected to gather the security configuration of all accelerator members via the TVMs on other nodes and verify that they have been configured correctly as per Tenant's security policy. An important check includes ensuring all accelerators in the Virtual Pod have unique identifiers. The lead TVM shall then generate the UALink master keys and distribute it to other TVMs over secure channel. All TVMs shall then program the keys into the UALink ports in the local accelerators via a secure interface.

The secure key programming interface shall provide confidentiality, integrity and replay protection of the keys in accordance with the security model described earlier. The secure key programming interface may be in the form of device specific interface exposed to the VF driver allowing TVM to program the keys through secure memory mapped interface in the virtual function hardware. Alternately, the keys may be delivered from TVM to the UALink port using SPDMM messages between TSM and DSM. These are implementation choices as long as the security requirements are met.

The accelerators shall be assigned accelerator identifiers by the Pod Controller when the pod comes up and is configured. This accelerator identifier serves as the identity of the transmitter and receiver in the UALink transactions and shall be unique across the virtual pod. Since Pod controller is outside the TCB and is not trusted to ensure uniqueness of accelerator identifiers, when a Virtual Pod is created, the lead TVM is responsible for getting accelerator identifiers of all accelerators in the Virtual Pod from all TVMs in the Virtual Pod and ensuring they are unique.

If the workload uses collectives, similar steps are taken to securely configure the accelerators and switches for accelerator-switch link protection as described in Section 8.6.

8.5 Encryption and authentication scheme for UALink

To meet the security objectives, UALink standard uses AES-GCM. AES-GCM data processing is performed in the protocol/functional layer as shown in Figure 8-2 below:

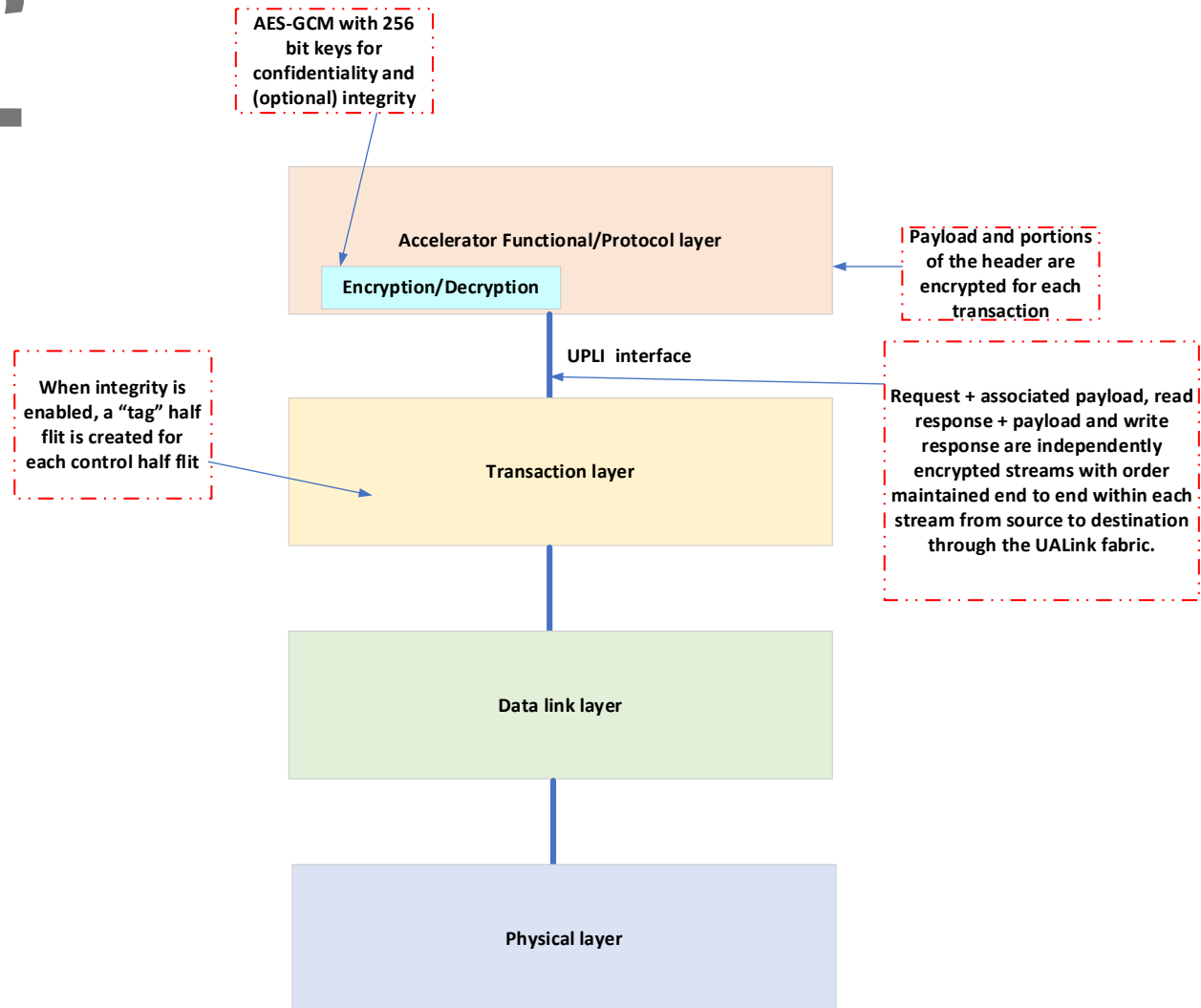


Figure 8-2 Encryption and Authentication touch points in UALink stack

AES-GCM for UALink will have the following parameters:

- A. Key size - 256 Bit
- B. Tag size - 8Bytes or 0B (i.e., Tag is optional).

Encryption is done on a per UALink transaction basis. The encrypted transaction along with the optional tag is sent over the UPLI interface to the transaction layer. The transaction layer shall send the transaction along with the tag by creating a tag half flit that accompanies the control half flit as illustrated in the example flit format in Figure 8-3 below:

	64-byte TL Flit															
	Upper TL Half-Flit								Lower TL Half-Flit							
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Flit																
0	AuthTag1		AuthTag0		AuthTagB		AuthTagA		CWReq1		CWReq0		CWR B	CWR A	NOP	FC
1	Req0.Data.1								Req0.Data.0							
2	Req0.Data.3								Req0.Data.2							
3	Req0.Data.5								Req0.Data.4							
4	Req0.Data.7								Req0.Data.6							
5	Req1.Data.1								Req1.Data.0							
6	Req1.Data.3								Req1.Data.2							
7	Req1.Data.5								Req1.Data.4							
8	Req1.Data.7								Req1.Data.6							

Figure 8-3 Example of UALink TL flit with the "Tag" half flit

The order of the tags in the tag half flit shall be exactly same as the order of the requests/responses in control half flit. Note that the tag is for request + payload or response + payload in case the request or response has an accompanying payload.

8.5.1 Control Half-Flit field encryption and authentication

UALink Control Half-Flit fields are only partially encrypted as certain sub-fields are necessary for routing and flow control. The only sub-fields that are encrypted are Address [17:2], UPLI transaction identifier Tag and ASI in requests. The only sub-field that is encrypted in responses is the UPLI transaction identifier Tag.

When the integrity mode is enabled, select Control Half-Flit sub-fields are integrity protected.

Request channel signals that are encrypted and authenticated are given in Table 8-3.

Table 8-3 Request channel signals that are encrypted and authenticated

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
ReqVld	No	No	No	Not visible on the wire	Not visible on the wire
ReqPortID	No	No	No	Not visible on the wire	Not visible on the wire

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
ReqASI	Yes	No	Yes	N/A	May be used by Switch.
ReqAuthTag	N/A	N/A	Yes	N/A	N/A
ReqSrcPhysAccID	Yes	No	Yes	N/A	Used by switch (used to index Address Compression Cache)
ReqDstPhysAccID	Yes	No	Yes	N/A	Used by switch (used to index Address Compression Cache and to route Request)
ReqTag	Yes	No	Yes	N/A	Tag being encrypted in request and unencrypted in response can lead to security vulnerability.
ReqNumBeats	Yes	No	Yes	N/A	Used by switch
ReqAddr	All bits are authenticated	[17:8] [5:0]	Yes	N/A	Bits [56:18] may be removed by TL due to the compression scheme. Bits [7:6] is left unencrypted to allow the TL to properly pack Byte

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
					Enable TL Half-Flits.
ReqCmd	Yes	No	Yes	N/A	May be used by Switch (possible at least with VDCs)
ReqLen	Yes	No	Yes	N/A	May be used by Switch (possible at least with VDCs)
ReqMetadata	Yes	No	Yes	N/A	Must be unencrypted to allow TL to compress a specific Metadata value. Encryption would mask the specific value and corrupt the compression.
ReqVC	Yes	No	Yes	N/A	May be used by Switch to order Requests. Used by Credit loops at Switch and TLs.
ReqPool	No	No	No	<u>Not visible on wire.</u>	Not visible on wire.

Read response channel signals that are encrypted and authenticated are given in Table 8-4.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Table 8-4 Read response channel signals that are authenticated and encrypted

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
RdRspVld	No	No	No	Not visible on the wire	Not visible on the wire
RdRspPortID	No	No	No	Not visible on the wire	Not visible on the wire
RdRspAuthTag	N/A	N/A	Yes	N/A	N/A
RdRspSrcPhysAccID	Yes	No	Yes	.	Must be unencrypted to allow requestor accelerator to index lookup table of tag & destination accelerator ID to determine correct IV value for decryption and authentication.
RdRspDstPhysAccID	Yes	No	Yes	N/A	Used by switch (Used to route Response)
RdRspTag	Yes	No	Yes	N/A	Must be unencrypted to allow master accelerator to index lookup table of tag to the Request Destination Accelerator to look up correct IV value.

**Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0**

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
RdRspNumBeats	Yes	No	Yes	N/A	Used by switch
RdRspData	Yes	Yes	Yes	N/A	N/A
RdRspStatus	Yes	No	Yes	N/A	Must be unencrypted to allow TL to compress the predominant “no error” responses. Encryption would corrupt compression decision.
RdRspOffset	Yes	No	Yes	N/A	May be used by switch
RdRspLast	Yes	No	Yes	N/A	May be used by switch
RdRspDataError	No	No	Yes	Not visible on the wire	Not visible on the wire
RdRspVC	Yes	No	Yes	N/A	May be used by Switch to order Requests. Used by Credit loops at Switch and TLs.
RdRspPool	No	No	No	Not visible on wire.	Not visible on wire.
dRspTypeInfo	TBD	TBD	Yes	New signals for collectives – waiting to find out implications for TL	New signals for collectives – waiting to find out implications for TL

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

For the write response channel, signals that are encrypted and authenticated are given in Table 8-5.

Table 8-5 Write response channel signals that are authenticated and encrypted

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
WrRspVld	No	No	No	Not visible on the wire	Not visible on the wire
WrRspPortID	No	No	No	Not visible on the wire	Not visible on the wire
WrRspAuthTag	N/A	N/A	Yes	N/A	N/A
WrRspSrcPhysAccID	Yes	No	Yes		Must be unencrypted to allow requestor accelerator to index lookup table of tag & destination accelerator ID to determine correct IV value for decryption and authentication.
WrRspDstPhysAccID	Yes	No	Yes	N/A	Used by switch (Used to route Response)

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
WrRspTag	Yes	No	Yes	N/A	Must be unencrypted to allow master accelerator to index lookup table of tag to the Request Destination Accelerator to look up correct IV value.
WrRspStatus	Yes	No	Yes	N/A	Must be unencrypted to allow TL to compress the predominant “no error” responses. Encryption would corrupt compression decision.
WrRspVC	Yes	No	Yes	N/A	May be used by Switch to order Requests. Used by Credit loops at Switch and TLs.
WrRspPool	No	No	No	Not visible on wire.	Not visible on wire.
WrRspTypeInfo	TBD	TBD	Yes	New signals for collectives – waiting to find out implications for TL	New signals for collectives – waiting to find out implications for TL

For the originator data channel, signals that are encrypted and authenticated are given in Table 8-6.

Security

204

Table 8-6 Originator data channel signals that are authenticated and encrypted

Name	Authenticate	Encrypt	Visible on the wire?	Reason for not authenticating	Reason for not encrypting
OrigDataVld	No	No	No	Not visible on the wire	Not visible on the wire
OrigDataPortID	No	No	No	Not visible on the wire	Not visible on the wire
OrigData	Yes	Yes	Yes	N/A	N/A
OrigDataByteEn	Yes	Yes	Yes	N/A	N/A
OrigDataOffset	Yes	No	Yes	N/A	May be used by switch
OrigDataLast	Yes	No	Yes	N/A	May be used by switch
OrigDataError	No	No	Yes	Can be modified by Switch or TL.	Can be modified by Switch or TL.
OrigDataVC	Yes	No	Yes	N/A	May be used by Switch to order Requests. Used by Credit loops at Switch and TLs.
OrigDataPool	No	No	No	Not visible on wire.	Not visible on wire.

8.5.1.1 Source ID Lookup for Response

The transmitter shall maintain a lookup table of outstanding Requests and the source ID of destination accelerators the Request was sent to. Using the RspTag, which contains the ReqTag field from the initial Request, the transmitter shall determine the source ID of the responder. This lookup function is not part of TL. It may be implemented in the crypto engine or the accelerator functional layer.

8.5.2 Data authentication and encryption

All bytes of data and associated Byte Enables (if any) are encrypted and (optionally) integrity protected for writes, atomic requests and read responses as described in Table 8-4 through Table 8-6.

8.5.3 ISOLATE response handling

Decryption engine shall ignore ISOLATE response – ISOLATE response shall not be decrypted or integrity checked.

8.5.4 Modes of operation

An implementation can choose to support:

1. No security features
2. Encryption only
3. Encryption and integrity

In case an implementation has encryption only, then the following modes shall be supported:

1. Encryption enabled
2. Encryption disabled

In case an implementation has encryption and integrity, then the following modes shall be supported:

1. Encryption enabled
2. Encryption and integrity enabled
3. Encryption and integrity disabled

8.5.5 Poisoned data handling with security enabled

Encryption engine will skip the data beat and Byte Enable that has poison indication set while doing encryption and authentication tag generation. The encryption engine shall set all bytes of the poisoned beat to 0 and Byte enable to 0 before sending it on UPLI interface. The poison indicator shall be passed along with each data beat to the transmit path of the TL. The TL shall replace each poisoned data beat with 2 32B “poison” messages.

On the receive path, TL will decode the poison messages and assert the poison indicator for the corresponding 64B data beat on the UPLI interface. TL shall ensure that all bytes of the 64B poisoned data beat is 0 and Byte enable is 0 on UPLI interface. The decryption engine will skip poisoned data bytes and Byte Enable during the decryption and authentication tag check process.

8.5.6 Ordering requirements imposed due to AES-GCM

AES-GCM requires the sender and receiver to keep the varying part of the IV (essentially the 32-bit Invocation field) in lockstep with each other.

To achieve lockstep operation of sender (encryption) and receiver side (decryption), UALink transactions are grouped into three independent streams with transactions belonging to the same stream being kept strictly in order while travelling from a given source port to a given destination port through the UALink fabric.

The streams are defined as follows:

- A. Request stream (Stream Index 0)– this stream has Read requests, Write requests + associated payload and Atomic requests + associated payload.
- B. Read response stream (Stream Index 1) – this stream has Read responses + associated payload and atomic requests + associated payload.
- C. Write response stream (Stream Index 2) – this stream has write responses

These streams are defined on a per VC basis where each VC has 3 streams.

8.5.6.1 Notes on ordering

- There are no ordering requirements between different streams.
- There are no ordering requirements between traffic from different source ports to a given destination port.
- There are no ordering requirements between traffic from a given source port to different destination ports.
- There are no ordering requirements between traffic belonging to different VCs when per VC state replication is present.

8.5.7 Authentication and Encryption/Decryption Implementation in an UALink port

8.5.7.1 Key and other security state management

An UALink port is assumed to have transmit logic (TX) and receive logic (RX). Figure 8-4 below summarizes the per destination accelerator state elements maintained in TX and RX of a port for authentication and encryption in a 1024 accelerator system:

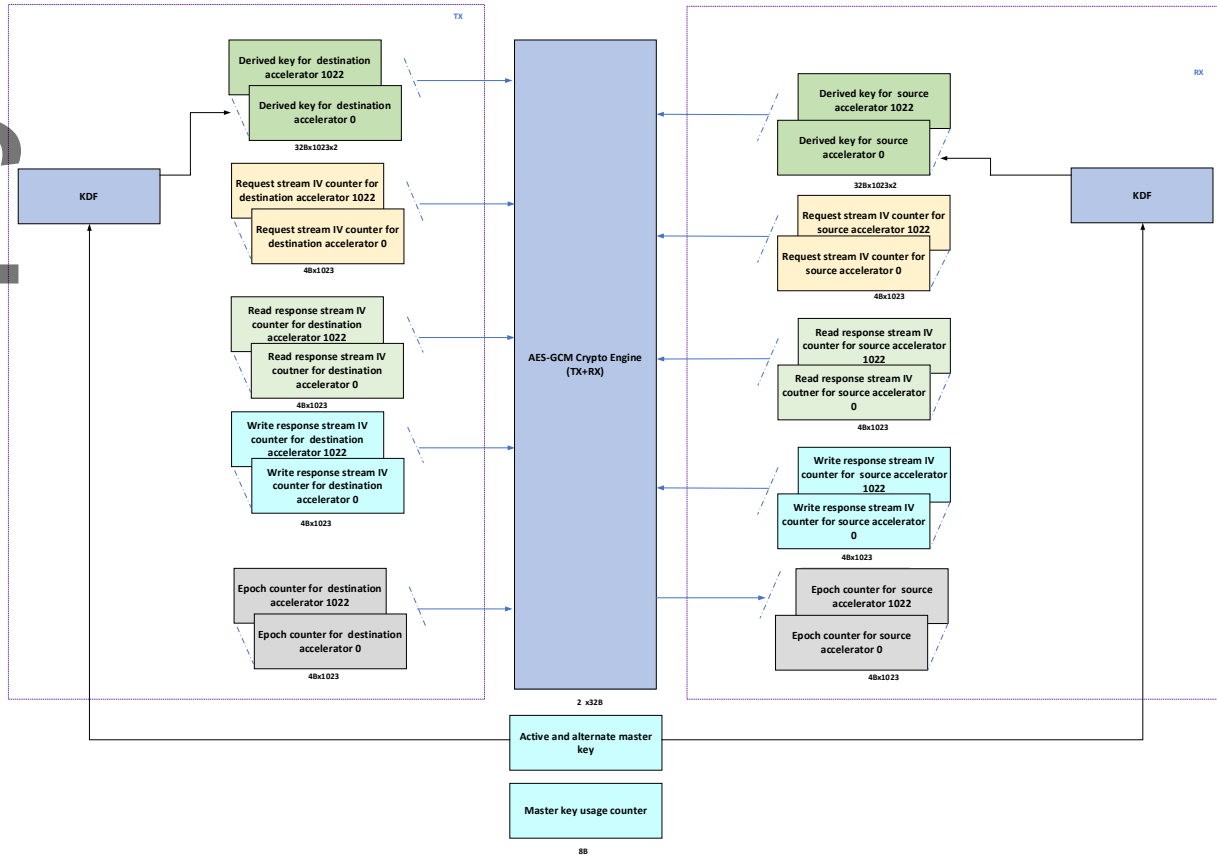


Figure 8-4 Security related state elements in UALink port TX and RX for a 1024 accelerator system

There is only one active master key and alternate master key per virtual Pod. It is assumed that a given UAL port's TX can only reach a single port on a remote accelerator; therefore, the UAL port TX only needs to maintain state on a per destination accelerator basis. As shown in Figure 8-4 the TX of each port shall maintain the following state:

1. A single master key [32B]
2. A single alternate master key [32B]
3. A per destination derived key [32B * 2*number of destinations]
4. A per destination derived key epoch counter [4B * number of destinations]
5. Per stream, per destination accelerator IV counter [4B*3*number of destinations]
6. A single master key usage counter [4B]

Note: For accelerator to accelerator link protection, it is assumed that an accelerator will not send transactions to itself through the UALink fabric.

As shown in Figure 8-4 Security related state elements in UALink port TX and RX for a 1024 accelerator system . the RX of each port shall maintain the following state:

1. A single master key [32B]
2. A single alternate master key [32B]

3. A per source derived key [32B * 2*number of destinations]
4. A per source derived key epoch counter [4B * number of destinations]
5. Per stream, per destination accelerator IV counter [4B*3*number of destinations]

8.5.7.1.1 Master key management

As mentioned previously, an UALink TX has an active master key and an alternate master key. Similarly, an UALink RX has an active master key and an alternate master key. These keys are configured by secure SW/FW. UALink RX and TX are required to offer a secure interface to SW/FW for programming the two master keys. At any instant, one of the two programmed keys is the “active” key and the other is the “alternate” key. Determination of whether a particular key is the active key or the alternate key in UALink TX/RX is determined in the following way:

- When the two keys are programmed in for the very first time and secure transmission enabled, hardware selects the first key as the “active” key.
- Later, when a key swap flow is triggered by SW, HW promotes the alternative key to become the active key. When this happens, the original active key is marked as stale by hardware and software is required to program a new key replacing the stale key.

Hardware shall have mechanisms to ensure that a stale key is never selected as an active key. If the key swap flow is triggered and the alternate key is marked as stale, the hardware shall stop processing new transactions for that destination/from that source and report an error event to the security processor in the accelerator. The security processor upon receiving this error event, shall report to the CCM(TVM) that manages the accelerator with information about the Port number that reported the event.

- Hardware shall provide mechanisms to inform SW that a key has become stale. HW reports the stale key once the key swap flow is complete for all destinations (TX).
- Hardware shall provide mechanisms for SW to replace a stale key with a fresh key.

8.5.7.1.2 Illustration of security State requirements in TX and RX

To understand the security state requirements in more detail, the 4-accelerator system example shown in Figure 8-5 is helpful:

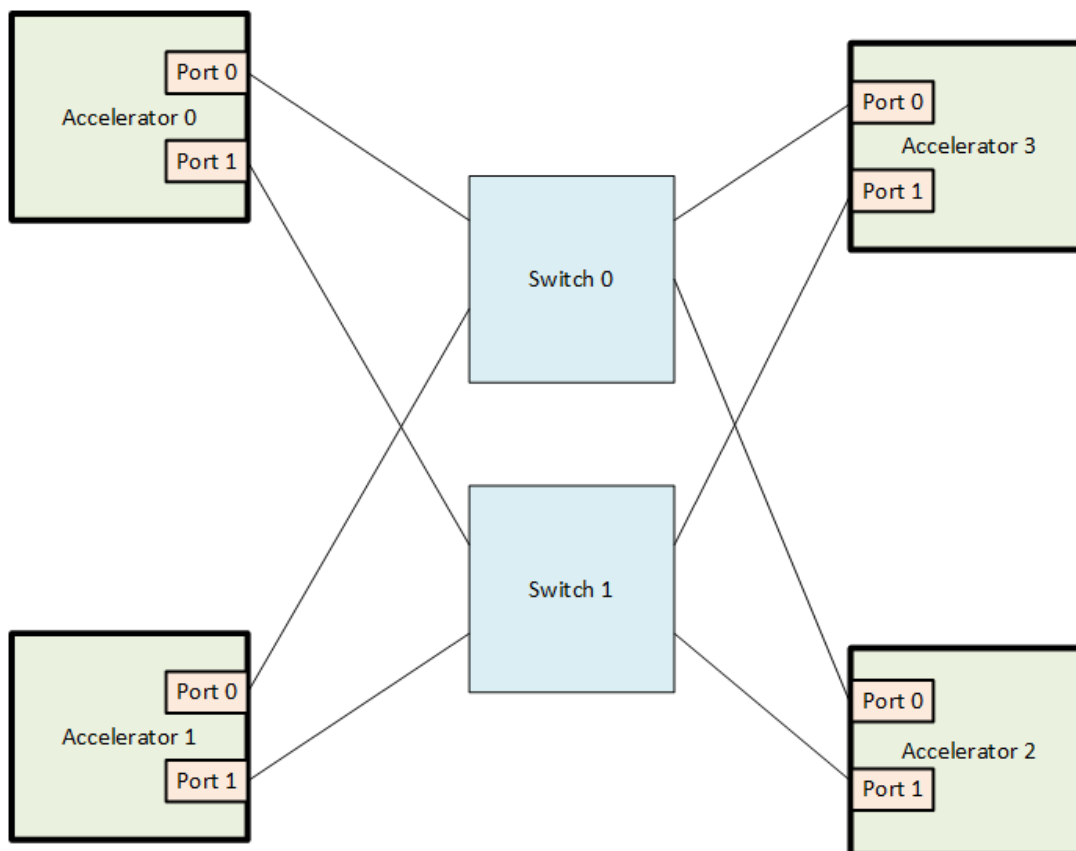


Figure 8-5 A 4 accelerator UALink system example

In the system shown in Figure 8-5, accelerator 0 Port 0 TX has the following state:

1. Active Master Key (32B)
2. Alternate Master Key (32B)
3. Invocation field for stream 0 to Accelerator 1, 2, and 3 (3x4B)
4. Invocation field for stream 1 to Accelerator 1, 2, and 3 (3x4B)
5. Invocation field for stream 2 to Accelerator 1, 2, and 3 (3x4B)
6. Epoch counter (3x4B)
7. Current derived key (3x32B)
8. Next derived key (3x32B)
9. Master key usage counter (4B)

And accelerator 0 Port 0 RX has the following state:

1. Active Master Key (32B)
2. Alternate Master Key f(32B)

3. Invocation field for stream 0 from Accelerator 1, 2, and 3 (3x4B)
4. Invocation field for stream 1 from Accelerator 1, 2, and 3 (3x4B)
5. Invocation field for stream 2 from Accelerator 1, 2, and 3 (3x4B)
6. Epoch counter for transactions from accelerator 1, 2, and 3 (3x4B)
7. Current Derived key for Accelerator 1, 2, and 3 (3x32B)
8. Next derived key for Accelerator 1, 2, and 3 (3x32B)

Similar state requirements exist for Accelerator 0 port 1, Accelerator 1 port 0, Accelerator 1 port 1, Accelerator 2 port 0, Accelerator 2 port 1, Accelerator 3 port 0 and Accelerator 3 port 1 in this example system.

8.5.7.2 Key Derivation Function

To ensure UALinkSec provides strong cryptographic protection, the derivation of key material used by the AES-GCM-256 encryption shall be performed using a strong and NIST approved Key Derivation Function (KDF).

The key derivation function “KDF-KMAC256”, or shortly “KDF” for UALinkSec, follows the definitions “KDF using KMAC” provided in Section 4.4 of NIST SP 800-108 Rev. 1 and “KMAC256” algorithm provided in Section 4 of NIST SP 800-185.

The KDF-KMAC256 (K, X) function shown in Figure 8-6 is defined as:

$\text{KMAC256}(K, X, L = 256, S = \text{“KDF”})$

The input parameters for KDF-KMAC256 are:

1. K: 256-bit key-derivation key
2. X: 88-bit context value
3. L: is fixed to 256 so that KDF outputs a 256-bit key value
4. S: is “KDF” in 8-bit ASCII characters as suggested in Section 4.4 of NIST SP 800-108 Rev. 1.

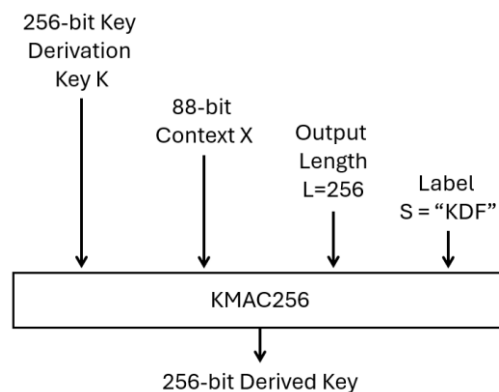


Figure 8-6 KDF-KMAC256 (K, X) function

The 256-bit key derivation key K is programmed into the accelerator as one of the active and alternate master key by software as described in the Master Key Management section.

8.5.7.2.1 Context value in KMAC256

The context value is constructed to provide uniqueness to the key reflecting TX and RX identities and the epoch counter that reflects the master key epoch. Table 8-7 shows the construction of the context value used in the KDF.

Table 8-7 88-bit Context value in KMAC256

Context [87:0]				
[87:72]	[71:56]	[55:40]	[39:32]	[31:0]
Padded Source Accelerator ID (16-bit)	Padded Destination Accelerator ID (16-bit)	Padded Source Port Number (16-bit)	Padded VC (8-bit)	Epoch Counter Value (32-bit)

8.5.7.3 Initialization Vector (IV) Construction for AES-GCM

UALink uses 96-bit Initialization Vector (IV) for AES-GCM. By following Section 8.2.1 of NIST SP 800-38D, the IV is constructed in a deterministic way so that it is the concatenation of 64-bit **fixed field** and the 32-bit **invocation field**

The IV construction is different for Accelerator to switch link protection which is described in Section 8.6.

The 96-bit IV is later concatenated with 32-bit block counter values and provided as 128-bit input to the AES engines in AES-GCM. In UALink accelerators, the block counter is reset to 0 for each new transaction.

8.5.7.3.1 IV construction for Accl-Accl link protection

For an accelerator-accelerator pair, the 64-bit fixed field contains the source & destination accelerator IDs, source accelerator port number, as well as the stream ID and VC ID between the accelerators. The 32-bit invocation field is defined as an integer counter incremented with each transaction from source accelerator to destination accelerator. The detailed IV fields at TX and RX are provided in Table 8-8.

Table 8-8 96-bit IV construction for Accl-Accl link protection with multi-path disabled

IV [95:0]						
Fixed Field						Invocation field
[95:66]	[65:56]	[55:46]	[45:36]	[35:34]	[33:32]	[31:0]
0	Source Accelerator ID (10-bit)	Destination Accelerator ID (10-bit)	Source Accelerator Port Number (10-bit)	Stream ID (2-bit)	VC ID (2-bit)	Counter (incremented with each transaction) (32-bit)

Note: At the receiver, the “destination accelerator ID” is receiver’s accelerator ID. Similarly, at the transmitter, “source accelerator ID” is transmitter accelerator ID and “source port ID” is transmitter’s port ID.

Note: Port number is unique for each port in an accelerator and is different from the UPLI port ID.

Note: In strongly ordered [SO] mode [see “Ordering requirements” section] , the VC ID field shall always be set to 0.

8.5.7.3.2 IV construction for Switch-Accl link protection

For an accelerator-switch pair, the 64-bit fixed field contains the accelerator and switch IDs and source accelerator (or switch) port number, as well as the stream ID and VC ID. The 32-bit invocation field is defined as an integer counter incremented with each transaction from source accelerator (or switch) to destination switch (resp. accelerator). The detailed IV fields for an accelerator-switch pair are provided in Table 8-9 (source accelerator – destination switch) and Table 8-10 (source switch – destination accelerator). In a switch-accelerator link, the IV includes 2-bits to identify the device pair type: 2b’01 for source accelerator – destination switch in Table 8-9, and 2b’10 for source switch – destination accelerator in Table 8-10.

Table 8-9 96-bit IV Construction when Accelerator is TX and Switch is RX

IV [95:0]						
Fixed Field						Invocation field
[95:58]	[57:56]	[55:46]	[45:36]	[35:34]	[33:32]	[31:0]
0	Source Device Type 1 = 2b’01 (2-bit)	Source Accelerator ID (10-bit)	Destination Switch ID (10-bit)	Stream ID (2-bit)	VC ID (2-bit)	Counter (incremented with each transaction) (32-bit)

Table 8-10 96-bit IV Construction when Switch is TX and Accelerator is RX

IV [95:0]						
Fixed Field						Invocation field
[95:58]	[57:56]	[55:46]	[45:36]	[35:34]	[33:32]	[31:0]

0	Source Device Type 2 = 2b'10 (2-bit)	Source Switch ID (10-bit)	Destination Accelerator ID (10-bit)	Stream ID (2-bit)	VC ID (2-bit)	Counter (incremented with each transaction) (32-bit)
---	---	---------------------------	-------------------------------------	-------------------	---------------	--

Whether virtual channel (VC) support is enabled or not, the 64-bit fixed field always includes the 2-bit VC ID as one of the fields. It is set to 0 when VC support is not enabled, or to the actual value of VC ID otherwise. Please refer to the Section 8.5.12.1 for the virtual channel support.

The integer counter in the invocation field is reset to 0 when a new AES-GCM key is derived. The maximum allowable number of AES-GCM invocations per derived key is restricted to $2^{32}-1$, thereby ensuring that the same IV value is not reused with the same key.

8.5.7.3.3 Key identification at Transmitter port and receiver port

For a 2.0 version compliant transmitter port, the derived key to use for transmitting a request or response is determined by the destination accelerator ID.

For a 2.0 version compliant receiver port, the derived key to use a received request is determined by the source accelerator ID.

For a 2.0 version compliant receiver port, the derived key to use for receiving a response is determined by the response tag.

8.5.7.4 TX implementation

As mentioned in Section 8.5, in an UALink port TX, there shall be an active master key and an alternate master key. The active key is fed into the KDF to generate a derived key. This key derivation is detailed in Section 8.5.7.2. The derived key is then used along with a destination and stream specific 96-bit IV defined earlier to encrypt transactions belonging to each stream. A derived key usage counter tracks the number of transactions encrypted using the current derived key. If the derived key usage counter becomes equal to a software configured “derived key expiry” threshold and that destination’s “Epoch” counter is incremented. The key derivation flow for TX is detailed in Section 8.5.7.4.1. The key derivation flow uses the “KeyDeriveMSGReqChannel”, “KeyDeriveMSGRdRspChannel” and “KeyDeriveMSGWrRspChannel” Requests.

If the master key usage counter values reach a software configured “Master key expiry” threshold and a valid active key is not available, then TX hardware shall stop encryption of new transactions and shall assert an error event to the security processor of the accelerator. The IV counter shall be 0 for the very first transaction that a TX stream encrypts using a new derived key.

8.5.7.4.1 TX Stream-Key derivation flow

The key derivation flow is done on a per TX, RX pair basis and is managed by the TX port. The key derivation flow can be triggered by SW during initialization by writing to a configuration register bit or by HW (whenever the derived key expiry threshold is hit) – the key derivation flow will take

current active master key as input to KDF and derive a new key. If a new derived key is required and the active key is not valid, then an error is signaled and all further processing of traffic to the destination accelerator is stopped. The key derivation flow involves co-ordination between TX and remote RX ports. This TX-RX handshake is illustrated in Figure 8-10

The Stream-key derivation flow in TX is illustrated in Figure 8-7 below

Evaluation Copy

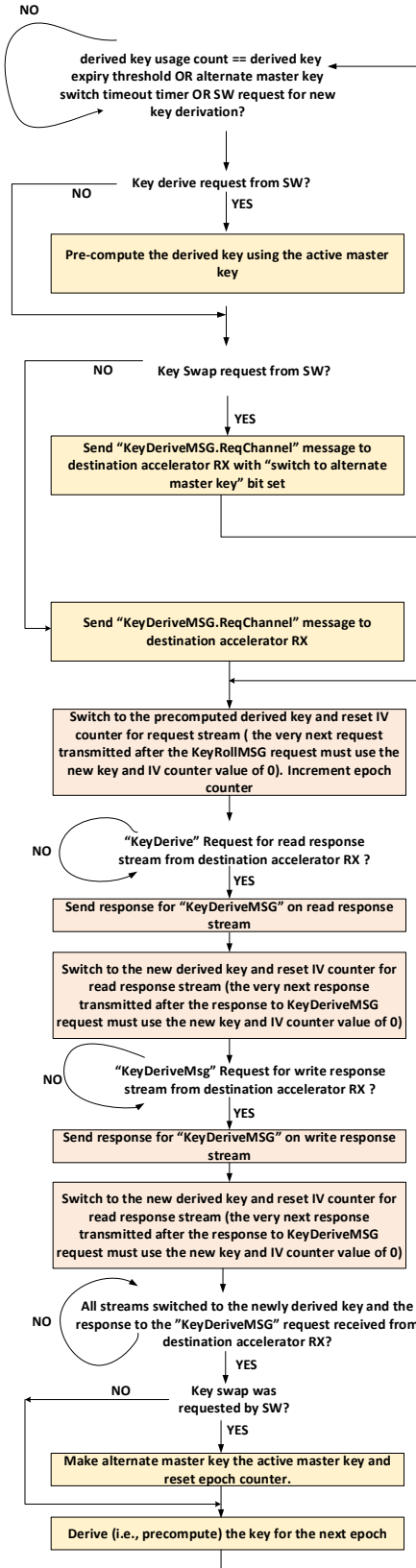


Figure 8-7 Stream Key derivation flow in UALink port TX
Security

Note: when SW is NOT triggering the key derivation flow, it is assumed that HW has already pre-computed the derived key as part of the overall key derivation as shown in the figure above.

Note: As per NIST recommendations, the maximum number of usages of a specific derived key value is limited to $2^{32}-1$.

Note: Programmers should ensure that the threshold is not programmed with too low a value – otherwise, key rolls and key derivations might be unsustainably frequent.

Note: programmers must ensure that the threshold value provides sufficient time to complete the key derivation handshake before the hard limit of $2^{32}-1$ transactions is hit.

Note: if a TX or RX times out on the KeyDeriveMSG requests it sends as part of the key derivation flow for a particular destination accelerator, then it will report an error to the security processor and stop processing of traffic to that destination accelerator and discard such traffic.

Note: For ease of implementation, it is expected that a Transaction tag value is reserved for use by the security engine for key derivation/key swap requests.

8.5.7.4.2 TX Master key swap flow

The master key swap flow is triggered by SW. The key swap flow is done by TX for each destination accelerator.

When SW triggers a key swap flow, TX will take the following actions:

1. TX will set a key swap message timeout timer.
2. If TX hits key derivation threshold for a particular destination, TX will set the “switch to alternate master key” bit in the KeyDeriveMSG.RegChannel that it sends to that destination. TX will track which all destinations have been sent the “switch to alternate master key”.
3. Once the key swap message timer expires, TX will check if there are any destinations for whom the “switch to alternate master key” was not sent. TX shall trigger a key derivation flow for all such destinations such that all destinations gets the “switch to alternate master key” indication.
4. Once a particular destination has been sent the “switch to alternate master key” indication, TX will switch to alternate master key as the input for all further master key derivations for that destination.
5. Once all destinations have been sent the “switch to alternate master key” indication, TX will set an alternate master key switch timeout timer. Once this timeout timer is set, TX will track which destinations have gone through a key derivation flow.
 - Note that SW is free to program different values for key swap message timeout timer and alternate master key switch timeout timer.
6. Once the master key switch timeout time expires, TX shall trigger a key derivation flow for all destinations for which a key derivation flow has not occurred. This ensures that all destinations have switched to the alternate master key.
7. Once all destinations have started using a derived key that used the alternate master key as input, the TX will inform the security processor in the accelerator that the key swap flow is complete. The security processor in turn will inform the node CCM SW.

This key swap flow is illustrated in Figure 8-8.

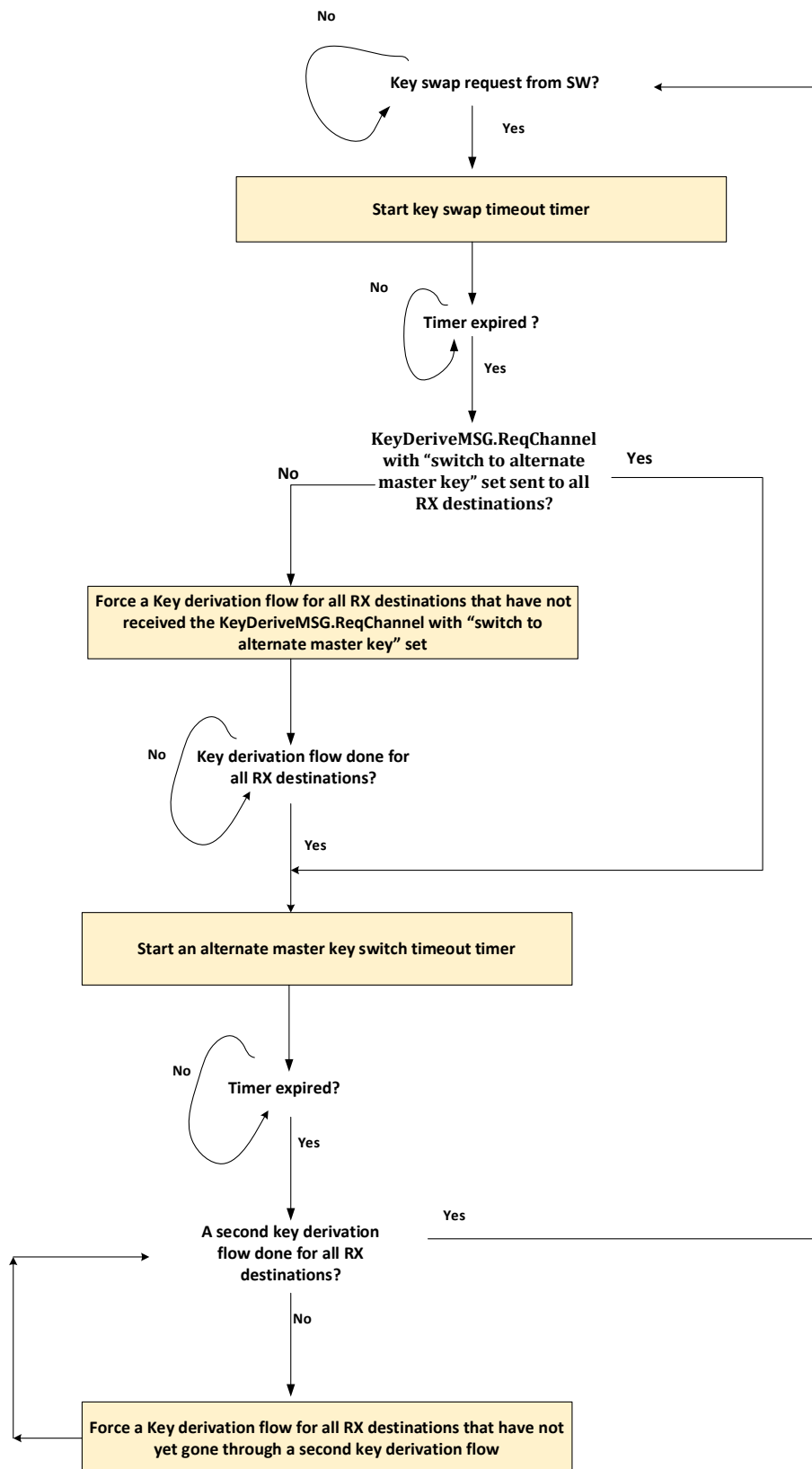


Figure 8-8 Master key swap flow in TX

Security

8.5.7.5 RX implementation

As mentioned in Section 8.5.7.1, in an UALink port RX, there will be an active master key and an alternate master key. The active key is fed into a NIST approved Key Derivation Function (KDF) defined in Section 8.5.7.2 to generate a derived key for each source accelerator. This is detailed in Section 8.5.7.2.1. This derived source specific key is then used along with a {per source accelerator, per stream} IV (format defined in section 8.5.7.3) to decrypt transactions received by the RX. New key derivation in the RX for each source accelerator is driven by the source accelerator TX. The key derivation flow is detailed in Section 8.5.7.2. The key derivation flow involves three “KeyDeriveMSG” request – response pairs – one for each stream from the source accelerator. When the RX receives a KeyDeriveMSG.ReqChannel request on the request stream, it shall switch to doing decryption using a new key derived from the active master key on the very first request received after the “KeyDeriveMSG.ReqChannel” request. For the read response channel, the RX shall switch to doing decryption using a new key derived from the active master key from the very first response received after the response for the “KeyDeriveMSG.RdRspChannel” request. For the write response channel, the RX shall switch to doing decryption using a new key derived from the active master key from the very first response received after the response for the “KeyDeriveMSG.WrRspChannel” request. These sequence of actions are illustrated in Figure 8-10. To achieve this instantaneous switch for a given source accelerator, RX is required to pre-compute and keep the derived key ready for the next epoch as detailed in Figure 8-9. The master key swap in RX for a particular source accelerator is triggered by the source accelerator TX. Since each source accelerator TX can request for a key swap independent of other source accelerators, at any given instant, the master key to be used in key derivation for some source accelerators could be the active key and the master key for the rest of the source accelerators could be alternate master key. The RX shall keep track of the master key to be used for each source accelerator. The IV counter shall be 0 for the very first transaction decrypted by RX using a new derived key.

8.5.7.5.1 RX Stream Key derivation flow

In the RX, if a new stream key needs to be derived and the active key is not valid, then an error is signaled and all further processing of traffic from the source accelerator is stopped.

The key derivation flow is illustrated in Figure 8-9 below:

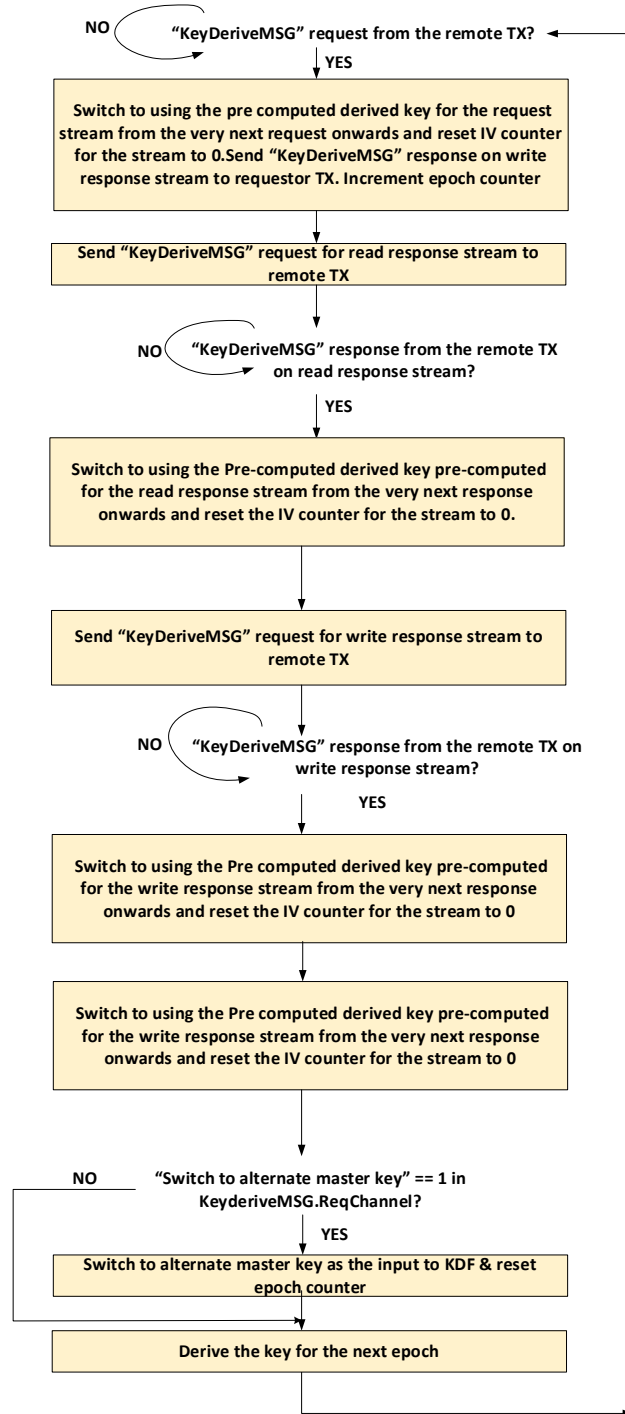


Figure 8-9 Master key swap flow in RX

The interactions between a source accelerator and a destination accelerator for the key swap flow is illustrated in a time sequence fashion in Figure 8-10:

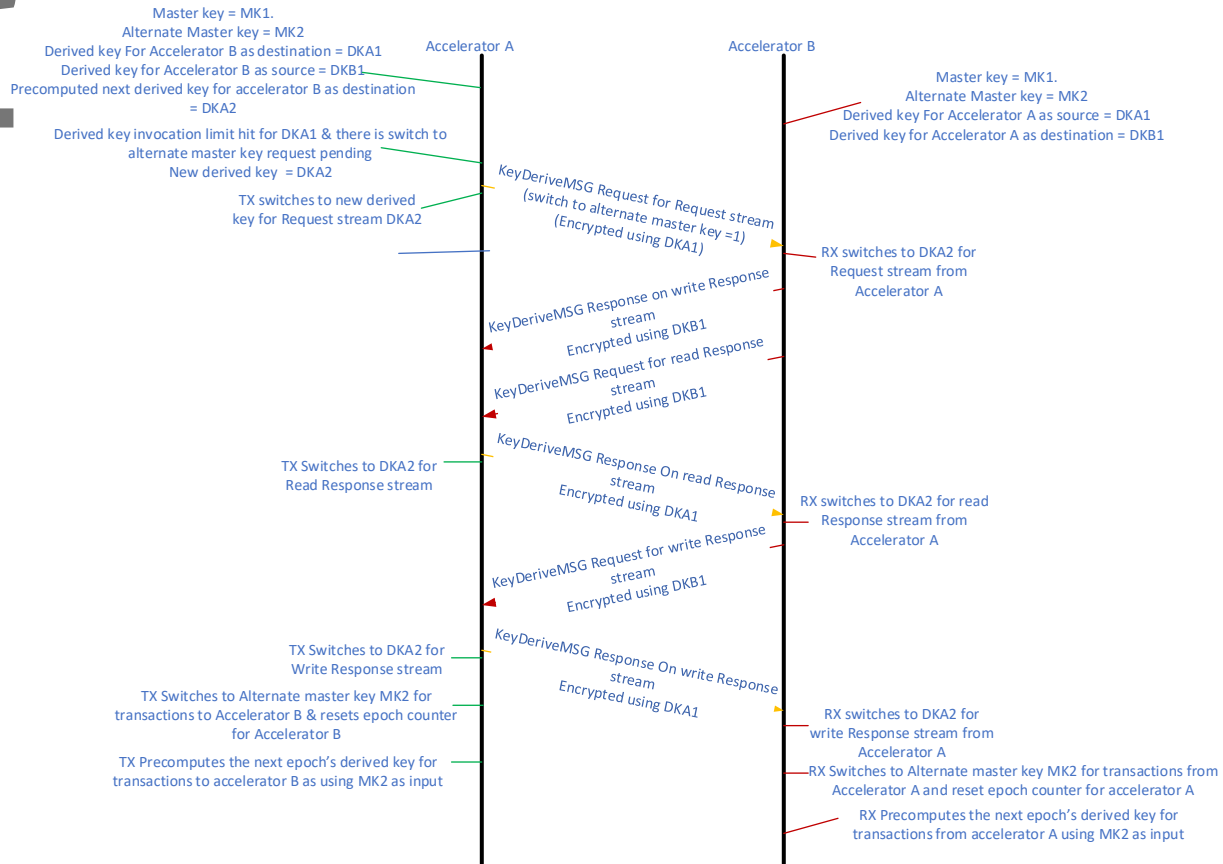


Figure 8-10 Source accelerator - Destination accelerator interactions for key derivation & swap flow

8.5.7.5.2 Allowing backpressure

RX is permitted to back pressure the remote TX if it is not able to complete key derivation in time for switching to the new derived key. In practice, this is expected to manifest as TX running out of credits since the crypto engine will not process incoming transactions till it has completed the key derivation. Note that this has performance implications and hence should be avoided unless necessary.

8.5.7.6 SW forced key swap

UALink Rx and Tx shall offer a secure interface to SW/FW to force key swap of a master key. To prevent the race condition when SW_forced_key swap is triggered just after an HW triggered key swap and before a new alternate Key has been installed, the UALink shall have a programmable watchdog counter to count the clock cycles after the last HW triggered key swap to allow the SW/FW enough time to program a new alternate Key.

8.5.7.7 Invalidating Master Key and Error recovery

There may be a need to invalidate the installed master keys either due to authentication failure on a payload received or other reasons that require clearing the installed master keys.

UALink Rx and Tx shall offer a secure interface to SW/FW to invalidate installed master keys (current key and alternate key if installed). Note: Traffic associated to with the master key to be invalidated must be quiesced before invalidation.

After an authentication failure, to re-enable a connection between AcceleratorX/PortA and AcceleratorY/PortB the SW/FW shall follow the following procedure:

- 1) Quiesce the traffic between AcceleratorX/PortA and AcceleratorY/PortB
- 2) Invalidate installed master keys on AcceleratorX for Tx and Rx connections to AcceleratorY/PortB
- 3) Invalidate installed master keys on AcceleratorY for Tx and Rx connections to AcceleratorX/PortA
- 4) Install Master Key on AcceleratorX for Tx and Rx connections to AcceleratorY/PortB
- 5) Install Master Key on AcceleratorY for Tx and Rx connections to AcceleratorX/PortA

Traffic between AcceleratorX/PortA and AcceleratorY/PortB can resume after the derived keys are generated.

8.5.7.8 UALink Security resources mapping

While a UALink system can have up to 1024 accelerators in a Pod, an individual UALink port may connect only to a smaller subset. Each UALink port has a set of "UALink security resource" instances. On devices with support for reduced storage, an "UALink_security resource" instance comprises of all the storage required for configuring and maintaining a secure connection to a target accelerator port:

This includes:

- Tx/Rx
- 1 Current Derived Key
- 1 Next Derived Key
- 3 IV values (1 for each stream)
- 1 Epoch counter

Note: Current Master Key, the Alternate Master Key and the master key usage counter are common resources and state to the whole UALink device and are not replicated for each UALink_security resource.

For each UALink port, the Virtual Pod Controller shall be responsible for assigning and configuring the required UALink security resources to the set of accelerators to which the port is supposed to be connected to. A HW-SW interface shall be provided by the UALink port HW to do the assignment and configuration. The number of UALink security resources for each port are required to be discoverable by security SW.

The UALink Security resources assigned to Rx (and Tx) shall have a consecutive UALink Security resources index.

8.5.7.9 AES-GCM Message and Additional Authentication Data

This section specifies the mapping order of the UPLI channels fields for AES-GCM Additional Authentication Data (AAD) and the Message (MSG) to be encrypted on TX or Decrypted on the RX.

The TAG is computed by using the AAD and the ciphertext after MSG Encryption.

The AAD is always a multiple of 32 bits. When the width of concatenated fields is not a multiple of 32 bits, it will be 0s-padded.

Table 8-11 through Table 8-15 show the fields contributing to AAD and MSG for all UPLI channels. Depending on the request type, the first beat of the originator data will be processed in parallel with the request.

Table 8-11 Request Channel Fields for AAD and MSG - Non Write or UPLI Write Message Request

	SUM_BITS	ReqASI	ReqSrcPhysAccID	ReqDstPhysAccID	ReqTag	ReqNumBeats	ReqAddr	ReqCmd	ReqLen	ReqAttr	ReqVC	ReqMeta data
AAD	106	2	10	10	11	2	[56:18], [7:6]	6	6	8	2	8
MSG	14						[17:2]					

Table 8-12 Originator Data Channel Fields for AAD and MSG (partial-word write or UPLI Write Message Request)

	SUM_BITS	OrigData	OrigDataByteEn	OrigDataOffset	OrigDataLast	OrigDataVC
AAD	5			2	1	2
MSG	576	512	64			

Table 8-13 Originator Data Channel Fields for AAD and MSG (Full Write)

	SUM_BITS	OrigData	OrigDataOffset	OrigDataLast	OrigDataVC
AAD	5		2	1	2
MSG	512	512			

Table 8-14 Read Response channel fields for AAD and MSG

	SUM_BITS	RdRspSrcPhysAccID	RdRspDstPhysAccID	RdRspTag	RdRspNumBeats	RdRspData	RdRspStatus	RdRspOffset	RdRspVC	RdRspLast
AAD	32	0	10	11	2		4	2	2	1
MSG	512					512				

Table 8-15 Write Response channel fields for AAD and MSG

	SUM_BITS	WrRspSrcPhysAccID	WrRspDstPhysAccID	WrRspTag	WrRspStatus	WrRspVC
AAD	32	0	10	11	4	2
MSG	0			0		

Table 8-16 describes the mapping of AAD bits for request.
Address field is double-word aligned and Addr[1:0] are always 0. These 2 bits are not encrypted and not authenticated. The AAD size is 128 bits with padded zeroes.

Table 8-16 Mapping Request Channel AAD bits

AAD for Req = 128-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes	0	ReqMetadata[7:0]							
	1	ReqAttr[7:0]							
	2	ReqCmd[1:0]		ReqLen[5:0]					
	3	ReqAddr[19:18]		ReqAddr[7:6]		ReqCmd[5:2]			
	4	ReqAddr[27:20]							
	5	ReqAddr[35:28]							
	6	ReqAddr[43:36]							
	7	ReqAddr[51:44]							
	8	ReqDst...ID[0]	ReqNBeats[1:0]		RAddr[56:52]				
	9		ReqDstPhysAccID[8:1]						
	10	ReqSrcPhysAccID[6:0]							ReqDst...ID[9]
	11	ReqTag[0]	ReqVC[1:0]		ReqASI[1:0]		ReqSrcPhysAccID[9:7]		
	12	ReqTag[8:1]							
	13	0`s Padding						ReqTag[10:9]	
	14	0`s Padding							
	15	0`s Padding							

For each UPLI transaction, the encrypted counter generated with IV-128-bit block counter (IV_Blk_Cnt) value 1 (IV|| 0³¹||1) is encrypting (XOR-ing) the 128-bit MAC.

Table 8-17 describes the mapping of MSG bits for non-write or UPLI non-write message request. The MSG size is 32 bits. If PCRC is enabled, the remainder 32-bit of the encrypted counters will be Encrypting (XOR-ing) the PCRC.

Table 8-17 Mapping Request Channel MSG bits for non write and non UPLI Write Message

MSG for Req (no write req.) =32-bit									
IV_Blk_Cnt = 2		bits							
		7	6	5	4	3	2	1	0
Bytes	0	ReqAddr[11:8]				ReqAddr[5:2]			
	1	0`s Padding		ReqAddr[17:12]					
	2	0`s Padding							
	3	0`s Padding							

Table 8-18 describes the mapping of AAD bits for Write Request or UPLI Write Message, with 1st beat of the Originator Data Channel. The AAD size is 128 bits containing Request Channel's fields and Originator Data Channel's fields.

Table 8-18 Mapping Request and Originator Data channels AAD bits for Wr Req or UPLI Write Message

AAD for Wr Req w. Byte Enable of the 1st beat = 128-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes		ReqMetadata[7:0]							
	1	ReqAttr[7:0]							
	2	ReqCmd[1:0]		ReqLen[5:0]					
	3	ReqAddr[19:18]		ReqAddr[7:6]		ReqCmd[5:2]			
	4	ReqAddr[27:20]							
	5	ReqAddr[35:28]							
	6	ReqAddr[43:36]							
	7	ReqAddr[51:44]							
	8	ReqDst...ID[0]	ReqNBeats[1:0]		RAddr[56:52]				
	9		ReqDstPhysAccID[8:1]						
	10	ReqSrcPhysAccID[6:0]							ReqDst...ID[9]
	11	ReqTag[0]	ReqVC[1:0]		ReqASI[1:0]		ReqSrcPhysAccID[9:7]		
		ReqTag[8:1]							
	13	0`s Padding					ReqTag[10:9]		
	14	0`s Padding			OrigDataVC[1:0]		OrigDataOffset[1:0]		Last
15	0`s Padding								

The total MSG size is 608 bits for 1st beat of Write Request w. Byte Enable or UPLI Write Message of the 1st beat or 544-bits for 1st beat of Full-word Wr Req (w/o Byte Enable). The 512-bit Originator data consume 4x 128-bit encrypted counters with IV-128-bit block counter value from 1 to 4. The last part of the MSG is 96-bit concatenating the Write Request or UPLI Write Message with the Byte Enable field (Table 8-19) or 32-bit for Full word write requests. (Table 8-20). It consumes 96-bit or 32-bits for Full word write requests of the encrypted counter with IV-128-bit block counter value 5.

If PCRC is enabled, after encrypting the last beat of the write request, the remainder 32-bit of the encrypted counters will be Encrypting (XOR-ing) the PCRC.

Table 8-19 Mapping MSG bits for Write Request with Byte Enable or UPLI Write Message of the 1st beat

MSG for Full-word Wr Req (w. Byte Enable) of the 1st beat = 608-bit									
IV_Blk_Cnt = 2		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[127-0]							
IV_Blk_Cnt = 3		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[255-127]							
IV_Blk_Cnt = 4		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[383-256]							
IV_Blk_Cnt = 5		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[511-384]							
IV_Blk_Cnt = 6		bits							
		7	6	5	4	3	2	1	0
Bytes	0	ReqAddr[11:8]				ReqAddr[5:2]			
	1	0`s Padding		ReqAddr[17:12]					
	2	0`s Padding							
	3	0`s Padding							
	4	ByteEnable[7:0]							
	5	ByteEnable[15:8]							
	6	ByteEnable[23:16]							
	7	ByteEnable[31:24]							
	8	ByteEnable[39:32]							
	9	ByteEnable[47:40]							
	10	ByteEnable[55:48]							
	11	ByteEnable[63:56]							

Table 8-20 Mapping MSG bits for Full-word Wr Req (w/o Byte Enable) of the 1st beat

MSG for Full-word Wr Req (w/o Byte Enable) of the 1st beat = 544-bit									
IV_Blkc_Cnt = 2		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[127-0]							
IV_Blkc_Cnt = 3		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[255-127]							
IV_Blkc_Cnt = 4		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[383-256]							
IV_Blkc_Cnt = 5		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		OrigData[511-384]							
IV_Blkc_Cnt = 6		bits							
		7	6	5	4	3	2	1	0
Bytes	0	ReqAddr[11:8]				ReqAddr[5:2]			
	1	0`s Padding		ReqAddr[17:12]					
	2	0`s Padding							
	3	0`s Padding							

The table below describes the mapping AAD bits of **non-1st beat of the originator data**.
The AAD size is 32-bits.

Table 8-21 Mapping AAD bits for Originator data non 1st beat

AAD Orig Data other beats (not 1st) = 32-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes	0	0`s Padding			OrigDataVC[1:0]		OrigDataOffset[1:0]		Last
	1	0`s Padding							
	2	0`s Padding							
	3	0`s Padding							

Ultra Accelerator Link Consortium Inc. (UALink) - UALink Common Specification
Rev 2.0

The table below describes the mapping MSG bits of non-1st beat of the originator data non-full write word. The MSG size is 576-bits

The 2nd and rest of the originator data beats will consume each 4x128-bit encrypted counters for the 512b-bit Originator data and 64-bit of the encrypted counter for write request requiring 64-bit Byte_enable information. Each consecutive 128-bit encrypted counter is calculated from incremented IV-128-bit block counter value and they are consumed back-to-back for each beat: i.e. non consumed encrypted counters for one beat are carried over to encrypt the next beat.

Table 8-22 shows the Mapping of the MSG bits for Full word write request with Byte Enable.

Table 8-23 shows the Mapping of the MSG bits for Full word write requests without Byte Enable. If PCRC is enabled, after encrypting the last beat of the write request, the remainder 32-bit of the encrypted counters will be Encrypting (XOR-ing) the PCRC.

Table 8-22 Mapping MSG bits for Full-word Wr Req (w. Byte Enable) non 1st beat

MSG Orig Data other beats (not 1st, w. Byte Enable) = 576-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]	OrigData[127-0]								
		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]	OrigData[255-127]								
		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]	OrigData[383-256]								
		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]	OrigData[511-384]								
IV_Blk_Cnt = 6		bits							
		7	6	5	4	3	2	1	0
Bytes	0	ByteEnable[7:0]							
	1	ByteEnable[15:8]							
	2	ByteEnable[23:16]							
	3	ByteEnable[31:24]							
	4	ByteEnable[39:32]							
	5	ByteEnable[47:40]							
	6	ByteEnable[55:48]							
	7	ByteEnable[63:56]							

Table 8-23 describes the mapping MSG bits of non-1st beat of the originator data full write word. The MSG size is 512-bits will consume 4 128-bits encrypted counters.

Table 8-23 Mapping MSG bits for Full-word Wr Req (w/o Byte Enable) non 1st beat

MSG Orig Data other beats (not 1st, w/o Byte Enable)									
= 512-bit									
	bits								
	7	6	5	4	3	2	1	0	
Bytes [0-15]	OrigData[127-0]								
	bits								
	7	6	5	4	3	2	1	0	
Bytes [0-15]	OrigData[255-127]								
	bits								
	7	6	5	4	3	2	1	0	
Bytes [0-15]	OrigData[383-256]								
	bits								
	7	6	5	4	3	2	1	0	
Bytes [0-15]	OrigData[511-384]								

Table 8-24 describes the mapping AAD bits for read response. The AAD size is 32-bits.

Table 8-24 Mapping Read Response Channel AAD bits

AAD Read Response = 32b-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes	0	Beats[0]	RdRspStatus[3:0]				RdrspOffset[1:0]		Last
	1	RdRspDstPhysAccID[6:0]							Beats[1]
	2	RdRspTag[4:0]					RdRspDstPhysAccID[9:7]		
	3	RdRspVC[1:0]		RdRspTag[10:5]					

Table 8-25 describes the mapping MSG bits for Read Response for single or multi beat. Each beat of the Read Response has MSG size of 512-bit. and it consumes the 4x128-bit encrypted counters. Each consecutive 128-bit encrypted counter is calculated from incremented IV-128-bit block counter value and they are consumed back-to- back for each beat.

If PCRC is enabled, after encrypting the last beat of the Read Response, 32-bit of the encrypted counters will be Encrypting (XOR-ing) the PCRC.

Table 8-25 Mapping MSG bits for Read Response

MSG Read Response = 512-bit									
IV_Blkc_Cnt = 2+4*Beats_idx		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		RdRspData[127-0]							
IV_Blkc_Cnt = 3+4*Beats_idx		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		RdRspData[255-127]							
IV_Blkc_Cnt = 4+4*Beats_idx		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		RdRspData[383-256]							
IV_Blkc_Cnt = 5+4*Beats_idx		bits							
		7	6	5	4	3	2	1	0
Bytes [0-15]		RdRspData[511-384]							

Table 8-26 describes the mapping AAD bits for **write response**. The AAD size is 32-bits.

Table 8-26 Mapping Read Response Channel AAD bits

AAD Write response = 32-bit									
		bits							
		7	6	5	4	3	2	1	0
Bytes	0	WrRspDstPhysAccID[3:0]				WrRspStatus[3:0]			
	1	WrRspTag[1:0]		WrRspDstPhysAccID[9:4]					
	2	WrRspTag[9:2]							
	3	0's Padding				WrRspVC[1:0]		WrRspTag[10]	

Write response does not have any field to be encrypted so MSG size is zero.

8.5.8 Initializing encrypted and authenticated transmission and reception

For UALink 2.0, It is assumed that encryption (and optionally, authentication) is enabled on a per virtual Pod basis. This means that all traffic transmitted or received by a port belonging to that pod

will be encrypted and optionally authenticated. This also means that all ports on an accelerator are required to have the same setting for encryption enable and authentication enable. In addition, it is assumed that no valid transactions are sent out by an UALink TX prior to configuring and enabling encryption and authentication. This ensures that the very first transaction that an RX receives is an encrypted (and optionally authenticated) transaction. With these assumptions, the trusted SW (i.e., CCM within a TVM) is expected to do the following for initialization of encrypted (and optionally authenticated) transmission and reception:

1. Generate active and alternate master keys for the virtual POD.
2. Program the active and alternate master keys into each TX and RX port in the virtual POD.
3. Program the derived key expiry threshold in each TX and RX. The same threshold is programmed into all TX and RX.
4. Program the master key usage limit in each TX. The same threshold is programmed into all TX.
5. Program the set of destination accelerator IDs in each TX port. This is used for IV formation and as part of the input to KDF.
6. Program the logical source port number in each TX port. This is used for IV formation and as part of the input to KDF. TX is required to know its own accelerator ID and port number
7. Program the set of {source accelerator ID, source port number} combinations in each RX port. This is used for IV formation and as part of the input to KDF. RX is required to know its own accelerator ID and port number

An example flow for initialization is as follows:

- For each UALink TX in the pod:
 - The CCM in the lead node of the virtual POD instructs the CCM of the sub-ordinate node which has the accelerator containing the UALink TX to do the security related programming. The lead CCM also provides the master keys (active and alternate), the master key usage limit and derived key expiry threshold
 - The sub-ordinate node CCM that manages the accelerator containing the UALink TX does the following:
 - The CCM SW configures the active master key value and alternate master key value in each UALink TX in the accelerator. CCM configures the derived key expiry threshold and the master key usage limit in each UALink TX in the accelerator.
 - Once these steps are completed, the sub-ordinate node CCM that manages the accelerator containing the UALink TX informs the lead CCM that programming is complete.
- For each UALink RX in the pod:
 - The CCM in the lead node of the virtual POD instructs the CCM of the sub-ordinate node which has the accelerator containing the UALink RX to do the security related programming. The lead CCM also provides the master keys (active and alternate), the master key usage limit and derived key expiry threshold

- The sub-ordinate node CCM that manages the accelerator containing the UALink TX does the following:
 - The CCM SW configures the active master key value and alternate master key value registers in each UALink RX in the accelerator .

Note that transmission shall not start until key derivation is complete.

- Once these steps are completed, the sub-ordinate node CCM that manages the accelerator containing the UALink RX informs the lead CCM that programming is complete.
- Once these steps are complete for all TX and RX pairs in the Virtual Pod, the lead CCM instructs the sub-ordinate CCMs in the Virtual Pod of all accelerators in the pod to enable encrypted and (optionally) authenticated transaction reception in all UALink ports.
- Once reception is enabled in all accelerators, the lead CCM instructs the sub-ordinate CCMs of all accelerators in the pod to enable encrypted and (optionally) authenticated transaction transmission in all UALink ports.
- The very first valid transaction that a RX receives will be encrypted and (optionally) authenticated transaction.

8.5.9 Refreshing an expired key

The node CCM and/or lead node CCM is expected to trigger a key refresh. For this, CCM can potentially trigger key refresh periodically or it can monitor the bandwidth usage of various ports and then trigger the key refresh if the Bandwidth usage crosses a specific threshold.

The following is an example flow for refreshing an expired key.

- The lead node CCM key refresh timer expires. The lead node CCM generates a new key.
- The lead node CCM will instruct all the node CCMs to trigger a key swap.
- The node CCMs will trigger key swaps in all UALink TXes and wait for the completion of the key swap to be reported from all UALink TXes in the node
- Once all node CCMs report to the lead node CCM that the key swap has been completed, the lead node CCM supplies the new key to all the node CCMs to replace the key that has become stale.
- The node CCMs will program the new key into all the UALink TXes and RXes to replace the stale key.
- Once the node CCMs report to the Lead node CCM that the new key programming is complete, lead node CCM will restart the key refresh timer.

8.5.10 Safeguarding UALink configuration to ensure confidentiality and integrity

From a security perspective, registers in the UALink port/station can be classified into two categories

- A. Registers that only trusted SW/FW shall configure. E.g., the Active master key register. To ensure confidentiality and integrity, such registers shall have restricted access such that only SW/FW that are trusted can write to them.

- B. Registers (and configuration storage arrays/lookup tables) that should be configured by untrusted SW/FW (e.g., address decoding related registers), but can cause loss of confidentiality and/or integrity and/or execution corruption if maliciously modified. These registers shall have a “lock” mechanism which ensures that only secure software/firmware has write access once the register/array is in locked state. This ensures that untrusted SW/FW can configure these registers and then trusted SW/FW can lock the register (preventing further updates by untrusted SW/FW), verify the configuration’s validity and then enable secure transmission/reception. The “lock” shall be set by the DSM by writing to one or more registers that only DSM can access.

It is the implementer’s responsibility to conduct a security audit of all configuration registers (and configuration storage arrays/look up tables) and decide which registers need to be in category A and which needs to be in category B.

8.5.11 Integrity failure handling

As soon as an integrity check failure is detected for a transaction within a stream, the receiver shall drop the failing transaction and stop processing new transactions from that source (all streams). In addition, the receiver will report a security failure to the security processor within the accelerator via hardware/firmware means that cannot be manipulated by untrusted HW/SW/FW. The security processor within the accelerator is responsible for informing the TVM (essentially CCM) of the failure.

There is no cross-port impact of authentication failures. For example, if port A on an accelerator receives a packet and it fails the authentication check, it does not impact the processing of incoming or outgoing traffic on other ports of the same accelerator. However, it is expected that the software will bring everything to stop since failure on one part of the Virtual Pod will likely have a cascading effect on the entire workload requiring a reset of the Virtual Pod. Software’s handling of the failure is outside the scope of this specification.

8.5.12 Switch requirements

Switches need to support the following:

1. Detecting that the control flit has an associated tag half flit at ingress ports.
2. Unpacking of tag half flits and associating each control half flit request/response with its tag
3. Packing the tag of a transaction into a tag half flit at egress.
4. Maintaining the order for each stream between a source accelerator port and destination accelerator port. Note that the traffic between a SrcID, DstID pair would include KeyRollMSG requests and corresponding responses.

8.5.12.1 Virtual channel support with security enabled

UALinkSec 2.0 specification supports link protection when virtual channel (VC) support is enabled in two modes: Strongly ordered [SO] and Strongly ordered lite [SOL].

In Strongly ordered [SO] mode, the traffic for each stream must be kept in order from a given TX accelerator port to a given RX accelerator port regardless of VC on which the transaction is sent.

In Strongly ordered Lite [SOL] mode, the traffic must kept in order for a given {stream, VC} pair – i.e. there is no ordering requirement between streams belonging to different VCs. i.e.,

8.5.12.1.1 State replication requirements for SOL mode

In a UALink network that supports SOL mode, each VC is expected to have the request stream, the write response stream and the read response stream. While within a VC the traffic per stream is required to be in-order, traffic belonging to different VCs are completely unordered with respect to each other. This requires each VC to maintain its own stream related crypto resources.

The crypto resources maintained per VC include:

- Derived key (one per destination)
- Epoch counter (one per destination)
- Next derived key (one per destination)
- IV (one per {destination, stream} pair)

Derived key generation shall include VC ID as one of the inputs to the KDF as described in 1.5.7.2.

IV construction will also include VC ID as one of the input to the KDF as described in 1.5.7.3.

The key derivation and key swap flows shall be done by each TX for all {VC, destination} pairs.

8.5.13 Datapath protection

This section specifies requirement for data path protection of the AES-GCM engines. This feature is optional and can be enabled during UALink security configuration when Authentication is enabled.

Data path protection provides robustness against hard and soft faults internal to the encryption and decryption engines. CRC-32 is calculated on the message Plaintext content (PCRC). The encrypted PCRC (ECRC) integrates into the standard Auth_TAG check mechanism and does not consume any incremental link bandwidth. It can be implemented without adding significant incremental latency.

A polynomial with the coefficients 1EDC 6F41h shall be used for PCRC calculation. PCRC calculation shall begin with the polynomial initialized to FFFF_FFFFh (CRC seed). The PCRC shall be calculated across all the bytes of plaintext messages that are part of the given Auth_TAG. PCRC calculation shall begin with bit0 byte0 of message plaintext content and sequentially include bits 0 - 7 for each byte of the message contents that are mapped to the plaintext. After accumulating the 32-bit value across the message contents, the PCRC value shall be finalized by taking 1's complement of the bits of accumulated value to obtain PCRC[31:0].

On the transmitter side, during the encryption flow, the PCRC value shall be appended to the end of the aggregated flit plaintext content, encrypted to form the Encrypted CRC (ECRC), and then included in the Auth_TAG calculation as shown in Figure 8-11.

Encryption

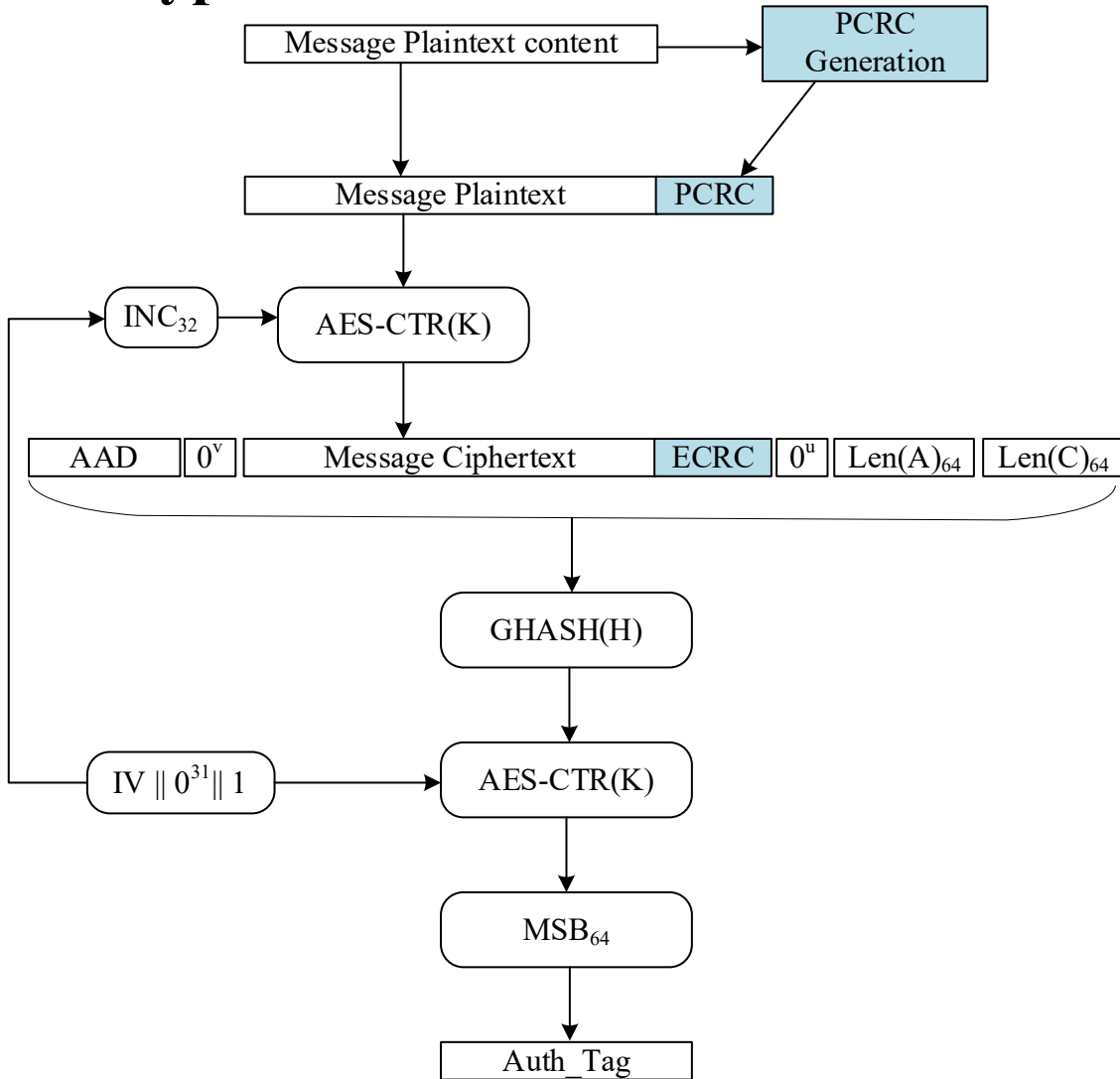


Figure 8-11 PCRC generation and integration during encryption flow on the transmitter side

On the receiver side, as shown in Figure 8-12, the PCRC value shall be recalculated based on the received, decrypted ciphertext message. When the last portion of the message of the current Auth_TAG has been processed, the accumulated PCRC value shall be XORed (encrypted) with the AES keystream bits that immediately follow the values that are used for decrypting the received cipher message. This encrypted PCRC value shall be appended to the end of the received ciphertext for the purposes of Auth_TAG computation.

Decryption

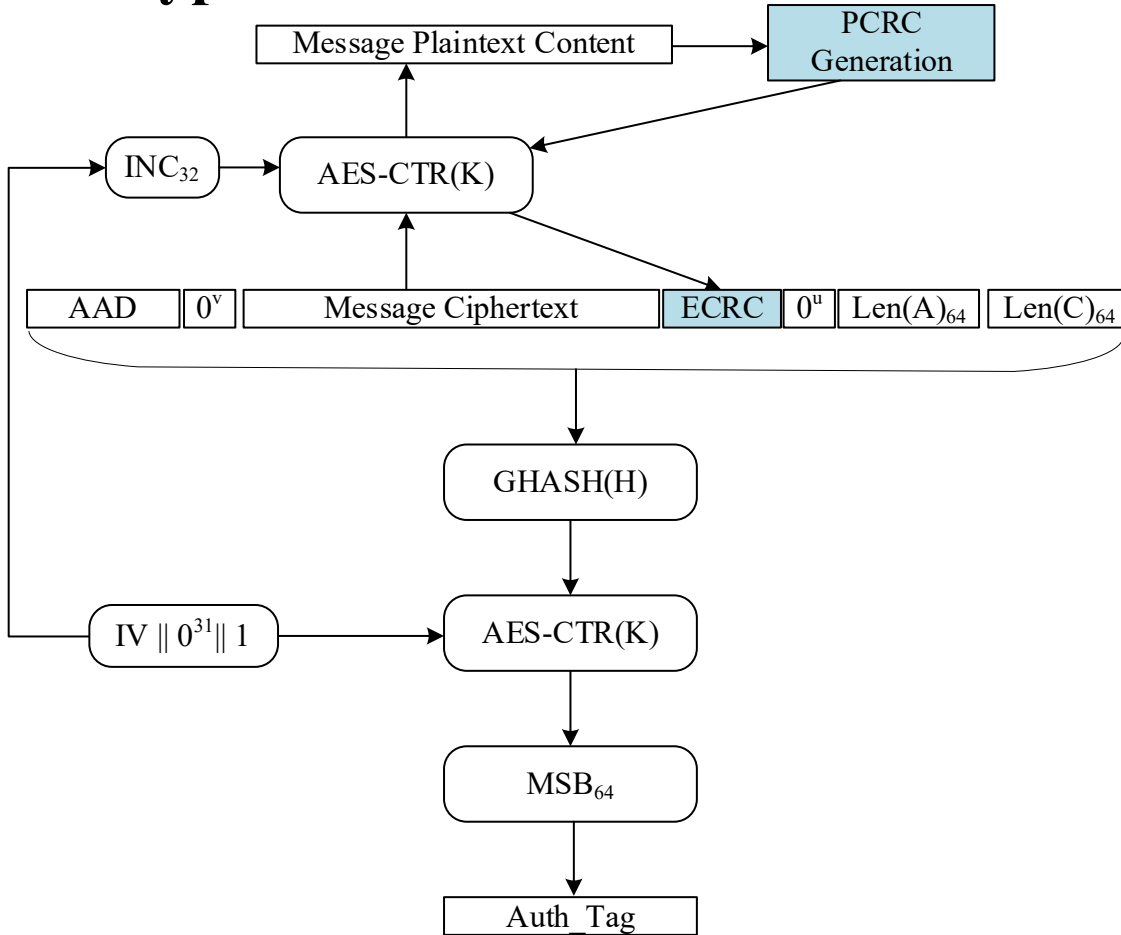


Figure 8-12 PCRC Calculation during the decryption flow on the receiver side

8.6 Security during collective operations

The UALink security allows protecting tenant data during collective operation from the adversaries defined in the security model in Section 8.3. This is an optional feature that can be enabled if a tenant that wants to perform confidential computing uses collectives.

The INC capable switch is assumed to be TEE capable and shall provide an interface to the trusted software on the node for secure configuration and management of tenant's Virtual Pod Partition in the switch. A Virtual Pod Partition in the switch refers to all resources assigned to the Virtual Pod and all configurations associated with the Virtual Pod in the switch. These include switch ports, routing tables, INC primitives, internal memory, debug and performance monitoring settings, and other components of a Virtual Pod in the switch.

Figure 8-13 shows the key elements of the security architecture for protecting tenant data during collective operation:

- A TEE capable switch that has Switch Security Manager (SSM) and a SPDM responder running on the security FW on the physical switch.

- An SPDm session between the TEE Security Manager on one of the nodes in the Virtual Pod and the SSM. Only one of the nodes, designated as the lead node in the Virtual Pod, shall establish an SPDm session with SSM on the switch and will perform security management of the Virtual Pod Partition on the switch. The designated lead node is assumed to perform the following tasks as part of bringing switch into the TCB and enforcing Virtual Pod Partition security.
 - Attest the switch to verify its authenticity and integrity.
 - Send TDISP equivalent commands such as LOCK, START and STOP/UNLOCK to transition the Virtual Pod Partition to different states.
 - Read Virtual Pod Partition configuration settings such as Virtual Pod's routing table to verify it against known good values.
 - Verify the security policy of Virtual Pod Partition to verify that debug is disabled, performance monitoring is off or restricted and other security settings are as expected.

Since a switch can be configured to support multiple Virtual Pods, SSM is required to support multiple SPDm sessions concurrently. The maximum number of concurrent SPDm sessions supported by the switch determines the number of concurrent confidential Virtual Pods (that use collectives) that can run.

The communication between trusted SW on the node and SSM can occur over UART interface between GPU and switch as defined in base specification. In an alternate implementation it may use ethernet channel such as the one used by Pod Controller to communicate with the switch management SW. In the latter case, the untrusted SW on the processor on the switch platform is required to forward the SPDm messages to SSM where the crypto operations shall be performed. UALSec does not dictate use of specific communication mechanism, however it does require that the SPDm crypto operations are performed in the security FW on the switch ASIC.

- A Crypto engine (CE) at the switch port decrypts the incoming collective request or response at the ingress and encrypts the outgoing collective request or response at the egress. The switch shall determine whether the incoming traffic is collective or non-collective traffic (meant for another accelerator) by looking at the command encoding or the tags and only decrypt the collective traffic. The non-collective traffic shall bypass the crypto engine and remain encrypted inside the switch while being routed to the destination accelerator. Likewise, only the outgoing collective traffic shall be encrypted at the egress, and the non-collective traffic shall bypass the crypto engine on the way out and will retain the end-to-end encryption between the accelerator ports. The accelerator-switch encryption keys are selected by the trusted software and sent to SSM over the SPDm session. The SSM shall program the encryption keys into the crypto engine of each switch port in the Virtual Pod Partition.
- The GroupTable in the switch, which is used by INC primitive to look up the group membership information when performing collective operations, is assumed to be programmed securely by the TVMs. The programming may happen via Load/Store over UALink after UALink protection between accelerator and switch has been enabled, in which case it would use the UALSec to protect GroupTable programming. If it is programmed by an untrusted entity such as Pod Controller, it must be verified by the TVM before use.
- The Virtual Pod's Routing Table is expected to be programmed by the Pod Controller. At the time the Virtual Pod Partition is transitioned to LOCK state, the Virtual Pod's Routing Table entry shall be locked and prevented from modification. The TVM is required to verify that

the accelerator ID-port ID mappings in the routing table are programmed correctly before transitioning the Virtual Pod Partition to RUN state via the TDISP equivalent START interface supported by SSM. In an example implementation the lead TVM, which already collects accelerator ID and port numbers for all accelerators via the TVMs on their respective nodes, gets the routing table information from the switch and verifies it matches what is reported by the TVMs.

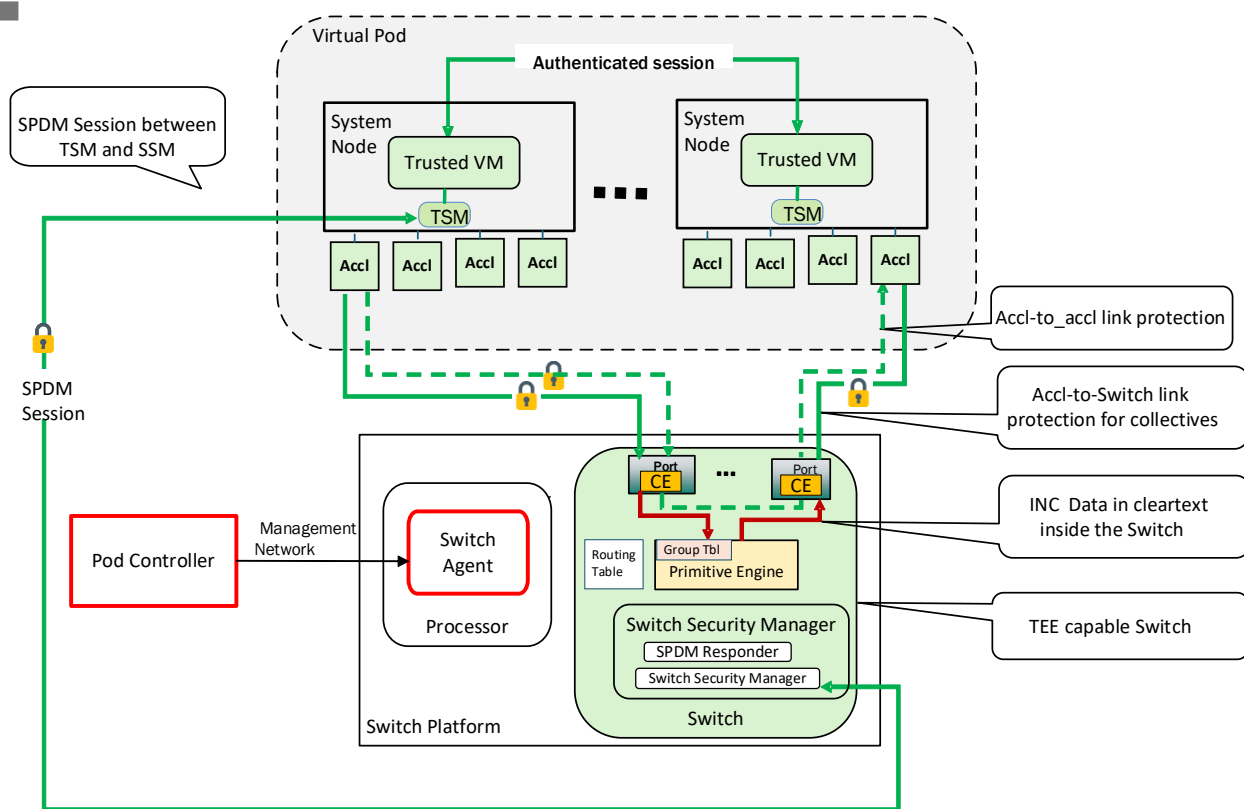


Figure 8-13 Security architecture of a Virtual Pod that uses collectives

8.6.1 Switch Port Identifier

Each switch port in a Pod shall be uniquely identifiable via (DevID/DevType/PortNum) same as an accelerator port. UALSec uses the device ID and port number to construct the Context value used in key derivation function in order to derive port specific unique link encryption key as described in Section 8.6.4.2.

8.6.1.1 Switch ID

While Accelerator ID serves as the device ID for the accelerators in the Pod, switch devices are identified by a Switch ID. Just like the accelerators, the switches are assigned a 10-bit switch ID by the Pod Controller which serves as its device ID in DL messages and neighbor discovery and is used in UALSec for deriving the unique TX and RX keys for encrypting collective transactions between accelerator and switch.

Accelerator IDs and Switch IDs have separate namespace, meaning they can have same IDs, however, within its own namespace the ID shall be unique. No two switches shall have the same switch ID, just as no two accelerators shall have the same accelerator ID. The lead TVM shall verify

the uniqueness of the switch IDs within its Virtual Pod same as how it verifies the uniqueness of accelerator ID within its Virtual Pod. TVM can get the switch ID from the switches over secure SPDM session and verify that they are unique across the Virtual Pod during configuration and verification flow described in Section 8.6.6.

8.6.1.2 Port Number

UALink architecture assumes that the accelerators and switch ASICs come up with their own port numbering. The Pod Controller receives that numbering when the accelerators and switches register with the Pod Controller. The port number, which is a 10-bit value to allow switches to support 1024 accelerators in a physical Pod, is unique within a switch and is verified by the TVM.

8.6.1.3 Neighbor programming, verification and locking

The Pod Controller programs each port on the accelerators and switches with neighbor port's (Device ID, Device Type, and Port Num) where the neighbor is the device on the other end of the cable. In a switched topology, the neighbor of any accelerator port is expected to be a switch port. Conversely, neighbor of an INC switch port is expected to be an accelerator in the current version of the UALink spec which supports only single layer INC switches.

Thus, each accelerator port is programmed with switch ID and PortNum of the switch port it is attached to. And each switch port is programmed with accelerator ID and accelerator port num of the accelerator Port it is connected to.

During port bring up, every port shall discover what DevID/DevType/PortNum is actually on the other side of the link ("Operational Neighbor") using the DL messages defined in the DL/PL specification and verify that the programming by the Pod Controller matches the physical topology. If it does not match, the link shall be prevented from progressing beyond the Data Link Layer (no TL/UPLI traffic), which means there can be no tenant traffic.

When TVM applies TDISP LOCK on the accelerator to lock TDI configuration and equivalent LOCK on the switch to lock Virtual Pod Partition configuration during the setup process, the neighbor information programmed by the Pod Controller is locked on both on the accelerator and the switch and cannot be modified until the Virtual Pod Partition goes to UNLOCK state.

8.6.1.4 Verifying and locking SrcAccID and destAccID at Ingress and Egress

Each switch port has a SrcAccID register at ingress and DestAccID at egress that is programmed by the Pod Controller. Switch Security Manager must enforce that these match the device ID programmed as part of neighbor information programmed at each port.

When the Virtual Pod transitions to LOCK state, these registers shall be locked and prevented from being modified until Virtual Pod Partition transitions to UNLOCK state when the workload terminates or the Virtual Pod Partition goes into an error state.

8.6.2 Encryption and authentication scheme for Accelerator-Switch link

A tenant that has UALink security enabled can optionally enable link protection between the accelerator port and switch ports to protect tenant's collective data over the network.

The current UALinkSec standard uses the exact same encryption and authentication scheme for protecting accelerator-switch link as the one used for end-to-end link protection between accelerators as described in Section 8.5. It uses the standard AES-GCM with crypto processing performed in the protocol/functional layer on the accelerator and the switch ports. It is thus just an instantiation of end-to-end encryption protocol defined between accelerator ports with the difference being that one of the endpoints is the directly attached switch port.

However, device specific performance characteristics must be considered during implementation. For example, if the accelerator as a TX initiates key rolling, the timeouts for response to key roll request may need to account for performance differences in key derivation on the switch ports.

Figure 8-14 shows an example system with two Virtual Pods with Accelerator-Switch link protection enabled for both the Virtual Pods. The accelerator and switch ports for the two virtual pods are programmed with separate set of master keys providing cryptographic isolation between the two mutually untrusted Virtual Pods.

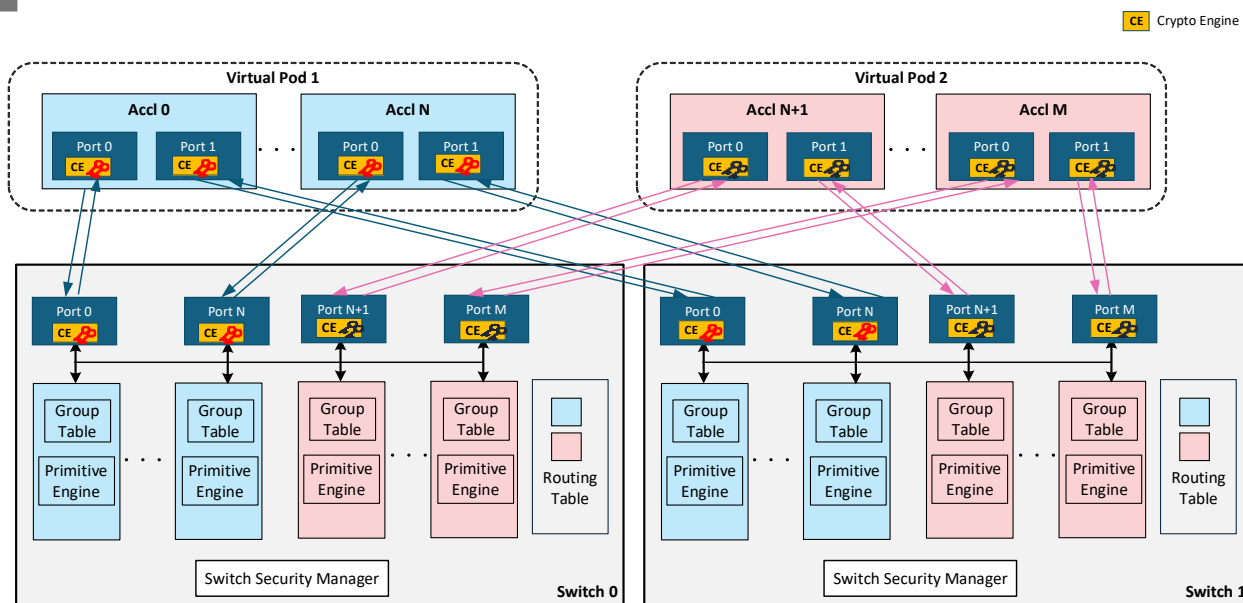


Figure 8-14 Accl-Switch link protection on a system with two Virtual Pods

8.6.3 Ordering Requirements

The current UALink spec (2.0) supports only single-level collectives with collectives supported only in the leaves. The collective transactions are assumed to maintain ordering between a given source and destination pair and within the same stream type.

There are no ordering requirements on collective transactions between different source-destination pairs and between different stream types.

8.6.4 Encryption Keys and Security State Management

There is a single INC active master key per Virtual Pod for deriving TX and RX keys for accelerator-switch links. The $\{accl, accl\}_{MasterKey} \neq \{accl, switch\}_{MasterKey}$ and the trusted software is required to select unique set of master keys (primary and alternate) for accelerator-accelerator links and for accelerator-switch links. The accl-switch master keys are used to derive TX and RX keys for encrypting accl-switch traffic.

There are two INC master keys, a primary and an alternate key that are programmed by the TVMs into each accelerator and switch over secure SPDH session.

While the software implementation of how the master keys are distributed and programmed is out of scope of this UALink specification, an example implementation might be where a lead TVM selects the INC master keys and distributes it securely to TVMs on the other nodes in the Virtual

Pod. Each TVM would then program the INC master key in their local accelerators over TDISP enabled SPDM session. The Device Security Manager (DSM) in each of the accelerator would then distribute the master key over secure HW channel to each port in the accelerator.

The same lead TVM would then distribute the INC master keys to the SSM in each of the switch over SPDM session between the lead TVM and the switch. The SSM in turn would then program the master keys into each of the switch ports that is assigned to the Virtual Pod.

The key management policy for INC keys is same as the one for the accelerator-to-accelerator encryption keys as described in Section 8.5.7.1.1.

8.6.4.1 Key States

Each accelerator port is attached to only 1 switch port and each switch port is attached to only one accelerator port. Therefore, each accelerator port has the following additional keys for accelerator to switch encryption/decryption.

1. 1 set of TX key material for encrypting collective transaction sent from the accelerator to the switch port it is attached to.
2. 1 set of RX key material for decrypting collective transaction received from the switch port it is attached to.

And each switch port has:

1. 1 set of TX key material for encrypting collective transactions sent from the switch to the accelerator port it is attached to
2. 1 set of RX key material for decrypting collective transactions received from the accelerator port it is attached to.

More specifically, each accelerator port and switch port will have the following key material for collective transactions, in addition to the key material described in Section 8.5.7.1.1 for accelerator-to-accelerator encryption.

1. 1 active INC Master Key (32B)
2. 1 alternate INC Master Key (32B)
3. 1 Master key usage counter (4B)

It will have the following TX key material for encrypting and authenticating collective requests:

1. 1 Current TX derived key (32B)
2. 1 Next TX derived key (32B)
3. 1 Invocation field for stream 0 (4B)
4. 1 Invocation field for stream 1 (4B)
5. 1 Invocation field for stream 2 (4B)
6. 1 Epoch counter (4B)

It will have the following RX key material for decrypting and verifying collective response:

1. 1 Current RX derived key (32B)
2. 1 Next RX derived key (32B)
3. 1 Invocation field for stream 0 (4B)
4. 1 Invocation field for stream 1 (4B)
5. 1 Invocation field for stream 2 (4B)
6. 1 Epoch counter (4B)

8.6.4.2 Key derivation for accelerator-switch link protection

The same KDF function as defined in Section 8.5.7.2 is used to derive the TX and RX encryption keys for collective traffic. However, since the endpoints are not symmetrical the Context value, which is an input to the KDF, is constructed using accelerator and switch device IDs as shown in the Table 8-27 and Table 8-28. In a switch-accelerator link, the context value includes 2-bits to identify the device pair type: 2b'01 for source accelerator – destination switch in Table 8-27, and 2b'10 for source switch – destination accelerator in Table 8-28.

Table 8-27 Context for Deriving Accelerator TX and Switch RX keys

Context [73:0]				
[73-72]	[71-56]	[55-40]	[39-32]	[31:0]
Source Device Type 1 = 2b'01 (2-bit)	Padded Source Accelerator ID (16-bit)	Padded Destination Switch ID (16-bit)	Padded VC (8-bit)	Epoch Counter Value (32-bit)

Table 8-28 Context for Deriving Switch TX and Accelerator RX keys

Context [73:0]				
[73-72]	[71-56]	[55-40]	[39-32]	[31:0]
Source Device Type 2 = 2b'10 (2-bit)	Padded Source Switch ID (16-bit)	Padded Destination Accelerator ID (16-bit)	Padded VC (8-bit)	Epoch Counter Value (32-bit)

IV construction for accelerator-switch link protection is described in Section 8.5.7.3.

8.6.5 Detecting collective traffic

The switch port needs to be able to detect if the incoming transaction is a collective transaction in order to decide whether it needs to be decrypted or it should bypass the crypto engine. Likewise, the accelerator needs to be able to detect which incoming transaction is a collective transaction in order to decide if it needs to use the switch key material to authenticate and decrypt it. The command encoding or tag is used to detect collective transactions depending on the requester and responder.

- A collective request from accelerator to switch has the collective command encoded in the ReqCmd field. Switch uses that to detect the incoming collective request. The ReqDstPhysAccID field is overloaded with a GroupID[9:0] value. However, the switch port already knows the requesting accelerator ID from the SrcAccID register at the ingress and has only one RX key (since a switch port is attached only one accelerator port) and does not need to deal with key selection.
- In response to a collective request received from an originating accelerator, the switch routes the collective request to destination accelerators. The switch uses the same ReqCmd encoding to issue a collective command to the Destination Accelerators as the one received

from the Requesting Accelerator allowing the Destination Accelerator to distinguish the collective request and use the switch key material for authentication and decryption.

- The Destination Accelerator Completer retains indications (tags) for whether the Request was a Collective Primitive (ReadReduce, WriteMulticast, WriteFullMulticast, AtomicNRMultiast), a Block Collective (BlockWriteFull, BlockRead), or neither of the above. The Completer uses the indications saved above to set the RdRspTypeInfo or WrRspTypeInfo fields in the Response. The switch port uses RdRspTypeInfo or WrRspTypeInfo fields in the Response to determine if it is a collective response and needs to be authenticated and decrypted at the ingress.
- After the INC primitive has received all the response, it will collate or reduce the response as needed and send the final response back to the originating accelerator. The switch uses the original command encoding in the final response, which indicates to the originating accelerator that the response is a collective response and must be authenticated and decrypted using the switch key material.

8.6.6 Configuring and starting Accelerator-Switch link protection

The UALink protection for collectives is enabled on a per Virtual Pod basis. When the accelerator-switch link protection is enabled, all collective traffic to and from accelerator ports to the neighbor switch ports in the Virtual Pod shall be encrypted and authenticated. All accelerator ports in the Virtual Pod and their neighbor switch ports, shall have security enabled and shall be programmed with the same INC master keys and key rotation and key swap thresholds. They shall all derive the TX and RX keys per the key derivation function described in Section 8.6.4.2. If link protection between accelerator and switch has been enabled, the Virtual Pod Partition in the switch shall not send or receive any tenant transaction until the Virtual Pod Partition has been transitioned to RUN state via TDISP equivalent START interface.

The trusted software that uses link protection for collectives is required to enable and configure both end-to-end link protection between the accelerators and accelerator-switch link protection upfront before starting workload execution. Once the accelerators are in the TDISP RUN state, link protection between accelerator and switch cannot be enabled without transitioning the accelerator to a STOP state via TDISP STOP interface which would require halting the workload execution.

An example workflow for configuring and starting both end-to-end to link protection and accelerator-switch link protection is described as follows:

1. Under the control of a lead TVM, all TVMs in the Virtual Pod enable and configure all accelerators in the Virtual Pod for end-to-end link protection per steps described in Section 8.5.8. However, lead TVM does not tell other TVMs to transition the accelerator to RUN state yet. Which means they do not begin to transmit or receive tenant transactions.
2. The lead TVM sets up SPDM session with SSM of all switches in the Virtual Pod and verifies their attestation report.
3. The lead TVM then transitions the Virtual Pod Partition in each switch to LOCK state via TDISP equivalent LOCK interface supported by SSM.
4. The lead TVM collects the neighbor port information from all TVMs, gets the status of Virtual Pod Partition from the switch and verifies:
 - a. Switch IDs and switch port numbers are unique
 - b. The routing table mappings for the Virtual Pod reported by the switch match the neighbor port mapping collected from the TVMs.

- c. Security settings such as debug state and performance monitoring are configured as expected.
5. The lead TVM selects primary and alternate INC master keys and other global key material and
 - a. Distributes the INC key material to all TVMs which program them in the accelerator ports.
 - b. Programs the INC key material in the switches via their respective SPDM sessions.
6. The lead TVM transitions the Virtual Pod to RUN state:
 - a. Sends SPDM message to the switches to transition the Virtual Pod Partition to RUN state
 - b. Tells all TVMs to transition their accelerators to RUN state.
7. After all of this has been successfully completed successfully, the lead TVM initiates secure workload execution.

8.6.7 Handling integrity failure in Accelerator-Switch link

If either accelerator or switch detect an integrity failure during authentication tag verification, it shall drop the failing transaction and stop processing new transactions (all streams) from that source, thus stalling the pipeline which will eventually result in timeout. Only the port that found the failure will take this action. Additionally, the associated transmitter on the same port shall stop sending any transaction to that neighbor.

If the failure is detected in the accelerator port, it shall isolate the neighbor switch and notify the accelerator security processor using a mechanism that is secure from tampering by the adversaries defined in the security model. The security processor is required to notify the software allowing software to take appropriate action for handling the failure.

If the failure is detected in the switch port, it shall isolate the neighbor accelerator and notify the SSM. The SSM shall report this failure to the Pod Controller software which is required to notify the TVM via the Node Agent. Note, the Pod Controller is not trusted to guarantee the delivery of failure notification to TVM. However, failing to inform the TVM prevents the TVM from handling the failure gracefully and can only result in denial of service attack which is outside the security scope of UALSec.

Failure in one of the accelerator-switch link does not cause other links to fail. The other links will continue to operate normally, though it is expected that the SW would likely need to stop and reset the Virtual Pod as the failure in one link may result in cascading failure across the workload execution,

8.7 Keyrollmsg formats

1.5 version of the specification modifies the Keyrollmsg in the following way:

1. The Keyrollmsg is used to trigger a key derivation in an accelerator endpoint if $\text{Attr}[0] == 0$ and $\text{Attr}[1] == 0$.
2. The keyrollmsg triggers a key derivation and a key swap in an accelerator endpoint if $\text{Attr}[0] == 1$ and $\text{Attr}[1] == 0$
3. The Keyrollmsg is used to trigger a key derivation in an INC capable switch that supports INC security if $\text{Attr}[0] == 0$ and $\text{Attr}[1] == 1$.
4. The keyrollmsg triggers a key derivation and a key swap in an INC capable switch that support INC security if $\text{Attr}[0] == 1$ and $\text{Attr}[1] == 1$

Security

244

Switches shall parse Keyrollmsg request only if it supports INC and INC security is enabled otherwise it shall forward it to the destination that is specified in the destination ID field.

If a requestor sends a keyrollmsg with Attr[1] = 1 to an accelerator Endpoint, the receiving endpoint shall drop it and raise an error event to the security processor in that accelerator.

8.8 Specification version compatibility

There is no expectation that a 2.0 compliant accelerator or switch needs to be compatible with 1.0 for security features. An implementation is permitted to be only 2.0 compliant or 1.0 compliant or both.

Possible operational modes are captured in the table below :

Table 29 Possible operational modes for various specification versions

Specification version	At least two end to end VCs enabled & SOL mode	At least two end to end VCs enabled & SO mode	Only default VC (VC0) enabled
2.0	YES	YES	YES
1.0	NO	YES	YES

9 UALink Switch Requirements

9.1 Overview

UALink Switches serve to relay UALink Requests and Responses between Accelerators.

All switch ports shall connect to Accelerators and Switch-to-switch links shall not be supported. Each Request or Response shall be produced by a Source Accelerator, shall enter the Switch via an Ingress Link, shall be relayed by the Switch to a single egress Link, where it shall be consumed by a Destination Accelerator. All Response and Request relaying for traffic other than In Network Collective traffic shall be unicast.

For traffic other than In Network Collective traffic, a UALink Switch is only responsible for the delivery of Requests and Responses between accelerators and shall not be required to decode the contents of these Requests or Responses beyond that necessary to deliver the Requests and Responses, nor shall the Switch track any Request/Response state.

Inbound DL flits arriving from the Source Accelerator shall be unpacked into TL flits, which shall then be further unpacked into individual Requests and Responses for forwarding between Ports. The Destination Accelerator ID field within each Request or Response shall index into a routing table to determine the egress Station and Port to which it must be forwarded. Once forwarded between ports, outbound Requests and Responses shall be packed into TL flits, which in turn shall be packed into DL flits and sent to the Destination Accelerator.

Each Request or Response shall be routed independently. Multiple Requests or Responses unpacked from the same inbound TL flit do not necessarily route to the same egress Station and Port and multiple Requests and Responses packed into the same outbound TL may have arrived from multiple ingress stations and ports.

The UALink Switch shall contain a complete implementation of the UALink DL and TL, similar to that found in an Accelerator. The nominal interface between TL and switch core shall be the UPLI Originator/Completer interface, although a Switch design may substitute any suitable vendor-defined interface. Any vendor-defined interface replacing UPLI shall mimic any UPLI mandated Originator or Completer behaviors in this Specification, including but not limited to UPLI Drop modes (see 3.1.3) and Port ID field manipulation (See 2.4).

Flow control across an individual link between accelerator and switch, including the required independence of Requests vs Responses, shall be handled by the TL layer logic on each end of the link. Requests or Responses stalled by flow control shall wait within the outbound TL until they are eligible, at which time they may be included in a TL flit being packed and sent. Flow control for Requests or Responses being forwarded between Source and Destination Stations also respects the independence of Requests versus Responses, as does the UPLI interface between TL and switch core.

9.2 Bifurcation support

Link bifurcation feature allows a station to split into multiple independent ports. Each ULS station has four lanes. Each station shall support independent configuration as a single four-lane port, two two-lane ports, or four single-lane ports.

9.3 Lossless Request and Response delivery

Except in error cases which explicitly require Requests or Responses to be dropped (see Section 3.1.3), all Requests or Responses shall be relayed through the switch in a lossless manner. Link

level Retry (LLR) shall be used to ensure delivery from source Accelerator to the switch ingress port, and from switch egress port to destination Accelerator. Flow control shall ensure that Requests and Responses are never dropped due to receive-buffer space limitations.

9.4 Non-blocking architecture

Traffic between any one pair of Ports shall operate independently of traffic between any other pair of Ports. However, switch core architecture may share resources across ports within a station or even amongst multiple stations.

Ingress and egress traffic of a port shall operate independently of each other. Ingress Requests shall operate independently of ingress Responses, and egress Requests shall operate independently of egress Responses.

Stalled egress Requests or Responses to one port of a station may block same station egress Requests or Responses headed towards another port of the same station. Similarly, stalled ingress Requests or Responses from one port of a station may block like ingress Requests or Responses from another port of the same station. Under no circumstances shall stalled Requests be permitted to block Responses.

While such blocking is permitted, it should be minimized by switch designs to the extent reasonably possible. Non-blocking architecture is a crucial property of network switch design. Switches should incorporate sufficient buffering at the ingress and/or egress ports to prevent congestion and blocking.

9.5 Forward progress guarantee

All arbitration within the ULS shall be starvation-free, to avoid livelock or starvation cases. This shall include but is not limited to arbitration between ingress ports within a station, arbitration between ingress stations competing for access to an egress station, and arbitration between Requests and Responses for access to available TL flit sectors.

Starvation-free arbitration may include simple round-robin, weighted and/or deficit round-robin, age-based oldest-first, etc.

9.6 Ordering and Virtual Channels

Each port-to-port path through ULS shall follow Request or Response delivery ordering and Virtual Channel handling as defined for UPLI (see 2.7.9). Support for Strict Ordering mode is mandatory, and support for non-Strict Ordering mode is optional. If non-Strict Ordering mode is supported, it shall be selected on a per-port basis, with identical settings being used on ingress and egress ports.

9.7 Routing Table Structure

Routing decisions shall be made by looking up an entry in a Route Table at each Port based on the 10-bit Destination Accelerator ID of inbound Requests and Responses (Destination Accelerator IDs shall be numbered from 0 to $n-1$ where “ n ” is the maximum number of Accelerators a Switch may support). The number of entries in the table shall equal or exceed the maximum number of supported ports in the Switch with single-lane bifurcation. Each table entry shall contain the following information:

- An indication of whether to allow or deny routing Requests or Responses received by the port based on the Destination Accelerator ID of the Request or Response. The Deny setting may be used to prevent routing Switches in a partitioned Physical Switch, and between Virtual Pods. Upon Switch reset, the Deny setting shall apply to all table entries.

- An indication of the Egress Port on the Switch (in terms of the Station and Port) to which the Request or Response shall be routed if allowed. For example, in a switch with 64 stations, each bifurcable into at most four ports, each route table entry shall supply a 6-bit station number and a 2-bit port number.

Where the table contains fewer than 1,024 entries, Requests or Responses whose Destination Accelerator ID value exceeds the size of the table shall be denied routing. Requests or Responses which are denied routing (either due to the allow/deny indication or exceeding the Route Table size) shall be silently dropped and shall be logged by the management controller.

9.7.1 Routing Table Instances

The switch shall contain a separate, independently programmable instance of the Route Table for each possible Port (due to bifurcation) per station (i.e. a station that can be bifurcated into four Ports shall have four independent Route Tables).

The independent programmability of different routing table instances, and of the independent deny controls for different ports within each bi-furcated station, allow a switch to be subdivided into multiple independent Virtual Switches , serving multiple Virtual Pods.

9.7.2 Egress port reachability

All egress ports of all stations shall be reachable from any ingress port, with no arbitrary restrictions. This shall include allowing route-to-self (a.k.a. U-turn) cases, where the Requests and Responses ingress and egress via the same port.

9.8 Configuration

Configurability options are left up to the implementation. The list below is an incomplete but a useful list:

- Accelerator ID value associated with each port.
- Routing table associated with each port, as described in Section (see 9.7)Bifurcation mode for each station.
- For each port, the Strict versus non-Strict ordering mode (see 9.6).
- For each port, whether or not Authentication is enabled, which affects TL flit packing and unpacking.
- Ability to determine which stations are in drop mode, and in the case of port-granular modes, ability to determine which ports are in drop mode.
- Ability to reset drop modes to resume normal operation, with station granular or port granular control, as appropriate.

9.9 UALink Switch Recommendations and Goals

The following subsections are recommendations and goals and are not meant to form UALink switch requirements but are meant to help guide implementors in areas of importance.

9.9.1 Debug

It is recommended that UALink switches support both transaction injection with and without errors towards switch core and towards the UALink. Other vendor defined debug support is allowed, but none is required under this section. UALink switches are recommended to support injection of key errors (i.e. link errors, link down, at least one silicon ECC error) to allow for platform/system level testing of RAS. The Injection, if supported, shall be disabled by default during mission mode and shall have a mechanism to allow enabling injection through a secure FW switch.

9.9.2 Latency goals

UALink Switches are recommended to target the following idle and unloaded pin-to-pin latency for a 64 byte Write Request on a 4-lane non bifurcated port with FEC enabled based on size of the switch:

- 128 lane switch: <200ns
- 256 lane switch: <250ns
- 512 lane switch: <300ns

9.9.3 Performance goals

It is recommended that UALink Switch core should maintain a post-FEC line rate of 200Gbps. Switches should be capable of maintaining line rate for pairwise communication as well as concurrent communication across a small group (8-to-32) of accelerators. DL and TL protocol overhead will lower the effective line rate based on the traffic pattern and security protocols.